

SDVoE Solution

API Developers Reference Guide

Table of Contents

1	PURPOSE	12
1.1	VERSION INFORMATION.....	12
1.2	TARGET AUDIENCE	12
1.3	REFERENCE MATERIALS	12
2	OUTLINE OF THE API.....	13
2.1	EVENTS	13
2.2	COMMANDS TYPES.....	13
2.3	DEVICE MODEL, LIFE CYCLE & COMMAND STRUCTURE.....	13
2.4	DESCRIPTION OF STREAMS & DESTINATIONS	13
2.5	SPECIFYING VIDEO OUTPUT FORMAT	14
2.5.1	Command Options.....	14
2.5.2	Video Display Mode Argument.....	14
2.5.3	Video Format Arguments.....	15
2.5.4	Uniquely Identifying a Video Format	17
3	CORE COMMANDS	18
3.1	COMMAND MODE HUMAN	18
3.1.1	Arguments	18
3.1.2	Return Value	19
3.1.3	Errors	19
3.1.4	Example	19
3.2	COMMAND REQUIRE	19
3.2.1	Arguments	19
3.2.2	Return Value	19
3.2.3	Errors	19
3.2.4	Example	20
3.3	COMMAND VERSION	21
3.3.1	Arguments	21
3.3.2	Return Value	21
3.3.3	Errors	21
3.3.4	Example	21
4	API COMMAND REFERENCE	23
4.1	SUMMARY LIST OF API COMMANDS	23
4.2	COMMANDS THAT MODIFY DEVICE SETTINGS	27
4.3	COMMANDS AUTH & UNAUTH.....	27
4.3.1	Arguments	28
4.3.2	Return Value	28
4.3.3	Errors	28
4.3.4	Examples.....	28
4.4	COMMANDS CLAIM & RELEASE	29
4.4.1	Arguments	30
4.4.2	Return Value	30
4.4.3	Errors	30
4.4.4	Examples.....	30
4.5	COMMAND EVENT	31
4.5.1	Arguments	31
4.5.2	Return Value	32

4.5.3	Errors	33
4.5.4	Example	34
4.6	COMMAND FACTORY	34
4.6.1	Arguments	34
4.6.2	Return Value	35
4.6.3	Errors	35
4.6.4	Example	35
4.7	COMMAND GET	35
4.7.1	Arguments	36
4.7.2	Return Value	37
4.7.3	Errors	38
4.7.4	Examples	38
4.8	COMMAND USB_EXTENDER	38
4.8.1	Arguments	38
4.8.2	Return Value	38
4.8.3	Errors	39
4.8.4	Example	39
4.9	COMMAND JOIN	39
4.9.1	Arguments	39
4.9.2	Return Value	40
4.9.3	Errors	40
4.9.4	Examples of Using the <i>join</i> Command	41
4.10	COMMAND LEAVE	44
4.10.1	Arguments	44
4.10.2	Return Value	44
4.10.3	Errors	44
4.10.4	Example	45
4.11	COMMAND LIST	45
4.11.1	Command <i>list firmware</i>	45
4.11.2	Command <i>list multicast</i>	46
4.12	COMMAND NETSTAT	48
4.12.1	Network Characterization Assumptions & Limitations	48
4.12.2	Command <i>netstat read</i>	49
4.12.3	Command <i>netstat clear</i>	63
4.12.4	Command <i>netstat select</i>	66
4.13	COMMAND PAIR/UNPAIR	69
4.13.1	Command <i>pair</i>	69
4.13.2	Command <i>pair_mac</i>	70
4.13.3	Command <i>unpair</i>	70
4.14	COMMAND REBOOT	72
4.14.1	Arguments	72
4.14.2	Return Value	72
4.14.3	Errors	73
4.14.4	Example	73
4.15	COMMAND RELEASE	73
4.16	COMMAND REQUEST	74
4.16.1	Arguments	74
4.16.2	Return Value	74
4.16.3	Errors	74
4.16.4	Examples	74
4.16.5	Requests Associated with Background Commands	74
4.16.6	Requests Associated with Event <i>CEC_RECEIVED</i>	76

4.16.7	Requests Associated with Event <i>DEVICE_CONNECTED</i>	77
4.16.8	Requests Associated with Event <i>DEVICE_DISCONNECTED</i>	77
4.16.9	Requests Associated with Event <i>DEVICE_INFO_AVAILABLE</i>	77
4.16.10	Requests Associated with Event <i>FIRMWARE_UPDATE_PROGRESS</i>	78
4.16.11	Requests Associated with Event <i>I2C</i>	79
4.16.12	Requests Associated with Event <i>INFRARED_RECEIVED</i>	81
4.16.13	Requests Associated with Event <i>MULTIVIEW_CHANGED</i>	82
4.16.14	Requests Associated with Event <i>RS232_RECEIVED</i>	84
4.16.15	Requests Associated with Event <i>SETTINGS_CHANGED</i>	85
4.17	COMMAND SEND	88
4.17.1	Command <i>send cec</i>	88
4.17.2	Command <i>send infrared</i>	89
4.17.3	Command <i>send rs232</i>	91
4.18	COMMAND SEND_WOL	94
4.18.1	Arguments	94
4.18.2	Return Value	94
4.18.3	Errors	95
4.18.4	Example	95
4.19	COMMAND SET	96
4.19.1	Command <i>set dot1x_certificates</i>	96
4.19.2	Command <i>set edid</i>	98
4.19.3	Command <i>set gpio</i>	99
4.19.4	Command <i>set ip</i>	101
4.19.5	Command <i>set overlay</i>	103
4.19.6	Command <i>set property</i>	106
4.19.7	Command <i>set scaler</i>	108
4.19.8	Command <i>set secure_av</i>	109
4.19.9	Command <i>set security_key</i>	111
4.19.10	Command <i>set thumbnail</i>	113
4.19.11	Command <i>set usb</i>	115
4.19.12	Command <i>set video</i>	118
4.20	COMMAND SFP_READ	123
4.20.1	Arguments	123
4.20.2	Return Value	123
4.20.3	Errors	124
4.20.4	Example	124
4.21	COMMAND START	124
4.21.1	Arguments	124
4.21.2	Return Value	126
4.21.3	Errors	126
4.21.4	Examples	126
4.22	COMMAND STOP	127
4.22.1	Arguments	127
4.22.2	Return Value	128
4.22.3	Errors	128
4.22.4	Examples	129
4.23	COMMAND SWITCH	130
4.23.1	Arguments	131
4.23.2	Return Value	132
4.23.3	Errors	132
4.23.4	Examples	132
4.24	COMMAND UPDATE	135

4.24.1	Arguments	135
4.24.2	Return Value	135
4.24.3	Errors	136
4.24.4	Examples.....	136
4.25	COMMAND UPDATE_STREAM	137
4.25.1	Arguments	137
4.25.2	Return Value	137
4.25.3	Errors	137
4.25.4	Examples.....	137
4.26	COMMAND USB	138
4.26.1	USB Descriptors	138
4.26.2	Command <i>usb descriptor read</i>	140
4.26.3	Command <i>usb descriptor write</i>	142
5	MULTIVIEW COMMAND REFERENCE	145
5.1	MULTIVIEW MODEL	145
5.1.1	Multiview Mode Guidelines & Limitations.....	145
5.2	MULTIVIEW COMMANDS.....	145
5.2.1	Summary of Multiview Commands.....	145
5.2.2	Command <i>list layout</i>	146
5.2.3	Command <i>layout create</i>	147
5.2.4	Command <i>layout delete</i>	151
5.2.5	Command <i>layout describe</i>	152
5.2.6	Command <i>layout surface</i>	158
5.2.7	Command <i>layout window</i>	160
5.2.8	Command <i>list layout</i>	163
5.2.9	Command <i>set multiview</i>	164
5.3	DEFAULT SURFACE CONFIGURATION	166
5.4	MULTIVIEW EXAMPLES.....	166
5.4.1	Picture-in-Picture Example	166
5.4.2	Quad Split Layout Example.....	167
6	DEVICE OBJECT DEFINITION (DEVICEOBJECT)	170
6.1	GENERIC TYPES.....	171
6.1.1	Video Format Description (<i>VideoFormat</i>).....	171
6.1.2	Video Format Detailed Information (<i>VideoFormatDetails</i>).....	172
6.1.3	Audio Format Detailed Information (<i>AudioFormatDetails</i>).....	174
6.1.4	Audio Format Extended Information (<i>AudioFormatExtended</i>).....	174
6.1.5	Source Reference Specification (<i>SourceReference</i>).....	175
6.1.6	Source Selection (<i>SourceSelection</i>).....	176
6.1.7	Source Selection Choice (<i>SourceSelectionChoice</i>).....	176
6.2	DEVICE IDENTITY (DEVICEIDENTITY)	176
6.3	DEVICE CAPABILITIES (DEVICECAPABILITIES)	179
6.4	DEVICE CONFIGURATION (DEVICECONFIG).....	182
6.5	DEVICE STATUS (DEVICESTATUS).....	183
6.6	STREAMS (STREAMOBJECT).....	184
6.6.1	Stream Configuration (<i>StreamConfig</i>).....	185
6.6.2	Stream Status (<i>StreamStatus</i>).....	185
6.7	SUBSCRIPTIONS (SUBSCRIPTIONOBJECT)	186
6.7.1	Subscription Configuration (<i>SubscriptionConfig</i>).....	186
6.7.2	Subscription Status (<i>SubscriptionStatus</i>).....	187
6.8	NODES (NODE).....	187

6.8.1	Node Inputs (<i>NodeInput</i>).....	188
6.8.2	Stereo Audio Input Node (<i>STEREO_AUDIO_INPUT</i>)	189
6.8.3	Stereo Audio Input/Output Node (<i>STEREO_AUDIO_INPUT_OUTPUT</i>).....	190
6.8.4	Stereo Audio Output Node (<i>STEREO_AUDIO_OUTPUT</i>)	192
6.8.5	Bitmap Overlay Node (<i>BITMAP_OVERLAY</i>)	194
6.8.6	CEC Node (<i>CEC</i>).....	195
6.8.7	Color Generator Node (<i>COLOR_GENERATOR</i>).....	197
6.8.8	Frame Buffer Node (<i>FRAME_BUFFER</i>).....	198
6.8.9	Frame Rate Converter Node (<i>FRAME_RATE_CONVERTER</i>)	202
6.8.10	HDMI Decoder Node (<i>HDMI_DECODER</i>)	203
6.8.11	HDMI Encoder Node (<i>HDMI_ENCODER</i>)	209
6.8.12	HDMI Monitor Node (<i>HDMI_MONITOR</i>)	214
6.8.13	Multichannel Audio Input Node (<i>MULTICH_AUDIO_INPUT</i>).....	216
6.8.14	Multichannel Audio Output Node (<i>MULTICH_AUDIO_OUTPUT</i>).....	226
6.8.15	Infrared Decoder Node (<i>INFRARED_DECODER</i>).....	229
6.8.16	Infrared Encoder Node (<i>INFRARED_ENCODER</i>)	230
6.8.17	Network Interface Node (<i>NETWORK_INTERFACE</i>).....	231
6.8.18	Network Port Node (<i>NETWORK_PORT</i>).....	233
6.8.19	Network Switch Node (<i>NETWORK_SWITCH</i>).....	234
6.8.20	Scaler Node (<i>SCALER</i>)	234
6.8.21	Thumbnail Generator Node (<i>THUMBNAIL_GENERATOR</i>)	236
6.8.22	UART Node (<i>UART</i>).....	238
6.8.23	USB Human Interface Device (HID) Extender Node (<i>USB_HID</i>)	239
6.8.24	USB Extender Node (<i>USB_EXTENDER</i>).....	244
6.8.25	Video Input Node (<i>VIDEO_INPUT</i>).....	250
6.8.26	Video Compressor Node (<i>VIDEO_COMPRESSOR</i>).....	251
6.8.27	Video Decompressor Node (<i>VIDEO_DECOMPRESSOR</i>)	252
7	CONFIGURABLE DEVICE PROPERTIES.....	254
7.1	DEVICE CONFIGURATION	254
7.2	STREAM CONFIGURATION	255
7.3	STEREO AUDIO INPUT/OUTPUT NODE CONFIGURATION	256
7.4	STEREO AUDIO OUTPUT NODE CONFIGURATION	256
7.5	CEC NODE CONFIGURATION	257
7.6	COLOR GENERATOR NODE CONFIGURATION.....	258
7.7	FRAME BUFFER NODE CONFIGURATION.....	259
7.8	HDMI AUDIO NODE CONFIGURATION	260
7.9	HDMI DECODER NODE CONFIGURATION	260
7.10	HDMI ENCODER NODE CONFIGURATION	261
7.11	MULTICHANNEL AUDIO INPUT NODE CONFIGURATION.....	262
7.12	MULTICHANNEL AUDIO OUTPUT NODE CONFIGURATION.....	263
7.13	NETWORK PORT NODE CONFIGURATION	264
7.14	NETWORK SWITCH NODE CONFIGURATION	264
7.15	SCALER NODE CONFIGURATION	264
7.16	UART NODE CONFIGURATION	265
7.17	USB EXTENDER NODE CONFIGURATION	265
7.18	VIDEO COMPRESSOR NODE CONFIGURATION	266
7.19	VIDEO DECOMPRESSOR NODE CONFIGURATION	266
	APPENDIX: GET COMMAND OUTPUT EXAMPLES	267
A.1	COMMAND “GET_DEVICE” FOR BLUERIVER TRANSCEIVER DEVICE	267
A.2	COMMAND “GET_DEVICE” FOR UEX2000 LOCAL DEVICE	316

A.3	COMMAND “GET EDID” FOR BLUERIVER TRANSMITTER DEVICE	318
A.4	COMMAND “GET EDID” FOR BLUERIVER RECEIVER DEVICE	319
A.5	COMMAND “GET HELLO” FOR BLUERIVER AND UEX DEVICES	320
A.6	COMMAND “GET IDENTITY” FOR BLUERIVER DEVICES	321
A.7	COMMAND “GET IDENTITY” FOR UEX DEVICES	322
A.8	COMMAND “GET LIST” FOR BLUERIVER AND UEX DEVICES	323
A.9	COMMAND “GET SETTINGS” FOR BLUERIVER TRANSCEIVER DEVICE	325
A.10	COMMAND “GET SETTINGS” FOR UEX DEVICE	357
A.11	COMMAND “GET PATH” FOR BLUERIVER RECEIVER DEVICE	359
A.12	COMMAND “GET USB_TREE” FOR BLUERIVER OR UEX DEVICE	360
APPENDIX: NON GET COMMAND OUTPUT EXAMPLES		362
A.1	COMMAND “LAYOUT DESCRIBE”	362
A.2	COMMAND “MULTIVIEW_CHANGED” EVENT	366
8	REVISION HISTORY	370

Semtech Confidential Information
Mark Li
ADV
02/03/2026

List of Tables

TABLE 1: VIDEO DISPLAY MODE SUPPORTED KEYWORDS	14
TABLE 2: SUPPORTED VIDEO IDENTIFICATION CODES (VIC).....	16
TABLE 3: API CORE COMMANDS.....	18
TABLE 4: VERSION COMMAND RETURN VALUES OBJECT MEMBERS	21
TABLE 5: VERSION JSON MODULE DESCRIPTION OBJECT MEMBERS	21
TABLE 6: LIST OF API COMMANDS.....	23
TABLE 7: RETURN VALUE SCHEMA OBJECT MEMBERS	27
TABLE 8: DEVICESUCCESS SCHEMA OBJECT MEMBER	27
TABLE 9: EVENT OBJECT MEMBERS.....	32
TABLE 10: SUBSET ARGUMENTS FOR "GET" COMMAND.....	37
TABLE 11: GET COMMAND OBJECT MEMBERS.....	38
TABLE 12: LIST FIRMWARE ARRAY MEMBER.....	45
TABLE 13: LIST FIRMWARE OBJECT MEMBERS	46
TABLE 14: LIST MULTICAST OBJECT ARRAY MEMBER	46
TABLE 15: LIST MULTICAST OBJECT MEMBERS.....	47
TABLE 16: NETSTAT READ OBJECT ARRAY MEMBERS	49
TABLE 17: NETSTAT DEVICE OBJECT ARRAY MEMBERS.....	50
TABLE 18: NETSTAT ERROR OBJECT ARRAY MEMBERS.....	50
TABLE 19: RESPONSES TO NETSTAT READ PER DEVICE TYPE.....	50
TABLE 20: 10G TX EGRESS DATA PATH OBJECT MEMBERS.....	51
TABLE 21: 10G TX EGRESS BANDWIDTH OBJECT MEMBERS.....	51
TABLE 22: 10G TX EGRESS BANDWIDTH SELECTION OBJECT MEMBERS	52
TABLE 23: 10G TX EGRESS BANDWIDTH USAGE OBJECT MEMBERS	53
TABLE 24: 10G TX EGRESS COUNTERS OBJECT MEMBER.....	53
TABLE 25: 10G TX EGRESS MISS FRAMES FLAGS OBJECT MEMBERS.....	54
TABLE 26: 10G TX INGRESS DATA PATH OBJECT MEMBERS.....	54
TABLE 27: 10G TX INGRESS COUNTERS OBJECT MEMBERS	55
TABLE 28: 10G TX INGRESS ERROR OBJECT MEMBERS	55
TABLE 29: 10G RX EGRESS DATA PATH OBJECT MEMBERS.....	56
TABLE 30: 10G RX EGRESS MISSED FRAMES FLAGS OBJECT MEMBERS.....	56
TABLE 31: 10G RX INGRESS DATA PATH OBJECT MEMBERS.....	57
TABLE 32: 10G RX INGRESS BANDWIDTH OBJECT MEMBER.....	57
TABLE 33: 10G RX INGRESS BANDWIDTH USAGE OBJECT MEMBER.....	57
TABLE 34: 10G RX INGRESS COUNTERS OBJECT MEMBERS.....	57
TABLE 35: 10G RX INGRESS ERROR FLAGS OBJECT MEMBERS	58
TABLE 36: 10G TXRX INGRESS DATAPATH OBJECT MEMBERS.....	58
TABLE 37: NETSTAT CLEAR RETURN VALUE OBJECT MEMBERS.....	64
TABLE 38: NETSTAT CLEAR DEVICE RESPONSE OBJECT MEMBER	64
TABLE 39: NETSTAT CLEAR DEVICE ERROR OBJECT MEMBERS.....	64
TABLE 40: SUPPORTED BANDWIDTH CHANNEL SELECTIONS	66
TABLE 41: NETSTAT SELECT DEVICE OBJECT MEMBERS.....	67
TABLE 42: NETSTAT SELECT DEVICE RESPONSE OBJECT MEMBER	67
TABLE 43: NETSTAT SELECT DEVICE ERROR OBJECT MEMBERS.....	67
TABLE 44: SUPPORTED PAIRINGS FOR SEMTECH UEX AND ICRON EXTENDERS	71
TABLE 45: REBOOT OBJECT ARRAY MEMBERS	72
TABLE 46: REBOOT OBJECT MEMBER.....	73
TABLE 47: CEC OBJECT ARRAY MEMBER	76
TABLE 48: CEC OBJECT MEMBERS	76
TABLE 49: UPDATE PROGRESS OBJECT ARRAY MEMBER	78
TABLE 50: UPDATE PROGRESS OBJECT MEMBERS.....	78

TABLE 51: I ² C DATA OBJECT ARRAY MEMBER	79
TABLE 52: I ² C DATA OBJECT MEMBERS.....	79
TABLE 53: INFRARED (IR) OBJECT ARRAY MEMBER.....	81
TABLE 54: INFRARED (IR) OBJECT MEMBERS	81
TABLE 55: MULTIVIEW LAYOUT OBJECT MEMBERS	82
TABLE 56: SURFACE OBJECT MEMBERS	83
TABLE 57: MULTIVIEW OBJECT ARRAY MEMBERS.....	83
TABLE 58: RS-232 OBJECT ARRAY MEMBER	84
TABLE 59: RS-232 OBJECT MEMBERS	84
TABLE 60: SETTINGS SUBSETS	86
TABLE 61: CEC OBJECT ARRAY MEMBERS	88
TABLE 62: CEC OBJECT MEMBER.....	89
TABLE 63: INFRARED (IR) OBJECT ARRAY MEMBERS	90
TABLE 64: INFRARED (IR) OBJECT MEMBER	90
TABLE 65: SEND RS232 OBJECT ARRAY MEMBERS	92
TABLE 66: SEND RS232 OBJECT MEMBER.....	92
TABLE 67: WOL OBJECT ARRAY MEMBERS	94
TABLE 68: WOL OBJECT MEMBERS	94
TABLE 69: LIST OF CONFIGURABLE PROPERTIES	107
TABLE 70: UPDATE OBJECT ARRAY MEMBERS.....	135
TABLE 71: UPDATE OBJECT MEMBER.....	136
TABLE 72: USB DESCRIPTORS HEADER	139
TABLE 73: REPORT DESCRIPTORS HEADING	140
TABLE 74: USB DESCRIPTOR READ OBJECT ARRAY MEMBERS	141
TABLE 75: USB DESCRIPTOR READ OBJECT MEMBERS.....	142
TABLE 76: USB DESCRIPTOR WRITE OBJECT ARRAY MEMBERS.....	143
TABLE 77: USB DESCRIPTOR WRITE RESULT OBJECT MEMBERS.....	143
TABLE 78: MULTIVIEW LAYOUT COMMAND SET	146
TABLE 79: LAYOUT ITEM MEMBER.....	146
TABLE 80: LAYOUT ITEM OBJECT MEMBER.....	146
TABLE 81: LAYOUT DESCRIBE OBJECT ARRAY MEMBER.....	152
TABLE 82: LAYOUT DESCRIBE OBJECT MEMBERS	152
TABLE 83: SURFACE DESCRIPTION OBJECT MEMBERS.....	153
TABLE 84: WINDOW DESCRIPTION OBJECT MEMBERS.....	153
TABLE 85: LIST LAYOUT OBJECT ARRAY MEMBER	163
TABLE 86: LIST LAYOUT OBJECT MEMBER	163
TABLE 87: LIST OF SUPPORTED DEVICE OBJECTS.....	170
TABLE 88: VIDEO FORMAT DESCRIPTION OBJECT MEMBERS.....	171
TABLE 89: VIDEO FORMAT DETAILED INFORMATION OBJECT MEMBERS.....	172
TABLE 90: AUDIO FORMAT DETAILED INFORMATION OBJECT MEMBERS	174
TABLE 91: VIDEO FORMAT EXTENDED INFORMATION OBJECT MEMBERS	175
TABLE 92: SOURCE REFERENCE SPECIFICATION OBJECT MEMBERS	175
TABLE 93: SOURCE SELECTION OBJECT MEMBERS	176
TABLE 94: SOURCE SELECTION CHOICE OBJECT MEMBERS	176
TABLE 95: DEVICE IDENTITY OBJECT MEMBERS.....	177
TABLE 96: FIRMWARE DETAILS OBJECT MEMBERS.....	178
TABLE 97: DEVICE CAPABILITIES OBJECT MEMBERS	180
TABLE 98: DEVICE CONFIGURATION OBJECT MEMBERS	182
TABLE 99: DEVICE STATUS OBJECT MEMBERS.....	183
TABLE 100: DEVICE ERROR STATUS OBJECT MEMBERS.....	184
TABLE 101: STREAM OBJECT MEMBERS	184
TABLE 102: STREAM CONFIGURATION OBJECT MEMBERS.....	185

TABLE 103: STREAM STATUS OBJECT MEMBER.....	186
TABLE 104: SUBSCRIPTIONS OBJECT MEMBERS.....	186
TABLE 105: SUBSCRIPTION CONFIGURATION OBJECT MEMBERS.....	186
TABLE 106: SUBSCRIPTION STATUS OBJECT MEMBERS.....	187
TABLE 107: NODE OBJECT MEMBERS.....	188
TABLE 108: NODE INPUT OBJECT MEMBERS.....	189
TABLE 109: NODE INPUT CONFIGURATION OBJECT MEMBER.....	189
TABLE 110: NODE INPUT STATUS OBJECT MEMBER.....	189
TABLE 111: STEREO AUDIO INPUT/OUTPUT OBJECT MEMBER.....	190
TABLE 112: INPUTS SUPPORTED FOR AUDIO INPUT/OUTPUT NODE.....	190
TABLE 113: OUTPUTS SUPPORTED FOR AUDIO OUTPUT NODE.....	192
TABLE 114: BITMAP OVERLAY NODE OBJECT MEMBERS.....	194
TABLE 115: CEC NODE P2P CONFIGURATION OBJECT MEMBER.....	195
TABLE 116: COLOR GENERATOR NODE OBJECT MEMBERS.....	197
TABLE 117: FRAME BUFFER NODE OBJECT MEMBERS.....	198
TABLE 118: FRAME BUFFER NODE STATUS OBJECT MEMBER.....	200
TABLE 119: FRAME BUFFER NODE INPUT OBJECT MEMBERS.....	200
TABLE 120: FRAME RATE CONVERTER NODE OBJECT MEMBER.....	203
TABLE 121: FRAME RATE CONVERTER NODE INPUT OBJECT MEMBER.....	203
TABLE 122: HDMI DECODER NODE OBJECT MEMBERS.....	204
TABLE 123: HDMI DECODER NODE STATUS OBJECT MEMBERS.....	206
TABLE 124: HDMI DECODER NODE INPUT OBJECT MEMBER.....	207
TABLE 125: HDMI ENCODER NODE STATUS OBJECT MEMBERS.....	209
TABLE 126: HDMI ENCODER NODE STATUS OBJECT MEMBERS.....	210
TABLE 127: HDMI ENCODER NODE INPUT OBJECT MEMBERS.....	211
TABLE 128: HDMI MONITOR NODE STATUS OBJECT MEMBERS.....	215
TABLE 129: HDMI MONITOR NODE INPUT OBJECT MEMBER.....	215
TABLE 130: MULTICHANNEL AUDIO INPUT NODE OBJECT MEMBERS.....	216
TABLE 131: MULTICHANNEL AUDIO SAMPLING FREQUENCY CONFIGURATION OBJECT MEMBERS.....	216
TABLE 132: MULTICHANNEL AUDIO SAMPLING FREQUENCY CHOICE NODE STATUS OBJECT MEMBERS.....	217
TABLE 133: MULTICHANNEL AUDIO NUMBER OF AUDIO CHANNELS NODE STATUS OBJECT MEMBERS.....	217
TABLE 134: CHANNELS NUMBER CHOICE NODE STATUS OBJECT MEMBERS.....	218
TABLE 135: CHANNEL ALLOCATION NODE OBJECT MEMBERS.....	218
TABLE 136: CHANNEL ALLOCATION CHOICE OBJECT MEMBERS.....	218
TABLE 137: AUDIO ENCODING OBJECT MEMBER.....	219
TABLE 138: MULTICHANNEL AUDIO OUTPUT NODE OBJECT MEMBER.....	226
TABLE 139: INFRARED ENCODER NODE OBJECT MEMBER.....	230
TABLE 140: INFRARED ENCODER INPUT NODE OBJECT MEMBER.....	230
TABLE 141: NETWORK INTERFACE NODE OBJECT MEMBERS.....	231
TABLE 142: NETWORK INTERFACE IP CONFIGURATION NODE OBJECT MEMBERS.....	231
TABLE 143: NETWORK INTERFACE NODE STATUS OBJECT MEMBERS.....	232
TABLE 144: NETWORK INTERFACE IP STATUS OBJECT MEMBER.....	232
TABLE 146: NETWORK PORT NODE STATUS OBJECT MEMBERS.....	233
TABLE 147: NETWORK SWITCH NODE OBJECT MEMBER.....	234
TABLE 148: SCALER NODE OBJECT MEMBERS.....	235
TABLE 149: SCALER NODE STATUS NODE OBJECT MEMBER.....	235
TABLE 150: SCALER INPUT NODE OBJECT MEMBER.....	235
TABLE 151: THUMBNAIL GENERATOR NODE OBJECT MEMBERS.....	237
TABLE 152: THUMBNAIL GENERATOR NODE STATUS MEMBERS.....	237
TABLE 153: UART NODE CONFIGURATION OBJECT MEMBERS.....	238
TABLE 154: UART NODE STATUS OBJECT MEMBER.....	238
TABLE 155: UART NODE INPUTS OBJECT MEMBER.....	238

TABLE 156: USB HID CONFIGURATION NODE OBJECT MEMBERS	239
TABLE 157: USB HID DEVICE STATUS NODE OBJECT MEMBERS	240
TABLE 158: USB HID DEVICE STATUS NODE OBJECT MEMBER	241
TABLE 159: USB HID HOST STATUS NODE OBJECT MEMBER.....	241
TABLE 160: USB HID PEER LINKED NODE OBJECT MEMBER	241
TABLE 161: USB HID REMOTE EXTENDER STATISTICS NODE OBJECT MEMBERS	242
TABLE 162: USB EXTENDER CONFIGURATION NODE OBJECT MEMBERS	245
TABLE 163: USB EXTENDER STATUS NODE OBJECT MEMBERS	247
TABLE 166: VIDEO INPUT NODE OBJECT MEMBER	250
TABLE 167: VIDEO COMPRESSOR NODE OBJECT MEMBERS.....	251
TABLE 168: VIDEO COMPRESSOR STATUS NODE OBJECT MEMBER	252
TABLE 169: VIDEO DECOMPRESSOR NODE OBJECT MEMBER.....	252
TABLE 170: VIDEO DECOMPRESSOR STATUS NODE OBJECT MEMBER.....	253
TABLE 171: DEVICE CONFIGURATION PROPERTIES	254
TABLE 172: VALID DEVICE MODE CHANGES	255
TABLE 173: STREAM CONFIGURATION PROPERTIES.....	255
TABLE 174: STEREO AUDIO INPUT/OUTPUT NODE CONFIGURATION PROPERTIES	256
TABLE 175: STEREO OUTPUT NODE CONFIGURATION PROPERTIES.....	256
TABLE 176: CEC P2P NODE CONFIGURATION PROPERTY	257
TABLE 177: COLOR GENERATOR NODE CONFIGURATION PROPERTIES	258
TABLE 178: FRAME BUFFER NODE CONFIGURATION PROPERTIES	259
TABLE 179: HDMI DECODER NODE CONFIGURATION PROPERTIES	260
TABLE 180: HDMI DECODER NODE CONFIGURATION PROPERTIES	260
TABLE 181: HDMI ENCODER NODE CONFIGURATION PROPERTIES	261
TABLE 182: MULTICHANNEL AUDIO INPUT NODE CONFIGURATION PROPERTIES.....	262
TABLE 183: MULTICHANNEL AUDIO OUTPUT NODE CONFIGURATION PROPERTIES	263
TABLE 184: NETWORK PORT NODE CONFIGURATION PROPERTIES	264
TABLE 185: NETWORK SWITCH NODE CONFIGURATION PROPERTIES.....	264
TABLE 186: SCALER NODE CONFIGURATION PROPERTIES	264
TABLE 187: UART NODE CONFIGURATION PROPERTIES	265
TABLE 189: USB EXTENDER NODE CONFIGURATION PROPERTIES	265
TABLE 190: VIDEO COMPRESSOR NODE CONFIGURATION PROPERTIES	266
TABLE 191: VIDEO DECOMPRESSOR NODE CONFIGURATION PROPERTIES.....	266

1 Purpose

This reference guide describes and explains the SDVoE API command set.

The SDVoE Developers Introduction to SDVoE API User Guide is recommended to be referenced in regards to understanding the requirements and application of the API.

1.1 Version Information

This version of the API Developers Reference Guide applies to the SDVoE API version 3.9. SDVoE endpoint devices incorporating the BlueRiver AVP ASIC firmware version 2.4.0.0 and the UEX USB 2.0 extender firmware version 1.2.0.0.

1.2 Target Audience

The target group is software designers and developers, project managers and support teams involved in the design, coding and/or deployment of client applications that are to control and manage SDVoE systems and endpoints (BlueRiver AVP) and/or UEX USB 2.0 extender endpoints (UEX1000/UEX2000 attached to BlueRiver AVP or UEX2000 used standalone without a BlueRiver AVP).

1.3 Reference Materials

Prior to beginning to design and implement the API, it is recommended to refer to the Developers Introduction to SDVoE API User Guide to gain a good understanding of the API platform and its requirements.

In addition, the SDVoE Control Server Administrators Guide (PDS-062492) and the SDVoE Developers API Application Note Library (PDS-062491) may prove useful, the latter contains information and examples that may prove useful in the development and design of software client solutions that will manage SDVoE and/or UEX USB 2.0 Extender endpoint devices.

2 Outline of the API

This API provides a way for control software to interact with BlueRiver-based or UEX-based devices and itself. It is designed to be flexible, supporting a wide variety of functionality allowing for creative Pro AV applications to be implemented. In addition, it simplifies programming and implementation of custom applications, which permits seamless operation of separate devices across a network.

Reminder: Details on the structure of the API is provided in the Introduction to the SDVoE API User Guide, refer to it for general information on the API platform and its requirements.

2.1 Events

Change of status or an occurrence of an event, such as a change of device state, trigger an API event. To have active clients remain up to date with state changes, these events need to be handled.

Such events can be managed either by polling for list of latest events using a WebSocket to monitor for asynchronous event notifications. Refer to the Introduction to the SDVoE API User Guide for more information on event types, tracking device states and event buffering.

2.2 Commands Types

The API server supports two types of commands:

- Immediate:** This type of command always completes immediately. Returns with either a `SUCCESS` or `ERROR` status.
- Background:** These commands complete a task in the background. The response of these commands is retrieved using a `request` command referring to the unique request ID assigned to each command of this type when it is issued.

Refer to the Introduction to the SDVoE API User Guide for more information on command types.

2.3 Device Model, Life Cycle & Command Structure

The API device model describes each devices status, configuration, data connectivity, as well as supported capabilities. The API device model is represented with a JSON object referred to as “Device Objects”. The device model is the heart of the API.

For details on the device model including its components and life cycle, refer to the Introduction to the SDVoE API User Guide. In addition, the introduction guide provides detailed information on Error Objects, HTTP Status and Device Error Objects.

2.4 Description of Streams & Destinations

Reminder, control of data routing between devices uses a stream and subscription abstraction:

- Streams are the sending endpoints, while subscriptions are the receiving endpoints.
- Each stream or subscription has an assigned type, only streams and subscriptions of the same type can be associated together.

Depending on their type, streams are either `joinable` or `switchable`.

Refer to the Introduction to the SDVoE API User Guide for more information on joinable and switchable streams, as well as how the API implements multicast IP address management.

2.5 Specifying Video Output Format

Managing the video output format is a major factor of the API command module and must be considered when managing SDVoE endpoint devices.

TIP: For background information on managing video display modes, refer to the Introduction to the SDVoE API User Guide, section Managing the Video Output Format provides background information on video output formats.

2.5.1 Command Options

Three command options are available for managing the video output format of a receiver (RX) device.

1. `set video` command.

This is generally the recommended command to use to set the video output format. Refer to section 4.19.12 Command `set video` for more information.

2. `join` command.

When joining a video stream, the video output format can optionally be specified, allowing both operations (joining and setting video output format) to be performed with a single command. Refer to section 4.9 Command `join` more information.

3. `set multiview` command.

Used to set the receiver (RX) to multiview mode with a specified multiview layout. Refer to section 5 Multiview Command Reference for more information on multiview.

2.5.2 Video Display Mode Argument

When a `set video` or `join` command (optional for the `join` command) is issued and video format are to be defined than display mode arguments are required, table below summarizes mode value to specify.

Reminder: Introduction to the SDVoE API User Guide, section Managing the Video Output Format provides additional information on video output formats.

Table 1: Video Display Mode Supported Keywords

Mode	API String	Description
Genlock	GENLOCKED	Specifies to set the video mode to genlock (i.e. synchronized zero-frame latency) full screen 1:1 display mode.
Genlock Scaling	GENLOCK_SCALING	Specifies to set the video mode to genlock (i.e. zero-frame latency) full screen mode with scaling.
Fast Switch	FAST_SWITCHED	Specifies to set the video mode to fast switched full screen mode, with aspect ratio preserved by adding black bars when needed.
Fast Switch - Crop	FAST_SWITCHED_CROP	This string specifies video mode to be set to fast switched full screen mode, with aspect ratio preserved by cropping the video.
Fast Switch - Stretch	FAST_SWITCHED_STRETCH	This string specifies the video mode to be set to fast switched full screen mode, with the video stretched to cover the entire screen without regards to preserving the aspect ratio.

Mode	API String	Description
Genlock Wall	WALL_GENLOCKED	Specifies to set the video mode set to genlocked (synchronized) display wall mode.
Fast Switch Wall	WALL_FAST_SWITCHED	Specifies to set the video mode to fast switched display wall mode.
Multiview	Refer to description.	Refer to section 5 Multiview Command Reference

2.5.3 Video Format Arguments

In an API request, the video arguments are specified as a `VideoFormatArguments` JSON object. The format of that object type is the following:

```
{
  "aspect_ratio" : String (optional),
  "fps" : Real or String (conditional),
  "hsync_front_porch" : Integer (conditional),
  "vsync_front_porch" : Integer (conditional),
  "hdmi_vic" : Integer (conditional),
  "keep_width" : Integer (conditional),
  "keep_height" : Integer (conditional),
  "horiz_offset" : Integer (conditional),
  "vert_offset" : Integer (conditional),
  "horiz_polarity" : String (conditional),
  "vert_polarity" : String (conditional),
  "hsync_pulse" : Integer (conditional),
  "vsync_pulse" : Integer (conditional),
  "width" : Integer (conditional),
  "height" : Integer (conditional),
  "total_width" : Integer (conditional),
  "total_height" : Integer (conditional),
  "vic" : Integer (conditional),
  "viewport_horiz" : Integer (conditional),
  "viewport_vert" : Integer (conditional),
  "viewport_width" : Integer (conditional),
  "viewport_height" : Integer (conditional),
  "quantization_range" : String (optional),
  "aspect_ratio_mode" : String (conditional),
  "grid_width" : Integer (conditional),
  "grid_height" : Integer (conditional),
  "grid_index_horiz" : Integer (conditional),
  "grid_index_vert" : Integer (conditional),
  "bezel_horiz" : Integer (conditional),
  "bezel_vert" : Integer (conditional),
  "wall_name" : String (optional)
}
```

For detailed information on the `set video` arguments, refer to the Developers Introduction to the SDVoE API User Guide, sections “Video Format Arguments” and “Uniquely Identifying a Video Format”. Unless otherwise stated, arguments generally apply to all display modes except Genlock, which does not require nor does it actually allow any video format arguments. In addition, unless otherwise stated, all arguments are individually optional, but to uniquely identify the video format a minimum set of arguments must be specified. For details of the multiview and wall modes, refer to the API Developers Application Note Library.

Below table provides the list of supported Video Identification Codes (VIC) for the API.

Table 2: Supported Video Identification Codes (VIC)

Video Identification Code	Video Format
1	640x480 60Hz (4:3)
2	720x480 59.94Hz or 60 Hz (4:3)
3	720x480 59.94Hz or 60 Hz (16:9)
4	1280x720 (720p) 59.94Hz or 60 Hz
16	1920x1080 (1080p) 59.94Hz or 60 Hz
17	720x576 50Hz (4:3)
18	720x576 50Hz (16:9)
19	1280x720 (720p) 50Hz
31	1920x1080 (1080p) 50Hz
32	1920x1080 (1080p) 24Hz or 23.98Hz
33	1920x1080 (1080p) 25Hz
34	1920x1080 (1080p) 29.97Hz or 30Hz
60	1280x720 (720p) 24Hz or 23.98Hz
61	1280x720 (720p) 25Hz
62	1280x720 (720p) 29.97Hz or 30Hz
93*	3840x2160 (2160p) 24Hz or 23.98Hz
94*	3840x2160 (2160p) 25Hz
95*	3840x2160 (2160p) 29.97Hz or 30Hz
96	3840x2160 (2160p) 50Hz
97	3840x2160 (2160p) 59.94Hz or 60 Hz
98*	4096x2160 24Hz or 23.98Hz (256:135)
99	4096x2160 25Hz (256:135)
100	4096x2160 29.97Hz or 30Hz (256:135)
101	4096x2160 50Hz (256:135)
102	4096x2160 59.94Hz or 60 Hz (256:135)

* Video Identification Codes (VIC) with an asterisk can alternatively be specified by their HDMI Video Identification Codes (HDMI_VIC).

In regards to the HDMI Video Identification Code (`hdmi_vic`) argument the supported values are:

`hdmi_vic` argument specifies HDMI Video Identification Code (HDMI_VIC).

The supported values are:

- 1 -- 3840x2160 (2160p) 29.97Hz or 30Hz
- 2 -- 3840x2160 (2160p) 25Hz
- 3 -- 3840x2160 (2160p) 24Hz or 23.98Hz
- 4 -- 4096x2160 24Hz or 23.98Hz (256:135)

Other values are inserted in the HDMI Vendor-Specific InfoFrame as requested but cannot be used to uniquely identify a video format.

2.5.4 Uniquely Identifying a Video Format

A minimum set of video format arguments are required to uniquely identify a video format. This section describes the various options for doing so.

2.5.4.1 By Video Identification Code (VIC or HDMI_VIC)

If the desired video format has a supported Video Identification Code (VIC) or HDMI Video Identification Code (HDMI_VIC), it is sufficient to only specify this information. Other parameters (such as video timings, etc.) are automatically retrieved from a table.

For some video formats, there is an ambiguity regarding the frame rate, which is resolved as follows:

- For formats with a frame rate of 24Hz or 23.98Hz (24Hz/1.001), 24Hz frame rate is selected.
- For formats with a frame rate of 30Hz or 29.97Hz (30Hz/1.001), 29.97Hz frame rate is selected.
- For formats with a frame rate of 60Hz or 59.94Hz (60Hz/1.001), 59.94Hz frame rate is selected.

Refer to Table 2: Supported Video Identification Codes (VIC) and under HDMI Video Identification Code for a list of supported HDMI Video Identification Codes (HDMI_VIC).

2.5.4.2 By Video Identification Code (VIC or HDMI_VIC) & Frame Rate

Same as by Video Identification Code (VIC or HDMI_VIC) as outlined in previous section but the frame rate ambiguity is resolved by explicitly specifying the frame rate.

2.5.4.3 By Video Resolution & Frame Rate

If the desired video format has a supported Video Identification Code (VIC) or HDMI Video Identification Code (HDMI_VIC), it is sufficient to only specify the video resolution and frame rate.

Refer to Table 2: Supported Video Identification Codes (VIC) under HDMI Video Identification Code for a list of supported HDMI Video Identification Codes (HDMI_VIC).

2.5.4.4 Completely Specified Video Formats

If the video format does **not** have a supported Video Identification Code (VIC) or HDMI Video Identification Code (HDMI_VIC), all video arguments must be specified to completely describe the video format, this includes the following:

- Picture aspect ratio if applicable
- Frame Rate
- Video Sync Front Porch
- Video Sync Polarity
- Video Sync Pulse Width
- Video Resolution
- Total Size

3 Core Commands

The API supports an extensive set of commands designed to support the functionality built into the SDVoE and UEX USB 2.0 Extender endpoint devices. The table provided below reviews the commands that can be issued “prior” to loading the API command module.

On a WebSocket connection, the effects of these commands are limited to a single API connection.

Note: For the full list of API commands, refer to section 4 API Command Reference

Table 3: API Core Commands

Command	Description	Return Value	Errors
<code>mode human</code>	Enable/disable human-readable output mode. When enabled, the JSON responses returned by the API are indented and formatted to be more easily read. When disabled, the JSON return contains no newline, space or other white space characters.	null	<code>ILLEGAL_ARGUMENT</code> occurs when an invalid value is specified.
<code>require</code>	Verifies if the requested version of API command module is available in current API Server and loads it if it is. TIP: Compatibility rules are outlined in Introduction to the SDVoE API User Guide.	null	<code>VERSION</code> returned when API not compatible with requested version. <code>UNSUPPORTED_FEATURE</code> returned when requested module unsupported by the API version.
<code>version</code>	Retrieve version of the command module that is currently loaded.	null	<code>ILLEGAL_COMMAND</code> occurs if the API version running is legacy version, prior to API 2.4.

The following sections detail, as well as provide examples, for each of the core commands outlined above.

3.1 Command `mode human`

`...?mode human=TRUE/FALSE`

Enable or disable the human-readable output mode, the `human` query string argument described is valid in any request. This is what the ellipsis (...) in the syntax shown above represents, it can be replaced with any API URI.

Whether this mode is enabled or disabled by default is dependent on the API Server configuration, refer to the Control Server (API) Administrators Guide for details.

In a WebSocket connection, the human-readable output mode is initially disabled.

3.1.1 Arguments

The argument is a Boolean argument specifying whether to enable (`TRUE`) or disable (`FALSE`) the human-readable output mode.

3.1.2 Return Value

null

3.1.3 Errors

ILLEGAL_ARGUMENT	Occurs when an invalid value is specified in the request syntax.
------------------	--

3.1.4 Example

```
GET /api?human=true HTTP/1.1
```

```
...
```

```
HTTP/1.1 200 OK
```

```
...
```

```
{
  "status": "SUCCESS",
  "request_id": null,
  "result": {
    "server": "Control Server",
    "vendor": "Semtech",
    "version": "3.3.0.0",
    "modules": [
      {
        "name": "api",
        "version": "3.3.0.0"
      }
    ]
  },
  "error": null
}
```

3.2 Command require

```
GET /api?require(module_name)=version
```

Verify the requested version of the command module is available from the API Server.

3.2.1 Arguments

The `module_name` argument is the name of the command module.

The `version` argument is the requested version.

3.2.2 Return Value

Refer to the command `version`.

3.2.3 Errors

VERSION	The requested version is not compatible.
---------	--

UNSUPPORTED_FEATURE	The requested module is not supported.
---------------------	--

3.2.4 Example

```
GET /api?require(api)=3.3.0.0 HTTP/1.1
```

```
...
```

```
HTTP/1.1 200 OK
```

```
...
```

```
{
  "status": "SUCCESS",
  "request_id": null,
  "result": {
    "server": "Control Server",
    "vendor": "Semtech",
    "version": "3.3.0.0",
    "modules": [
      {
        "name": "api",
        "version": "3.3.0.0"
      }
    ]
  },
  "error": null
}
```

```
GET /api?require(api)=3.9999.0.0 HTTP/1.1
```

```
...
```

```
HTTP/1.1 404 Not Found
```

```
...
```

```
{
  "status" : "ERROR",
  "request_id" : null,
  "result" : null,
  "error" : {
    "reason" : "VERSION",
    "message" : "Incompatible version"
  }
}
```

```
GET /api?require(unsupported)=1.2.3.4 HTTP/1.1
```

```
...
```

```
HTTP/1.1 404 Not Found
```

```
...
```

```
{
  "status" : "ERROR",
  "request_id" : null,
  "result" : null,
  "error" : {
    "reason" : "UNSUPPORTED_FEATURE",
    "message" : "Unknown module 'unsupported' in command 'require'"
  }
}
```

3.3 Command version

GET /api

Retrieve version of the command module currently loaded.

Note: This command was introduced in API version 3.0.

3.3.1 Arguments

None

3.3.2 Return Value

```
{
  "server" : String,
  "vendor" : String,
  "version" : String,
  "modules" : String
}
```

Table 4: Version Command Return Values Object Members

Member Name	Since	Type	Description
server	3.0	String	Name of API server software.
vendor	3.0	String	Vendor of API server software.
version	3.0	String	Release version of API server software.
modules	3.0	Array of ModuleDescriptionObject	Supported API command modules.

Each ModuleDescriptionObject JSON object has the following structure:

```
{
  "name" : String,
  "version" : String
}
```

Table 5: Version JSON Module Description Object Members

Member Name	Since	Type	Description
name	3.0	string	Name of API command module.
version	3.0	string	Version of API command module.

3.3.3 Errors

ILLEGAL_COMMAND

Occurs if API version is before 2.4.

3.3.4 Example

```
GET /api HTTP/1.1
...
HTTP/1.1 200 OK
...
```

```
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "server" : "Control Server",
    "vendor" : "Semtech",
    "version" : "3.3.0.0",
    "modules" : [
      {
        "name" : "api",
        "version" : "3.3.0.0"
      }
    ]
  },
  "error" : null
}
```

Semtech Confidential Information
Mark Li
ADV
02/03/2026

4 API Command Reference

4.1 Summary List of API Commands

The SDVoE API command module contains commands used for controlling and querying the status of BlueRiver transceiver, transmitter and receiver as well as UEX USB 2.0 extender local and remote endpoint devices.

Note: As of API version 3.5, it is possible to specify the `device_name` or `device_id` of an individual device as the `target` in the URL of a POST command request. No change regarding the current device groups, `ALL`, `ALL_RX` and `ALL_TX`. For details on managing device names refer to the SDVoE Developers API Application Note Library section Managing a Device Name.

Table 6: List of API Commands

Command	Description
auth	Endpoint device is authenticated to an API server, applicable to BlueRiver AVP ASIC devices.
claim	Associates a device(s) with the API Server from which the command was issued, so it is controlled exclusively through this API instance. An unauthorized API server on the network will not be able to control a claimed device. Applicable to BlueRiver and UEX devices. The effect of this command is undone with the <code>release</code> command. Performing a factory reset of the target device(s) associated to the API Server also releases the claim.
event	Get a list of events. Applicable to BlueRiver and UEX devices. Query arguments supported to allow for filtering of event types. This includes: <code>after=event_id</code> <code>limit=limit</code> <code>type=types</code> <code>settings_change=subsets</code>
factory	Restore the factory default settings of a device or group of devices. If the target device(s) was associated with the API server by use of <code>auth</code> or <code>claim</code> requests, it is released. Applicable to BlueRiver and UEX devices.
get	The <code>get</code> command fetches information from a single device or group of devices. Applicable to BlueRiver and UEX devices. For each device, information retrieved is a subset of the device object, which contains all configuration, status, supported features and other information about the device.
join	Subscribe one or more decoder (RX) devices to a video or audio stream. Applicable to BlueRiver devices. Four stream types supported: <code>STEREO_AUDIO</code> Stereo audio stream <code>HDMI</code> HDMI video stream <code>HDMI_AUDIO</code> HDMI audio stream <code>MULTICH_AUDIO</code> Multichannel audio stream
leave	Unsubscribe one or more decoder (RX) devices from a subscribed AV stream. Applicable to BlueRiver devices.
list	Two options available for the <code>list</code> command: 1. <code>list firmware</code> List all firmware update files available to the API server. Applicable to BlueRiver and UEX

Command	Description
	devices. 2. <code>list multicast</code> List information regarding the multicast IP addresses currently in use.
<code>netstat</code>	<code>netstat read</code> retrieves network characteristics for one or more devices. Data useful in investigating possible cause of a network problem. Applicable to BlueRiver and attached UEX devices. <code>netstat clear</code> resets all counters. Refer to section 4.12 Command <code>netstat</code> for more information.
<code>pair</code>	Pair a Semtech UEX (attached to BlueRiver AVP or standalone) or up to seven (if SUI enabled) Icron (attached to BlueRiver AVP) remote extenders (REX) to a single local extender (LEX). A request is sent for each pair to be paired to the local extender.
<code>reboot</code>	Restarts the target device(s). Applicable to BlueRiver and UEX devices. Note: The target device(s) is unavailable for a short time while the reboot is completed.
<code>release</code>	Undo the effect of the <code>claim</code> command. This can only be done on the API Server instance on which the device was claimed. Applicable to BlueRiver and UEX devices.
<code>request</code>	Either this function retrieves the return value of a <code>background</code> command or the data associated with an event. Applicable to BlueRiver and UEX devices. Following outlines event types that can be received: <code>REQUEST_COMPLETE</code> event indicates completion of a <code>background</code> command, response status <code>PROCESSING</code> and a request ID instead of status <code>SUCCESS</code>. Reminder: Background commands typically respond with <code>PROCESSING</code> status and a request ID. Refer the Introduction to the SDVoE API User Guide for more information. <code>CEC_RECEIVED</code> event indicates that Consumer Electronics Control (CEC) data received by a specific device. <code>DEVICE_CONNECTED</code> event indicates that a new device has joined the network. <code>DEVICE_DISCONNECTED</code> event indicates that a device has left the network. <code>DEVICE_INFO_AVAILABLE</code> event indicates that device information is available for a newly discovered device. <code>FIRMWARE_UPDATE_PROGRESS</code> event indicates a firmware update in progress. - Information associated with this event can be used to report progress information to the user, for example by displaying a progress bar. <code>I2C</code> event indicates that an I²C bus master (e.g. a MCU) has generated an event through the I²C interface. <code>INFRARED_RECEIVED</code> event indicates Infrared data received by a specific device. <code>RS_232_RECEIVED</code> event indicates RS-232 data received by a specific device. <code>SETTINGS_CHANGED</code> event indicates that new settings are returned. - Format is identical to output of the “<code>get settings</code>” command called on a device associated with a <code>SETTINGS_CHANGED</code> event. <code>USB_TREE_CHANGED</code> event indicates that a USB device connection tree element was

Command	Description
	added (USB device connect) or removed (USB device disconnect). Format is identical to output of the “get_usb_tree” command called on a device associated with a USB_TREE_CHANGED event.
send	The <code>send</code> command provides three sub-commands that can be used to send the following data types to one or more devices. Applicable to BlueRiver devices. - Send Consumer Electronics Control (CEC) data to one or more devices. - Send infrared remote (IR) control data to one or more devices. - Send RS-232 data to one or more devices.
send_wol	The <code>send_wol</code> command is used to send a Wake-on-Lan (WoL) packet from the API to an AVP device, for the AVP device to relay it on the network (acts as a WoL proxy) by broadcasting the message to the local subnet to wake a powered-off target machine. This feature was added in API revision 3.7 and ASIC firmware version 2.2. Applicable to BlueRiver AVP devices.
set	The <code>set</code> command is a powerful command that has several options provided to manage transmitters and receivers. A brief description of each option provided below: <code>set dot1x_certificates</code> Upload 802.1X certificate chain to BlueRiver device(s). Applicable to BlueRiver AVP devices. <code>set edid</code> Set the EDID of one or more transmitter devices. Applicable to BlueRiver devices. <code>set gpio</code> Set the GPIO pins direction and/or value. Applicable to BlueRiver devices. <code>set ip</code> Set the Internet Protocol (IP) configuration of the primary network interface of a target device. If USB 2.0 present, this command is also used to set the USB extender (Semtech UEX or Icron) network interface. Applicable to BlueRiver and UEX devices. <code>set property</code> Set a configuration property for one or more devices. Applicable to BlueRiver and UEX devices. Refer to section 7 Configurable Device Properties for more information on configuration properties. <code>set scaler</code> Set the scaler video resolution of one or more transmitter devices. Applicable to BlueRiver devices. <code>set secure_av</code> Set the <code>secure_av</code> key of a target device. Applicable to BlueRiver devices. <code>set security_key</code> Configure the encryption key for USB HID IP communication for the specified device(s). Applicable to BlueRiver devices. <code>set usb</code> Configure USB HID extension on an BlueRiver device. <code>set video</code> Set the video mode and video output parameters of one or more receiver devices. Applicable to BlueRiver devices.
sfp	The <code>sfp</code> command is used to read SFP+ optical module diagnostic information from one or more endpoint devices. This feature was added in API revision 3.6 and ASIC firmware 2.1. Applicable to BlueRiver AVP devices.

Command	Description								
start	Used to start one or more stream(s) on one or more devices. Applicable to BlueRiver devices. <table> <tr> <td>STEREO_AUDIO</td><td>Stereo audio stream</td></tr> <tr> <td>HDMI</td><td>HDMI video stream</td></tr> <tr> <td>HDMI_AUDIO</td><td>HDMI audio stream</td></tr> <tr> <td>MULTICH_AUDIO</td><td>Multichannel audio stream</td></tr> </table>	STEREO_AUDIO	Stereo audio stream	HDMI	HDMI video stream	HDMI_AUDIO	HDMI audio stream	MULTICH_AUDIO	Multichannel audio stream
STEREO_AUDIO	Stereo audio stream								
HDMI	HDMI video stream								
HDMI_AUDIO	HDMI audio stream								
MULTICH_AUDIO	Multichannel audio stream								
stop	Stops one or more stream(s) on one or more devices and optionally frees the multicast address associated with each stopped stream, allowing it to be available for reassignment. Applicable to BlueRiver and UEX devices.								
switch	Set stream destination for a device. Applicable to BlueRiver and UEX devices. The <code>switch</code> command is used to establish a one-way connection between a source and the destination devices. To establish two-way connection between any two devices, the command must be issued twice with source and destination reversed. There are three options that can be applied to specify the destination. These options are: 1. Using device ID and stream index when destination is a device. 2. Using the keyword “api” when destination is API Server. 3. Specifying the destination IP address.								
unauth	Removes authentication of an endpoint device(s) from API server to which it was bound. Applicable to BlueRiver AVP devices.								
update	Update the firmware of one or more devices. Applicable to BlueRiver and UEX devices. While a firmware update is in progress, no other operation can be performed on the device. For this reason, any command sent to the API Server that targets the device will fail with <code>INVALID_STATE</code> for this device until the firmware update completes. All connected clients are notified of the start and end of a firmware update with a <code>FIRMWARE_UPDATE_START</code> event and a <code>FIRMWARE_UPDATE_COMPLETE</code> event, respectively. Note: A reboot command must be issued to the device once the firmware update is complete for the changes to take effect. The device is in an undefined state until that reboot.								
unpair	Used to unpair a Semtech UEX (attached to a BlueRiver AVP or standalone) or Icron (attached to BlueRiver AVP) remote USB extender (REX) from a local extender (LEX).								
usb	The <code>set</code> command has several sub-commands to manage USB HID extension on a device. Applicable to BlueRiver AVP devices. Below outlines these sub-commands providing a brief description of each. <table> <tr> <td><code>usb descriptor read</code></td><td>Read USB descriptors.</td></tr> <tr> <td><code>usb descriptor write</code></td><td>Write USB descriptors.</td></tr> </table>	<code>usb descriptor read</code>	Read USB descriptors.	<code>usb descriptor write</code>	Write USB descriptors.				
<code>usb descriptor read</code>	Read USB descriptors.								
<code>usb descriptor write</code>	Write USB descriptors.								

Following subsections review each of the supported commands included in API version 3.3. Each command includes details of command arguments, return values and examples.

4.2 Commands that Modify Device Settings

Most—though not all—commands described in the sections that follow aim to change the settings of one or more devices. Examples include the `start`, `stop`, `join` and `leave` commands that control audio and video streams and subscriptions, as well as the various variations of the `set` command, etc.

All commands that modify device settings have an identical return value. The return value is an object with two array members: `devices` and `error`.

For each device targeted by the command, there is either:

1. An entry in the `devices` array if the operation was fetched successfully; or
2. A device error object in the `error` array if the operation was unsuccessful.

```
{
  "devices" : [DeviceSuccess, ...],
  "error" : [DeviceError, ...]
}
```

Table 7: Return Value Schema Object Members

Member	Since	Type	Description
<code>devices</code>	3.0	Array of DeviceSuccess	Array of objects that are subset of DeviceSuccess.
<code>error</code>	3.0	Array of DeviceError	An array of DeviceError objects. Indicates devices that command failed for, as well as the reason for failure.

Device error objects (`DeviceError`) are described in the Introduction to the SDVoE API User Guide.

DeviceSuccess objects have the following schema:

```
{
  "device_id" : String
}
```

Table 8: DeviceSuccess Schema Object Member

Member Name	Since	Type	Description
<code>device_id</code>	3.0	String	Device ID of a device for which the operation succeeded.

4.3 Commands `auth` & `unauth`

The `auth` command is a security feature that allows for secure communication between an API server and SDVoE endpoint devices based on the BlueRiver AVP ASIC series. It enables endpoint devices to be bound-authenticated to an API server. When an endpoint is “bound-authenticated” it communicates only with a specific API server because they share a specified security key. For details on the configuring the security key refer to the SDVoE Control Server Administrators Guide. For certain use cases, the specific API server can be a pair of API servers, for more information refer to the SDVoE API Developers Application Note Library.

The `unauth` command returns the device(s) to a non-secured, unauthenticated operational status. Request will succeed only if the device(s) is currently bound-authenticated to the API server.

The command syntax for each `auth` and `unauth` is as follows:

```
POST /api/device/{target}
{
```

```
    "op" : "auth"
  }
```

```
POST /api/device/{target}
{
  "op" : "unauth"
}
```

Note: This feature was enabled in API version 3.3, BlueRiver firmware 1.3.

4.3.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group.

The allowed device groups are:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

4.3.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.3.3 Errors

The `auth/unauth` command can fail with the following error:

CONFIGURATION	If this feature has not been enabled in the API server configuration.
PROTOCOL NOT FOUND	This message is provided when the command is not supported by the API.

The `auth/unauth` command can fail for a specific device with the following error:

DEVICE_TYPE	The firmware does not support this feature.
-------------	---

4.3.4 Examples

Below is an example of enabling the bound-authentication of a single endpoint device, 001ec0f03668.

```
POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "auth"
}
HTTP/1.1 201 Created
Location: /api/request/144
...
{
  "status" : "PROCESSING",
  "request_id" : 144,
  "result" : null,
  "error" : null
}
```

This example disables the bound-authentication of a single endpoint device, 001ec0f03668.

```
POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "unauth"
}
HTTP/1.1 201 Created
Location: /api/request/145
{
  "status" : "PROCESSING",
  "request_id" : 145,
  "result" : null,
  "error" : null
}
```

Example of the protocol error occurring when an API doesn't support bound-authentication.

```
{
  "status": "ERROR",
  "request_id": null,
  "result": null,
  "error": {
    "reason": "PROTOCOL",
    "message": "Not Found"
  }
}
```

4.4 Commands `claim` & `release`

The `claim` command is a security feature aiming to prevent an adversary from using an unauthorized API server instance—for example running on a laptop connected to a network port in an installation—to control BlueRiver and/or UEX USB 2.0 Extender devices. When a device is claimed, it is controlled exclusively by the API server instance from which the command was issued.

- The `claim` command only succeeds if the device(s) is currently unclaimed or already claimed by the API Server issuing the command.
- The `release` command returns the device(s) to unclaimed state. It only succeeds if the device(s) is/are claimed by the current API server or already unclaimed.

The command syntax for `claim` and `release` is as follows:

```
POST /api/device/{target}
{
  "op" : "claim"
}
```

```
POST /api/device/{target}
{
  "op" : "release"
}
```

```
}
```

4.4.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group.

- If a UEX (UEX1000 or UEX2000) is attached to the BlueRiver AVP, the target shall be the AVP device. In this case, the claim and release commands apply to both the AVP and its attached UEX device(s). Setting UEX as the target is not supported when attached to an AVP for these commands.

If a UEX is used standalone (UEX2000 only), the target shall be the UEX device.

The allowed device groups are:

<code>ALL</code>	All devices
<code>ALL_TX</code>	All transmitter devices
<code>ALL_RX</code>	All receiver devices

4.4.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.4.3 Errors

The `claim` command can fail with the following error:

<code>CONFIGURATION</code>	If this feature has not been enabled in the API server configuration.
----------------------------	---

The `claim` command can fail for a specific device with the following error:

<code>DEVICE_TYPE</code>	The firmware does not support this feature.
--------------------------	---

4.4.4 Examples

```
POST /api/device/001ec0f03668 HTTP/1.1
```

```
...
```

```
{
  "op" : "claim"
}
HTTP/1.1 201 Created
Location: /api/request/42
...
{
  "status" : "PROCESSING",
  "request_id" : 42,
  "result" : null,
  "error" : null
}
```

```
POST /api/device/001ec0f03668 HTTP/1.1
```

```
...
```

```
{
  "op" : "release"
}
```

```
HTTP/1.1 201 Created
```

```
Location: /api/request/43
{
  "status" : "PROCESSING",
  "request_id" : 43,
  "result" : null,
  "error" : null
}
```

4.5 Command event

GET /api/event

Query arguments:

```
after=event_id
limit=limit
type=types
settings_change=subsets
```

This command retrieves a list of all the events that recently occurred.

TIP: An alternative to polling for events is to use a WebSocket connection to receive asynchronous event notifications. Refer to the Introduction to the SDVoE API User Guide for information on polling for new events and the asynchronous event notification mode.

4.5.1 Arguments

The *event_id* argument is the event ID of the last event (event ID with the highest numerical value) retrieved by the client. The API then lists all events that occurred since the specified event ID.

To ensure consistency of device lifecycle events in the API, the *event_id* argument:

- Must be omitted or, equivalently, specified with the value 0 the first time this command is invoked by a client since they connected to the API.
- Moreover, must then be specified with the correct, non-zero value when it is subsequently invoked.

As of API 3.7, the support for *event_id* value -1 has been added and allows the user to get only the last event that occurred. When this value is used, there are no more subsequent arguments to add to the API command.

The *limit* argument specifies maximum number of events to return. Remaining events can be retrieved using a subsequent request with the *event_id* argument set appropriately. If argument omitted, there is no limit.

The *types* argument specifies which types of events are returned. Monitoring for some types of events but not others reduces the total number of events.

Types of events monitored include the following:

- A comma-separated list of event types, refer to Table 9: Event Object Members below for list.
- The **ALL** keyword specifies that the client is interested in all events.
- The **NONE** keyword specifies the client will not handle any events.

The *subsets* argument specifies for **SETTINGS_CHANGED** events the subsets a client is interested in monitoring for changes for the *event* command, as well as reported as asynchronous event notifications.

This allows monitoring specific subsets of device settings for changes, while not receiving notifications of changes not affecting these subsets, thereby reducing the number of **SETTINGS_CHANGED** events that need

to be handled.

The `subsets` argument is ignored if `SETTINGS_CHANGED` events are filtered out (i.e. not selected) by the `types` argument.

The `subsets` argument can be one of the following:

- A comma-separated list of settings subset names. These subset names are defined in section 4.16.15 Requests Associated with Event `SETTINGS_CHANGED`.
- The `ALL` keyword specifies that the client is interested in changes in any subset.
- The `NONE` keyword specifies the client will not handle any `SETTINGS_CHANGED` events.

For both `types` and `subsets`, if omitted, the default is `ALL`.

Note: Depending on the application, as an example if RS-232 is used to perform a firmware update of components connected to a BlueRiver device, the rate at which `RS232_RECEIVED` notifications are produced can be significant.

4.5.2 Return Value

```
{
  "events" : [EventObject ...]
}
```

`EventObject` is an object with a single member named "events", which is an array of event objects. An event object (`EventObject`) has the following structure:

```
{
  "device_id" : String
  "event_id" : Integer,
  "event_type" : String,
  "timestamp" : Integer,
  "request_id" : Integer
}
```

The table below defines the members incorporated in an API event object.

Table 9: Event Object Members

Member Name	Since	Type	Description
device_id	3.0	String	The ID of the device for which an event occurred. Note: This member is null for <code>REQUEST_COMPLETE</code> events since background command requests can target multiple devices.
event_id	3.0	Integer	The ID of the event.
event_type	3.0	String	The type of event, which is one of the following: DEVICE_CONNECTED Device joined the network. DEVICE_DISCONNECTED Device left the network. DEVICE_INFO_AVAILABLE Initial device data available for a newly connected device. FIRMWARE_UPDATE_COMPLETE Firmware update of device has completed. FIRMWARE_UPDATE_PROGRESS Firmware update of device

Member Name	Since	Type	Description
			in progress. FIRMWARE_UPDATE_START Firmware update started. No operations can be performed on device until update done. I2C An I²C bus master, such as a microcontroller) has generated an event through the I²C interface. INFRARED_RECEIVED Infrared data has been received. REQUEST_COMPLETE Background command complete. RS232_RECEIVED RS-232 data has been received. SETTINGS_CHANGED The settings of a device changed. USB_TREE_CHANGED The tree of downstream connected USB devices changed. Applies to UEX, in case of the UEX attached to a BlueRiver AVP device there will be 2 events (one from the UEX device and one from the AVP). TIP: As new versions of the API can introduce new event types, clients should accept and ignore other event types to ensure compatibility with future versions.
timestamp	3.0	Integer	Time that the event occurred. The precision of this time stamp is in seconds and its origin (time 0) is unspecified.
request_id	30	Integer	Request ID associated with the event, null if not applicable. Refer to section 4.16 Command request for more information. DEVICE_CONNECTED: The request ID used to retrieve basic device data. DEVICE_DISCONNECTED: The request ID used to retrieve basic device data. DEVICE_INFO_AVAILABLE The request ID used to retrieve available data. SETTINGS_CHANGED: The request ID used to retrieve available data. I2C: The request ID used to identify the source of an event. INFRARED_RECEIVED The request ID is used to retrieve Infrared (IR) data. RS232_RECEIVED: Request ID used to retrieve RS-232 data. REQUEST_COMPLETE: Includes the request ID associated to the request that completed.

4.5.3 Errors

None

4.5.4 Example

```
GET /api/event?after=49 HTTP/1.1
...
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "events" : [{
      "device_id" : "d8803962abcd",
      "event_id" : 50,
      "event_type" : "SETTINGS_CHANGED",
      "timestamp" : 1450386633,
      "request_id" : 29
    },
    {
      "device_id" : "d8803962b2d9",
      "event_id" : 51,
      "event_type" : "DEVICE_CONNECTED",
      "timestamp" : 1450386633,
      "request_id" : null
    },
    {
      "device_id" : "d8803962b2d9",
      "event_id" : 52,
      "event_type" : "DEVICE_INFO_AVAILABLE",
      "timestamp" : 1450386633,
      "request_id" : 30
    }
  ]
},
  "error" : null
}
```

4.6 Command factory

```
POST /api/device/{target}
{
  "op" : "factory"
}
```

Reminder: Command used to restore the factory default settings of a single device (BlueRiver or UEX) or a group of devices.

4.6.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group.

- If a UEX (UEX1000 or UEX2000) is attached to the BlueRiver AVP, the target shall be the AVP device. In this case, the factory command applies to both the AVP and its attached UEX device(s). Setting UEX as the target is not supported when attached to an AVP for this command.

- If a UEX is used standalone (UEX2000 only), the target shall be the UEX device.

The allowed device groups are:

ALL:	All devices
ALL_TX:	All transmitter devices
ALL_RX:	All receiver devices

4.6.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for the description of the return value.

4.6.3 Errors

ILLEGAL_ARGUMENT	Occurs if the target argument is not a valid device ID or group name.
------------------	---

4.6.4 Example

```
POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "factory"
}
HTTP/1.1 201 Created
Location: /api/request/18772
...
{
  "status" : "PROCESSING",
  "request_id" : 18772,
  "result" : null,
  "error" : null
}
```

4.7 Command get

List of discovered devices:

```
GET /api/device
```

The hello subset:

```
GET /api/device/{target}
```

All other subsets:

```
POST /api/device/{target}
{
  "op" : "get",
  "subset" : "String"
}
```

Reminder: The `get` command retrieves information from a single device (BlueRiver or UEX if UEX2000 is used standalone) or from a group of devices.

When the `subset` argument is either `list`, `hello` or `capabilities`, this command is guaranteed to return immediately. For all other subsets, the command may return with status `PROCESSING`.

TIP: Client applications must be capable of handling the case where a `get settings` request returns with status `PROCESSING`. For subset `settings`, the API Server caches information that allows it to return the requested information immediately, however, occasionally it does return with a `PROCESSING` status.

4.7.1 Arguments

The `target` argument must be either the device ID of a single device (BlueRiver or UEX if UEX2000 is used standalone) or the name of a device group.

The allowed device groups are:

<code>ALL:</code>	All devices
<code>ALL_TX:</code>	All transmitter devices
<code>ALL_RX:</code>	All receiver devices

The valid values for the `subset` argument and their meaning are provided in the table below.

Semtech Confidential Information
Mark Li
ADV
02/03/2026

Table 10: Subset Arguments for "get" Command

Subset Name	Since	Description
capabilities	3.0	Retrieves the capabilities of the target device(s). Applicable to BlueRiver and UEX devices.
device	3.0	Retrieves all available information regarding a target device(s). Applicable to BlueRiver and UEX devices.
dot1x_certificates	3.6.0.1	Retrieves details of the previously uploaded certificate chain. Applicable to BlueRiver AVP devices.
edid	3.0	Retrieves the EDID from a target device(s). Applicable to BlueRiver devices.
gpio	3.0	Retrieves the device GPIO mode (input or output) and value. Applicable to BlueRiver devices.
hello	3.0	Retrieves basic status information regarding a target device(s) that is available when it is first detected on the network, such as device type and firmware version. Applicable to BlueRiver and UEX devices.
identity	3.0	Retrieves basic identity information regarding a target device(s), such as device name, type and firmware version. Applicable to BlueRiver and UEX devices.
list	3.0	Lists target devices with no information other than device ID. Most useful with a device group as the target argument. Applicable to BlueRiver and UEX devices.
multiview	3.4	Retrieve details of the layout currently loaded on a receiver (RX) device operating in multiview mode. Includes same info as a layout describe request but the data is retrieved directly from the receiver(s). Applicable to BlueRiver devices.
path	3.6	Returns all the paths applicable to the specified target(s). Applicable to BlueRiver devices.
settings	3.0	Return detailed information regarding current configuration and state of the target device(s). The return is similar the return from get device. Most of the device object is returned, excluding the EDID, temperature and the enumeration of available choices for configuration members. Applicable to BlueRiver and UEX devices.
temperature	3.0	Retrieves the temperature of the target device(s). Applicable to BlueRiver devices.
timestamp	3.7	Retrieves the API time stamp.
usb_tree	3.8	Retrieves from a LOCAL or REMOTE UEX USB Extender the tree of the USB devices connected downstream. Applicable to UEX (attached to AVP and standalone).

4.7.2 Return Value

For each device targeted by the `get` command, there is either:

1. An entry in the devices array if the information was fetched successfully.
2. Alternatively, a device error object in the error array if the information was not successfully retrieved.

```
{
  "devices" : [DeviceObject, ...],
  "error" : [DeviceError, ...]
}
```

Table 11: Get Command Object Members

Member Name	Since	Type	Description
devices	3.0	Array of DeviceObject	Array of objects that are subset of DeviceObject.
error	3.0	Array of DeviceError	Array of DeviceError objects. Indicates devices command failed for and reason for failure.

Refer to section 6 Device Object Definition (DeviceObject) for descriptions of device objects and for details on device error objects (DeviceError) refer to the Introduction to the SDVoE API User Guide.

4.7.3 Errors

PROTOCOL Occurs if the target argument is not a valid device ID or group name.
 ILLEGAL_ARGUMENT Occurs if the subset argument is not a valid subset name.

4.7.4 Examples

For example data of various get returns refer to Appendix: Get Command Output Examples.

4.8 Command usb_extender

```
POST /api/device/0016c035566e
{
  "op" : "usb_extender:reset",
  "index" : 0 (optional)
}
```

Reminder: usb_extender:reset is used to reset an USB extender device that has been incorporated into a BlueRiver endpoint device, for example if the USB Icron extender module's reported state is unreachable.

4.8.1 Arguments

The target argument must be either the device ID of a single device or the name of a device group.

The allowed device groups are:

ALL: All devices
 ALL_TX: All transmitter devices
 ALL_RX: All receiver devices

The index argument represents the USB extender device, which can be 0 or 1. If the index is not specified the default is 0.

4.8.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for the description of the return value.

4.8.3 Errors

If the USB extender does not support the Reset command, an error message like "request not supported by device" is returned.

4.8.4 Example

The example below resets the USB extender device of endpoint device 0016c035566e.

```
POST /api/device/0016c035566e
{
  "op" : "usb_extender:reset",
  "index" : 0
}
HTTP/1.1 201 Created
Location: /api/request/101
...
{
  "status" : "PROCESSING",
  "request_id" : 101,
  "result" : null,
  "error" : null
}
```

4.9 Command join

```
POST /api/device/{target}
{
  "op" : "join",
  "source_device" : String,
  "stream_type" : String,
  "stream_index" : Integer,
  "subscription_type" : String (optional),
  "subscription_index" : Integer,
  "display_mode" : String (conditional),
  "video" : VideoFormatArguments (conditional)
}
```

Reminder: `join` is used to subscribe one or more receivers to a transmitter's video or audio stream.

Before joining an HDMI video stream (type equals `HDMI`), the video processing mode and video output parameters must have previously been set on the receiver device(s). This can be accomplished by either:

1. Issuing a separate `set video` command. Refer to section 4.19.12 Command `set video` for more information on using the `set video` command.
2. Or specify the video processing mode and video output parameters directly in the `join` command as described in this section.

4.9.1 Arguments

The `target` argument must be either the device ID of a single receiver device or the name of a device group.

The allowed device groups for the `join` command are:

ALL	All devices
ALL_RX	All receiver devices

The `source_device` argument must be the device ID of a single transmitter device.

The `stream_type` argument must be a valid stream type. The following types are supported:

STEREO_AUDIO	Stereo audio stream
HDMI	HDMI video stream
HDMI_AUDIO	HDMI audio stream
MULTICH_AUDIO	Multichannel audio stream

The `index_tx` argument must be the valid index of a stream of the specified type available on the source device.

TIP: For the command to succeed, the `source` stream must currently be started/running. Refer to section 4.21 Command `start` for information on starting device streams.

The `subscription_type` argument is optional. If specified, it must be identical to the `stream_type` argument. Subscriptions can only join streams of the same type.

The `subscription_index` argument must be a valid index of a subscription of specified type on the destination device.

The `display_mode` and `video` arguments can only be specified when joining a stream whose type is HDMI that is the `stream_type` argument is set to HDMI. When these arguments are specified, the command has the effect of combining the `join/set video` commands.

The `display_mode` argument specifies the display mode.

The `video` argument specifies video format. As mentioned earlier, no video argument needed nor permitted for genlock display mode. In this case, an empty object can be provided or argument can be omitted entirely.

TIP: Refer to section 2.5.2 Video Display Mode Argument for information on video modes.

4.9.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for the description of the return value.

4.9.3 Errors

It is possible that an error could be returned when a join command is issued. Possible errors include:

ILLEGAL_ARGUMENT	Indicates the arguments entered are invalid.
INVALID_STATE	Occurs if the source device is disconnected.

The request may also fail for individual devices for the following reasons:

INVALID_STATE	Indicates that the source stream is stopped
DEVICE_DISCONNECTED	if the source or destination device is disconnected

If source device is not connected when a `join` command is issued, the command fails with the error, `INVALID_STATE`.

If the source device is disconnected while the command is being processed, the command may fail for some destination devices with `DEVICE_DISCONNECTED`.

4.9.4 Examples of Using the join Command

Below are some examples on the use of the `join` command to subscribe receiver devices to transmitter devices for both video and audio streams. Some of these examples also combine the `join` and `set video` command arguments issuing both sets of commands simultaneously.

```
POST /api/device/001ec0f04d9c HTTP/1.1
```

```
...
```

```
{
  "op" : "join",
  "source_device" : "001ec0f03668",
  "stream_type" : "STEREO_AUDIO",
  "stream_index" : 0,
  "subscription_index" : 0
}
```

```
HTTP/1.1 201 Created
```

```
Location: /api/request/60
```

```
...
```

```
{
  "status" : "PROCESSING",
  "request_id" : 60,
  "result" : null,
  "error" : null
}
```

```
POST /api/device/001ec0f04d9c HTTP/1.1
```

```
...
```

```
{
  "op" : "join",
  "source_device" : "001ec0f03668",
  "stream_type" : "HDMI",
  "stream_index" : 0,
  "subscription_index" : 0
}
```

```
HTTP/1.1 201 Created
```

```
Location: /api/request/61
```

```
...
```

```
{
  "status" : "PROCESSING",
  "request_id" : 61,
  "result" : null,
  "error" : null
}
```

```
POST /api/device/001ec0f04d9c HTTP/1.1
```

```
...
```

```
{
  "op" : "join",
```

```
"source_device" : "001ec0f03668",
"stream_type" : "HDMI",
"stream_index" : 0,
"subscription_index" : 0,
"display_mode" : "GENLOCKED"
}
HTTP/1.1 201 Created
Location: /api/request/62
...
{
  "status" : "PROCESSING",
  "request_id" : 62,
  "result" : null,
  "error" : null
}
POST /api/device/001ec0f04d9c HTTP/1.1
...
{
  "op" : "join",
  "source_device" : "001ec0f03668",
  "stream_type" : "HDMI",
  "stream_index" : 0,
  "subscription_index" : 0,
  "display_mode" : "FAST_SWITCHED",
  "video" : {
    "width" : 1920,
    "height" : 1080,
    "fps" : 60
  }
}
HTTP/1.1 201 Created
Location: /api/request/63
...
{
  "status" : "PROCESSING",
  "request_id" : 63,
  "result" : null,
  "error" : null
}
POST /api/device/001ec0f04d9c HTTP/1.1
...
{
  "op" : "join",
  "source_device" : "001ec0f03668",
  "stream_type" : "HDMI",
```

```
"stream_index" : 0,
"subscription_index" : 0,
"display_mode" : "WALL_GENLOCKED",
"video" : {
    "width" : 1920,
    "height" : 1080,
    "fps" : 60,
    "horiz_offset" : 0,
    "vert_offset" : 1080,
    "keep_width" : 1920,
    "keep_height" : 1080,
}
}
HTTP/1.1 201 Created
Location: /api/request/64
...
{
    "status" : "PROCESSING",
    "request_id" : 64,
    "result" : null,
    "error" : null
}
POST /api/device/001ec0f04d9c HTTP/1.1
...
{
    "op" : "join",
    "source_device" : "001ec0f03668",
    "stream_type" : "HDMI",
    "stream_index" : 0,
    "subscription_index" : 0,
    "display_mode" : "WALL_FAST_SWITCHED",
    "video" : {
        "width" : 1920,
        "height" : 1080,
        "fps" : 60,
        "horiz_offset" : 0,
        "vert_offset" : 1080,
        "keep_width" : 1920,
        "keep_height" : 1080,
    }
}
HTTP/1.1 201 Created
Location: /api/request/65
...
{
    "status" : "PROCESSING",
```

```

    "request_id" : 65,
    "result" : null,
    "error" : null
}

```

4.10 Command leave

```

POST /api/device/{target}

{
    "op" : "leave",
    "subscription_type" : String (optional),
    "subscription_index" : Integer (optional)
}

```

Reminder: Command used to unsubscribe one or more receiver devices from a transmitter device stream. The leave command was introduced in API version 3.0.

4.10.1 Arguments

The `target` argument must be either the device ID of a single receiver device or the name of a device group. The allowed device groups are:

ALL: All devices
 ALL_RX: All receiver devices

Note: The `target` argument can also be a transmitter (TX) device in relation to stereo and multichannel audio stream types. So in addition to the above, either the device ID of a single transmitter (TX) or the ALL_TX device group can be specified when managing either of these audio stream types.

The `subscription_type` argument must be a valid subscription type.

The following types are supported:

STEREO_AUDIO Stereo audio stream
 HDMI HDMI video stream
 HDMI_AUDIO HDMI audio stream
 MULTICH_AUDIO Multichannel audio stream

Additional information:

- i. The `index` argument must be the valid index of a subscription of the specified type on the target device.
- ii. If the `index` is omitted, all subscriptions of the given type are affected.
- iii. If `subscription_type` is omitted, all subscriptions of all types listed above are affected.

4.10.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.10.3 Errors

ILLEGAL_ARGUMENT Occurs if the arguments entered are invalid.

The request may also fail for individual devices for the following reason:

DEVICE_TYPE

Occurs if device receiving the command is not a receiver device.

4.10.4 Example

```
POST /api/device/001ec0f04d9c HTTP/1.1
...
{
  "op" : "leave",
  "subscription_type" : "HDMI",
  "subscription_index" : 0
}
HTTP/1.1 201 Created
Location: /api/request/3929
...
{
  "status" : "PROCESSING",
  "request_id" : 3929,
  "result" : null,
  "error" : null
}
```

4.11 Command list

The `list` command provides sub-commands to query lists from the API server. Below each of the sub-commands are reviewed separately.

4.11.1 Command list firmware

```
GET /api/firmware
```

This command was introduced in API version 3.0.

Reminder: The `list firmware` sub-command, lists all currently available firmware update files (for BlueRiver and UEX devices) to the API server.

4.11.1.1 Return Value

```
{
  "firmware" : [FirmwareItemObject, ...]
}
```

Table 12: List Firmware Array Member

Member Name	Since	Type	Description
firmware	3.0	Array of <i>FirmwareItemObject</i>	Array of objects that represent the firmware file.

Each item in the firmware array (*FirmwareItemObject*) has the following structure:

```
{
  "file_name" : String,
  "supports_production" : String,
}
```

Table 13: List Firmware Object Members

Member Name	Since	Type	Description
firmware	3.0	String	File name containing the firmware.
supports_production	3.0	Boolean	If member false, file only supports device upgrades (update command).

4.11.1.2 Example

```

GET /api/firmware HTTP/1.1
...
HTTP/1.1 200 OK
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "firmware" : [
      {
        "file_name" : "blueriver_rdk_2-0-0-0.apz",
        "supports_production" : false
      },
      {
        "file_name" : "blueriver_rdk_2-0-0-0.apz ",
        "supports_production" : true
      }
    ]
  },
  "error" : null
}

```

4.11.2 Command list multicast

```
GET /api/multicast
```

This command was introduced in API version 3.0.

Reminder: This command retrieves information regarding the multicast IP addresses currently in use.

4.11.2.1 Return Value

```

{
  "multicast" : [MulticastItemObject, ...]
}

```

Table 14: List Multicast Object Array Member

Member Name	Since	Type	Description
multicast	3.0	Array of MulticastItemObject	An array of objects that each represent a multicast IP address.

Each item in the multicast array (MulticastItemObject) has the following structure:

```

{
  "address" : String,
  "state" : String,
  "device_id" : String,

```

```

    "stream_type" : String,
    "stream_index" : Integer
}

```

Table 15: List Multicast Object Members

Member Name	Since	Type	Description
address	3.0	String	Multicast IP address in dotted decimal notation
state	3.0	String	Current state of the stream, which is one of the following: - STREAMING device is actively transmitting on specified multicast address. - STOPPED the stream is stopped.
device_id	3.0	String	Device ID of device that is using the multicast address.
stream_type	3.0	String	Type of stream the multicast address is being used for.
steam_index	3.0	Integer	Index of stream the multicast address is being used for.

4.11.2.2 Example

```
GET /api/multicast
```

```
...
```

```
HTTP/1.1 200 OK
```

```

{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "multicast" : [
      {
        "address" : "224.1.1.1",
        "state" : "STREAMING",
        "device_id" : "d8803962feae",
        "stream_type" : "HDMI",
        "stream_index" : 0
      },
      {
        "address" : "224.1.1.2",
        "state" : "STREAMING",
        "device_id" : "d8803962dbf3",
        "stream_type" : "HDMI",
        "stream_index" : 0
      },
      {
        "address" : "224.1.1.3",
        "state" : "STREAMING",
        "device_id" : "001ec0f01818",
        "stream_type" : "STEREO_AUDIO",
        "stream_index" : 0
      }
    ]
  },
  "error" : null
}

```

4.12 Command `netstat`

By monitoring network characteristics on a constant basis, network diagnostic information provides historical data, which in turn assists users in investigating the possible cause of a network problem. Using a `netstat` request allows for tracking the uptime and data usage since boot-up. API also can provide the bandwidth usage of a encoder (TX) on a per stream basis, including:

- Total bandwidth usage
- CPU control
- Auxiliary channel
- Native and Scaled video streams (stream 0 and stream 1).
- Audio stream bandwidth usage for HDMI audio, Stereo audio and Multichannel audio.
- HDMI AUX stream that carries all HDMI aux data including AV info packets and audio data.
- 1GbE stream
- USB Streams (Semtech UEX or ICRON)
- Thumbnail preview stream (NT2000/NT1000 devices only).

For a decoder (RX) device, the total bandwidth usage is available. Network data is not available on a per stream type as it is for transmitter streams.

Notes:

- i. Refer to Table 30: 10G RX Egress Missed Frames Flags Object Members for descriptions of each of the above object stream members.
- ii. The `netstat` API command is used to retrieve network characteristic information.
- iii. Three `netstat` commands (`read`, `clear` and `select`) are supported refer to the appropriate subsection for details on the use of each.
- iv. Possible to clear/reset all the counters.
- v. Possible to select (mask) specific channels.

4.12.1 Network Characterization Assumptions & Limitations

This subset of feature specifications apply to BlueRiver-based endpoint devices.

When a device reboots, all internal statistics are reset for the device.

The received bandwidth measured as a whole, while the transmitted bandwidth can be measured either as a whole or per transmitted stream element. Potentially up to 11 streams can be monitored on a transceiver or transmitter device.

Bandwidth channels are measured one at a time and each measurement takes two seconds. Only currently selected channels are measured.

The following applies to bandwidth measurements:

- Transmitter devices allow `egress` (i.e. transmitted traffic) bandwidth measurements only.
- Receiver devices allow `ingress` (i.e. received traffic) bandwidth measurements only.
- Transceiver devices allow both `egress/ingress` bandwidth measurements (there is two complete bandwidth measurement modules present, both modules can be read in parallel).

4.12.2 Command `netstat read`

This command allows statistics and up time of a device or a group of devices to be obtained.

```
POST /api/device/{target}
{
  "op" : "netstat",
  "option" : "read",
  "filter" : String (optional)
}
```

4.12.2.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group. The allowed device groups are the following:

ALL	All devices
ALL_RX	All receiver devices
ALL_TX	All transmitter devices

This command retrieves all statistics that are available for the specified device or a subset of the statistics if an optional `filter` argument is used.

The supported `filter` arguments are:

counter	Returns all the counters in the counter class.
bandwidth	Returns all bandwidth counters in the bandwidth class.
error	Returns error flags in the "dropped_frame" class (10G_EGRESS) and the error class (10G_INGRESS).

4.12.2.2 Return Value

For each device that is targeted by this command, there is either a statistics entry in the result if the statistics were fetched successfully or a device error object if the information was not retrieved.

```
{
  "statistics" : [DeviceStatistics, ...],
  "error" : [DeviceError, ...]
}
```

Table 16: Netstat Read Object Array Members

Member Name	Since	Type	Description
statistics	3.0	Array of DeviceStatistics	Array of objects containing requested statistics.
error	3.0	Array of DeviceError	Array of DeviceError objects indicating the devices for which the command failed and the reason for the failure.

A `DeviceStatistics` object has the following format:

```
{
  "device_id" : String,
  "device_up_time" : Integer,
  "data_paths" : [DataPath, ...]
}
```

Table 17: Netstat Device Object Array members

Member Name	Since	Type	Description
device_id	3.0	String	Device ID of the device to which statistics belong.
device_up_time	3.0	Integer	Device up time since last reset or power up, in seconds.
data_paths	3.0	Array of DataPath	Array of objects containing statistics for a given data path.

A DeviceError has the following format:

```
{
  "device_id" : String,
  "reason" : String,
  "message" : String
}
```

Table 18: Netstat Error Object Array Members

Member Name	Since	Type	Description
device_id	3.0	String	The device ID of the device to which the error belongs.
reason	3.0	String	Specific error code. For example "DEVICE_TYPE"
message	3.0	String	Message with details of the error. For example: "Statistics is not supported on this device."

DataPath is one of the following data path objects:

10GTXEgressDataPath	On a transmitter (TX) device, egress traffic (exiting) of a 10G link
10GTXIngressDataPath	On a transmitter (TX) device, ingress traffic (entering) of a 10G link
10GRXEgressDataPath	On a receiver (RX) device, egress traffic (exiting) of a 10G link
10GRXIngressDataPath	On a receiver (RX) device, ingress traffic (entering) of a 10G link
10GTXRXIngressDataPath	On a transceiver (TR) device, ingress traffic (entering) of a 10G link

Table 19: Responses to Netstat Read per Device Type

Device Type	Egress	Ingress
Transmitter (TX)	10GTXEgressDataPath	10GTXIngressDataPath
Receiver (RX)	10GRXEgressDataPath	10GRXIngressDataPath
Transceiver (TR)	10GTXEgressDataPath	10GTXRXIngressDataPath

10GTXEgressDataPath

A 10GTXEgressDataPath has the following format:

```
{
  "type" : String,
  "index" : Integer,
  "bandwidth" : 10GTXEgressBandwidth,
  "counters" : 10GTXEgressCounters,
  "dropped_frames" : 10GTXEgressMissedFramesFlags
}
```

Table 20: 10G TX Egress Data Path Object Members

Member Name	Since	Type	Description
type	3.0	String	Data path type for this object: "10G_TRANSMITTER_EGRESS".
index	3.0	Integer	Index for an object of this data path type. Is always 0 since there is only one 10GbE link.
bandwidth	3.0	10GTXEgressBandwidth	Object contains all the bandwidth counters applicable to the associated data path type.
counters	3.0	10GTXEgressCounters	Object with all the other counters applicable to this data path type that are not bandwidth counters.
dropped_frames	3.0	10GTXEgressMissedFramesFlags	Object with applicable missed frame flags (Booleans).

A 10GTXEgressBandwidth has the following format:

```
{
  "selection" : 10GTXEgressBandwidthSelection,
  "usage" : 10GTXEgressBandwidthUsage
}
```

Table 21: 10G TX Egress Bandwidth Object Members

Member Name	Since	Type	Description
selection	3.0	10GTXEgressBandwidthSelection	Object with the selected state of each applicable channel.
usage	3.0	10GTXEgressBandwidthUsage	Object with the usage of each applicable channel.

A 10GTXEgressBandwidthSelection has the following format:

```
{
  "total" : Boolean,
  "cpu_control" : Boolean,
  "aux_channel" : Boolean,
  "hdmi_stream_0" : Boolean,
  "hdmi_stream_1" : Boolean,
  "stereo_audio" : Boolean,
  "hdmi_audio" : Boolean,
  "multi_channel_audio" : Boolean,
  "thumbnail" : Boolean,
  "usb_extender" : Boolean,
  "network_1g" : Boolean
}
```

Table 22: 10G TX Egress Bandwidth Selection Object Members

Member Name	Since	Type	Description
total	3.0	Boolean	Select filter (true/false) for total measured bandwidth.
cpu_control	3.0	Boolean	Select filter (true/false) for the measured bandwidth of the CPU communications.
aux_channel	3.0	Boolean	Select filter (true/false) for the measured bandwidth of the first HDMI A&V description and audio channel.
hdmi_stream_0	3.0	Boolean	Select filter (true/false) for the measured bandwidth of by the first HDMI video channel.
hdmi_stream_1	3.0	Boolean	Select filter (true/false) for the measured bandwidth of the second HDMI video channel.
stereo_audio	3.0	Boolean	Select filter (true/false) for the measured bandwidth of the stereo audio channel.
hdmi_audio	3.0	Boolean	Select filter (true/false) for the measured bandwidth of the HDMI audio channel.
multi_channel_audio	3.0	Boolean	Select filter (true/false) for the measured bandwidth of the multichannel audio channel.
thumbnail	3.0	Boolean	Select filter (true/false) for the measured bandwidth of the thumbnail channel.
usb_extender	3.8	Boolean	Select filter (true/false) for the measured bandwidth of the USB Extender (Semtech UEX or Icron) channel.
network_1g	3.0	Boolean	Select filter (true/false) for the measured bandwidth of the 1G network channel.

A 10GTXEgressBandwidthUsage has the following format:

```
{
  "total" : Integer,
  "cpu_control" : Integer,
  "aux_channel" : Integer,
  "hdmi_stream_0" : Integer,
  "hdmi_stream_1" : Integer,
  "stereo_audio" : Integer,
  "hdmi_audio" : Integer,
  "multi_channel_audio" : Integer,
  "thumbnail" : Integer,
  "usb_extender" : Integer,
  "network_1g" : Integer
}
```

Table 23: 10G TX Egress Bandwidth Usage Object Members

Member Name	Since	Type	Description
total	3.0	Integer	Total bandwidth usage.
cpu_control	3.0	Integer	Usage of the CPU communications.
aux_channel	3.0	Integer	Usage of the first HDMI A&V description and audio channel.
hdmi_stream_0	3.0	Integer	Usage of the first HDMI video channel.
hdmi_stream_1	3.0	Integer	Usage of the second HDMI video channel.
stereo_audio	3.0	Integer	Usage of the stereo audio channel.
hdmi_audio	3.0	Integer	Usage of the HDMI audio channel.
multi_channel_audio	3.0	Integer	Usage of the multichannel audio channel.
thumbnail	3.0	Integer	Usage of the thumbnail channel.
usb_extender	3.8	Integer	Usage of the USB Extender (Semtech UEX or Icron) channel.
network_1g	3.0	Integer	Usage of the 1G network channel.

A 10GTXEgressCounters has the following format:

```
{
    "ethernet_flow_control_frames" : Integer
}
```

Table 24: 10G TX Egress Counters Object Member

Member Name	Since	Type	Description
ethernet_flow_control_frames	3.0	Integer	Number of Ethernet flow control frames received since the last reset

A 10GTXEgressMissedFramesFlags has the following format:

```
{
    "cpu_control" : Boolean,
    "aux_channel" : Boolean,
    "hdmi_stream_0" : Boolean,
    "hdmi_stream_1" : Boolean,
    "stereo_audio" : Boolean,
    "hdmi_audio" : Boolean,
    "multi_channel_audio" : Boolean,
    "thumbnail" : Boolean,
    "usb_extender" : Boolean,
    "network_1g" : Boolean
}
```

Table 25: 10G TX Egress Miss Frames Flags Object Members

Member Name	Since	Type	Description
cpu_control	3.0	Boolean	Indicates that at least one frame was missed from the CPU control channel.
aux_channel	3.0	Boolean	Indicates that at least one frame missed from AUX channel.
hdmi_stream_0	3.0	Boolean	Indicates that at least one frame was missed from the HDMI receiver output.
hdmi_stream_1	3.0	Boolean	Indicates that at least one frame was missed from the scaler output.
stereo_audio	3.0	Boolean	Indicates that at least one frame was missed from the stereo audio source.
hdmi_audio	3.0	Boolean	Indicates that at least one frame was missed from the HDMI audio source.
multi_channel_audio	3.0	Boolean	Indicates that at least one frame was missed from the multichannel audio source.
thumbnail	3.0	Boolean	Indicates that at least one frame was missed from the thumbnail source.
usb_extender	3.8	Boolean	Indicates that at least one frame was missed from the USB Extender (Semtech UEX or Icron) source.
network_1g	3.0	Boolean	Indicates at least one frame was missed from 1G source.

10GTxIngressDataPath

A 10GTxIngressDataPath has the following format:

```
{
    "type" : String,
    "index" : Integer,
    "counters" : 10GTxIngressCounters,
    "errors" : 10GTxIngressErrorFlags
}
```

Table 26: 10G TX Ingress Data Path Object Members

Member Name	Since	Type	Description
type	3.0	String	Data path type for this object: "10G_TRANSMITTER_INGRESS".
index	3.0	Integer	Index for an object of this data path type. Is always 0 since there is only one 10GbE link.
counters	3.0	10GTxIngressCounters	Object with all other counters applicable to the data path type that are not bandwidth counters.
errors	3.0	10GTxIngressErrorFlags	Object with applicable error flags (Booleans).

Note: A 10GTxIngressDataPath object has no bandwidth object.

A 10GTXIngressCounters has the following format:

```
{
    "ethernet_flow_control_frames" : Integer,
    "xgmii_phy_errors" : Integer,
    "xgmii_preamble_errors" : Integer,
    "xgmii_crc_errors" : Integer,
    "xgmii_frame_errors" : Integer,
    "jumbo_frames" : Integer,
    "runt_frame_errors" : Integer
}
```

Table 27: 10G TX Ingress Counters Object Members

Member Name	Since	Type	Description
ethernet_flow_control_frames	3.0	Integer	Number of flow control frames received.
xgmii_phy_errors	3.0	Integer	Number of errors from the 10G MII chip (PHY layer).
xgmii_preamble_errors	3.0	Integer	Number of preamble errors from the 10G MII chip (PHY layer).
xgmii_crc_errors	3.0	Integer	Number of Cyclic Redundancy Check (CRC) errors from the 10G MII chip (PHY layer).
xgmii_frame_errors	3.0	Integer	Number of framing errors from the 10G MII chip (PHY layer).
jumbo_frames	3.0	Integer	Number of frames over 1522 bytes, not an error.
runt_frame_errors	3.0	Integer	Number of frames smaller than 60 bytes (error).

A 10GTXIngressErrorFlags has the following format:

```
{
    "network_frame_dropped" : Boolean,
    "xgmii_decoder_error" : Boolean,
    "xgmii_decoder_wrong_start_code" : Boolean,
    "xgmii_decoder_wrong_stop_code" : Boolean,
    "xgmii_decoder_invalid_control_code" : Boolean
}
```

Table 28: 10G TX Ingress Error Object Members

Member Name	Since	Type	Description
network_frame_dropped	3.0	Boolean	Frame(s) dropped in receive on the network (true/false).
xgmii_decoder_error	3.0	Boolean	Decoder error from the 10G MII chip (PHY layer) (true/false).
xgmii_decoder_wrong_start_code	3.0	Boolean	Decoder wrong start code error from the 10G MII chip (PHY layer) (true/false).
xgmii_decoder_wrong_stop_code	3.0	Boolean	Decoder wrong stop code error from the 10G MII chip (PHY layer) (true/false).
xgmii_decoder_invalid_control_code	3.0	Boolean	Decoder invalid control code error from 10G MII chip (PHY layer) (true/false).

10GRXEgressDataPath

A 10GRXEgressDataPath has the following format:

```
{
  "type" : String,
  "index" : Integer,
  "dropped_frames" : 10GRXEgressMissedFramesFlags
}
```

Table 29: 10G RX Egress Data Path Object Members

Member Name	Since	Type	Description
type	3.0	String	Data path type for this object is 10G_RECEIVER_EGRESS.
index	3.0	Integer	Index for an object of this data path type. Is always 0 since there is only one 10GbE link.
dropped_frames	3.0	10GRXEgressMissedFramesFlags	Object with applicable missed frame flags (Boolean).

Note: A 10GRXEgressDataPath object has no bandwidth counters.

A 10GRXEgressMissedFramesFlags has the following format:

```
{
  "cpu_control" : Boolean,
  "multi_channel_audio" : Boolean,
  "usb_extender" : Boolean,
  "network_1g" : Boolean
}
```

Table 30: 10G RX Egress Missed Frames Flags Object Members

Member Name	Since	Type	Description
cpu_control	3.0	Boolean	Frame(s) dropped on CPU control channel (true/false).
multi_channel_audio	3.0	Boolean	Frame(s) dropped on multichannel audio channel (true/false).
usb_extender	3.8	Boolean	Frame(s) dropped on USB Extender (Semtech UEX or Icron) channel (true/false).
network_1g	3.0	Boolean	Frame(s) dropped on 1G network channel (true/false).

10GRXIngressDataPath

A 10GRXIngressDataPath has the following format:

```
{
  "type" : String,
  "index" : Integer,
  "bandwidth" : 10GRXIngressBandwidth,
  "counters" : 10GRXIngressCounters,
  "errors" : 10GRXIngressErrorFlags
}
```


Table 31: 10G RX Ingress Data Path Object Members

Member Name	Since	Type	Description
type	3.0	String	Data path type for this object: "10G_RECEIVER_INGRESS"
index	3.0	Integer	Index for an object of this data path type. Is always 0 since there is only one 10GbE link.
bandwidth	3.0	10GRXIngressBandwidth	Object with all the bandwidth counters applicable to this data path type.
counters	3.0	10GRXIngressCounters	Object with all the other counters applicable to this data path type that are not bandwidth counters
errors	3.0	10GRXIngressErrorFlags	Object with applicable error flags (Booleans)

A 10GRXIngressBandwidth has the following format:

```
{
  "usage" : 10GRXIngressBandwidthUsage
}
```

Table 32: 10G RX Ingress Bandwidth Object Member

Member Name	Since	Type	Description
usage	3.0	10GRXIngressBandwidthUsage	Object with usage of each applicable channel.

A 10GRXIngressBandwidthUsage has the following format:

```
{
  "total" : Integer
}
```

Table 33: 10G RX Ingress Bandwidth Usage Object Member

Member Name	Since	Type	Description
total	3.0	Integer	Total usage

A 10GRXIngressCounters has the following format:

```
{
  "xgmii_phy_errors" : Integer,
  "xgmii_preamble_errors" : Integer,
  "xgmii_crc_errors" : Integer,
  "xgmii_frame_errors" : Integer,
  "jumbo_frames" : Integer,
  "runt_frame_errors" : Integer
}
```

Table 34: 10G RX Ingress Counters Object Members

Member Name	Since	Type	Description
xgmii_phy_errors	3.0	Integer	Number of errors from the 10G MII chip (PHY layer)
xgmii_preamble_errors	3.0	Integer	Number of preamble errors from 10G MII chip (PHY layer)
xgmii_crc_errors	3.0	Integer	Number of Cyclic Redundancy Check (CRC) errors from the 10G MII chip (PHY layer)
xgmii_frame_errors	3.0	Integer	Number of framing errors from the 10G MII chip (PHY layer)
jumbo_frames	3.0	Integer	Number of frames larger than 1522 bytes (not an error)
runt_frame_errors	3.0	Integer	Number of frames smaller than 60 bytes (error)

A 10GRXIngressErrorFlags has the following format:

```
{
  "network_frame_dropped" : Boolean,
  "xgmii_decoder_error" : Boolean,
  "xgmii_decoder_wrong_start_code" : Boolean,
  "xgmii_decoder_wrong_stop_code" : Boolean,
  "xgmii_decoder_invalid_control_code" : Boolean
}
```

Table 35: 10G RX Ingress Error Flags Object Members

Member Name	Since	Type	Description
network_frame_dropped	3.0	Boolean	Frame(s) dropped in receive on the network (true/false).
xgmii_decoder_error	3.0	Boolean	Decoder error from the 10G MII chip (PHY layer) (true/false).
xgmii_decoder_wrong_start_code	3.0	Boolean	Decoder wrong start code error from the 10G MII chip (PHY layer) (true/false).
xgmii_decoder_wrong_stop_code	3.0	Boolean	Decoder wrong stop code error from the 10G MII chip (PHY layer) (true/false).
xgmii_decoder_invalid_control_code	3.0	Boolean	Decoder invalid control code error from the 10G MII chip (PHY layer) (true/false).

10GTXXIngressDataPath

A 10GTXXIngressDataPath has the following format:

```
{
  "type" : String,
  "index" : Integer,
  "bandwidth" : 10GRXIngressBandwidth,
  "counters" : 10GTXXIngressCounters,
  "errors" : 10GRXIngressErrorFlags
}
```

The only difference between a 10GRXIngressDataPath and a 10GTXXIngressDataPath is the "counters" object, which is a 10GRXIngressCounters for the receiver device and 10GTXXIngressCounters for the transceiver device. Please refer to the section about 10GTXXIngressCounters for more details.

Table 36: 10G TXX Ingress Datapath Object Members

Member Name	Since	Type	Description
type	3.0	String	Object data path type: 10G_TRANSCEIVER_INGRESS"
index	3.0	Integer	Index for an object of this data path type. Is always 0 since there is only one 10GbE link.
bandwidth	3.0	10GRXIngressBandwidth	Refer to section 10GRXIngressDataPath
counters	3.0	10GTXXIngressCounters	Refer to section 10GTXXIngressDataPath
errors	3.0	10GRXIngressErrorFlags	Refer to section 10GRXIngressDataPath

4.12.2.3 Errors

`ILLEGAL_ARGUMENT` Returned if target argument is not a valid device ID or group name.

The request may also fail for individual devices for the following reasons:

`DEVICE_TYPE` Returned when the device does not support network statistics.

4.12.2.4 Example

The following example shows the response from two transmitters (TX) (first and second devices) and from a receiver (RX) (third device) to a read command without a subset.

Note: If a subset used, response has the same structure but some statistics objects will appear to be empty.

```
POST /api/device/all HTTP/1.1
...
{
  "op" : "netstat",
  "option" : "read"
}
HTTP/1.1 201 Created
Location: /api/request/16
{
  "status" : "PROCESSING",
  "request_id" : 16,
  "result" : null,
  "error" : null
}
GET /api/request/16 HTTP/1.1
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : 16,
  "result" : {
    "statistics" : [
      {
        "device_id" : "d8803962e132",
        "device_up_time" : 256596,
        "data_paths" : [
          {
            "type" : "10G_TRANSMITTER_EGRESS",
            "index" : 0,
            "bandwidth" : {
              "selection" : {
                "total" : false,
                "cpu_control" : false,
                "aux_channel" : false,
                "hdmi_stream_0" : false,
                "hdmi_stream_1" : true,
                "stereo_audio" : false,
```

```

        "hdmi_audio" : false,
        "multi_channel_audio" : false,
        "thumbnail" : true,
        "usb_extender" : true,
        "network_1g" : false
    },
    "usage" : {
        "total" : 0,
        "cpu_control" : 0,
        "aux_channel" : 0,
        "hdmi_stream_0" : 0,
        "hdmi_stream_1" : 391393830,
        "stereo_audio" : 0,
        "hdmi_audio" : 0,
        "multi_channel_audio" : 0,
        "thumbnail" : 0,
        "usb_extender" : 0,
        "network_1g" : 0
    }
},
"counters" : {
    "ethernet_flow_control_frames" : 0
},
"dropped_frames" : {
    "cpu_control" : false,
    "aux_channel" : false,
    "hdmi_stream_0" : false,
    "hdmi_stream_1" : false,
    "stereo_audio" : false,
    "hdmi_audio" : false,
    "multi_channel_audio" : false,
    "thumbnail" : false,
    "usb_extender" : false,
    "network_1g" : false
}
},
{
    "type" : "10G_TRANSMITTER_INGRESS",
    "index" : 0,
    "counters" : {
        "ethernet_flow_control_frames" : 0,
        "xgmii_phy_errors" : 0,
        "xgmii_preamble_errors" : 0,
        "xgmii_crc_errors" : 0,
        "xgmii_frame_errors" : 7741,
        "jumbo_frames" : 0,
        "runt_frame_errors" : 0
    },
    "errors" : {
        "network_frame_dropped" : false,
        "xgmii_decoder_error" : false,

```

```

        "xgmii_decoder_wrong_start_code" : false,
        "xgmii_decoder_wrong_stop_code" : false,
        "xgmii_decoder_invalid_control_code" : false
    }
}
]
},
{
    "device_id" : "d8803962b3c3",
    "device_up_time" : 269180,
    "data_paths" : [
        {
            "type" : "10G_TRANSMITTER_EGRESS",
            "index" : 0,
            "bandwidth" : {
                "selection" : {
                    "total" : true,
                    "cpu_control" : true,
                    "aux_channel" : true,
                    "hdmi_stream_0" : true,
                    "hdmi_stream_1" : true,
                    "stereo_audio" : true,
                    "hdmi_audio" : true,
                    "multi_channel_audio" : true,
                    "thumbnail" : true,
                    "usb_extender" : true,
                    "network_1g" : true
                },
                "usage" : {
                    "total" : 574925174,
                    "cpu_control" : 3296,
                    "aux_channel" : 6553226,
                    "hdmi_stream_0" : 393304912,
                    "hdmi_stream_1" : 174769650,
                    "stereo_audio" : 0,
                    "hdmi_audio" : 0,
                    "multi_channel_audio" : 0,
                    "thumbnail" : 0,
                    "usb_extender" : 72,
                    "network_1g" : 0
                }
            },
            "counters" : {
                "ethernet_flow_control_frames" : 0
            },
            "dropped_frames" : {
                "cpu_control" : false,
                "aux_channel" : false,
                "hdmi_stream_0" : false,
                "hdmi_stream_1" : false,
                "stereo_audio" : false,

```

```

        "hdmi_audio" : false,
        "multi_channel_audio" : false,
        "thumbnail" : false,
        "usb_extender" : false,
        "network_1g" : false
    }
},
{
    "type" : "10G_TRANSMITTER_INGRESS",
    "index" : 0,
    "counters" : {
        "ethernet_flow_control_frames" : 0,
        "xgmii_phy_errors" : 0,
        "xgmii_preamble_errors" : 0,
        "xgmii_crc_errors" : 0,
        "xgmii_frame_errors" : 0,
        "jumbo_frames" : 0,
        "runt_frame_errors" : 0
    },
    "errors" : {
        "network_frame_dropped" : false,
        "xgmii_decoder_error" : false,
        "xgmii_decoder_wrong_start_code" : false,
        "xgmii_decoder_wrong_stop_code" : false,
        "xgmii_decoder_invalid_control_code" : false
    }
}
]
},
{
    "device_id" : "d880396317ef",
    "device_up_time" : 269184,
    "data_paths" : [
        {
            "type" : "10G_RECEIVER_EGRESS",
            "index" : 0,
            "dropped_frames" : {
                "cpu_control" : false,
                "multi_channel_audio" : false,
                "usb_extender" : false,
                "network_1g" : false
            }
        },
        {
            "type" : "10G_RECEIVER_INGRESS",
            "index" : 0,
            "bandwidth" : {
                "usage" : {
                    "total" : 400579266
                }
            }
        }
    ],

```

```

        "counters" : {
            "xgmii_phy_errors" : 0,
            "xgmii_preamble_errors" : 0,
            "xgmii_crc_errors" : 0,
            "xgmii_frame_errors" : 0,
            "jumbo_frames" : 0,
            "runt_frame_errors" : 0
        },
        "errors" : {
            "network_frame_dropped" : false,
            "xgmii_decoder_error" : false,
            "xgmii_decoder_wrong_start_code" : false,
            "xgmii_decoder_wrong_stop_code" : false,
            "xgmii_decoder_invalid_control_code" : false
        }
    }
]
},
"error" : [
    {
        "device_id" : "001ec0f03668",
        "reason" : "DEVICE_TYPE",
        "message" : "Statistics is not supported on this device"
    },
    {
        "device_id" : "001ec0f03cc1",
        "reason" : "DEVICE_TYPE",
        "message" : "Statistics is not supported on this device"
    },
    {
        "device_id" : "d88039e7fa53",
        "reason" : "DEVICE_TYPE",
        "message" : "Statistics is not supported on this device"
    }
]
},
"error" : null
}

```

4.12.3 Command `netstat clear`

The `netstat clear` command clears/resets all the current statistics for a device or group of devices.

```

POST /api/device/{target}

{
    "op" : "netstat",
    "option" : "clear"
}

```

4.12.3.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group.

The allowed device groups are:

ALL: All devices
ALL_TX: All transmitter devices
ALL_RX: All receiver devices

4.12.3.2 Return Value

For each device targeted by this command, there is `clear_statistics` entry in the result if the statistics were cleared successfully, or a device error object if `clear` is not supported.

```
{
  "clear_statistics" : [DeviceResponse, ...],
  "error" : [DeviceError, ...]
}
```

Table 37: Netstat Clear Return Value Object Members

Member Name	Since	Type	Description
DeviceResponse	3.0	Array of DeviceResponse	Array of objects containing a <code>device_id</code> indicating the statistics that were cleared.
DeviceError	3.0	Array of DeviceError	An array of <code>DeviceError</code> objects indicating the devices for which command failed and the reason for the failure.

A `DeviceResponse` has the following format:

```
{
  "device_id" : String
}
```

Table 38: Netstat Clear Device Response Object Member

Member Name	Since	Type	Description
<code>device_id</code>	3.0	String	The device ID of the device that cleared its statistics

A `DeviceError` has the following format:

```
{
  "device_id" : String,
  "reason" : String,
  "message" : String
}
```

Table 39: Netstat Clear Device Error Object Members

Member Name	Since	Type	Description
<code>device_id</code>	3.0	String	The device ID of the device to which the error belongs.
<code>reason</code>	3.0	String	Specific error code. For example "DEVICE_TYPE".
<code>message</code>	3.0	String	Message with details of an error. For example, "Statistics is not supported on this device."

4.12.3.3 Errors

`ILLEGAL_ARGUMENT` Returned if target argument is not a valid device ID or group name.

The request may also fail for individual devices for the following reasons:

`DEVICE_TYPE` Returned when the device does not support network statistics.

4.12.3.4 Examples

```
POST /api/device/all HTTP/1.1
```

```
...
```

```
{
  "op" : "netstat",
  "option" : "clear"
}
```

```
HTTP/1.1 201 Created
```

```
Location: /api/request/17
```

```
{
  "status" : "PROCESSING",
  "request_id" : 17,
  "result" : null,
  "error" : null
}
```

```
GET /api/request/17 HTTP/1.1
```

```
HTTP/1.1 200 OK
```

```
...
```

```
{
  "status" : "SUCCESS",
  "request_id" : 17,
  "result" : {
    "clear_statistics" : [
      {
        "device_id" : "d8803962b3c3"
      },
      {
        "device_id" : "d8803962e132"
      },
      {
        "device_id" : "d880396317ef"
      }
    ],
    "error" : [
      {
        "device_id" : "001ec0f03668",
        "reason" : "DEVICE_TYPE",
        "message" : "Statistics is not supported on this device"
      },
      {
        "device_id" : "001ec0f03cc1",
        "reason" : "DEVICE_TYPE",

```

```

        "message" : "Statistics is not supported on this device"
    },
    {
        "device_id" : "d88039e7fa53",
        "reason" : "DEVICE_TYPE",
        "message" : "Statistics is not supported on this device"
    }
]
},
"error" : null
}

```

4.12.4 Command `netstat select`

The `netstat select` command allows the selection of bandwidth channels for bandwidth reports made as a response to a `netstat read` command.

```

POST /api/device/{target}

{
    "op" : "netstat",
    "option" : "select",
    "select_list" : Array of Strings
}

```

4.12.4.1 Arguments

The `target` argument must be either the device ID of a single transmitter or the name of a device group.

The allowed device groups are the following:

ALL	All devices
ALL_TX	All transmitter devices

The `select_list` argument is mandatory. It can be a comma-separated list of selected bandwidth channels, ALL for all channels or NONE for no channels.

Table 40: Supported Bandwidth Channel Selections

Selection Name	Output
All	All bandwidth registers are selected. Must be alone.
None	All bandwidth registers are deselected. Must be alone.
Total	Total bandwidth measurement.
cpu_control	CPU bandwidth measurement.
aux_channel	First HDMI A&V description and audio measurement.
hdmi_stream_0	First HDMI video channel measurement.
hdmi_stream_1	Second HDMI video channel measurement.
stereo_audio	Stereo audio channel measurement.
hdmi_audio	Bandwidth HDMI audio channel measurement.
multi_channel_audio	Multichannel audio channel measurement.
Thumbnail	Thumbnail channel measurement.
usb_extender	USB Extender (Semtech UEX or Icron) channel measurement.
network_1g	1G network channel measurement.

4.12.4.2 Return Value

For each device targeted by this command, there is a `select_statistics` entry in the result if requested bandwidth channels were selected successfully or a device error object if bandwidth selection is not supported.

```
{
  "select_statistics" : [DeviceResponse, ...],
  "error" : [DeviceError, ...]
}
```

Table 41: Netstat Select Device Object Members

Member Name	Since	Type	Description
DeviceResponse	3.0	Array of DeviceResponse	Array of objects containing a <code>device_id</code> related to the selected bandwidth channels.
DeviceError	3.0	Array of DeviceError	Array of DeviceError objects indicating devices for which command failed and reason for the failure.

A DeviceResponse has the following format:

```
{
  "device_id" : String
}
```

Table 42: Netstat Select Device Response Object Member

Member Name	Since	Type	Description
device_id	3.0	String	Device ID of device that accepted and applied the bandwidth selection.

A DeviceError has the following format:

```
{
  "device_id" : String,
  "reason" : String,
  "message" : String
}
```

Table 43: Netstat Select Device Error Object Members

Member Name	Since	Type	Description
device_id	3.0	String	Device ID of the device to which the error belongs
reason	3.0	String	Specific error code. For example "DEVICE_TYPE"
message	3.0	String	Error message, such as "Bandwidth selection not supported on this device".

4.12.4.3 Errors

ILLEGAL_ARGUMENT Returned if target argument is not a valid device ID or group name.
ILLEGAL_ARGUMENT Returned if the comma separated select list is not valid.

The request may also fail for individual devices for the following reasons:

DEVICE_TYPE Returned when the device does not support network statistics.
DEVICE_TYPE Returned when the device does not support bandwidth selection.

4.12.4.4 Examples

```
POST /api/device/all HTTP/1.1
...
{
  "op" : "netstat",
  "option" : "select",
  "select_list" : ["hdmi_stream_0","hdmi_stream_1","network_lg"]
}
HTTP/1.1 201 Created
Location: /api/request/15
{
  "status" : "PROCESSING",
  "request_id" : 15,
  "result" : null,
  "error" : null
}
GET /api/request/15 HTTP/1.1
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : 15,
  "result" : {
    "select_statistics" : [
      {
        "device_id" : "d8803962e132"
      },
      {
        "device_id" : "d8803962b3c3"
      }
    ],
    "error" : [
      {
        "device_id" : "001ec0f03668",
        "reason" : "DEVICE_TYPE",
        "message" : "Statistics is not supported on this device"
      },
      {
        "device_id" : "001ec0f03cc1",
        "reason" : "DEVICE_TYPE",
        "message" : "Statistics is not supported on this device"
      },
      {
        "device_id" : "d880396317ef",
        "reason" : "DEVICE_TYPE",
        "message" : "Bandwidth selection not supported on this device"
      }
    ]
  }
},
```

```

    {
        "device_id" : "d88039e7fa53",
        "reason" : "DEVICE_TYPE",
        "message" : "Statistics is not supported on this device"
    },
    {
        "device_id" : "d88039e7a4ee",
        "reason" : "DEVICE_TYPE",
        "message" : "Bandwidth selection is not supported on this device"
    }
]
},
"error" : null
}

```

4.13 Command pair/unpair

The `pair` and `unpair` commands are used to manage pairings between a USB Extender (Semtech UEX or Icron) Local Extender (LEX) and a USB Extender (Semtech UEX or Icron) Remote Extenders (REX). When the Simultaneous User Interaction (SUI) feature is enabled for the Icron modules up to seven REX can be paired to a single LEX.

4.13.1 Command pair

Note: The SDVoE Developers API Application Note Library (PDS-062491) section “Managing Semtech UEX or Icron USB 2.0 Extender” provides details and use cases for the Simultaneous User Interaction (SUI) feature.

```

POST /api/device/{target}
{
    "op": "usb_extender:pair",
    "stream_index": index (0 or 1), (optional, 0 if omitted)
    "dest_device": "device_id",
    "subscription_index": index (0 or 1), (optional, 0 if omitted)
}

```

4.13.1.1 Arguments

The *target* is the associated device ID of the endpoint device (BlueRiver if attached to the UEX or Icron, UEX if UEX2000 is used standalone) whose Semtech UEX or Icron extender is to be paired.

- When the *target* is associated to a local extender (LEX) and SUI is enabled on the Icron extender, then multiple pairings are allowed. Reminder, each pairing request requires a separate command.
- When the *target* is the endpoint device associated to a remote extender (REX) then only a single pairing is allowed.
- Regarding the *dest_device* argument, when pairing extenders the device ID is the device (BlueRiver if attached, UEX if standalone) of either a local extender (LEX) or remote extender (REX) depending on the extender type of the target specified.

4.13.1.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a

description of the return value.

4.13.1.3 Errors

If a pairing request is sent and the endpoint device has previously reached its maximum allotted pairings, for example seven pairs for Icron, an error is generated. Below is an error example for Icron:

ICRON_EXTENDER: Icron device does not support more than 7 simultaneous peer(s). Consider unpairing from previous peer(s).

TIP:

- i. If the error “Device d88039e83991: (INVALID_STATE) Icron extender is not reachable” is received add a two-second delay between pairing commands.
- ii. The error message received will then be, “Device d88039e83991: (INVALID_STATE) The icron extender did not acknowledge the 'pairing' process”, indicating the pairing request was unsuccessful.

4.13.1.4 Example

The following is an example illustrating a pair request, where the local extender (LEX) is device ID d88039e83991 is being paired to the remote extender (REX) 0016c04c46d7:

```
POST /api/device/d88039e83991
{
  "op": "usb_extender:pair",
  "stream_index": 0,
  "dest_device": "0016c04c46d7",
  "subscription_index": 0,
}
```

4.13.2 Command pair_mac

Note: The pair_mac command is only meant for advanced users that specifically want to pair a UEX to an Icron that is not managed by the API. To use with care.

```
POST /api/device/{target}
{
  "op": "usb_extender:pair_mac",
  "stream_index": index (0 or 1), (optional, 0 if omitted)
  "dest_device": "mac_address"
}
```

4.13.3 Command unpair

The API unpair command is used to unpair a remote extender (REX) from a local extender (LEX).

```
POST /api/device/{target}
{
  "op": "usb_extender:unpair",
  "stream_index": index (0 or 1), (optional, 0 if omitted)
  "dest_device": "device_id", (optional)
  "subscription_index": index (0 or 1) (optional, 0 if omitted)
}
```

4.13.3.1 Arguments

The `target` is the associated device ID of the endpoint device (BlueRiver if attached, UEX if standalone) whose Semtech UEX or Icron extender is to be unpaired.

- When the `target` specified is associated to a local extender (LEX) and SUI is enabled on the Icron extender, then multiple unpairings are allowed.
- If the `target` specified is associated to a remote extender (REX) then only single unpairing is completed.

The `dest_device` argument is the device ID of the endpoint device (BlueRiver if attached, UEX if standalone) that the local extender (LEX) or remote extender (REX) is to be unpaired from.

- When the `target` specified is associated to a local extender (LEX) and the `dest_device` is a remote extender (REX) the current pairing of the specified pair is removed.
- When the `target` specified is associated to a local extender (LEX) and the `dest_device` is not specified then all current existing pairings to remote extenders (REX) are removed.
- When the `target` specified is associated to a remote extender (REX) then the `dest_device` must be of the paired local extender (LEX) or omitted from the command and in both cases only this pairing is removed.

4.13.3.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.13.3.3 Errors

If a pairing request is sent and the endpoint device has previously reached its maximum allotted pairings, e.g. seven pairs for Icron, an error is generated. Below is an example for Icron:

```
ICRON EXTENDER: Icron device does not support more than 7 simultaneous peer(s). Consider unpairing from previous peer(s).
```

Table 44: Supported Pairings for Semtech UEX and Icron Extenders

Extender Type	Simultaneous User Interaction (SUI)	Number of Supported Pairings
Remote (REX)	Enabled	1
Remote (REX)	Disabled	1
Local (LEX)	Enabled	7 for Icron, 1 for Semtech UEX
Local (LEX)	Disabled	1

Warnings!

- When a `factory reset` is issued to a device, all current pairings are removed for the device(s) as it is reset to default settings.
- A `stop` request with the stream type set to `USB_EXTENDER` will unpair all current Semtech UEX or Icron pairings.

Reminder: If a `switch` request is used to establish a pairing between a local extender (LEX) and remote extender (REX), if a pairing currently exists it is removed and the new pairing established.

4.13.3.4 Example

The following example is an `unpair` request to remove the pairing of a Semtech UEX or Icron extender currently paired, were the local extender (LEX) is device ID `d88039e83991` and the remote extender (REX)

0016c04c46d7:

```
POST /api/device/d88039e83991
{
  "op": "usb_extender:unpair",
  "dest_device": "0016c04c46d7",
}
```

4.14 Command reboot

```
POST /api/device/{target}
{
  "op" : "reboot"
}
```

Reminder: Causes target device(s) (BlueRiver and/or UEX) to restart. Device is unavailable for a short time while restart completes.

4.14.1 Arguments

The target argument must be either the device ID of a single device (BlueRiver or UEX if UEX2000 is used standalone) or the name of a device group.

- If a UEX (UEX1000 or UEX2000) is attached to the BlueRiver AVP, the target shall be the AVP device. In this case, the reboot command to the AVP will also have the effect of rebooting its attached UEX device(s). Setting UEX as the target is not recommended when attached to an AVP for this command.
- If a UEX is used standalone (UEX2000 only), the target shall be the UEX device.

The allowed device groups are:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

4.14.2 Return Value

For each device targeted by the reboot command, there is either an entry in the reboot array if the data successfully sent or a device error object in the error array if the operation failed.

```
{
  "reboot" : [RebootObject, ...],
  "error" : [DeviceError, ...]
}
```

Table 45: Reboot Object Array Members

Member Name	Since	Type	Description
reboot	3.0	Array of RebootObject	An array of objects that represent the devices for which the request succeeded.
error	3.0	Array of DeviceError	An array of DeviceError objects indicating the devices for which the command failed and the reason why it failed

The `RebootObject` object has the following structure:

```
{
  "device_id" : String
}
```

Table 46: Reboot Object Member

Member Name	Since	Type	Description
device_id	3.0	String	Device ID

Refer to the Introduction to the SDVoE API User Guide for details on the device error object (`DeviceError`).

Comment: Prior to API 2.3, the return value for this command was `null`.

4.14.3 Errors

`ILLEGAL_ARGUMENT` If the target argument is not a valid device ID or group name.

4.14.4 Example

```
POST /api/device/001ec0f03668
...
{
  "op" : "reboot"
}
HTTP/1.0 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "reboot" : [
      {
        "device_id" : "001ec0f03668"
      }
    ],
    "error" : []
  },
  "error" : null
}
```

4.15 Command release

Refer to section 4.4 Commands `claim` & `release` for details on the release command.

4.16 Command request

GET /api/request/{request_id}

Query arguments:

reduced_settings_change=(on|off)

Notes:

- i. This function retrieves either:
 - a. The return value of a background command. That is a command that responded with `PROCESSING` status and a request ID;
 - b. Or, the data associated with an event.
- ii. Request numbers expire 30 seconds or more after they are created.

4.16.1 Arguments

The `request_id` is a request identification number included in either:

1. A `PROCESSING` command response.
2. Alternatively, the associated event object.

The `reduced_settings_change` query argument specifies whether reduced `SETTINGS_CHANGED` events are enabled (`on`) or disabled (`off`). This is only relevant for `SETTINGS_CHANGED` events and ignored otherwise. Refer to section 4.16.15 Requests Associated with Event `SETTINGS_CHANGED` for details.

4.16.2 Return Value

The return value is dependent on the request type. Refer to each sub-section below for details.

4.16.3 Errors

<code>ILLEGAL_ARGUMENT</code>	If specified request ID does not exist. This also includes calling the request command with an expired request number.
-------------------------------	--

4.16.4 Examples

For details, refer to examples provided in the following subsections.

4.16.5 Requests Associated with Background Commands

Reminder: The `REQUEST_COMPLETE` event indicates the completion of a background command.

Once a request completes, the return type is as documented for a `background` command.

If the request is called before the request completes, without first checking for `REQUEST_COMPLETE` event type, the request command responds with status `PROCESSING` and the same request ID as the one specified when calling the request.

4.16.5.1 Example

The examples below illustrate how the `request` command is used to confirm successful execution of a `stop` command that was sent to a specific device to stop the native HDMI video stream (stream 0).

```
POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "stop",
  "subscription_type" : "HDMI",
  "index" : 0,
  "free" : false
}
HTTP/1.1 201 Created
Location: /api/request/85521
...
{
  "status" : "PROCESSING",
  "request_id" : 85521,
  "result" : null,
  "error" : null
}
GET /api/event?lastseen=85528 HTTP/1.1
...
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "events" : [
      {
        "device_id" : null,
        "event_id" : 85529,
        "event_type" : "REQUEST_COMPLETE",
        "timestamp" : 1462902262,
        "request_id" : 85521
      }
    ]
  },
  "error" : null
}
GET /api/request/85521 HTTP/1.1
...
HTTP/1.1 200 OK
...
```

```

{
  "status" : "SUCCESS",
  "request_id" : 85521,
  "result" : {
    "devices" : [
      {
        "device_id" : "001ec0f03668",
        ...
      }
    ],
    "error" : []
  },
  "error" : null
}

```

4.16.6 Requests Associated with Event CEC_RECEIVED

Reminder: The CEC_RECEIVED event indicates that Consumer Electronics Control (CEC) data was received by a specific device.

```

{
  "cec" : [CECDataObject, ...]
}

```

Table 47: CEC Object Array Member

Member Name	Since	Type	Description
cec	3.0	Array of CECDataObject	Array of CEC data objects, described below

The CECDataObject includes two strings:

1. One string representing the device that the CEC data was received from.
2. In addition, a second that contains the CEC data itself.

```

{
  "device_id" : String,
  "cec_data" : String
}

```

Table 48: CEC Object Members

Member Name	Since	Type	Description
device_id	3.0	String	Device ID of the device from which the data was received
cec_data	3.0	String	Hexadecimal string that represents the received CEC data.

4.16.6.1 Example

```

event 85750
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "events" : [

```

```

        {
            "device_id" : "001ec0f04d9c",
            "event_id" : 85752,
            "event_type" : "CEC_RECEIVED",
            "timestamp" : 1462903915,
            "request_id" : 85744
        }
    ],
    "error" : null
}
request 85744
{
    "status" : "SUCCESS",
    "request_id" : 85744,
    "result" : {
        "cec" : [
            {
                "device_id" : "001ec0f04d9c",
                "cec_data" : "4f36"
            }
        ],
        "error" : []
    },
    "error" : null
}

```

4.16.7 Requests Associated with Event **DEVICE_CONNECTED**

Reminder: The **DEVICE_CONNECTED** event indicates that a new device (BlueRiver or UEX) has joined the network.

Basic device information is returned. The format is identical to the output of the “get hello” command called on the device associated with the **DEVICE_CONNECTED** event.

4.16.8 Requests Associated with Event **DEVICE_DISCONNECTED**

Reminder: The **DEVICE_DISCONNECTED** event indicates that a device (BlueRiver or UEX) has left the network.

Basic device information is returned. The format is identical to the output of the “get hello” command called on the device associated with the **DEVICE_DISCONNECTED** event.

4.16.9 Requests Associated with Event **DEVICE_INFO_AVAILABLE**

Reminder: The **DEVICE_INFO_AVAILABLE** event indicates that device information is available for a newly discovered device (BlueRiver or UEX).

Device information of a newly connected device is returned. The format is identical to the output of the “get device” command called on a device associated with the **DEVICE_INFO_AVAILABLE** event.

4.16.10 Requests Associated with Event FIRMWARE_UPDATE_PROGRESS

Reminder: The FIRMWARE_UPDATE_PROGRESS event indicates that a firmware update is in progress (BlueRiver or UEX).

Information associated with this event can be used to report progress information to the client software, for example display a progress bar.

4.16.10.1 Return Value

```
{
  "update_progress" : [UpdateProgressObject, ...]
}
```

Table 49: Update Progress Object Array Member

Member Name	Since	Type	Description
update_progress	3.0	Array of UpdateProgressObject	Array of firmware update progress objects, described below.

The UpdateProgressObject includes the following:

```
{
  "device_id" : String,
  "progress" : Integer,
  "total" : Integer
}
```

Table 50: Update Progress Object Members

Member Name	Since	Type	Description
device_id	3.0	String	Device ID of the device from which the data was received
progress	3.0	Integer	Number of progress units accomplished so far. These units arbitrary and subject to change: they are only meaningful relative to value of total member.
total	3.0	Integer	The number of progress units for the whole operation

4.16.10.2 Example

```
event 22128
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "events" : [
      {
        "device_id" : "001ec0f04d9c",
        "event_id" : 22129,
        "event_type" : "FIRMWARE_UPDATE_PROGRESS",
        "timestamp" : 1462471715,
        "request_id" : 22150
      }
    ]
  },
  "error" : null
}
```

```

}
request 22150
{
  "status" : "SUCCESS",
  "request_id" : 22150,
  "result" : {
    "update_progress" : [
      {
        "device_id" : "001ec0f04d9c",
        "progress" : 3603552,
        "total" : 8643204
      }
    ],
    "error" : []
  },
  "error" : null
}

```

4.16.11 Requests Associated with Event I2C

Reminder: The I2C event indicates that an I2C bus master, such as an external microcontroller, has generated an event through the I2C interface.

4.16.11.1 Return Value

```

{
  "i2c_event" : [I2CEventObject, ...]
}

```

Table 51: I2C Data Object Array Member

Member Name	Since	Type	Description
i2c_event	3.0	Array of I2CEventObject	Array of I2C event objects, described below.

The I2CDataObject includes two strings:

1. One string that represents the device that the I2C data was received from.
2. The second string contains the I2C data itself.

```

{
  "device_id" : String,
  "event_source" : Integer
}

```

Table 52: I2C Data Object Members

Member Name	Since	Type	Description
device_id	3.0	String	Device ID of the device from which the data was received.
event_source	3.0	Integer	Number of event source. This number specified by the I2C bus master generating the event. Event sources 0 to 7 currently supported. Recommended event source 0 be reserved for a feature that allows a device to be identified.

Member Name	Since	Type	Description
			For example, a user physically presses a button on a device that then triggers it to be highlighted in the client interface. The semantics of other event sources is product-specific. Recommended practices: • If product implements an “identify device” feature as described above, use event source 0 for that purpose. • If product does not implement an “identify device” feature, do not use event source 0. • Except for event source 0, software should not interpret I²C events unless it is for a recognized device type, basing that decision on the devices vendor ID and product ID.

4.16.11.2 Example

```
event 1233
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "events" : [
      {
        "device_id" : "001ec0f03668",
        "event_id" : 1234,
        "event_type" : "I2C",
        "timestamp" : 1462903093,
        "request_id" : 1300
      }
    ]
  },
  "error" : null
}
```

```
request 1300
update progress object array
{
  "status" : "SUCCESS",
  "request_id" : 1300,
  "result" : {
    "i2c_event" : [
      {
        "device_id" : "001ec0f03668",
        "event_source" : 4
      }
    ],
    "error" : []
  },
  "error" : null
}
```


4.16.12 Requests Associated with Event INFRARED_RECEIVED

Reminder: The INFRARED_RECEIVED event indicates Infrared data has been received by a specific device.

4.16.12.1 Return Value

```
{
  "infrared" : [IRDataObject, ...]
}
```

Table 53: Infrared (IR) Object Array Member

Member Name	Since	Type	Description
Infrared	3.0	Array of IRDataObject	Array of infrared data objects, described below

The IRDataObject includes two strings:

1. One string represents the device that the infrared (IR) data was received from.
2. The other contains the IR data itself.

```
{
  "device_id" : String,
  "infrared_data" : String
}
```

Table 54: Infrared (IR) Object Members

Member Name	Since	Type	Description
device_id	3.0	String	Device ID of the device from which the data was received
infrared_data	3.0	String	Hexadecimal string which represent the received Pronto code.

4.16.12.2 Example

```
event 85609
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "events" : [
      {
        "device_id" : "001ec0f03668",
        "event_id" : 85611,
        "event_type" : "INFRARED_RECEIVED",
        "timestamp" : 1462903093,
        "request_id" : 85604
      }
    ]
  },
  "error" : null
}

request 85604
{
  "status" : "SUCCESS",
  "request_id" : 85604,
```

```

"result" : {
  "infrared" : [
    {
      "device_id" : "001ec0f03668",
      "infrared_data" :
      "0000006d0000002700ac00a90016000f0016000f001500100015001
      00014001100140037001500100014001100150010001500100015001
      00015001000150010001500100014001100150010001400a90015003
      60015003600140037001500100015003600150036001500100015003
      6001500100016000f001500100016000f00150010001500100015003
      60016000f001500360015003600150036001400370015017c"
    }
  ],
  "error" : []
},
"error" : null
}

```

4.16.13 Requests Associated with Event MULTIVIEW_CHANGED

4.16.13.1 Return Value

The `MVDataObject` includes a string that represents the device that the MV data was received from and details of the layout currently loaded. The tables provided below outline details of the data returned.

```

"devices" : [
  {
    "device_id" : String,
    "status" : {
      "active" : Boolean
    },
    "nodes" : [
      {
        "type" : "FRAME_BUFFER",
        "index" : 0,
        "configuration" : {
          "display_mode" : "MULTIVIEW",
          "multiview_layout" : MVLayoutObject
        }
      }
    ]
  }
]

```

Table 55: Multiview Layout Object Members

Member Name	Since	Type	Description
name	3.0	String	Layout name
width	3.0	Integer	Layout width
height	3.0	Integer	Layout height
read_only	3.0	Boolean	If layout is read-only
surfaces	3.0	Array of SurfaceObject	Layout surfaces
windows	3.0	Array of WindowObject	Layout windows

Table 56: Surface Object Members

Member Name	Since	Type	Description
index	3.0	Integer	Surface index
horizontal_position	3.0	Integer	Surface horizontal position
vertical_position	3.0	Integer	Surface vertical position
width	3.0	Integer	Surface width
height	3.0	Integer	Surface height

Table 57: Multiview Object Array Members

Member Name	Since	Type	Description
index	3.0	Integer	Window index
horizontal_position	3.0	Integer	Window horizontal position
vertical_position	3.0	Integer	Window vertical position
width	3.0	Integer	Window width
height	3.0	Integer	Window height
horizontal_offset	3.0	Integer	Window horizontal offset
vertical_offset	3.0	Integer	Window vertical offset
content	3.0	String	Type of content, example "VIDEO"
target_surface	3.0	Integer	Window target surface index

4.16.13.2 Example

Example of the event generated for a MULTIVIEW_CHANGED event.

```
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "events" : [
      {
        "device_id" : "d8deadbeef07",
        "event_id" : 69053,
        "event_type" : " MULTIVIEW_CHANGED",
        "timestamp" : 1638995235,
        "request_id" : 123}
    ]
  },
  "error" : null
}
```

An example of the return for the above MULTIVIEW_CHANGED event is available at the end of this document in the section Appendix: Non Get Command Output Examples.

4.16.14 Requests Associated with Event RS232_RECEIVED

4.16.14.1 Return Value

```
{
  "rs232" : [RS232DataObject, ...]
}
```

Table 58: RS-232 Object Array Member

Member Name	Since	Type	Description
rs232	3.0	Array of RS232DataObject	Array of RS-232 data objects, described below

The RS232DataObject includes two strings:

1. One string represents the device that the RS-232 data was received from.
2. The other contains the RS-232 data itself.

```
{
  "device_id" : String,
  "rs232_data" : String,
  "rs232_data_hex" : String
  "stream_idx" : Integer
}
```

Table 59: RS-232 Object Members

Member Name	Since	Type	Description
device_id	3.0	String	Device ID of the device from which the data was received
rs232_data	3.0	String	ASCII string representation of RS-232 data. The binary data is escaped using the following rules: \b replaces the ASCII backspace character (hex. 08) \f replaces the ASCII form feed (hex. 0c) \n replaces the ASCII line feed (hex. 0a) \r replaces the ASCII carriage return (hex. 0d) \t replaces the ASCII tab character (hex. 09) \\ replaces a single backslash (\) character \xnn, where nn is the hexadecimal value (two hexadecimal digits) of the character, replaces: • All ASCII control characters (hex. 1f and below) not already handled above; • The ASCII DEL character (hex. 7f); and • All non-ASCII characters (hex. 80 and above). This is in addition to JSON escaping rules. Relevant JSON rules are as follows: \" replaces a double quote \\ replaces a single backslash (\) character A JSON parser will handle JSON escaping rules, leaving only the first set of rules to be handled.
rs232_data_hex	3.6	String	HEX string representation of RS-232 data.
stream_idx	3.0	Integer	RS-232 port number that the data was sent from. This is also the RS-232 stream/subscription index.

4.16.14.2 Example

```
event 85750
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "events" : [
      {
        "device_id" : "001ec0f04d9c",
        "event_id" : 85752,
        "event_type" : "RS232_RECEIVED",
        "timestamp" : 1462903915,
        "request_id" : 85744
      }
    ]
  },
  "error" : null
}

request 85744
{
  "status" : "SUCCESS",
  "request_id" : 85744,
  "result" : {
    "rs232" : [
      {
        "device_id" : "001ec0f04d9c",
        "rs232_data" : "Hello",
        "rs232_data_hex" : "48656c6c66"
        "stream_idx" : 0
      }
    ]
  },
  "error" : []
},
"error" : null
}
```

4.16.15 Requests Associated with Event `SETTINGS_CHANGED`

4.16.15.1 Return Value

The return value depends on whether reduced `SETTINGS_CHANGED` events are enabled or disabled. If reduced `SETTINGS_CHANGED` events are disabled, the new settings are returned with a format identical to the return value of the `get settings` command. Refer to section 4.7 Command `get` for details.

If reduced `SETTINGS_CHANGED` events are enabled, the format is the same but only the specified subset(s) of nodes, streams and/or subscriptions are included, thereby reducing the amount of data to be parsed. The subset definitions are in the table below.

TIP: Refer to the Introduction to the SDVoE API User Guide for more information on event types, tracking device states and event buffering.

Table 60: Settings Subsets

Subset Name	Since	Description
AUDIO	3.0	Includes the following stream and subscription types: STEREO_AUDIO MULTICH_AUDIO Includes the following node types: STEREO_AUDIO_INPUT_OUTPUT STEREO_AUDIO_INPUT STEREO_AUDIO_OUTPUT MULTICH_AUDIO_INPUT MULTICH_AUDIO_OUTPUT
CONTROL_DATA	3.0	RS-232, infrared and CEC configuration and status. Includes the following stream and subscription types: CEC INFRARED RS232 Includes the following node types: CEC INFRARED_DECODER INFRARED_ENCODER UART
GENERAL	3.0	SETTINGS_CHANGED event raised for this subset if one or more of the following change: Members of the devices configuration member: Device name Locate mode Secure A/V Members of the devices status member: Point-to-Point Update in progress Error code A SETTINGS_CHANGED event always includes above data regardless whether it has changed or not.
HDMI_AUDIO	3.0	HDMI audio. Includes the following stream types: HDMI HDMI_AUDIO Includes the following subscription types: HDMI_AUDIO A SETTINGS_CHANGED event is raised for this subset if one of more of the following changes: • The <code>remove_audio</code> member of the HDMI stream. • The <code>audio_details</code> and <code>audio_details_extended</code> are members of the status member of HDMI_ENCODER.

Subset Name	Since	Description
NETWORK	3.0	Network configuration and status, notably IP address and mode (DHCP versus Static IP address). Includes the following node types: NETWORK_INTERFACE NETWORK_PORT NETWORK_SWITCH
USB	3.0	USB configuration. Includes the following stream and subscription types: USB_HID USB_EXTENDER since API 3.8 Includes the following node types: USB_HID USB_EXTENDER since API 3.8
VIDEO	3.0	Base video configuration and status. Includes the following stream and subscription type, HDMI. Includes the following node types: FRAME_BUFFER FRAME_RATE_CONVERTER HDMI_DECODER HDMI_ENCODER HDMI_MONITOR SCALER VIDEO_COMPRESSOR VIDEO_DECOMPRESSOR VIDEO_INPUT A SETTINGS_CHANGED event is not raised if only elements that changed are the ones covered by HDMI_AUDIO or VIDEO_DETAILS.
VIDEO_DETAILS	3.0	Video format details. Includes the following nodes: HDMI_DECODER HDMI_ENCODER SETTINGS_CHANGES event raised only when specified parts of following nodes change: • The video_details member of the status member of HDMI_DECODER. • The video_details member of the status member of HDMI_ENCODER.
VIDEO_MISC	3.0	Includes the following stream types: THUMBNAIL Includes the following node types: BITMAP_OVERLAY COLOR_GENERATOR THUMBNAIL_GENERATOR

4.17 Command send

The `send` command has a set of sub-commands used to send data to one or more devices. There are three types of data that can be sent, Consumer Electronics Control (CEC), Infrared (IR) and RS-232.

4.17.1 Command send cec

```
POST /api/device/{target}
{
  "op" : "send:cec",
  "port_index" : Integer (optional),
  "data_hex" : String
}
```

Reminder: CEC option used to send Consumer Electronics Control (CEC) data to one or more devices.

4.17.1.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group.

The allowed device groups are:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

Notes:

- For transceiver (TR) devices, the `port_index` is set to 0 for an input and 1 for an output.
- When the device is a transmitter or receiver, the `port_index` is set to 0.
- When the device is a transceiver configured in either transmitter or receiver device mode, the `port_index` is also set to 0.
- If port index is not specified, the default value is 0.

The `data_hex` argument is a hexadecimal string that represents the CEC data.

4.17.1.2 Return Value

For each device targeted by the `send cec` command, there is either an entry in the `send_cec` array when the data sent successfully or a device error object in the `error` array if the operation failed.

```
{
  "send_cec" : [SendCECObject, ...],
  "error" : [DeviceError, ...]
}
```

Table 61: CEC Object Array Members

Member Name	Since	Type	Description
send_cec	3.0	Array of SendInfraredObject	Aarray of objects representing devices for which request succeeded.
error	3.0	Array of DeviceError	An array of DeviceError objects indicating devices for which the command failed and the reason for failure.

The `SendCECObject` object has the following structure:

```
{
```



```

        "device_id" : String
    }

```

Table 62: CEC Object Member

Member Name	Since	Type	Description
device_id	3.0	String	Device ID

Refer to the Introduction to the SDVoE API User Guide for details on the device error object (`DeviceError`).

4.17.1.3 Errors

`ILLEGAL_ARGUMENT` Returned if target argument is not a valid device ID or group name.
`ILLEGAL_ARGUMENT` Also returned if the data is not in the format described above.

The request may also fail for individual devices for the following reasons:

`DEVICE_TYPE` Returned if the device does not support CEC.

4.17.1.4 Example

```

POST /api/device/001ec0f03668 HTTP/1.1
...
{
    "op" : "send:cec",
    "data_hex" : "0f36"
}
HTTP/1.1 200 OK
...
{
    "status" : "SUCCESS",
    "request_id" : null,
    "result" : {
        "send_cec" : [
            {
                "device_id" : "001ec0f03668"
            }
        ],
        "error" : []
    },
    "error" : null
}

```

4.17.2 Command send infrared

```

POST /api/device/{target}
{
    "op" : "send:infrared",
    "data_hex" : String
}

```

Reminder: Send infrared remote-control data to one or more devices.

4.17.2.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group. The allowed device groups are:

<code>ALL</code>	All devices
<code>ALL_TX</code>	All transmitter devices
<code>ALL_RX</code>	All receiver devices

The `data_hex` argument is a hexadecimal string representing the Pronto infrared code to be sent. The length of this string must be a multiple of four bytes (example, eight hexadecimal characters) and at most 1032 bytes (example, 2064 hexadecimal characters or 256 Pronto burst pairs).

Note: Depending on firmware version, some devices support a maximum code length of 512 bytes. Refer to `data_length_max` member in section 6.8.16.2 Status to determine if this applies.

4.17.2.2 Return Value

For each device targeted by this command, there is either an entry in the `send_infrared` array if the data sent successfully or a device error object in the `error` array if the operation failed.

```
{
  "send_infrared" : [SendInfraredObject, ...],
  "error" : [DeviceError, ...]
}
```

Table 63: Infrared (IR) Object Array Members

Member Name	Since	Type	Description
<code>send_infrared</code>	3.0	Array of SendInfraredObject	Array of objects representing devices for which the request succeeded.
<code>error</code>	3.0	Array of DeviceError	Array of DeviceError objects indicating devices the command failed for and reason why it failed

The `SendInfraredObject` object has the following structure:

```
{
  "device_id" : String
}
```

Table 64: Infrared (IR) Object Member

Member Name	Since	Type	Description
<code>device_id</code>	3.0	String	Device ID

Refer to the Introduction to the SDVoE API User Guide for details on the device error object (`DeviceError`).

Note: Prior to API 2.2, the return value for this command was `null`.

4.17.2.3 Errors

<code>ILLEGAL_ARGUMENT</code>	Returned if <code>target</code> argument is not a valid device ID or group name.
<code>ILLEGAL_ARGUMENT</code>	Also returned if the data is not in the format described above.

The request may also fail for individual devices for the following reasons:

<code>DEVICE_TYPE</code>	Returned when a device does not have an infrared output.
--------------------------	--

4.17.2.4 Example

```
POST /api/device/010203040506 HTTP/1.1
...
{
  "op" : "send:infrared",
  "data_hex" :
    "0000006d00220002015700ab00160015001600150016003f001600150016001
    50016001500160015001600150016003f0016003f001600150016003f0016003
    f0016003f0016003f0016003f0016003f0016003f00160015001600150016001
    500160015001600150016001500160015001600150016003f0016003f0016003
    f0016003f0016003f0016003f001605f30156005500160e4d"
}
HTTP/1.1 200 OK
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "send_infrared" : [
      {
        "device_id" : "001ec0f03668"
      }
    ],
    "error" : []
  },
  "error" : null
}
```

4.17.3 Command send rs232

```
POST /api/device/{target}
{
  "op" : "send:rs232",
  "port_index" : Integer (optional),
  "data_type" : "hex" (optional)
  "data_string" : String
}
```

Reminder: Send RS-232 data to one or more devices.

Note: Starting in API 3.6 HEX data is supported. The default data type is ASCII. If data to be sent as HEX the optional `data_type` argument must be specified.

4.17.3.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group. The allowed device groups are:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

The optional `port_index` argument is the RS-232 port number. If omitted, the default value for port is 0. Not all endpoint devices support multiple RS-232 ports.

The `data_string` argument is by default a printable ASCII text string.

Notes:

- i. This string must be escaped if it contains ASCII control characters or non-ASCII byte values.
- ii. Space characters must also be escaped.
- iii. These escaping rules are in addition to the JSON escape requirements. JSON escape sequences are typically generated automatically by the JSON Serializer and are transparent to the application.

The following escape sequences are recognized:

<code>\0</code>	null character
<code>\n</code>	line feed
<code>\r</code>	carriage return
<code>\t</code>	horizontal tab
<code>\xnn</code>	(where <i>nn</i> are two hexadecimal characters) hexadecimal representation of byte
<code>\\</code>	single backslash
<code>\ (space)</code>	space character

Reminder: If it is desired to send HEX data then the optional `data_type` : "hex" argument is required to be included in the API request. This argument is optional either `ascii` or `hex` can be specified, the default is "ascii".

4.17.3.2 Return Value

For each device targeted by the `send rs232` command, there is either an entry in the `send_rs232` array if the data was sent successfully or a device error object in the `error` array if the operation failed.

```
{
  "send_rs232" : [SendRs232Object, ...],
  "error" : [DeviceError, ...]
}
```

Table 65: Send rs232 Object Array Members

Member Name	Since	Type	Description
<code>send_rs232</code>	3.0	Array of SendRS232Object	Array of objects representing devices for which the request succeeded.
<code>error</code>	3.0	Array of DeviceError	Array of objects indicating devices that returned an error and the reason for the failure

The `SendRs232Object` object has the following structure:

```
{
  "device_id" : String
}
```

Table 66: Send rs232 Object Member

Member Name	Since	Type	Description
<code>device_id</code>	3.0	String	Device ID

Refer to the Introduction to the SDVoE API User Guide for details on the device error object (`DeviceError`).

4.17.3.3 Errors

ILLEGAL_ARGUMENT	Returned when target argument is not valid device ID or group name.
ILLEGAL_ARGUMENT	Also returned if the data is not in the format described above.
ILLEGAL_ARGUMENT	If the data_type specified is "hex" but the data_string is not in this format error returned.

The request may also fail for individual devices for the following reasons:

DEVICE_TYPE	Returned when device does not have the RS-232 port number specified.
-------------	--

4.17.3.4 Example

Example of a send rs232 request for port 2 using default data_type of ascii.

```
POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "send:rs232",
  "port_index" : 2,
  "data_string" : "Hello\\ World!\\r\\n"
}
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "send_rs232" : [
      {
        "device_id" : "001ec0f03668"
      }
    ],
    "error" : []
  },
  "error" : null
}
```

Example of a send rs232 request for port 2 using the data_type argument set to hex.

```
POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "send:rs232",
  "port_index" : 2,
  "data_type" : hex
  "data_string" : "9041ff9051ff"
}
```

Reminder: If the port_index is not specified the default port used is 0. If the data_type is not specified the default is ASCII.

4.18 Command `send_wol`

```
POST /api/device/{target}

{
  "op" : "send_wol",
  "wol_mac_address" : String,
  "wol_password" : String (optional)
}
```

Reminder: This command is used to send a Wake-on-Lan packet to the network via an AVP device.

4.18.1 Arguments

The `target` argument must be the device ID of a single device.

The `wol_mac_address` argument is a 6 bytes hexadecimal string representing the mac address of the machine to wake up. The leftmost byte is transmitted first in the WoL packet.

The `wol_password` argument is a 4 or 6 bytes hexadecimal string. The leftmost byte is transmitted first in the WoL packet. This argument is optional.

4.18.2 Return Value

For the device targeted by the `send_wol` command, there is either an entry in the `send_wol` array when the packet is sent successfully or a device error object in the `error` array if the operation failed.

```
{
  "send_wol" : [SendWoLObject],
  "error" : [DeviceError]
}
```

Table 67: WoL Object Array Members

Member Name	Since	Type	Description
<code>send_wol</code>	3.7	Array of <code>SendWoLObject</code>	Array of objects representing devices for which request succeeded (in this cas there is only 1 device).
<code>error</code>	3.0	Array of <code>DeviceError</code>	An array of <code>DeviceError</code> objects indicating devices for which the command failed and the reason for failure (in this cas there is only 1 device).

The `SendWoLObject` object has the following structure:

```
{
  "device_id" : String,
  "device_name" : String,
}
```

Table 68: WoL Object Members

Member Name	Since	Type	Description
<code>device_id</code>	3.0	String	Device ID
<code>device_name</code>	3.5	String	Device name

Refer to the Introduction to the SDVoE API User Guide for details on the device error object (`DeviceError`).

4.18.3 Errors

ILLEGAL_ARGUMENT	Returned if target argument is groupe name ALL, ALL_TX or ALL_RX.
ILLEGAL_ARGUMENT	Returned if target argument is not a valid device ID.
ILLEGAL_ARGUMENT	Returned if the wol_mac_address is not in the format described above.
ILLEGAL_ARGUMENT	Returned if the wol_password is not in the format described above.

The request may also fail for individual devices for the following reasons:

DEVICE_TYPE	Returned if the device does not support sending Wake-on-Lan packets.
-------------	--

4.18.4 Example

```
POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "send_wol",
  "wol_mac_address" : "010203040506",
  "wol_password" : "0708090A0B0C"
}
HTTP/1.1 201 Created
Location: /api/request/16
{
  "status" : "PROCESSING",
  "request_id" : 16,
  "result" : null,
  "error" : null
}
GET /api/request/16 HTTP/1.1
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "send_wol" : [
      {
        "device_id" : "001ec0f03668",
        "device_name" : "my-AVP-03668",
      }
    ],
    "error" : []
  },
  "error" : null
}
```

4.19 Command set

The `set` command provides a series of sub-commands to individually manage multiple options. This section reviews each sub-command including their purpose, command syntax, arguments and examples.

4.19.1 Command set `dot1x_certificates`

```
POST /api/device/{target}
{
  "op" : "set:dot1x_certificates",
  "password": String, (optional),
  "file_names": String
}
```

Reminder: API command to store the machine certificate with its private key, as well as intermediate certificates on the SDVoE endpoint device.

- The complete certificate chain must be uploaded in a single API request.
- The machine and intermediate certificates are to be X509 certificates in "PEM" format (base-64).
- The machine private key needs also to be in "PEM" format (base-64). Optionally, this file can be password protected (space character not supported).
- When API request sent, existing key and certificate chains are overwritten. This is required for certificate renewal, for example.
- 802.1x support added in API version 3.6.0.1.

4.19.1.1 Arguments

The `target` argument must be either the device ID of a single transmitter or the name of a device group. The allowed device groups are:

<code>ALL</code>	All devices
<code>ALL_RX</code>	All receiver devices
<code>ALL_TX</code>	All transmitter devices

The `password` argument is specified only required when the key file is encrypted.

The `file_names` argument specifies the list of certificate file names, starts with the `key file`, followed by the `leaf certificate` and then the `intermediate certificates`, in this order. A comma (,) is used to separate each `file_name` in the string.

Note: Using the `https` protocol is recommended, but `http` is also possible.

4.19.1.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.19.1.3 Errors

ILLEGAL_ARGUMENT	Returned when target argument is not a valid device ID or group name.
IO_ERROR	Occurs when one of the the listed files cannot be read.
DOT1X	Returned when one of the following conditions occur: • If the specified password is invalid. • If one of the files are not in pem format. • If the key is not ecdsa secp256r1. • If the certificate chain exceeds the allowed 4k device-limit • If one of the file is not in PEM format. • If the key is not ECDSA secp256r1. • If the certificate chain exceeds the allowed 4K device-limit.

4.19.1.4 Example

```
POST /api/device/010203040506

{
  "op" : "set:dot1x_certificates",
  "password": "semtech",
  "file_names": "client.key, client.pem, intermediate.pem, root.pem"
}

HTTP/1.1 201 Created
Location: /api/request/57
...
{
  "status": "SUCCESS",
  "request_id": id,
  "result": {
    "dot1x_certificates": [
      {
        "device_id": "010203040506",
        "device_name": "010203040506"
      }
    ],
    "error": []
  },
  "error": null
}
```

4.19.2 Command set edid

```
POST /api/device/{target}
{
  "op" : "set:edid",
  "data_hex" : String
}
```

Reminder: set edid option used to set the EDID of one or more transmitter devices.

4.19.2.1 Arguments

The `target` argument must be either the device ID of a single transmitter or the name of a device group.

The allowed device groups are:

ALL	All devices
ALL_TX	All transmitter devices

The `data` argument is a hexadecimal string representing the binary data to be set. It must be 512 hexadecimal characters (example, length must be 256 bytes). Since data is representing an EDID, the string must start with "00fffffffffffff00", this is, bytes 00 ff ff ff ff ff ff 00, which represent the EDID header.

4.19.2.2 Return Value

The return value is the same as if the "get edid" command is called on the same target right after new settings are applied. Refer to section 4.7 Command get for details.

4.19.2.3 Errors

ILLEGAL_ARGUMENT	Returned when target argument is not a valid device ID or group name.
ILLEGAL_ARGUMENT	In addition, returned if the data argument is not the proper length.
ILLEGAL_ARGUMENT	Also, returned if data argument does not begin with valid EDID header.

4.19.2.4 Example

```
POST /api/device/001ec0f03668 HTTP/1.1
{
  "op" : "set:edid",
  "data_hex" :
  "00fffffffffffff00410c2fc0c52d00000c140103802f1a782e3585a656489a241
  250542f6f00714f8180818a9500950fb3000101d1c0023a801871382d40582c450
  0dc0c1100001e00000ff004155353130313230313137313700000fd00384c1e5
  315000a20202020202000000fc005068696c69707320323231450a01b602031ef
  04b100501020304061213141f230907078301000065030c001100023a801871382
  d40582c4500dc0c1100001e8c0ad08a20e02d10103e9600dc0c11000018011d007
  251d01e206e285500dc0c1100001e8c0ad090204031200c405500dc0c110000180
  00000000000000000000000000000000000000000000000000000000000000ac"
}
HTTP/1.1 201 Created
Location: /api/request/57
...
{
```

```

    "status" : "PROCESSING",
    "request_id" : 57,
    "result" : null,
    "error" : null
}

```

4.19.3 Command set gpio

```

POST /api/device/{target}

{
  "op" : "set:gpio",
  "index" : Integer,
  "mode" : String,
  "value" : Integer (conditional)
}

```

Reminder: set gpio is used to set the direction and output value of a general-purpose input/output (GPIO) pin of one or more devices.

4.19.3.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group.

The allowed device groups are:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

The `index` argument specifies the index of the GPIO pin, must match the index of a GPIO node on the device.

The `mode` argument specifies the direction of the GPIO pin:

INPUT	Set GPIO pin as an input.
OUTPUT	Set GPIO pin as an output.

The `value` argument specifies the output value of the GPIO pin:

- It can be 0 or 1.
- Must be present if the GPIO pin is set as an output. That is the `mode` argument is `OUTPUT`.
- Must not be present if the GPIO pin is set as an input. That is the `mode` argument is `INPUT`.

4.19.3.2 Return Value

For each device targeted by the `set gpio` command, there is either an entry in the GPIO array if the operation was performed successfully on the device or a device error object in the error array if the operation failed for some reason.

```

{
  "gpio" : [GpioResultObject, ...],
  "error" : [DeviceError, ...]
}

```

4.19.3.3 Errors

ILLEGAL_ARGUMENT	Occurs when the arguments entered are invalid.
------------------	--

4.19.3.4 Example

```
POST /api/device/d88039631d77 HTTP/1.1
...
{
  "op" : "set:gpio",
  "index" : 7,
  "mode" : "INPUT"
}
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "gpio" : [
      {
        "device_id" : "d88039631d77"
      }
    ],
    "error" : []
  },
  "error" : null
}
POST /api/device/d88039631d77 HTTP/1.1
...
{
  "op" : "set:gpio",
  "index" : 7,
  "mode" : "OUTPUT",
  "value" : 1
}
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "gpio" : [
      {
        "device_id" : "d88039631d77"
      }
    ],
    "error" : []
  },
  "error" : null
}
```

4.19.4 Command `set ip`

For DHCP IP mode:

```
POST /api/device/{target}
{
  "op" : "set:ip:dhcp"
}
```

For AUTO_IP mode:

```
POST /api/device/{target}
{
  "op" : "set:ip:auto_ip"
}
```

For STATIC (MANUAL) IP mode:

```
POST /api/device/{target}
{
  "op" : "set:ip:static",
  "address" : String (conditional),
  "usb_extender_address" : String (optional),
  "mask" : String (conditional),
  "gateway" : String (conditional)
}
```

Reminder: `set ip` is used to set the Internet Protocol (IP) configuration of the primary network interface of a target device, as well as the Semtech UEX or Icron network interface (when appropriate).

4.19.4.1 Arguments

The `target` argument must be the device ID of a single device.

- BlueRiver device as the target supports the configuration of both the BlueRiver chipset's network interface node and if attached, the Semtech UEX or Icron device(s) network interface node.
- UEX device as the target, only allowed for UEX2000 used standalone, supports the configuration of the UEX network interface node.

In an API request the mode is implicit from the `op` argument:

DHCP	<code>set:ip:dhcp</code> Sets the device to DHCP/Auto-IP mode. If DHCP server is not present, then the Auto IP mechanism is used to assign an address.
AUTO_IP	<code>set:ip:auto_ip</code> This is the Force Auto-IP mode, when selected Auto IP mechanism implements Automatic IP assignment, whether a DHCP server is present or not. This is not the recommended mode, should be implemented with caution. Not applied to the Semtech UEX nor Icron IC, if present. Note, also only supported for BlueRiver ASIC devices, not available for NT1000/NT2000.
MANUAL	<code>set:ip:static</code> Configure the IP settings manually.

If the mode is set to `MANUAL`, the `address`, `mask` and `gateway` arguments must be specified.

- The `address` argument is the IP address of the BlueRiver chipset if the target is a BlueRiver device, otherwise is the IP address of the UEX chipset if the target is a UEX2000 standalone device.
- For UEX or Icron attached to AVP, the UEX or Icron IP mode will follow the one of the AVP; if required to specify a manual IP address for the UEX or Icron specify it with the field `usb_extender0_address` argument (or older argument `usb_extender_address` and with the argument `usb_extender1_address` if there is a second UEX attached to the AVP).
- The `mask` argument is the subnet mask.
- The `gateway` argument is the IP address of the default network gateway.

Regarding the IP address arguments, if only the `usb_extender0/1_address` address is specified, since the BlueRiver address is required an error is returned indicating the missing data. If both AVP and USB extender fields are specified but the `USB_EXTENDER` node(s) show the `chip_present` as false, the `usb_extender0/1_address` is ignored.

If mode is set to either `DHCP` or `AUTO_IP` these arguments are not specified.

4.19.4.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.19.4.3 Errors

<code>ILLEGAL_ARGUMENT</code>	Occurs if the specified target argument not valid device ID or group name.
<code>ILLEGAL_ARGUMENT</code>	Occurs if the specified target argument does not support <code>Auto_IP</code> mode and it was specified as the IP mode.

4.19.4.4 Examples

Example of setting a device to DHCP mode:

```
POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "set:ip:dhcp"
}
HTTP/1.1 201 Created
Location: /api/request/58
...
{
  "status" : "PROCESSING",
  "request_id" : 58,
  "result" : null,
  "error" : null
}
```

Example of setting device manually, specifying a static IP address and associated network settings:

```
POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "set:ip:static",
```

```

        "address" : "10.1.1.5",
        "usb_extender0_address" : "10.1.1.6",
        "usb_extender1_address" : "10.1.1.7",
        "mask" : "255.255.255.0",
        "gateway" : "10.1.1.1"
    }

HTTP/1.1 201 Created
Location: /api/request/59
...
{
    "status" : "PROCESSING",
    "request_id" : 59,
    "result" : null,
    "error" : null
}

```

4.19.5 Command set overlay

```

POST /api/device/{target}
{
    "op" : "set:overlay:on",
    "horiz_position" : Integer (conditional),
    "vert_position" : Integer (conditional),
    "scaling_factor" : Integer (optional),
    "fg_color" : String (conditional),
    "bg_color" : String (conditional),
    "fg_transparency" : Integer (conditional),
    "bg_transparency " : Integer (conditional),
    "bmp_hex" : String (optional),
    "invert" : Boolean (conditional)
}

POST /api/device/{target}
...
{
    "op" : "set:overlay:off"
}

```

Two forms of the command are supported with the first being used to enable and configure the bitmap overlay of a device and the second form disables the overlay. A bitmap overlay is supported on the scaler video stream of a transmitter device, such as HDMI stream 1 and on the HDMI output of a receiver device.

TIP: For details on the use and configuration of the Bitmap Overlay feature, refer to the API Developers Application Note Library, specifically the section “Bitmap Overlay” section.

Important to note that this command is only applicable to BlueRiver NT2000 endpoint devices and that include the AV Processor.

Notes:

- i. Refer to the following sources for information about the BMP file format:

Bitmaps: <https://docs.microsoft.com/en-ca/windows/desktop/gdi/bitmaps>

BMP file format: https://en.wikipedia.org/wiki/BMP_file_format

- ii. If the overlay bitmap is generated “on-the-fly”, such as by a text generator, instead of loading from an actual BMP file, the `BITMAPCOREHEADER` variant of the file format is recommended, as it is the simplest to implement.

4.19.5.1 Arguments:

The `target` argument must be either the device ID of a single device or the name of a device group. The allowed device groups are the following:

<code>ALL</code>	All devices
<code>ALL_RX</code>	All receiver devices
<code>ALL_TX</code>	All transmitter devices

WARNING! Many of the following arguments are optional and the corresponding parameters are left unchanged if omitted. This allows disabling and re-enabling the overlay without having to re-specify the full configuration.

However, a complete configuration must be set by specifying all arguments the first time the overlay is enabled on a given device. The API does not report an error if this requirement is not followed. Important to note, failing to initially apply a complete configuration will lead to undefined behaviour.

The `horiz_position` and `vert_position` arguments respectively specify the horizontal and vertical position of the overlay, in pixels. This is the position of the top-left corner of the overlay relative to the top-left corner of the picture. If either of these arguments are specified, both must be. If these parameters are omitted, then they are left unchanged from the previous call of this command on the target device.

The `scaling_factor` argument specifies the factor by which the overlay bitmap is to be scaled before being overlaid. This argument can be 1 (no scaling), 2 or 4. Scaling is performed by simple pixel replication. If omitted, the scaling factor remains unchanged.

The `fg_color` and `bg_color` arguments respectively specify the foreground and background colors. Each must be a string with format `RRGGBB` where each of `RR`, `GG` and `BB` is a pair of hexadecimal digits, respectively representing the 8-bit value of the red, green and blue components of the specified colour. If either are specified, then both must be. If omitted, these parameters are left unchanged.

The `fg_transparency` and `bg_transparency` arguments respectively specify the level of transparency of the foreground and background of the overlay. Each is an integer between 0 (completely opaque) and 100 (completely transparent). If either is specified, then both must be. If omitted, these parameters are unchanged.

The `bmp_hex` argument is a monochrome, that is a one-bit per pixel, bitmap file in BMP format encoded as a hexadecimal string. The size of the overlay bitmap is implicit from this argument. This argument is optional. If not specified the bitmap remains unchanged, which is useful for disabling/re-enabling or repositioning the overlay without reloading the bitmap.

If the the `invert` Boolean argument is set to `true`, the colors are inverted when interpreting the source bitmap file such that white becomes the background and black becomes the foreground. Otherwise, the default is applied, that is black or the darkest colour is the background and white or lightest colour is the foreground.

Note: In an API request, the `invert` argument is specified when the `bmp_hex` argument is and is not specified if the `bmp_hex` is not..

The picture size and the overlay position (`horiz_position` and `vert_position` arguments), bitmap size

and scaling factor (`scaling_factor` argument) must be such that the overlay is at least partially within the picture. In this context, the picture size refers to either:

- The output resolution for a receiver device; or
- The scaler resolution (HDMI stream 1) for a transmitter device.

4.19.5.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.19.5.3 Errors

The command may fail for the following reasons:

ILLEGAL_ARGUMENT	Occurs when any of the argument values entered are invalid.
ILLEGAL_ARGUMENT	Occurs when the <code>bmp_hex</code> argument does not represent a valid monochrome BMP bitmap file.

The command may also fail for individual devices for the following reasons:

DEVICE_TYPE	Occurs if the device does not support a bitmap overlay.
DEVICE_TYPE	Occurs if the device is a transmitter and does not have a scaler video stream.
INVALID_STATE	Occurs if the device is a transmitter and its scaler resolution has not been set.
INVALID_STATE	Occurs if the overlay is positioned outside the picture.

4.19.5.4 Examples

```
POST /api/device/d80102030405 HTTP/1.1
...
{
  "op" : "set:overlay:on",
  "horiz_position" : 320,
  "vert_position" : 200,
  "scaling_factor" : 2,
  "fg_color" : "00695B",
  "bg_color" : "000000",
  "fg_transparency" : 0),
  "bg_transparency" : 70,
  "bmp_hex" : "424d62160000000000000820000...ffffffff000000",
  "invert" : false
}

HTTP/1.1 201 Created
Location: /api/request/123
...
{
  "status" : "PROCESSING",
  "request_id" : 123,
  "result" : null,
  "error" : null
}
```

```
POST /api/device/d80102030405 HTTP/1.1
```

```
...
```

```
{
  "op" : "set:overlay:on"
}
```

```
HTTP/1.1 201 Created
```

```
Location: /api/request/124
```

```
...
```

```
{
  "status" : "PROCESSING",
  "request_id" : 124,
  "result" : null,
  "error" : null
}
```

```
POST /api/device/d80102030405 HTTP/1.1
```

```
...
```

```
{
  "op" : "set:overlay:off"
}
```

```
HTTP/1.1 201 Created
```

```
Location: /api/request/125
```

```
...
```

```
{
  "status" : "PROCESSING",
  "request_id" : 125,
  "result" : null,
  "error" : null
}
```

4.19.6 Command set property

```
POST /api/device/{target}
```

```
{
  "op" : "set:property",
  "key" : "String",
  "value" : Varies
}
```

Reminder: The `set property` option is used to set a configuration property for one or more devices (BlueRiver or UEX).

The table below lists the available configurable device, stream and nodes properties, as well as supplies a link to the section containing details about the `key` and `value` settings.

Table 69: List of Configurable Properties

Property	Section
Device Configuration	7.1 Device Configuration
Stream Configuration	7.2 Stream Configuration
Configurable Nodes	
Stereo Audio Input/Output	7.3 Stereo Audio Input/Output Node Configuration
Stereo Audio Output	7.4 Stereo Audio Output Node Configuration
CEC	7.5 CEC Node Configuration
Color Generator	7.6 Color Generator Node Configuration
Frame Buffer	7.7 Frame Buffer Node Configuration
HDMI Audio	7.8 HDMI Audio Node Configuration
HDMI Decoder	7.9 HDMI Decoder Node Configuration
HDMI Encoder	7.10 HDMI Encoder Node Configuration
Multichannel Input	7.11 Multichannel Audio Input Node Configuration
Multichannel Output	7.12 Multichannel Audio Output Node Configuration
Network Port	7.13 Network Port Node Configuration
Network Switch	7.14 Network Switch Node Configuration
Scaler	7.15 Scaler Node Configuration
UART	7.16 UART Node Configuration
USB Extender	7.17 USB Extender Node Configuration
Video Compressor	7.18 Video Compressor Node Configuration
Video Decompressor	7.19 Video Decompressor Node Configuration

For more information on configuration properties and a list of configurable device properties, refer to section 7 Configurable Device Properties.

4.19.6.1 Arguments

The `target` argument varies depending on the specific property being configured, but it is always either the device ID of a single device (BlueRiver or UEX if UEX2000 is used standalone) or the name of one of the following device groups:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

Refer to the specific property of interest for more details on the supported targets for the configurable property; links are included in the table above.

The `key` argument identifies a configurable property of a device and the `value` argument is a value of a type appropriate for that property.

The `key` argument is a path referring to an object member inside the device object, starting at the root of the device object. Refer to section 6 Device Object Definition (`DeviceObject`) for more details on device objects.

There are some syntax rules that need to be considered for the `key` argument, they are as follows:

- The `key` starts with the name of one of the members of the device object.
- Object members are accessed by appending a period (`.`) followed by the member name.
- Array elements are accessed by appending an array selector between square brackets (`[` and `]`).

- An array selector is a name and an index separated by a colon. Example, [name:3].
 - The array must be an array of objects, that is each element is an object.
 - The matched element is one with a member named "type" or "name" (checked in that order) matching the provided name and member named "index" that matches the provided index.

4.19.6.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.19.6.3 Errors

`ILLEGAL_ARGUMENT` If target argument entered is not a valid device ID or group name.

The request may also fail for individual devices for the following reasons:

`DEVICE_TYPE` Returned if the property being set or the chosen value for that property is not supported by the device.

4.19.6.4 Example

```
POST /api/device/001ec0f04d9c HTTP/1.1
...
{
  "op" : "set:property",
  "key" :
    "nodes[STEREO_AUDIO_OUTPUT:0].inputs[main:0].configuration.source.value",
  "value" : 3
}
HTTP/1.1 201 Created
Location: /api/request/60
...
{
  "status" : "PROCESSING",
  "request_id" : 60,
  "result" : null,
  "error" : null
}
```

TIP: In regards to quotes, when the value is not a boolean or number, it must be double-quoted.

4.19.7 Command set scaler

```
POST /api/device/{target}
{
  "op" : "set:scaler",
  "width" : Integer,
  "height" : Integer
}
```

Reminders:

- i. Used to set the scaler video resolution of one or more transmitter devices.
- ii. This command was introduced in API version 3.0.
- iii. To use this command, the scaler resolution must be set before starting a scaled video stream.

4.19.7.1 Arguments

The `target` argument must be either the device ID of a single transmitter device or the name of a device group.

The allowed device groups are:

ALL	All devices
ALL_TX	All transmitter devices

The `width` and `height` arguments respectively specify the width and height of the scaled video, in pixels.

4.19.7.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.19.7.3 Errors

ILLEGAL_ARGUMENT	Occurs when the arguments entered are invalid.
------------------	--

4.19.7.4 Example

```
POST /api/device/010203040506
{
  "op" : "set:scaler",
  "width" : 1920,
  "height" : 1080
}
HTTP/1.1 201 Created
Location: /api/request/42
...
{
  "status" : "PROCESSING",
  "request_id" : 42,
  "result" : null,
  "error" : null
}
```

4.19.8 Command set `secure_av`

```
POST /api/device/{target}
{
  "op" : "set:secure_av",
  "default" : Boolean,
  "key" : String (conditional)
}
```

Reminder: The `secure av key` command is used to set a secure key for the specified device.

Note: The value entered for the secure av key is applied when the AES128 encryption is applied to the HDMI video and HDMI audio streams during transmission.

4.19.8.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group.

The allowed device groups are:

ALL	All devices
-----	-------------

If `default` argument `true`, the key reverts to its default. In this case, a `key` argument must not be specified.

If `default` argument is `false`, then `key` argument is required.

The `key` argument specifies a user defined key, it must be a string of exactly eight (8) non-control ASCII characters (ASCII code range 32 to 126 inclusive).

4.19.8.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.19.8.3 Errors

ILLEGAL_ARGUMENT	Occurs if <code>target</code> argument entered is not valid device ID or group.
------------------	---

4.19.8.4 Example

The example shown sets a manual key that in turn is used to when the HDMI AV streams are encrypted for transmission between endpoint devices.

```
POST /api/device/d88039e7fa53 HTTP/1.1
...
{
  "op" : "set:secure_av",
  "default" : false,
  "key" : "4A4nVbQK"
}
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "secure_av" : [
      {
        "device_id" : "d88039e7fa53"
      }
    ],
    "error" : []
  },
  "error" : null
}
```

```
POST /api/device/d88039e7fa53 HTTP/1.1
```

```
...
```

```
{
  "op" : "set:secure_av",
  "default" : true
}
```

```
HTTP/1.1 200 OK
```

```
...
```

```
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "secure_av" : [
      {
        "device_id" : "d88039e7fa53"
      }
    ],
    "error" : []
  },
  "error" : null
}
```

4.19.9 Command set security_key

```
POST /api/device/{target}
```

```
{
  "op": "set:security_key",
  "module": "usb",
  "key_type": "DEFAULT"
}
```

```
POST /api/device/{target}
```

```
{
  "op": "set:security_key",
  "module": "usb",
  "key_type": "CUSTOM",
  "key_phrase": "<custom encryption phrase key>"
}
```

Reminder: The `security_key` command is used to configure the encryption key for USB HID IP communication for the specified device(s). This API command is only applicable to endpoint devices based on the BlueRiver ASIC series, not applicable to NT1000/NT2000 devices.

4.19.9.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group.

The `target` argument varies depending on the specific property being configured, but it is always either the device ID of a single device or the name of one of the following device groups:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

If `DEFAULT` argument `default`, encryption is activated using the default key. If this argument is `custom`, then encryption is activated using custom (user defined) key.

The `key_type` argument sets the source of the encryption to be implemented by the endpoint device, it can be either `DEFAULT` or `CUSTOM`, were:

DEFAULT	USB HID IP communication encryption uses default encryption key.
CUSTOM	USB HID IP communication encryption uses custom (user provided) encryption key. This command must be followed by the custom key phrase (24 to 64 characters), no space and no special characters. Using a random string is a good choice.

Refer to the SDVoE Control Server Administrators Guide for details on the API server configuration.

4.19.9.2 Return Value

The return value is the same as if the “`get settings`” command had been called on the same target right after the new settings were applied. Refer to section 4.7 Command `get` for details.

4.19.9.3 Errors

ILLEGAL_ARGUMENT	Occurs if the target argument entered is not a valid device ID (including if it is a group name).
------------------	---

The request may fail for individual devices for the following reasons:

DEVICE_TYPE	Occurs if the device does not support USB extension.
INVALID_STATE	Occurs if the communication is not secured between the API and the device
CONFIGURATION	Occurs if default key requested, but this key is not set in API configuration file. Refer to SDVoE Control Server Administrators Guide for details.

4.19.9.4 Example

```
POST /api/device/d88039e7fa53
{
  "op": "set:security_key",
  "module": "usb",
  "key_type": "DEFAULT"
}
HTTP/1.1 200 OK

...
{
  "status": "PROCESSING",
  "request_id": 465,
  "result": null,
```



```

    "error": null
  }

POST /api/device/d88039e7fa53
{
  "op": "set:security_key",
  "module": "usb",
  "key_type": "CUSTOM",
  "key_phrase": "7fwdXVqxYypkkvpN8HCF63Nd"
}
{
  "status": "PROCESSING",
  "request_id": 466,
  "result": null,
  "error": null
}

```

4.19.10 Command set thumbnail

```

POST /api/device/{target}
{
  "op" : "set:thumbnail",
  "fps" : Real or String,
  "ssrc" : Integer,
  "udp" : Integer (optional)
}

```

Configure the thumbnail generator of a **BlueRiver NT1000 or NT2000 endpoint device**. This is necessary before the THUMBNAIL stream can be started.

Reminder: Available only to NT1000/NT2000 endpoint devices, not applicable to the BlueRiver ASIC series.

IMPORTANT CUSTOMER NOTICE

It is your responsibility to comply with all laws, rules, and regulations applicable to the operation of this product, including the processing and distribution of electronic content and, if you use Semtech's thumbnail preview feature, the display of such content in a thumbnail format viewable on a monitor.

You are responsible for entering into, performing, and obtaining any necessary authorizations with content providers whose electronic content you opt to process, distribute or display. All interaction between you and third parties providing electronic content or other third-party products or services is between you and the third party. Semtech assumes no liability for any of your activities involving your use of this product for the processing, distribution or display of third-party electronic content. IN NO EVENT SHALL SEMTECH BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES IN CONNECTION WITH OR ARISING OUT OF THE USE OF THIS PRODUCT REGARDLESS OF WHETHER SEMTECH WAS ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Rev. 042019

4.19.10.1 Arguments

The **target** argument must be the device ID of a single transmitter device or the following device group name:

ALL_TX for all transmitter devices

The `fps` argument specifies the maximum frame rate of the thumbnail stream in frames per second.

The `ssrc` argument is a 32-bit integer (range 0..4294967295) that represents the RTP synchronization source (SSRC) identifier. This value must be unique within an RTP session, refer to the RFC-3550, section 8 for details.

The `port` argument is the destination UDP port number. If this argument is omitted, the default port number 6972 is used.

4.19.10.2 Return Value

The return value is the same as if the “`get settings`” command had been called on the same target right after the new settings were applied. Refer to section 4.7 Command `get` for details.

4.19.10.3 Errors

The command may fail for the following reasons:

ILLEGAL_ARGUMENT if any argument is invalid (type, range, etc.).

The command may also fail for individual devices for the following reasons:

DEVICE_TYPE if the device does not support a thumbnail stream.

4.19.10.4 Examples

```
POST /api/device/d88001020405 HTTP/1.1
```

```
...
```

```
{
  "op" : "set:thumbnail",
  "fps" : 0.5,
  "ssrc" : 12236,
  "udp" : 20000
}
```

```
HTTP/1.1 201 Created
Location: /api/request/13133
```

```
...
```

```
{
  "status" : "PROCESSING",
  "request_id" : 13133,
  "result" : null,
  "error" : null
}
```

```
POST /api/device/d88001020405 HTTP/1.1
```

```
...
```

```
{
  "op" : "set:thumbnail",
  "fps" : "60m",
  "ssrc" : 12236
}
```

```
HTTP/1.1 201 Created
```

Location: /api/request/13134

...

```
{
  "status" : "PROCESSING",
  "request_id" : 13134,
  "result" : null,
  "error" : null
}
```

4.19.11 Command set usb

POST /api/device/{target}

```
{
  "op" : "set:usb",
  "role" : String (optional)
  "security" : {
    "level" : "security_level_value" (optional)
  }
}
```

Reminders:

- i. set usb is used to configure a device(s) to serve as USB HID extender(s) and stop any active USB_HID streams.
- ii. Both arguments, role and security_level are optional, possible to set one, the other or both
- iii. Applicable to BlueRiver ASIC series.
- iv. Only supported on a device if the device JSON object for that device reports a USB_HID node.

A set usb request is also used to configure the security settings regarding USB HID device IP communication. If this security option is implemented it must be applied to appropriate endpoints, that is both the LOCAL and REMOTE USB HID extenders.

Note: To enable USB HID encryption, this feature must have been enabled in the product configuration at the time of production. Contact the hardware manufacturer to confirm if option selected in the hardware configuration settings. For details on managing security of an SDVoE system, refer to the SDVoE API Application Note Library.

4.19.11.1 Arguments

The target argument must be either the device ID of a single device or the name of a device group.

The allowed device groups are:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

The role argument sets role of device when acting as a USB HID extender, can be one of the following:

DISABLED	USB HID support is disabled.
OTG	The device automatically detects the type of the connected USB HID device and set its role to LOCAL or REMOTE accordingly.
LOCAL	The device acts as USB device that connects to a USB host (e.g. a PC).
REMOTE	The device acts as USB host to which USB HID devices connect (keyboard, mouse, etc.)

The `security_level_value` argument is used to enable or disable the USB HID security of the device when acting as a USB extender. It can be one of the following:

DISABLED	USB HID IP communication encryption is disabled.
ENABLED	USB HID IP communication encryption for all HID devices (keyboard, mouse, etc.).

4.19.11.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.19.11.3 Errors

The request may fail for individual devices for the following reason:

DEVICE_TYPE	If the device does not support USB HID extension.
CONFIGURATION	USB HID Default Encryption key cannot be set by this API server because of current configuration - must set 'usb_hid_encryption_default_key' configuration file option.
INVALID_OPERATION	Communication between API and the Device is not secured. See 'auth' command details for more information. USB HID security configurations are transmitted from the API server to an endpoint device is encrypted, it is required that the API server and BlueRiver ASIC devices be bound-authenticated to successfully transfer the security key. If authentication hasn't been enabled then the error message outlined above is received. Refer to section 4.3 Commands <code>auth</code> & <code>unauth</code> for details on implementing bound-authentication, as well as the SDVoE Control Server Administrators Guide.

4.19.11.4 Examples

```
POST /api/device/010203040506 HTTP/1.1
...
{
  "op" : "set:usb",
  "role" : "DISABLED"
}
HTTP/1.1 201 Created
Location: /api/request/123
...
{
  "status" : "PROCESSING",
  "request_id" : 123,
  "result" : null,
  "error" : null
}

POST /api/device/010203040506 HTTP/1.1
...
```

```
{
  "op" : "set:usb",
  "role" : "LOCAL"
}
HTTP/1.1 201 Created
Location: /api/request/124
...
{
  "status" : "PROCESSING",
  "request_id" : 124,
  "result" : null,
  "error" : null
}

POST /api/device/010203040506 HTTP/1.1
...
{
  "op" : "set:usb",
  "role" : "REMOTE"
}
HTTP/1.1 201 Created
Location: /api/request/125
...
{
  "status" : "PROCESSING",
  "request_id" : 125,
  "result" : null,
  "error" : null
}
```

To set the USB HID to autodetect if it should act as a LOCAL or REMOTE extender, enable the On-the-Go (OTG) option:

```
POST /api/device/010203040506 HTTP/1.1
...
{
  "op" : "set:usb",
  "role" : "OTG"
}
HTTP/1.1 201 Created
Location: /api/request/126
...
{
  "status" : "PROCESSING",
  "request_id" : 126,
  "result" : null,

```

```
    "error" : null
  }
```

The following examples, show the syntax for API requests enabling and disabling USB HID encryption, as well an example setting the role and encryption through a single API request.

Reminder: It is required that this option be enabled by the hardware manufacturer in the product configuration. Contact the manufacturer for more information.

```
POST /api/device/010203040506
{
  "op" : "set:usb",
  "security" :
    "level" : "ENABLED"
}
```

```
POST /api/device/010203040506
{
  "op" : "set:usb",
  "security" :
    "level" : "DISABLED"
}
```

Request below sets the extender role to REMOTE and enables USB HID security encryption for a specific device.

```
POST /api/device/010203040506
{
  "op" : "set:usb",
  "role" : REMOTE
  "security" : {
    "level" : "ENABLED"
  }
}
```

4.19.12 Command set video

```
POST /api/device/{target}
{
  "op" : "set:video",
  "display_mode" : String,
  "video" : VideoFormatArgument (conditional),
}
```

Reminders:

- i. The `set video` command is used to manage the display mode and video output arguments of one or more receiver devices.
- ii. The `set video` command was introduced in the API version 3.0.
- iii. For details on setting a device to the `multiview` video mode and applying a specific multiview layout, refer to section 5 Multiview Command Reference.

4.19.12.1 Arguments

The `target` argument must be either the device ID of a single receiver device or the name of a device group.

The allowed device groups are:

<code>ALL</code>	All devices
<code>ALL_RX</code>	All receiver devices

The `display_mode` argument specifies the display mode. Refer to section 2.5.2 Video Display Mode Argument for more information.

The `video` argument specifies the video format. Refer to section 2.5.3 Video Format Arguments, as well as the Introduction to the SDVoE API User Guide for more information on video arguments. As mentioned, no video arguments are needed or permitted for the genlock display mode.

Note: For details on managing multiview layouts, refer to section 5.2.9 Command `set multiview`.

TIP: For the `wall` processing mode, if the `wall_name` is being set using a TCI command it must be last argument specified in the API request. Refer to the Wall section of the SDVoE Developers API Application Note Library for details on creating and managing Video Walls.

4.19.12.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.19.12.3 Errors

<code>ILLEGAL_ARGUMENT</code>	Occurs when the arguments entered are invalid
-------------------------------	---

4.19.12.4 Examples

This first example places the receiver in `genlock` display mode:

```
POST /api/device/010203040506 HTTP/1.1
...
{
  "op" : "set:video",
  "display_mode" : "GENLOCKED"
}
HTTP/1.1 201 Created
Location: /api/request/62
...
{
  "status" : "PROCESSING",
  "request_id" : 62,
  "result" : null,
  "error" : null
}
```

This example applies the supported video format 1080p60 and frame rate in fast switch full screen display mode:

```
POST /api/device/010203040506 HTTP/1.1
...
{
  "op" : "set:video",
  "display_mode" : "FAST_SWITCHED",
  "video" : {
    "width" : 1920,
    "height" : 1080,
    "fps" : 60
  }
}
HTTP/1.1 201 Created
Location: /api/request/63
...
{
  "status" : "PROCESSING",
  "request_id" : 63,
  "result" : null,
  "error" : null
}
```

Slightly different example here, supported video format 1080p60 but is specified by the Video Identification Code (VIC) in fast switch full screen display mode:

```
POST /api/device/010203040506 HTTP/1.1
...
{
  "op" : "set:video",
  "display_mode" : "FAST_SWITCHED",
  "video" : {
    "vic" : 16
  }
}
HTTP/1.1 201 Created
Location: /api/request/64
...
{
  "status" : "PROCESSING",
  "request_id" : 64,
  "result" : null,
  "error" : null
}
```

In this example, the supported video format applied is 3840x2160 (2160p) 59.94Hz (60Hz/1.001) in stretched fast switch full screen display mode:

```
POST /api/device/010203040506 HTTP/1.1
...
{
  "op" : "set:video",
  "display_mode" : "FAST_SWITCHED_STRETCH",
  "video" : {
    "width" : 3840,
    "height" : 2160,
    "fps" : "60m"
  }
}
HTTP/1.1 201 Created
Location: /api/request/65
...
{
  "status" : "PROCESSING",
  "request_id" : 65,
  "result" : null,
  "error" : null
}
```

This example, fully specifies an 800x600 video format in fast switch full screen display mode:

```
POST /api/device/010203040506 HTTP/1.1
...
{
  "op" : "set:video",
  "display_mode" : "FAST_SWITCHED",
  "video" : {
    "width" : 800,
    "height" : 600,
    "fps" : 60.316541,
    "total_height" : 1056,
    "total_height" : 628,
    "hsync_pulse" : 128,
    "vsync_pulse" : 4,
    "hsync_front_porch" : 40,
    "vsync_front_porch" : 1,
    "horiz_polarity" : "POSITIVE",
    "vert_polarity" : "POSITIVE"
  }
}
HTTP/1.1 201 Created
Location: /api/request/66
...
```

```
{
  "status" : "PROCESSING",
  "request_id" : 66,
  "result" : null,
  "error" : null
}
```

Genlock wall mode (synchronized wall display mode) with the a Source Independent Wall arguments for a 2x2 wall with aspect ratio set to KEEP and the bezel compensation to 24 pixels:

```
POST /api/device/010203040506 HTTP/1.1
```

```
...
```

```
{
  "op" : "set:video",
  "display_mode" : "WALL_GENLOCKED",
  "video" : {
    "width" : 1920,
    "height" : 1080,
    "fps" : 60,
    "aspect_ratio_mode" : KEEP,
    "grid_width" : 2,
    "grid_height" : 2,
    "grid_index_horiz" : 2,
    "grid_index_vert" : 2,
    "bezel_horiz" : 24,
    "bezel_vert" : 24
  }
}
```

```
HTTP/1.1 201 Created
```

```
Location: /api/request/67
```

```
...
```

```
{
  "status" : "PROCESSING",
  "request_id" : 67,
  "result" : null,
  "error" : null
}
```

This example simpler to above but the Fast Switch Wall mode is being applied with a 1080p 59.94Hz output video format:

```
POST /api/device/010203040506 HTTP/1.1
```

```
...
```

```
{
  "op" : "set:video",
  "display_mode" : "WALL_FAST_SWITCHED",
  "video" : {
    "width" : 1920,
    "height" : 1080,
    "fps" : 59.94,

```

```

        "aspect_ratio_mode" : KEEP,
        "grid_width" : 2,
        "grid_height" : 2,
        "grid_index_horiz" : 2,
        "grid_index_vert" : 2,
        "bezel_horiz" : 24,
        "bezel_vert" : 24
    }
}
HTTP/1.1 201 Created
Location: /api/request/68
...
{
    "status" : "PROCESSING",
    "request_id" : 68,
    "result" : null,
    "error" : null
}

```

4.20 Command `sfp read`

```

POST /api/device/{target}
{
    "op" : "sfp:read",
    "physical_address" : "String",
}

```

Reminder: The `sfp` command is used to read SFP+ optical module diagnostic information from one or more endpoint devices. This feature was added in API revision 3.6 and ASIC firmware 2.1.

4.20.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group.

The allowed device groups are:

ALL	All devices
ALL_RX	All receiver devices
ALL_TX	All transmitter devices

The `physical_address` to be read: A0h or A2h

4.20.2 Return Value

A `sfp` request returns 128 bytes of hexadecimal information. The return will include the following:

```

"sfp_read": [
    {
        "device_id": "target",
        "device_name": "target name",
        "physical_address": "A0|A2",
        "values": "000102030405...FDFF" (128 bytes, hexadecimal)
    }
]

```

```
],
```

4.20.3 Errors

ILLEGAL_ARGUMENT Occurs when the arguments entered are invalid.

4.20.4 Example

```
POST /api/device/001ec0f04d9c
{
  "op" : "sfp:read",
  "physical_address" : "A0",
}
```

4.21 Command start

```
POST /api/device/{target}
{
  "op" : "start",
  "stream_type" : String,
  "stream_index" : Integer,
  "dest_address" : String (optional),
  "hdcg_thumbnail" : Boolean (optional) (NT1000/NT2000 devices only),
}
```

Reminder: The `start` command is used to initiate/start one or more stream(s) on one or more devices.

4.21.1 Arguments

The `target` argument must be either the device ID of a single transmitter device or the name of a device group.

The allowed device groups are:

<code>ALL</code>	All devices
<code>ALL_TX</code>	All transmitter devices

Exception: If the `dest_address` argument is specified the `target` must then be the device ID of a single transmitter device.

Notes:

- The `stream_type` argument specified must be a valid stream type.
- This command applies only to transmitter devices.

The following stream types are supported:

<code>STEREO_AUDIO</code>	Start the Stereo audio stream, sometimes referred to as analog audio.
<code>HDMI</code>	Start an HDMI video stream.
<code>HDMI_AUDIO</code>	Start an HDMI audio streams.
<code>MULTICH_AUDIO</code>	Start a Multichannel audio streams.
<code>THUMBNAIL</code>	Use to start a video thumbnail stream.

For RS-232, Infrared and other non-AV stream types, use the `switch` command. For details refer to section 4.23 Command `switch`.

The `stream_index` argument must be the valid index of a stream of the specified type.

The `dest_address` argument, if present, specifies the destination IP address of the stream.

- Must be an available and unused multicast IP address within the allocation range configured in the configuration file of the API Server. The default range is to [224.1.1.1, 224.1.3.255]).

The following are invalid for the `dest_address` argument:

- The “all TX” multicast IP address (specified in the configuration file, defaults to 224.1.1.253).
- The legacy “all TX” multicast IP address 224.1.1.253 (which may or may not be identical to the above depending on API configuration file content).
- The “all RX” multicast IP address (specified in the configuration file, defaults to 224.1.1.254).
- The legacy “all RX” multicast IP address 224.1.1.254 (which might or might not be identical to the above depending on configuration file content).
- Multicast IP address 225.225.225.225.
- Any multicast address in the following ranges defined in IETF RFC 3171:
 - An address in the Local Network Control Block (224.0.0.0/24).
 - An address in the Internetwork Control Block (224.0.1.0/24).
- Any of the following special use IPv4 addresses as defined in IETF RFC 5735:
 - An address on the “This” network (0.0.0.0/8).
 - A loopback address (127.0.0.0/8).

TIP: For more information on managing the default multicast range refer to the Control Server (API) Administrators Guide.

If `dest_address` is not specified, the API Server uses either the address already assigned to each stream by a previous invocation of the `start` command that has not yet been released by the `stop` command, if applicable, or otherwise it allocates an unused multicast address. For a `THUMBNAIL` stream, destination can be either an available multicast address or a unicast IP address.

The `thumbnail` argument is applicable only to NT1000/NT2000 endpoint devices. It specifies whether thumbnail support is enabled for HDCP sources. If omitted, it defaults to `false`. This argument is only relevant if the `stream_type` is `THUMBNAIL`, otherwise it is ignored. Refer to section 4.19.10 Command `set thumbnail` and the SDVoE API Developers API Application Note Library for more information on this option.

Additional information:

When starting an `HDMI` stream 0, the `HDMI_AUDIO` stream 0 from the same transmitter is NOT implicitly started, the audio streams are managed independently from the video stream.

An exception to the above does however exist. If a receiver is operating in Genlock mode and there is HDMI audio present, it is carried automatically as part of the HDMI video. If a different audio source is desired then separate join and start requests are required. For information on default audio behavior, refer to the Introduction to the SDVoE API User Guide.

Reminders:

- i. When the client directly manages the allocation of multicast IP addresses using the `dest_address` argument, the multicast IP address of the `HDMI` and `HDMI_AUDIO` streams are specified independently

even if decoder (RX) is operating in Genlock mode.

- ii. Some streams have preconditions that must be fulfilled before they can be successfully started. Otherwise, the `start` command fails with error `INVALID_STATE` for the affected devices.
 - Before the scaler video stream (HDMI stream 1) can be started, the video scaler must be configured with the `set_scaler` command. Refer to section 4.19.7 Command `set_scaler` for more information on managing the video scaler.
 - Before a `THUMBNAIL` stream can be started, the thumbnail generator be configured with the `set_thumbnail` command, refer to section 4.19.10 Command `set_thumbnail` for information on configuring a NT1000 or NT2000 thumbnail stream.

4.21.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.21.3 Errors

<code>ILLEGAL_ARGUMENT</code>	Returned if target argument is an invalid device ID or group name. Also returned if the target argument is not a valid device ID and a multicast IP address was specified.
<code>ILLEGAL_ARGUMENT</code>	In addition, this error is reported if no stream type or index was specified but a multicast IP address was specified.
<code>ILLEGAL_ARGUMENT</code>	Also, is reported when a multicast IP address outside the configurable allocation range is specified.
<code>ILLEGAL_ARGUMENT</code>	Finally, is also reported if a reserved multicast IP address is specified.

The operation may also fail for individual devices. Errors returned include the following:

<code>DEVICE_TYPE</code>	If the device does not have the specified stream type or index.
<code>DEVICE_TYPE</code>	If the specified device is a receiver.
<code>INVALID_STATE</code>	If the specified multicast IP address is currently in use by another stream.
<code>INVALID_STATE</code>	If there are no more multicast IP addresses available.
<code>INVALID_STATE</code>	If starting scaled stream (stream 1) but scaler resolution was not specified.
<code>INVALID_STATE</code>	If starting a thumbnail stream but the thumbnail generator is not configured.

4.21.4 Examples

Below is an example of starting the HDMI video stream for an individual device.

```
POST /api/device/001ec0f04d9c HTTP/1.1
...
{
  "op" : "start",
  "stream_type" : "HDMI",
  "stream_index" : 0
}
HTTP/1.1 201 Created
Location: /api/request/61
...
{
```

```

    "status" : "PROCESSING",
    "request_id" : 61,
    "result" : null,
    "error" : null
}

```

Below is the same example with a destination address specified.

```

POST /api/device/001ec0f04d9c HTTP/1.1
...
{
    "op" : "start",
    "stream_type" : "HDMI",
    "stream_index" : 0,
    "dest_address" : "239.1.2.3"
}
HTTP/1.1 201 Created
Location: /api/request/61
...
{
    "status" : "PROCESSING",
    "request_id" : 61,
    "result" : null,
    "error" : null
}

```

4.22 Command stop

```

POST /api/device/{target}
{
    "op" : "stop",
    "stream_type" : String (optional),
    "stream_index" : Integer (optional),
    "free" : Boolean
}

```

Reminder: Stops one or more stream(s) on one or more devices and optionally free the multicast address associated with each stopped stream so it can be re-used for another stream.

4.22.1 Arguments

The `target` argument must be either the device ID of a single device (BlueRiver or UEX if UEX2000 is used standalone) or the name of a device group. The allowed device groups are:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

The `stream_type` argument entered for the `stop` command must be a valid stream type. The following types are supported for all device types:

CEC	For a CEC data stream.
-----	------------------------

INFRARED	For an infrared data stream.
MULTICH_AUDIO	For a multichannel audio stream.
RS232	For an RS-232 data stream.
USB_HID	For a USB HID data stream.
USB_EXTENDER	For USB 2.0 stream.

Warning! Regarding the Icron SUI feature if this is being used and a `stop` command is issued for the `USB_EXTENDER` stream type, all pairings are removed. Refer to the SDVoE Developers API Application Note Library (PDS-062491) in the “Managing Semtech UEX or Icron USB 2.0 Extender” section for details.

In addition, the following stream types are supported on transmitter devices only:

STEREO_AUDIO	For a stereo audio stream.
HDMI	For an HDMI video stream.
HDMI_AUDIO	For an HDMI audio stream.
THUMBNAIL	For a video thumbnail stream.

Note: For above stream types the target argument cannot be a receiver (RX) device or the `ALL_RX` device group.

The `stream_index` argument must be the valid index of a stream of the specified type.

If the `stream_index` argument is omitted, then all streams of the given type are stopped.

Reminder: For command to target being an AVP, with `usb_extender` stream type, the `stream_index` shall be 0 for the first AVP attached UEX device or 1 for the second AVP attached UEX device. For command to target being a UEX, the `stream_index` shall be 0.

If the `stream_type` argument is omitted, then all streams of the following types are stopped:

HDMI	For an HDMI video stream.
HDMI_AUDIO	For an HDMI audio stream.
STEREO_AUDIO	For a stereo audio stream.
MULTICH_AUDIO	For a multichannel audio stream.

If `stream_type` is omitted, the target argument cannot be a receiver (RX) device or the `ALL_RX` group.

If the `free` argument is `true` then the multicast IP address associated with each target stream is freed/unassigned.

For streams with a type of either `RS232` or `INFRARED` where no multicast IP address is associated with the stream, the `free` keyword is allowed but ignored. This also applies to a `THUMBNAIL` stream (NT1000/NT2000 devcies) when a unicast address is associated with the thumbnail stream instead of a multicast address.

Note: Refer to the Introduction to the SDVoE API User Guide for more information on audio routing.

4.22.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.22.3 Errors

ILLEGAL_ARGUMENT	Occurs when the target argument is not a valid device ID or group name.
------------------	---

The operation may also fail for devices individually for the following reasons:

DEVICE_TYPE
DEVICE_TYPE

Occurs if device does not have the specified stream type or index.
Occurs if stream_type argument omitted and specified device a receiver.

4.22.4 Examples

```
POST /api/device/001ec0f04d9c HTTP/1.1
```

```
...
```

```
{
  "op" : "stop",
  "stream_type" : "HDMI",
  "stream_index" : 0,
  "free" : false
}
```

```
HTTP/1.1 201 Created
```

```
Location: /api/request/61
```

```
...
```

```
{
  "status" : "PROCESSING",
  "request_id" : 61,
  "result" : null,
  "error" : null
}
```

```
POST /api/device/001ec0f04d9c HTTP/1.1
```

```
...
```

```
{
  "op" : "stop",
  "stream_type" : "HDMI",
  "stream_index" : 0,
  "free" : true
}
```

```
HTTP/1.1 201 Created
```

```
Location: /api/request/62
```

```
...
```

```
{
  "status" : "PROCESSING",
  "request_id" : 62,
  "result" : null,
  "error" : null
}
```

```
POST /api/device/001ec0f04d9c HTTP/1.1
```

```
...
```

```
{
  "op" : "stop",
```

```

        "stream_type" : "HDMI",
        "free" : false
    }
HTTP/1.1 201 Created
Location: /api/request/63
...
{
    "status" : "PROCESSING",
    "request_id" : 63,
    "result" : null,
    "error" : null
}

POST /api/device/001ec0f04d9c HTTP/1.1
...
{
    "op" : "stop",
    "free" : false
}
HTTP/1.1 201 Created
Location: /api/request/64
...
{
    "status" : "PROCESSING",
    "request_id" : 64,
    "result" : null,
    "error" : null
}

```

4.23 Command switch

The destination is specified in one of three ways, therefore three forms of the command are available:

1. Using a device ID and a stream index (first form) when the destination is a device.
2. Using the keyword `api` (second form) when the destination is the API Server.
3. Using an IP address (third form). Supported for backwards compatibility with NT1000/NT2000 pre-3.4.0.0 firmware as it did not support the second form.

```

POST /api/device/{target}
{
    "op" : "switch: device",
    "stream_type" : String,
    "stream_index" : Integer,
    "dest_device" : String,
    "subscription_type" : String (optional),
    "subscription_index" : Integer
}

```

```

POST /api/device/{target}
{
  "op" : "switch: api",
  "stream_type" : String,
  "stream_index" : Integer
}

POST /api/device/{target}
{
  "op" : "switch:address",
  "stream_type" : String,
  "stream_index" : Integer,
  "dest_address" : String,
  "subscription_index" : Integer (optional)
}

```

Reminders:

- i. The `switch` command is used to set the stream destination for a device. It establishes a one-way connection between the source and destination devices.
- ii. To establish a two-way connection between any two devices, the command must be issued a second time with the source and destination reversed.

Notes:

- i. A device cannot send data to itself.
- ii. The second form of the command with the `api` keyword requires firmware support for NT1000/NT2000 devices of 3.4.0.0 or later and for BlueRiver series device firmware 1.0.0.0 or higher.

4.23.1 Arguments

The `target` argument must be either the device ID (BlueRiver or UEX if UEX2000 is used standalone) of a single device or the name of a device group.

The allowed device groups are:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

The `stream_type` argument entered must one of the following stream types:

CEC	CEC data stream.
INFRARED	Infrared (IR) data stream.
RS232	RS-232 data stream.
USB_HID	USB HID data stream.
USB_EXTENDER	For USB 2.0 stream (UEX or Icron).

Note: Issuing a `switch` command for `USB_EXTENDER` streams removes all current pairings, refer to section 4.13 Command `pair/unpair` for more information on pairing Icron modules using the Simultaneous User Interaction (SUI) pairing feature.

Reminder: For audio and video stream types, refer to the `start` command, section 4.20.

The `stream_index` argument must be the valid index of a stream of the specified type on the source device.

In the first form of the command, the `dest_device` argument must be either the device ID of a single device or the name of a device group.

The allowed device groups are:

<code>ALL</code>	All devices
<code>ALL_TX</code>	All transmitter devices
<code>ALL_RX</code>	All receiver devices

The `subscription_type` argument is optional. If specified, it must be identical to the `stream_type` argument.

Reminder: A stream can only be switched to a subscription of the same type.

The `subscription_index` argument must be the valid index of a subscription of the specified type on the destination device.

Reminder: For command to target and/or `dest_device` being an AVP, with `usb_extender` stream/subscription type, the `stream_index` and/or `subscription_index` shall be 0 for the first AVP attached UEX device or 1 for the second AVP attached UEX device. For command to target and/or `dest_device` being an UEX, the `stream_index` and/or `subscription_index` shall be 0.

In the third form of the command, the `dest_address` must be the IP address to of the API Server to which the data is to be sent.

4.23.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.23.3 Errors

<code>ILLEGAL_ARGUMENT</code>	Occurs when the arguments entered are invalid.
<code>INVALID_STATE</code>	This occurs if the destination device is disconnected.

The request may also fail for individual devices for the following reasons:

<code>INVALID_OPERATION</code>	Occurs when the device is being configured to send data to itself.
<code>DEVICE_TYPE</code>	Occurs when second form of the command is used and the firmware is not recent enough to support request.

4.23.4 Examples

Routing the third RS-232 port from a device to another:

```
POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "switch:device",
  "stream_type" : "RS232",
  "stream_index" : 2,
  "dest_device" : "001ec0f04d9c",
  "subscription_index" : 2
}
```

```
HTTP/1.1 201 Created
Location: /api/request/61
```

```
...
{
  "status" : "PROCESSING",
  "request_id" : 61,
  "result" : null,
  "error" : null
}
```

Routing the first RS-232 port from a device to the API server:

```
POST /api/device/001ec0f03668 HTTP/1.1
```

```
...
{
  "op" : "switch:api",
  "stream_type" : "RS232",
  "stream_index" : 0
}
```

```
HTTP/1.1 201 Created
Location: /api/request/62
```

```
...
{
  "status" : "PROCESSING",
  "request_id" : 62,
  "result" : null,
  "error" : null
}
```

Routing the first RS-232 port from a device to the API server by IP address:

```
POST /api/device/001ec0f03668 HTTP/1.1
```

```
...
{
  "op" : "switch:address",
  "stream_type" : "RS232",
  "stream_index" : 0,
  "dest_address" : "169.254.1.0"
}
```

```
HTTP/1.1 201 Created
Location: /api/request/63
```

```
...
{
  "status" : "PROCESSING",
  "request_id" : 63,
  "result" : null,
  "error" : null
}
```

Routing infrared data between two devices:

```
POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "switch:device",
  "stream_type" : "INFRARED",
  "stream_index" : 0,
  "dest_device" : "001ec0f04d9c",
  "subscription_index" : 0
}
HTTP/1.1 201 Created
Location: /api/request/64
...
{
  "status" : "PROCESSING",
  "request_id" : 64,
  "result" : null,
  "error" : null
}
```

Example shown below pairs two USB extenders (Semtech UEX or Icron) where device1 has a MAC address of 0016c035566f and device2 has a MAC address of 0017c0355675. Reminder, two switch requests are required to pair the extenders first in one direction and then a second to pair them in the other direction. For this example, the two requests would be as follows:

```
POST /api/device/0016c035566f
{
  "op" : "switch:device",
  "stream_type" : "USB_EXTENDER",
  "stream_index" : 0,
  "dest_device" : "0017c0355675",
  "subscription_index" : 0
}

POST /api/device/0017c0355675
{
  "op" : "switch:device",
  "stream_type" : "USB_EXTENDER",
  "stream_index" : 0,
  "dest_device" : "0016c035566f",
  "subscription_index" : 0
}
```

4.24 Command update

```
POST /api/device/{target}

{
  "op" : "update",
  "file_name" : "file_name.apz"
}
```

Reminders:

- i. This command updates the BlueRiver or UEX firmware of one or more devices.
Note: When an update is in progress, no other operations can be performed on a device.
- ii. This command was introduced in API version 3.0.
- iii. A reboot command must be issued to the device once the firmware update is complete for the changes to take effect. The device is in an undefined state until that reboot.

4.24.1 Arguments

The `target` argument must be either the device ID of a single device (BlueRiver or UEX no matter if attached to AVP or standalone) or the name of a device group.

The allowed device groups are:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

The `file_name` argument is the file name of a firmware file. The available firmware files can be listed with the `list firmware` command, refer to section 4.11.1 Command `list firmware`.

4.24.2 Return Value

For each device targeted by the update command, there is either an entry in the update array if the device was updated successfully or a device error object in the error array if the operation failed for some reason.

```
{
  "update" : [UpdateResultObject, ...],
  "error" : [DeviceError, ...]
}
```

Table 70: Update Object Array Members

Member Name	Since	Type	Description
update	3.0	Array of UpdateResultObject	An array of objects representing the devices for which the firmware update succeeded.
error	3.0	Array of DeviceError	An array of DeviceError objects indicating the devices for that the command failed and the reason for the failure.

The UpdateResultObject object has the following structure:

```
{
  "device_id" : String
}
```

Table 71: Update Object Member

Member Name	Since	Type	Description
device_id	3.0	String	Device ID

Refer to the Introduction to the SDVoE API User Guide for details on the device error object (`DeviceError`).

4.24.3 Errors

ILLEGAL_ARGUMENT	Occurs when target argument entered is not valid device ID or group name.
IO_ERROR	Occurs when the firmware file cannot be opened.

The request may also fail for individual devices for the following reasons:

IO_ERROR	Occurs when the firmware file cannot be read.
RESOURCE_NOT_FOUND	Specified firmware file does not contain a firmware valid for the device.

4.24.4 Examples

```
POST /api/device/001ec0f04d9c HTTP/1.1
...
{
  "op" : "update",
  "file_name" : "blueriver_rdk_2-0-0-0.apz"
}
HTTP/1.1 201 Created
Location: /api/request/32170
...
{
  "status" : "PROCESSING",
  "request_id" : 32223,
  "result" : null,
  "error" : null
}
GET /api/request/32223 HTTP/1.1
...
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : 32223,
  "result" : {
    "update" : [
      {
        "device_id" : "001ec0f04d9c"
      }
    ],
    "error" : []
  },
  "error" : null
}
```

4.25 Command `update_stream`

```
POST /api/device/{target}

{
  "op" : "update_stream",
  "stream_index" : integer,
  "fps" : integer,
  "dest_address" : String (optional),
  "stream_state" : String (optional),
  "file_path" : "file_path"
}
```

Reminders:

This command only applies to BlueRiver Bitmap Streamer software devices (version 1.1.0.0 and above).

4.25.1 Arguments

The `target` argument must be the device ID of a single BlueRiver Bitmap Streamer device.

The `stream_index` argument is the index of the video stream. If the stream index is a stream that exists, then the stream properties such as source file, framerate and destination address will be replaced with the new information. If the stream index is a stream that does not exist, the stream index shall be the next available index or the API will return an error.

The `fps` argument is the framerate per second of the video stream.

The `dest_address` argument is the multicast IP address of the video stream.

The `file_path` argument is the file path of the bitmap image file.

4.25.2 Return Value

This command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

4.25.3 Errors

`ILLEGAL_ARGUMENT` Occurs if the `stream_index` argument is not a valid index.

4.25.4 Examples

```
POST /api/device/001ec0f04d9c HTTP/1.1
...
{
  "op" : "update_stream",
  "stream_index" : 20,
  "fps" : 5,
  "dest_address" : "224.1.1.2",
  "stream_state": "streaming",
  "file_path": "/usr/games/1280-720-sample.bmp"
}

HTTP/1.1 201 Created
Location: /api/request/32170
...
```

```

{
  "status" : "PROCESSING",
  "request_id" : 32223,
  "result" : null,
  "error" : null
}
GET /api/request/32223 HTTP/1.1
...
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : 32223,
  "result" : {
    "update_stream" : [
      {
        "device_id" : "001ec0f04d9c"
      }
    ],
    "error" : []
  },
  "error" : null
}

```

4.26 Command USB

4.26.1 USB Descriptors

BlueRiver ASIC devices support extension of USB devices such as keyboards and mice that follow the USB Human Interface Device (HID) specification.

For more information on HID USB devices, refer to Device Class Definition for Human Interface Devices (HID):

<https://www.usb.org/document-library/device-class-definition-hid-111>

A device can act as a local or a remote extender. This role can be configured using the `set usb` command. For more information on this command refer to section 4.19.11 Command `set usb`.

- When configured as a remote extender (`REMOTE`), the BlueRiver device acts as a USB hub to which HID USB devices can be connected.
- When configured as a local extender (`LOCAL`), the BlueRiver device connects to a USB host and emulates a USB hub.
- The USB devices connected to this hub as seen by the USB host are the ones connected to the remote extender with which the local extender is communicating.

The `usb` command allows USB descriptors to be read or written. For more details on USB descriptors, refer to the USB 2.0 Specification:

<https://www.usb.org/document-library/usb-20-specification>

This API manipulates all descriptors packed together as binary data with a well-defined format. When included as part of an API command or its return value, this packed binary data is encoded as a hexadecimal string.

The binary representation of packed USB descriptors has the following four parts, in the same order as shown:

1. A `header` that contains sufficient information to locate the other parts.
2. `USB descriptors`, with the exclusion of string descriptors and report descriptors. Standard descriptors as defined by the USB 2.0 Specification and class-specific HID descriptors as defined in the Class Definition for HID are included here.
3. The `string descriptors` as defined by the USB 2.0 Specification (optional).
4. The `report descriptors` as defined in the Class Definition for HID.

Header
USB Descriptors
String Descriptors (optional)
Report Descriptors

These items are described in the following sub-sections.

4.26.1.1 Packed Descriptors Header

Table below outlines the format of the header.

Table 72: USB Descriptors Header

Offset (Byte)	Name	Size (Bytes)	Description
0	magic	1	Contains the hexadecimal value 5B.
1	version	1	Contains value 1, which identifies this version of header and descriptor packing.
2	size	2	The size, in bytes, of the packed binary data, including the header.
4	descsize	2	The size, in bytes, of the USB descriptors.
6	stroff	2	The offset of the string descriptors from the start of the packed binary data (including the header). Undefined (i.e. can be set to any value) if the string descriptors are omitted.
8	strsize	2	The size, in bytes, of the string descriptors. Must be set to zero (0) if the string descriptors are omitted.
10	reportoff	2	The offset of the report descriptors from the start of the packed binary data (including the header).
12	reportnum	2	The number of report descriptors.
14	checksum	2	A checksum computed on the whole packed binary data.

The checksum is computed as follow: the checksum field in the header is set to zero, then the value of all bytes, including the header and everything that follows, are added together.

4.26.1.2 Standard USB & HID Descriptors

The descriptors start immediately after the header. All descriptors, with the exception of string descriptors and class-specific report descriptors, are concatenated together without padding. Their total size is indicated by the `descsize` field in the header.

Each descriptor is formatted as specified in the USB 2.0 Specification. Among other things, this means each descriptor starts with a one-byte length, followed by a one-byte descriptor type. With this information, it is possible to enumerate descriptors and identify their type.

4.26.1.3 String Descriptors

If present, the string descriptors start at the offset specified in the `stroff` field of the header. String descriptors are concatenated together without padding. Their total size is indicated by the `strsize` field in the header.

String descriptors are optional. If they are omitted, the `strsize` field is set to zero. In that case, the value of `stroff` is undefined (i.e. it can have any value).

The string descriptors follow the format specified by the USB 2.0 Specification.

4.26.1.4 Report Descriptors

Report descriptors are class-specific descriptors specified by the Device Class Definition for Human Interface Devices (HID). They differ from standard USB descriptors in that they are variable in size.

Report descriptors start at the offset specified in the `reportoff` field of the header. The number of report descriptors is indicated by the `reportnum` field in the header.

Each report descriptor must start on a 4-byte boundary, with padding added as needed to satisfy this constraint. This implies that `reportoff` must be a multiple of four.

Each report descriptor is prepended with a header that has the format outlined in the table below.

Table 73: Report Descriptors Heading

Offset (Byte)	Name	Size (Bytes)	Description
0	size	2	The size of the report descriptor, excluding this header.
2	config	1	Index of the USB configuration to which this report descriptor applies.
3	interface	1	The index of the USB interface to which this report descriptor applies.

4.26.1.5 Other Constraints

The total length of the packed descriptors, cannot be more than 1024 bytes.

4.26.2 Command `usb descriptor read`

```
POST /api/device/{target}
{
  "op" : "usb:descriptor:read",
  "index" : Integer
}
```

This command is used to `read` the USB descriptors of a USB device.

An endpoint device that is configured to serve as a USB host (REMOTE role) has one or more USB HID devices connected to it. This command reads the USB descriptors of one of these devices.

An endpoint device configured to serve as a USB device (LOCAL role) connects to a USB host (e.g. a PC) and is seen by this host as a USB hub to which one or more USB devices are connected. This command allows the USB descriptors of the USB device(s) to be read.

Notes:

- i. Is only supported on a device if the device JSON object for that device reports a USB node.
- ii. Reading descriptors from a device is only allowed if its role is configured as HOST or DEVICE that is not DISABLED.

4.26.2.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group. The allowed device groups are:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

The `index` argument identifies the USB device. The first USB device has index 0 and USB devices are indexed sequentially. The number of devices is given by the number of elements in the `usb_devices` array member of the `status` member of the JSON device object's `USB_HID` node.

4.26.2.2 Return Value

For each device targeted by this command, there is either an entry in the `usb_descriptors` array if the descriptors were fetched successfully or a device error object in the `error` array if the information was not able to be fetched.

```
{
  "usb_descriptors" : [USBDescriptor, ...],
  "error" : [DeviceError, ...]
}
```

Table 74: USB Descriptor Read Object Array Members

Member Name	Since	Type	Description
usb_descriptors	3.0	Array of USBDescriptor	An array of objects containing the requested descriptors.
error	3.0	Array of DeviceError	An array of DeviceError objects indicating the devices for which the command failed and reason for failure.

A `USBDescriptor` object has the following format:

```
{
  "device_id" : String,
  "index" : Integer,
  "descriptors" : String,
}
```

Table 75: USB descriptor Read Object Members

Member Name	Since	Type	Description
device_id	3.0	String	The device ID of the device to which these descriptors belong.
index	3.0	Integer	The index of the USB device to which these descriptors belong.
descriptors	3.0	String	The packed descriptors encoded as a hexadecimal string.

4.26.2.3 Errors

The request may also fail for individual devices for the following reasons:

INVALID_STATE	If the device is not configured in the correct role.
DEVICE_TYPE	If the target device does not support USB extension.
DEVICE_TYPE	If the USB device index is out of range.

4.26.2.4 Examples

```
POST /api/device/010203040506 HTTP/1.1
...
{
  "op" : "usb:descriptor:read",
  "index" : 0
}
HTTP/1.1 201 Created
Location: /api/request/61
...
{
  "status" : "PROCESSING",
  "request_id" : 61,
  "result" : null,
  "error" : null
}
```

4.26.3 Command `usb descriptor write`

```
POST /api/device/{target}
{
  "op" : "usb:descriptor:write",
  "index" : Integer,
  "data_hex" : String
}
```

This command is used to write the USB descriptors of a USB device.

A BlueRiver device configured to serve as a USB device (LOCAL role) connects to a USB host, such as a computer. It is then seen by this host as a USB hub to which one or more USB HID devices are connected. This command allows the USB descriptors of the USB device(s) to be written.

Notes:

- Is only supported on a device if the device JSON object for that device reports a USB node.
- Writing descriptors to a device is only allowed if its role is configured as LOCAL.

4.26.3.1 Arguments

The `target` argument must be either the device ID of a single device or the name of a device group. The allowed device groups are:

<code>ALL</code>	All devices
<code>ALL_TX</code>	All transmitter devices
<code>ALL_RX</code>	All receiver devices

The `index` argument identifies the USB device. The first USB device has index 0 and USB devices are indexed sequentially. The number of devices is given by the number of elements in the `usb_devices` array member of the `status` member of the JSON device object's `USB_HID` node.

The `data_hex` argument is a string containing an hexadecimal encoding of the packed descriptors.

4.26.3.2 Return Value

For each device targeted by this command, there is either an entry in the `descriptors_write` array if the descriptors written successfully or a device error object in the `error` array if descriptors were not written.

```
{
  "descriptors_write" : [USBDescriptorWriteResult, ...],
  "error" : [DeviceError, ...]
}
```

Table 76: USB Descriptor Write Object Array Members

Member Name	Since	Type	Description
<code>descriptors_write</code>	3.0	Array of USBDescriptorWriteResult	Array of objects containing results of successful writes.
<code>error</code>	3.0	Array of DeviceError	Array of DeviceError objects indicating devices command failed for and reason for failure.

A `USBDescriptorWriteResult` object has the following format:

```
{
  "device_id" : String,
  "index" : Integer
}
```

Table 77: USB Descriptor Write Result Object Members

Member Name	Since	Type	Description
<code>device_id</code>	3.0	String	Device ID of the device to which the descriptors were successfully written.
<code>index</code>	3.0	Integer	Index of the USB device to which the descriptors were successfully written.

4.26.3.3 Errors

The request may fail for individual devices for the following reasons:

<code>INVALID_STATE</code>	If the device is not configured in the correct role (i.e. LOCAL).
<code>DEVICE_TYPE</code>	If the target device does not support USB HID extension.
<code>DEVICE_TYPE</code>	If the USB device index is out of range.

4.26.3.4 Examples

```
POST /api/device/010203040506
...
{
  "op" : "usb:descriptor:write",
  "index" : 0,
  "data_hex" :
    "5b01c00034004400440088000100261b120100020000000086d040ec00011010200
    0109022200010100a03109040000010301020009211001000122340007058103040
    00a0403090412034c006f006700690074006500630068002e035500530042002d00
    500053002f00320020004f00700074006900630061006c0020004d006f007500730
    065003400010005010902a1010901a1000509190129031500250195037501810295
    017505810305010930093109381581257f750895038106c0c0"
}
```

Semtech Confidential Information
Mark Li
ADV
02/03/2026

5 Multiview Command Reference

Multiview is the ability to take multiple independent sources and composite them into a single output stream that can then be displayed on a single display. This section describes the commands applicable to multiview.

5.1 Multiview Model

In multiview mode, the screen is divided in a grid of at most 16×16 rectangular tiles. Each tile displays video from a rectangular area of the receiver devices frame buffer. Alternatively, a tile can also be black. A layout is first created and configured and then applied to one or more receiver devices.

TIP: Refer to the API Developers API Application Note Library, “Multiview mode” section for information on managing multiview layouts. The article includes terminology, bandwidth usage and API command examples, as well as use cases and script examples.

5.1.1 Multiview Mode Guidelines & Limitations

There is some guidelines and limitations that apply to multiview layouts.

They are as follows:

- The maximum number of windows supported is 32.
- The maximum number of distinct video sources (i.e. surfaces) is also 32.
- The number of tiles cannot exceed 15 in either the horizontal or vertical direction, and 16 in the other direction (i.e. a grid of 15×16 or 16×15 tiles are both acceptable but 16×16 is not).
- Tile widths and horizontal read offsets (equivalently window widths, horizontal positions and horizontal offsets) must be even.
- Tile width has a minimum width of 8-pixels.
- A tile width cannot be equal to the following values 1366, 1368, 2732, 2734, 4098 or 4100, in pixels.
- The total size of the frame buffer is 10880×4096.
- Surfaces must be horizontally aligned on a 32-pixel boundary.
- The total bandwidth sent by an individual transmitter and received by an individual receiver is recommended to not exceed 9 Gbps.
- Source video must be progressive and fully Chroma sampled (RGB, YCbCr 4:4:4).

5.2 Multiview Commands

5.2.1 Summary of Multiview Commands

The `layout` command provides a series of sub-commands to create and modify multiview layouts.

Reminder: Multiview mode requires the AV Processing Engine, applicable only to endpoint devices that include this feature.

The multiview mode is used to create multiview configuration setups that include various layouts, including:

- Picture-in-Picture (PiP)
- Picture-and-Picture (PaP)
- Matrix grid
- L-Shape
- Customized layouts

The table below lists the API commands that are relevant to the multiview mode, specifically the commands and arguments used in the creation and management of multiview layouts, windows and surfaces.

Table 78: Multiview Layout Command Set

API Command	Description
<code>list layout</code>	List of the currently existing layouts.
<code>layout create</code>	Create a layout. If there is an existing layout with the same name it is overwritten.
<code>layout describe</code>	Retrieves details of an existing layout including windows and their positions.
<code>layout surface</code>	When the default layout surface configuration is not appropriate for a desired layout it is possible to modify it.
<code>layout window</code>	Specify the content of a window/tile.
<code>layout delete</code>	Delete an existing layout.
<code>set multiview</code>	Set a receiver device(s) (RX) into multiview mode and load an existing layout.
<code>set scaler</code>	Set scaler resolution of the source of an encoder device(s).
<code>Join</code>	Join a source to the multiview layout destination.
<code>start/stop</code>	Manage streams of the transmitters (TX) that are the sources for the layout.

The following subsections review the commands used to complete these functions.

5.2.2 Command `list layout`

The `list layout` command retrieves a list of layouts that currently exist from the API server.

```
GET /api/multiview/layout
```

5.2.2.1 Arguments

None

5.2.2.2 Return Value

An array of `LayoutItem` objects.

```
{
  "layout" : [LayoutItem, ...]
}
```

Table 79: Layout Item Member

Member Name	Since	Type	Description
<code>Layout</code>	3.0	Array of <code>LayoutItem</code>	Array of objects which each represent a multiview layout

Each item in the layout array (`LayoutItemObject`) has the following structure:

```
{
  "name" : String
}
```

Table 80: Layout Item Object Member

Member Name	Since	Type	Description
<code>Name</code>	3.0	String	The name of the layout.

5.2.2.3 Errors

None.

5.2.2.4 Example

```
GET /api/multiview/layout HTTP/1.1
...
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "layout" : [
      {
        "name" : "compatibility_4k_2x2"
      },
      {
        "name" : "example_4k_single_source_2x2_1080p"
      }
    ]
  }
}
```

5.2.3 Command `layout create`

The `layout create` command is used to create a new multiview layout or reset an existing layout with the specified name.

```
POST /api/multiview/layout/{name}
{
  "op" : "create",
  "width" : Integer,
  "height" : Integer
}
```

5.2.3.1 Arguments

The `name` argument is the name of the multiview layout.

The `width` and `height` arguments respectively specify total width and height of the displayed video, in pixels.

Note: The `width` argument must be a multiple of 8.

5.2.3.2 Return Value

Description of the layout refer to section 5.2.5 Command `layout describe`.

HTTP status is:

- 201 -- Created when a new if a new layout was created.
- 200 -- OK if an existing layout with that name was overwritten.

5.2.3.3 Errors

ILLEGAL_ARGUMENT	Occurs when the arguments entered are invalid.
MULTIVIEW_LAYOUT_STATE	Occurs when the specified layout exists and is read-only.

5.2.3.4 Example

```
POST /api/multiview/layout/some_layout HTTP/1.1
...
{
  "op" : "create",
  "width" : 1920,
  "height" : 1080
}
HTTP/1.1 201 Created
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "layout_description" : {
      "name" : "some_layout",
      "width" : 1920,
      "height" : 1080,
      "read_only" : false,
      "surfaces" : [
        {
          "index" : 0,
          "horizontal_position" : 0,
          "vertical_position" : 0,
          "width" : 4096,
          "height" : 2160
        },
        {
          "index" : 1,
          "horizontal_position" : 4160,
          "vertical_position" : 0,
          "width" : 2048,
          "height" : 1080
        },
        {
          "index" : 2,
          "horizontal_position" : 6240,
          "vertical_position" : 0,
          "width" : 2048,
          "height" : 1080
        },
        {
          "index" : 3,
          "horizontal_position" : 8320,
```

```

        "vertical_position" : 0,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 4,
        "horizontal_position" : 4160,
        "vertical_position" : 1080,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 5,
        "horizontal_position" : 6240,
        "vertical_position" : 1080,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 6,
        "horizontal_position" : 8320,
        "vertical_position" : 1080,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 7,
        "horizontal_position" : 0,
        "vertical_position" : 2160,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 8,
        "horizontal_position" : 2080,
        "vertical_position" : 2160,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 9,
        "horizontal_position" : 4160,
        "vertical_position" : 2160,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 10,
        "horizontal_position" : 6240,
        "vertical_position" : 2160,
        "width" : 2048,

```

```

        "height" : 1080
    },
    {
        "index" : 11,
        "horizontal_position" : 8320,
        "vertical_position" : 2160,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 12,
        "horizontal_position" : 0,
        "vertical_position" : 3240,
        "width" : 2048,
        "height" : 856
    },
    {
        "index" : 13,
        "horizontal_position" : 2080,
        "vertical_position" : 3240,
        "width" : 2048,
        "height" : 856
    },
    {
        "index" : 14,
        "horizontal_position" : 4160,
        "vertical_position" : 3240,
        "width" : 2048,
        "height" : 856
    },
    {
        "index" : 15,
        "horizontal_position" : 6240,
        "vertical_position" : 3240,
        "width" : 2048,
        "height" : 856
    },
    {
        "index" : 16,
        "horizontal_position" : 8320,
        "vertical_position" : 3240,
        "width" : 2048,
        "height" : 856
    },
    {
        "index" : 17,
        "horizontal_position" : 0,
        "vertical_position" : 0,
        "width" : 2048,
        "height" : 1080
    },
    },

```

```

    {
      "index" : 18,
      "horizontal_position" : 2080,
      "vertical_position" : 0,
      "width" : 2048,
      "height" : 1080
    },
    {
      "index" : 19,
      "horizontal_position" : 0,
      "vertical_position" : 1080,
      "width" : 2048,
      "height" : 1080
    },
    {
      "index" : 20,
      "horizontal_position" : 2080,
      "vertical_position" : 1080,
      "width" : 2048,
      "height" : 1080
    }
  ],
  "windows" : []
}
},
"error" : null
}

```

5.2.4 Command layout delete

The `layout delete` command is used to delete the multiview layout specified if it exists.

```
DELETE /api/multiview/layout/{name}
```

5.2.4.1 Arguments

The `name` argument is the name of the multiview layout to be deleted.

5.2.4.2 Return Value

None

5.2.4.3 Errors

ILLEGAL_ARGUMENT	Occurs when if the arguments entered are invalid.
MULTIVIEW_LAYOUT_STATE	Occurs if the specified layout is read-only.

5.2.4.4 Example

```

DELETE /api/multiview/layout/some_layout HTTP/1.1
...
HTTP/1.1 200 OK
...

```

```
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : null,
  "error" : null
}
```

5.2.5 Command layout describe

The `layout describe` command provides a complete description of the specified multiview layout.

```
GET /api/multiview/layout/{name}
```

5.2.5.1 Arguments

The `name` argument is the name of the multiview layout.

5.2.5.2 Return Value

```
{
  "layout_description" : LayoutDescription
}
```

Table 81: Layout Describe Object Array Member

Member Name	Since	Type	Description
<code>layout_description</code>	3.0	<code>LayoutDescription</code>	A layout description object, described below

A layout description object (`LayoutDescription`) has the following structure:

```
{
  "name" : String,
  "width" : Integer,
  "height" : Integer,
  "read_only" : Boolean,
  "surfaces" : [SurfaceDescription, ...],
  "windows" : [WindowDescription, ...]
}
```

Table 82: Layout Describe Object Members

Member Name	Since	Type	Description
<code>name</code>	3.0	<code>String</code>	Name of the layout.
<code>width</code>	3.0	<code>Integer</code>	Display width in pixels.
<code>height</code>	3.0	<code>Integer</code>	Display height in pixels.
<code>read_only</code>	3.0	<code>Boolean</code>	Layout protected against modifications. Only predefined <code>compatibility_4k_2x2</code> layout currently read-only.
<code>surfaces</code>	3.0	Array of <code>SurfaceDescription</code>	Description of all surfaces in this layout.
<code>windows</code>	3.0	Array of <code>WindowDescription</code>	Description of all windows in this surface.

A surface description object (SurfaceDescription) has the following structure:

```
{
    "index" : Integer,
    "horizontal_position" : Integer,
    "vertical_position" : Integer,
    "width" : Integer,
    "height" : Integer
}
```

Table 83: Surface Description Object Members

Member Name	Since	Type	Description
index	3.0	Integer	The surface index.
horizontal_position	3.0	Integer	Horizontal start position of surface in pixels.
vertical_position	3.0	Integer	Vertical start position of surface in pixels.
width	3.0	Integer	Surface width in pixels.
height	3.0	Integer	Surface height in pixels.

A window description object (WindowDescription) has the following structure:

```
{
    "index" : Integer,
    "horizontal_position" : Integer,
    "vertical_position" : Integer,
    "width" : Integer,
    "height" : Integer,
    "horizontal_offset" : Integer,
    "vertical_offset" : Integer,
    "content" : String,
    "target_surface" : Integer,
}
```

Table 84: Window Description Object Members

Member Name	Since	Type	Description
index	3.0	Integer	The surface index.
horizontal_position	3.0	Integer	Horizontal start position of window on screen, in pixels.
vertical_position	3.0	Integer	Vertical start position of window on screen, in pixels.
width	3.0	Integer	Window width, in pixels.
height	3.0	Integer	Window height, in pixels.
horizontal_offset	3.0	Integer	Horizontal offset from the start of the surface.
vertical_offset	3.0	Integer	Vertical offset from the start of the surface.
content	3.0	String	Type of content of the window, either VIDEO window has video content or BLACK window is black
target_surface	3.0	Integer	Index of surface that window referring to. Member is only meaningful if the content member is VIDEO.

5.2.5.3 Errors

ILLEGAL_ARGUMENT
RESOURCE_NOT_FOUND

Occurs when the arguments entered are invalid.
Occurs when the specified layout does not exist.

5.2.5.4 Example

The following example are the return for the output of the command `layout describe` for the `compatibility_4k_2x2` layout.

```
GET /api/multiview/layout/compatibility_4k_2x2 HTTP/1.1
...
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "layout_description" : {
      "name" : "compatibility_4k_2x2",
      "width" : 3840,
      "height" : 2160,
      "read_only" : true,
      "surfaces" : [
        {
          "index" : 0,
          "horizontal_position" : 0,
          "vertical_position" : 0,
          "width" : 4096,
          "height" : 2160
        },
        {
          "index" : 1,
          "horizontal_position" : 4160,
          "vertical_position" : 0,
          "width" : 2048,
          "height" : 1080
        },
        {
          "index" : 2,
          "horizontal_position" : 6240,
          "vertical_position" : 0,
          "width" : 2048,
          "height" : 1080
        },
        {
          "index" : 3,
          "horizontal_position" : 8320,
          "vertical_position" : 0,
          "width" : 2048,
          "height" : 1080
        },
        {
          "index" : 4,
          "horizontal_position" : 4160,
          "vertical_position" : 1080,

```

```

        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 5,
        "horizontal_position" : 6240,
        "vertical_position" : 1080,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 6,
        "horizontal_position" : 8320,
        "vertical_position" : 1080,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 7,
        "horizontal_position" : 0,
        "vertical_position" : 2160,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 8,
        "horizontal_position" : 2080,
        "vertical_position" : 2160,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 9,
        "horizontal_position" : 4160,
        "vertical_position" : 2160,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 10,
        "horizontal_position" : 6240,
        "vertical_position" : 2160,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 11,
        "horizontal_position" : 8320,
        "vertical_position" : 2160,
        "width" : 2048,
        "height" : 1080
    }

```

```

    },
    {
      "index" : 12,
      "horizontal_position" : 0,
      "vertical_position" : 3240,
      "width" : 2048,
      "height" : 856
    },
    {
      "index" : 13,
      "horizontal_position" : 2080,
      "vertical_position" : 3240,
      "width" : 2048,
      "height" : 856
    },
    {
      "index" : 14,
      "horizontal_position" : 4160,
      "vertical_position" : 3240,
      "width" : 2048,
      "height" : 856
    },
    {
      "index" : 15,
      "horizontal_position" : 6240,
      "vertical_position" : 3240,
      "width" : 2048,
      "height" : 856
    },
    {
      "index" : 16,
      "horizontal_position" : 8320,
      "vertical_position" : 3240,
      "width" : 2048,
      "height" : 856
    },
    {
      "index" : 17,
      "horizontal_position" : 0,
      "vertical_position" : 0,
      "width" : 2048,
      "height" : 1080
    },
    {
      "index" : 18,
      "horizontal_position" : 2080,
      "vertical_position" : 0,
      "width" : 2048,
      "height" : 1080
    },
  },
  {

```

```

        "index" : 19,
        "horizontal_position" : 0,
        "vertical_position" : 1080,
        "width" : 2048,
        "height" : 1080
    },
    {
        "index" : 20,
        "horizontal_position" : 2080,
        "vertical_position" : 1080,
        "width" : 2048,
        "height" : 1080
    }
],
"windows" : [
    {
        "index" : 0,
        "horizontal_position" : 0,
        "vertical_position" : 0,
        "width" : 1920,
        "height" : 1080,
        "horizontal_offset" : 0,
        "vertical_offset" : 0,
        "content" : "VIDEO",
        "target_surface" : 0
    },
    {
        "index" : 1,
        "horizontal_position" : 1920,
        "vertical_position" : 0,
        "width" : 1920,
        "height" : 1080,
        "horizontal_offset" : 0,
        "vertical_offset" : 0,
        "content" : "VIDEO",
        "target_surface" : 1
    },
    {
        "index" : 2,
        "horizontal_position" : 0,
        "vertical_position" : 1080,
        "width" : 1920,
        "height" : 1080,
        "horizontal_offset" : 0,
        "vertical_offset" : 0,
        "content" : "VIDEO",
        "target_surface" : 2
    },
    {
        "index" : 3,
        "horizontal_position" : 1920,

```

```

        "vertical_position" : 1080,
        "width" : 1920,
        "height" : 1080,
        "horizontal_offset" : 0,
        "vertical_offset" : 0,
        "content" : "VIDEO",
        "target_surface" : 3
    }
}
},
"error" : null
}

```

5.2.6 Command layout surface

The `layout surface` command is used to create, modify or delete a surface in the specified layout.

TIP: Default surfaces are provided when a layout is created. Typically, the default surfaces are adequate for most multiview applications. Refer to section 5.3 Default Surface Configuration.

POST /api/multiview/layout/{name}/surface/{index}

```

{
    "op" : "create",
    "horizontal_position" : Integer,
    "vertical_position" : Integer,
    "width" : Integer,
    "height" : Integer
}

```

DELETE /api/multiview/layout/{name}/surface/{index}

5.2.6.1 Arguments

The `name` argument is the name of the multiview layout.

The `index` argument is the index of the surface, must be numerical number between 0 and 31.

The `horizontal_position` and `vertical_position` arguments are respectively the horizontal and vertical start positions of the surface in the frame buffer, in pixels.

Note: The `horiz_position` argument cannot be less than 8-pixels and is applied in increments of two pixels, so for example 8, 10, 12, etc.

The `width` and `height` arguments are respectively the width and height of the surface, in pixels.

Note: The `width` argument can also not be less than 8-pixels and is applied in increments of two pixels, so for example 8, 10, 12, etc.

If the `delete` keyword is present, the surface is deleted if it exists.

5.2.6.2 Return Value

Description of the surface when creating. Refer to section 5.2.5 Command layout describe.

Return value is `null` when deleting a layout.

HTTP status is:

201 -- Created when a new surface was created (POST).
200 -- OK if an existing surface was overwritten (POST).
200 -- OK on successful delete (DELETE).

5.2.6.3 Errors

ILLEGAL_ARGUMENT	Occurs when the arguments entered are invalid.
MULTIVIEW_LAYOUT_STATE	Occurs when the specified layout exists but is read-only.
RESOURCE_NOT_FOUND	Occurs when the specified layout does not exist.

5.2.6.4 Examples

This first example specifies the surface, position and size of surface 23 for the layout named `some_layout`.

```
POST /api/multiview/layout/some_layout/surface/23 HTTP/1.1
...
{
  "op" : "create",
  "horizontal_position" : 3872,
  "vertical_position" : 0,
  "width" : 1920,
  "height" : 1080
}
HTTP/1.1 201 Created
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "layout_surface" : {
      "index" : 23,
      "horizontal_position" : 3872,
      "vertical_position" : 0,
      "width" : 1920,
      "height" : 1080
    }
  },
  "error" : null
}
```

This second example deletes surface 23 from the layout named `some_layout`.

```
DELETE /api/multiview/layout/some_layout/surface/23 HTTP/1.1
...
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : null,
  "error" : null
}
```

5.2.7 Command `layout window`

The `layout window` command is used to create, modify or delete a window in the specified layout. There is two modes of the command, where the first form of the command is used for creation and modification, while the second form is used for deletion.

```
POST /api/multiview/layout/{name}/window/{index}
{
  "op" : "create",
  "horizontal_position" : Integer,
  "vertical_position" : Integer,
  "width" : Integer,
  "height" : Integer,
  "horizontal_offset" : Integer (optional),
  "vertical_offset" : Integer (optional),
  "target_surface" : Integer (optional)
}

DELETE /api/multiview/layout/{name}/window/{index}
```

5.2.7.1 Arguments

The `name` argument is the name of the multiview layout.

The `index` argument is the index of the window, which must be between 0 and 31.

The `horizontal_position` and `vertical_position` arguments are respectively the horizontal and vertical start positions of the window on the screen, in pixels.

Note: The `horizontal_position` argument cannot be less than 8-pixels and is applied in increments of two pixels, so for example 8, 10, 12, etc.

The `width` and `height` arguments are respectively the width and height of the window in pixels.

Note: The `width` argument also cannot be less than 8-pixels and is applied in increments of two pixels, so for example 8, 10, 12, etc.

The `horizontal_offset` and `vertical_offset` arguments are respectively horizontal and vertical start positions of a window relative to start of a target surface, in pixels. For simple use cases, these are typically zero.

Note: The `horizontal_offset` argument is also a minimum of 8-pixels and is applied in increments of two pixels, so for example 8, 10, 12, etc.

The `horizontal_offset` and `vertical_offset` arguments can each be omitted, which is equivalent to specifying zero.

The `target_surface` argument (optional) is the index of the surface to which the window refers. If this `target_surface` argument is omitted or null, the window is black.

All arguments should be present when creating a window. There is two exceptions, the first is the `offset` argument which can be assumed to be zero (horizontal and vertical) when a new window is created and the second is the `target_surface` argument which is optional.

5.2.7.2 Return Value

Description of the window when creating refer to section 5.2.5 Command layout describe.

None when deleting.

HTTP status is:

201 -- Created when a new window was created (POST).
200 -- OK if an existing window was overwritten (POST).
200 -- OK on successful delete (DELETE).

5.2.7.3 Errors

<code>ILLEGAL_ARGUMENT</code>	Occurs when the arguments entered are invalid.
<code>MULTIVIEW_LAYOUT_STATE</code>	Occurs when the specified layout exists but is read-only.
<code>RESOURCE_NOT_FOUND</code>	Occurs if the specified layout does not exist.

5.2.7.4 Examples

Create a window with video content:

```
POST /api/multiview/layout/some_layout/window/0 HTTP/1.1
...
{
  "op" : "create",
  "horizontal_position" : 480,
  "vertical_position" : 270,
  "width" : 1920,
  "height" : 1080,
  "horizontal_offset" : 0,
  "vertical_offset" : 0,
  "target_surface " : 3
}
HTTP/1.1 201 Created
...
{
  "status" : "SUCCESS",
  "request_id" : null,
```

```

    "result" : {
      "layout_window" : {
        "index" : 0,
        "horizontal_position" : 480,
        "vertical_position" : 270,
        "width" : 1920,
        "height" : 1080,
        "horizontal_offset" : 0,
        "vertical_offset" : 0,
        "content" : "VIDEO",
        "target_surface" : 3
      }
    },
    "error" : null
  }
}

```

Modify an existing window and make it a black window (surface index omitted):

```

POST /api/multiview/layout/some_layout/window/0 HTTP/1.1
...
{
  "op" : "create",
  "horizontal_position" : 480,
  "vertical_position" : 270,
  "width" : 1920,
  "height" : 1080,
  "horizontal_offset" : 0,
  "vertical_offset" : 0
}
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "layout_window" : {
      "index" : 0,
      "horizontal_position" : 480,
      "vertical_position" : 270,
      "width" : 1920,
      "height" : 1080,
      "horizontal_offset" : 0,
      "vertical_offset" : 0,
      "content" : "BLACK",
      "target_surface" : 0
    }
  },
  "error" : null
}

```

Delete a window:

```
DELETE /api/multiview/layout/some_layout/window/0 HTTP/1.1
...
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : null,
  "error" : null
}
```

5.2.8 Command list layout

The `list` command provides sub-commands to query lists from the API server. The `command list layout` is described below. The `list layout` option lists all multiview layouts available on the API server.

Note: The `list firmware` and `list multicast` sub-commands were covered in section 4.11 Command list.

```
GET /api/multiview/layout
```

5.2.8.1 Return Value

```
{
  "layout" : [LayoutItemObject, ...]
}
```

Table 85: List Layout Object Array Member

Member Name	Since	Type	Description
layout	3.0	Array of LayoutItemObject	Array of objects representing the multiview layout.

Each item in the layout array (`LayoutItemObject`) has the following structure:

```
{
  "name" : String
}
```

Table 86: List Layout Object Member

Member Name	Since	Type	Description
name	3.0	String	Name of layout.

5.2.8.2 Example

```
GET /api/multiview/layout HTTP/1.1
...
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
```

```

        "layout" : [
            {
                "name" : "compatibility_4k_2x2"
            },
            {
                "name" : "example_4k_single_source_2x2_1080p"
            }
        ]
    }
}

```

5.2.9 Command set multiview

The `set multiview` command is used to set one or more receivers in multiview mode and load a layout.

```

POST /api/device/{target}

{
    "op" : "set:multiview",
    "layout" : String,
    "video" : VideoFormatArguments (conditional),
    "subscriptions" : [SubscriptionObject,...] (optional)
}

```

Notes:

- i. This command applies only to receiver devices.
- ii. Legacy BlueRiver NT devices have limited multiview capabilities. Only the predefined `compatibility_4k_2x2` layout is supported by these devices.

5.2.9.1 Arguments

The `target` argument must be either the device ID of a single receiver or the name of a device group. The allowed device groups are:

ALL	All devices
ALL_RX	All receiver devices

The `layout` argument specifies the multiview layout to be loaded.

The `video` argument specifies video format arguments, in regards to managing layouts this argument must be implicitly specified as `multiview`. Refer to 2.5 Specifying Video Output Format for more information on video output formats.

The optional `subscriptions` argument specifies an array of HDMI `SubscriptionObject` to be apply to the multiview layout.

Each item in the subscriptions array (`SubscriptionObject`) has the following structure:

```

{
    "source_device" : String,
    "stream_index" : Integer,
    "subscription_index" : Integer
}

```

Source_device	The source (transmitter) device ID
Stream_index	Index of the source HDMI stream
Subscription_index	Index of the destination subscription

5.2.9.2 Return Value

This is command modifies device settings. Refer to section 4.2 Commands that Modify Device Settings for a description of the return value.

5.2.9.3 Errors

ILLEGAL_ARGUMENT	Occurs when target argument entered is not valid device ID or group name.
ILLEGAL_ARGUMENT	Also occurs in the following cases: • There is no subscriptions in the subscription array (i.e. empty list) • A subscription source is not a valid device ID • A subscription source does not exist, is disconnected or is not a transmitter device • The source stream index is missing or is invalid • The subscription index is missing or is invalid • A source stream or a subscription index is duplicated
RESOURCE_NOT_FOUND	Occurs when the specified layout does not exist.
MULTIVIEW_LAYOUT_STATE	Occurs when the specified layout does not satisfy either the hardware or API limitations. Refer to section 5.1.1 Multiview Mode Guidelines & Limitations for details.

The request may also fail for individual devices for the following reasons:

DEVICE_TYPE	This error occurs if the specified device is not a receiver device.
DEVICE_TYPE	Also occurs when the device does not support the full multiview capabilities, such as a AVP1000-based device. In addition, this error is received if device is a legacy BlueRiver NT device and the specified layout is not the compatibility_4k_2x2 layout.
INVALID_STATE	Indicates that the source stream of a subscription is stopped.

5.2.9.4 Example

```
POST /api/device/010203040506 HTTP/1.1
...
{
  "op" : "set:multiview",
  "layout" : "compatibility_4k_2x2",
  "video" : {
    "width" : 1920,
    "height" : 1080,
    "fps" : 60
  }
}

HTTP/1.1 201 Created

Location: /api/request/100
```

```

...
{
    "status" : "PROCESSING",
    "request_id" : 100,
    "result" : null,
    "error" : null
}

```

5.3 Default Surface Configuration

For a specific multiview layout, the allocation of surfaces in a devices frame buffer can be customized with the layout surface command.

However, when a layout is first created (or reset) using the layout create command, a default surface configuration is applied. This default configuration , which is sufficient for most purposes, is as follows:

Surface 0 can receive video up to resolution of 4096 x 2160.

Surfaces 1 to 11 can receive video up to a resolution of 2048x1080.

Surfaces 12 to 16 can receive video up to resolution of 2048x856.

Surfaces 17 to 20 can receive video up to a resolution of 2048x1080.

- However, surfaces 17 to 20 use the same area of the frame buffer as surface 0.
- This means that surfaces 17 to 20 can be used instead of surface 0 if the multiview layout does not contain a source with a resolution over 2048x1080 but they cannot be used at the same time as surface 0.

5.4 Multiview Examples

Reminder: The command `set multiview` is applicable only to BlueRiver devices that support advanced video processing modes.

TIP: For more information on the implementation of multiview commands including outline of managing bandwidth and sample use cases, refer to the API Developers API Application Note Library.

Below is a couple of layout implementations, including creating the layout, configuring the windows size and positions and assigning the targets. In addition, these examples include the starting and subscribing of streams.

5.4.1 Picture-in-Picture Example

The example provided shows the implementation of a picture-in-picture layout, the script provided stops and starts all appropriate streams, sets the properties of the scaled stream of the appropriate transmitters, as well as creates and loads the layout. The layout being created is named, "PiP_Sample" and has a total output resolution of 3840 x 2160.

The receiver has the device ID of 0016c04c46d7. The larger window is set to a size of 3840 x 2160 to match the output resolution with a position of 0,0. Since we want the larger window to be behind the smaller window, its index is set to 1 and the smaller window's index is set to 0. This smaller window's size is 960 x 540 and its position is 400 x 400. Reminder the size and position are specified in pixels.

```

multiview/layout/PiP_Sample
{"op" : "create", "width" : 3840, "height" : 2160}
device/d88039e83991
{"op" : "stop", "stream_type" : "HDMI", "stream_index" : 0, "free" : True}

```

```

device/d88039e83991
{"op" : "set:scaler", "width" : 3840, "height" : 2160}
device/d88039e83991
{"op" : "start", "stream_type" : "HDMI", "stream_index" : 1}
device/d88039e55d9a
{"op" : "stop", "stream_type" : "HDMI", "stream_index" : 0, "free" : True}
device/d88039e55d9a
{"op" : "set:scaler", "width" : 960, "height" : 540}
device/d88039e55d9a
{"op" : "start", "stream_type" : "HDMI", "stream_index" : 1}
multiview/layout/PiP_Sample/window/1
{"op" : "create", "horizontal_position" : 0, "vertical_position" : 0, "width" :
3840, "height" : 2160, "horizontal_offset" : 0, "vertical_offset" : 0,
"target_surface" : 0}
multiview/layout/PiP_Sample/window/0
{"op" : "create", "horizontal_position" : 400, "vertical_position" : 400,
"width" : 960, "height" : 540, "horizontal_offset" : 0, "vertical_offset" : 0,
"target_surface" : 1}
device/0016c04c46d7
{"op" : "set:multiview", "layout" : "PiP_Sample", "video" : {"width" : 3840,
"height" : 2160, "fps" : 30}, "subscriptions" : [{"source_device" :
"d88039e83991", "stream_index":1,"subscription_index":0}, {"source_device" :
"d88039e55d9a", "stream_index":1,"subscription_index":1}]}

```

5.4.2 Quad Split Layout Example

In this second example, the implementation is of a layout containing four windows setup in a quad split view. Again the script provided stops and starts all appropriate streams, sets the scaled stream properties, as well as creates and loads the layout. The layout is named, “Quad_Sample” and has an output resolution of 3840 x 2160.

The receiver has the device ID of 0016c04c487e. Note that although each of the four windows is assigned the same window size, each has a unique position and surface ID assigned.

```

multiview/layout/Quad_Sample
{"op" : "create", "width" : 3840, "height" : 2160}
device/d88039e4c0fb
{"op" : "stop", "stream_type" : "HDMI", "stream_index" : 0, "free" : True}
device/d88039e4c0fb
{"op" : "set:scaler", "width" : 1920, "height" : 1080}
device/d88039e4c0fb
{"op" : "start", "stream_type" : "HDMI", "stream_index" : 1}
device/d88039e4c0fb
{"op" : "set:property", "key" :
"nodes[SCALER:0].inputs[main:0].configuration.source.value", "value" : 1}
device/d88039e5a57b
{"op" : "stop", "stream_type" : "HDMI", "stream_index" : 0, "free" : True}
device/d88039e5a57b
{"op" : "set:scaler", "width" : 1920, "height" : 1080}

```

```

device/d88039e5a57b
{"op" : "start", "stream_type" : "HDMI", "stream_index" : 1}
device/d88039e5a57b
{"op" : "set:property", "key" :
"nodes[SCALER:0].inputs[main:0].configuration.source.value", "value" : 1}
device/d88039e58621
{"op" : "stop", "stream_type" : "HDMI", "stream_index" : 0, "free" : True}
device/d88039e58621
{"op" : "set:scaler", "width" : 1920, "height" : 1080}
device/d88039e58621
{"op" : "start", "stream_type" : "HDMI", "stream_index" : 1}
device/d88039e58621
{"op" : "set:property", "key" :
"nodes[SCALER:0].inputs[main:0].configuration.source.value", "value" : 1}
device/d88039e83991
{"op" : "stop", "stream_type" : "HDMI", "stream_index" : 0, "free" : True}
device/d88039e83991
{"op" : "set:scaler", "width" : 1920, "height" : 1080}
device/d88039e83991
{"op" : "start", "stream_type" : "HDMI", "stream_index" : 1}
device/d88039e83991
{"op" : "set:property", "key" :
"nodes[SCALER:0].inputs[main:0].configuration.source.value", "value" : 1}
multiview/layout/Quad_Sample/window/0
{"op" : "create", "horizontal_position" : 0, "vertical_position" : 0, "width" :
1920, "height" : 1080, "horizontal_offset" : 0, "vertical_offset" : 0,
"target_surface" : 17}
multiview/layout/Quad_Sample/window/1
{"op" : "create", "horizontal_position" : 1920, "vertical_position" : 0,
"width" : 1920, "height" : 1080, "horizontal_offset" : 0, "vertical_offset" :
0, "target_surface" : 18}
multiview/layout/Quad_Sample/window/2
{"op" : "create", "horizontal_position" : 0, "vertical_position" : 1920,
"width" : 1920, "height" : 1080, "horizontal_offset" : 0, "vertical_offset" :
0, "target_surface" : 19}
multiview/layout/Quad_Sample/window/3
{"op" : "create", "horizontal_position" : 1920, "vertical_position" : 1080,
"width" : 1920, "height" : 1080, "horizontal_offset" : 0, "vertical_offset" :
0, "target_surface" : 20}
device/0016c04c46d7
{"op" : "set:multiview", "layout" : "Quad_Sample", "video" : {"width" : 3840,
"height" : 2160, "fps" : 30}, "subscriptions" : [{"source_device" :
"d88039e4c0fb", "stream_index":1,"subscription_index":17},{ "source_device" :
"d88039e5a57b", "stream_index":1,"subscription_index":18},{ "source_device" :
"d88039e58621", "stream_index":1,"subscription_index":19},{ "source_device" :
"d88039e83991", "stream_index":1,"subscription_index":20}]}
device/0016c04c4cc0
{"op" : "set:multiview", "layout" : "Quad_Sample", "video" : {"width" : 3840,

```



```
"height" : 2160, "fps" : 30}, "subscriptions" : [{"source_device" :  
"d88039e4c0fb", "stream_index":1,"subscription_index":17}, {"source_device" :  
"d88039e5a57b", "stream_index":1,"subscription_index":18}, {"source_device" :  
"d88039e58621", "stream_index":1,"subscription_index":19}, {"source_device" :  
"d88039e83991", "stream_index":1,"subscription_index":20}]]  
device/0016c04c3d9a  
{"op" : "set:multiview", "layout" : "Quad_Sample", "video" : {"width" : 3840,  
"height" : 2160, "fps" : 30}, "subscriptions" : [{"source_device" :  
"d88039e4c0fb", "stream_index":1,"subscription_index":17}, {"source_device" :  
"d88039e5a57b", "stream_index":1,"subscription_index":18}, {"source_device" :  
"d88039e58621", "stream_index":1,"subscription_index":19}, {"source_device" :  
"d88039e83991", "stream_index":1,"subscription_index":20}]]  
device/0016c04c487e  
{"op" : "set:multiview", "layout" : "Quad_Sample", "video" : {"width" : 3840,  
"height" : 2160, "fps" : 30}, "subscriptions" : [{"source_device" :  
"d88039e4c0fb", "stream_index":1,"subscription_index":17}, {"source_device" :  
"d88039e5a57b", "stream_index":1,"subscription_index":18}, {"source_device" :  
"d88039e58621", "stream_index":1,"subscription_index":19}, {"source_device" :  
"d88039e83991", "stream_index":1,"subscription_index":20}]]
```

6 Device Object Definition (DeviceObject)

A `device` object completely describes a device, including its configuration, state and supported features. A subset of this object is returned as part of the response to the `get` command. For more information, refer to the `get` command description in section 4.7 Command `get`.

```
{
  "device_id" : String,
  "identity" : DeviceIdentity,
  "capabilities" : DeviceCapabilities,
  "configuration" : DeviceConfig,
  "status" : DeviceStatus
  "streams" : [StreamObject, ...],
  "subscriptions" : [SubscriptionObject, ...]
  "nodes" : [Node, ...]
}
```

Table 87: List of Supported Device Objects

Member Name	Since	Type	Included In	Description
device_id	3.0	String	(always)	Device ID
identity	3.0	DeviceIdentity	get identity get settings	Provides information regarding the identity of the device.
capabilities	3.0	DeviceCapabilities	get device get capabilities	Provides information regarding the capabilities of the device.
configuration	3.0	DeviceConfig	get identity get settings	Provides information regarding the configurable settings of the device.
status	3.0	DeviceStatus	get settings	Provides information regarding the current status of the device.
streams	3.0	Array of StreamObject	get settings	Describes details regarding the streams of data being sent on the network.
subscriptions	3.0	Array of SubscriptionObject	get settings	Describes details regarding the streams of data being received from the network.
nodes	3.0	Array of Node	get settings get edid get gpio	Describes devices hardware modules including their status and configuration. • Included in response to <code>get settings</code>, <code>get edid</code> and <code>get gpio</code> commands. • However, in response to a <code>get edid</code>, only following node types are included as only nodes containing EDID data: HDMI_DECODER HDMI_MONITOR • The response of a <code>get gpio</code> command, only GPIO nodes are included. These nodes are not included in the response of a <code>get settings</code> command.

6.1 Generic Types

This section describes device objects that are generic JSON types that are re-used in various parts of a device object.

6.1.1 Video Format Description (VideoFormat)

A `video_format` object describes video format, including the resolution, bits per pixel, color space, frame rate, height, scan mode and width.

```
{
  "bits_per_pixel" : Integer,
  "color_space" : String,
  "frames_per_second" : Integer,
  "height" : Integer,
  "scan_mode" : String,
  "width" : Integer
}
```

Table 88: Video Format Description Object Members

Member Name	Since	Type	Description
bits_per_pixel	3.0	Integer	Number of bits per video component, one of: 8-bits 10-bits 12-bits
color_space	3.0	String	Color space, which is one of the following: RGB RGB graphics YCBCR_444 YcbCr 4:4:4 video YCBCR_422 YcbCr 4:2:2 video YCBCR_420 YcbCr 4:2:0 video
frames_per_second	3.0	Integer	Frame rate. Note: The value of this member is always an integer. This means for example, 59.94 fps is reported as 60. For more precision, consider using the <code>frame_rate</code> member of the <code>VideoFormatDetails</code> object instead.
height	3.0	Integer	Video height
scan_mode	3.0	String	Video scan mode, which is one of the following: PROGRESSIVE Returned for progressive video INTERLACED Returned for interlaced video
width	3.0	Integer	Video width

6.1.2 Video Format Detailed Information (VideoFormatDetails)

The video format detail information object contains detailed information about the video format. Some of the reported information is taken directly from the Auxiliary Video Information (AVI) InfoFrame. For additional information regarding the meaning of reported members, refer to CEA-861-F in section 6.4 “Format of Version 2 & 3 AVI InfoFrames”.

```
{
  "frame_rate" : Real
  "pixel_clock" : Integer,
  "total_width" : Integer,
  "total_height" : Integer,
  "hsync_width" : Integer,
  "hsync_front_porch" : Integer,
  "hsync_negative" : Boolean,
  "vsync_width" : Integer,
  "vsync_front_porch" : Integer,
  "vsync_negative" : Boolean,
  "vic" : Integer,
  "has_hdmi_vic" : Boolean,
  "hdmi_vic" : Integer,
  "picture_aspect" : String,
  "has_active_format" : Boolean,
  "active_format" : Integer,
  "it_content_type" : String,
  "scan_information" : String,
  "colorimetry" : String,
  "rgb_range" : String,
  "ycc_range" : String,
  "has_hdr" : Boolean,
  "hdr" : String
}
```

Table 89: Video Format Detailed Information Object Members

Member Name	Since	Type	Description
frame_rate	3.0	Real	Frame rate, includes fractional values (e.g. 59.54).
pixel_clock	3.0	Integer	Pixel clock, in hertz (Hz).
total_width	3.0	Integer	Total horizontal resolution including blanking.
total_height	3.0	Integer	Total vertical resolution including blanking.
hsync_width	3.0	Integer	Horizontal sync pulse width in pixels.
hsync_front_porch	3.0	Integer	Horizontal sync front porch in pixels.
hsync_negative	3.0	Boolean	Indicates whether horizontal sync has negative polarity.
vsync_width	3.0	Integer	Vertical sync pulse width, in pixels.
vsync_front_porch	3.0	Integer	Vertical sync front porch, in pixels.
vsync_negative	3.0	Boolean	Indicates whether vertical sync has negative polarity.
vic	3.0	Integer	AVI InfoFrame Video Information Code (VIC).
has_hdmi_vic	3.0	Boolean	Indicates whether there is an HDMI Vendor-Specific InfoFrame with an HDMI_VIC.

Member Name	Since	Type	Description
hdmi_vic	3.0	Integer	HDMI Vendor-Specific InfoFrame HDMI_VIC. Undefined if has_hdmi_vic is false.
picture_aspect	3.0	String	AVI InfoFrame picture aspect ratio: ASPECT_NO_DATA for No Data ASPECT_4_3 for a 4:3 aspect ratio ASPECT_16_9 for a 16:9 aspect ratio
has_active_format	3.0	Boolean	Indicates if the AVI InfoFrame specifies active format information.
active_format	3.0	Integer	AVI InfoFrame Active Format Description (AFD) code. For more information refer to the CEA-861-F Annex H.
it_content_type	3.0	String	AVI InfoFrame IT content type: NO_DATA for No Data GRAPHICS for Graphics type PHOTO for Photo type CINEMA for Cinema type GAME for Game type
scan_information	3.0	String	AVI InfoFrame scan information: NO_DATA for No Data OVERSCAN for overscan UNDERSCAN for underscan
colorimetry	3.0	String	AVI InfoFrame colorimetry: NO_DATA for No Data RGB for RGB BT601 for SMPTE 170M/ITU-R BT.601 BT709 for ITU-R BT.709 XVYCC601 for xvYCC ₆₀₁ XVYCC709 for xvYCC ₇₀₉ SYCC601 for sYCC ₆₀₁ ADOBE_YCC601 for Adobe _{YCC601} ADOBE_RGB for Adobe _{RGB} BT2020_YCC_CONST for ITU-R BT.2020 Y'C'B'C' _{RC} BT2020_YCC for ITU-R BT.2020 Y'C'B'C' _R BT2020_RGB for ITU-R BT.2020 R'G'B' This member combines information from the “RGB or YC _B C _R ” (Y), Colorimetry (C) and Extended Colorimetry (EC) fields of the AVI InfoFrame. See CEA-861-F Table 13.
rgb_range	3.0	String	AVI InfoFrame RGB quantization range: DEFAULT LIMITED FULL
ycc_range	3.0	String	AVI InfoFrame YCbCr quantization range: LIMITED FULL

Member Name	Since	Type	Description
has_hdr	3.0	Boolean	Indicates if High-Dynamic Range video format information is available from the device. Note: Regarding BlueRiver ASIC devices, HDR format detection is not available and has_hdr always returns false with hdr always reporting SDR.
hdr	3.0	String	High-Dynamic Range video format. Can be one of the following: SDR Standard dynamic range DV_LLAT_RGB10_12 Dolby Vision Low Latency mode 10/12-bit RGB 4:4:4 DV_LLAT_YCBCR10_12_444 Dolby Vision Low Latency mode 10/12-bit YCbCr 4:4:4 DV_LLAT_YCBCR12_422 Dolby Vision Low Latency mode 12-bit YCbCr 4:2:2 DV_STD_RGB8 Dolby Vision Standard mode Tunnel 8-bit RGB 4:4:4 DV_STD_YCBCR12 Dolby Vision Standard mode Native 12-bit YCbCr 4:2:2 HDR10 HDR10 - SMPTE ST 2084 HLG Hybrid Log Gamma – ITU-R BT.2100-0 This member is undefined if has_hdr member reported false. Reminder: Regarding BlueRiver ASIC devices, HDR format detection is not available and has_hdr always returns false with hdr always reporting SDR.

6.1.3 Audio Format Detailed Information (AudioFormatDetails)

The audio format detailed information object contains information concerning the audio format.

```
{
    "sampling_frequency" : Integer,
    "number_of_channels" : Integer
}
```

Table 90: Audio Format Detailed Information Object Members

Member Name	Since	Type	Description
sampling_frequency	3.0	Integer	The sampling frequency, in hertz (Hz).
number_of_channels	3.0	Integer	The number of audio channels

6.1.4 Audio Format Extended Information (AudioFormatExtended)

The audio format extended information object contains information describing the audio format.

```
{
    "channel_allocation" : Integer,
    "sample_size" : Integer,
    "level_shift_value" : Integer,
    "lfe_playback_level" : Integer,
    "audio_encoding_type" : String,
```

```

        "is_downmix_inhibited" : Boolean
    }

```

Table 91: Video Format Extended Information Object Members

Member Name	Since	Type	Description
channel_allocation	3.0	Integer	Channel allocation as reported in CEA-861-F audio InfoFrame. Note: Valid values and their meanings are defined in CEA-861-F Table 31.
sample_size	3.0	Integer	The size of each audio sample in bits. An integer value between 16 and 24 inclusive or zero (0) if it is undefined.
level_shift_value	3.0	Integer	Level Shift Value as reported in the CEA-861-F audio InfoFrame. Provides information on the downmix coefficients attenuation. Note: Valid values and their meanings are defined in CEA-861-F Table 32.
lfe_playback_level	3.0	Integer	The Low-Frequency Effect (LFE) playback level as reported in the CEA-861-F audio InfoFrame. Note: Refer to CEA-861-F Table 34 for valid values and their meanings.
audio_encoding_type	3.0	String	Specifies the type of audio encoding. LPCM Linear PCM audio. ENCODED_HBR High-Bitrate audio. ENCODED_OTHER Other type of audio, such as compressed.
is_downmix_inhibited	3.0	Boolean	Downmix inhibit as reported in CEA-861-F audio InfoFrame. If true, audio output is prohibited from being downmixed; otherwise downmixing permitted or no information available.

6.1.5 Source Reference Specification (SourceReference)

The source reference specification object is used in both streams and nodes to specify the origin of the data.

```

{
    "ref_class" : String,
    "ref_type" : String,
    "ref_index" : Integer
}

```

Table 92: Source Reference Specification Object Members

Member Name	Since	Type	Description
ref_class	3.0	String	The reference class reported, is one of the following: NODE The origin of the data is a node. SUBSCRIPTION The origin of the data is a subscription. In addition, the following value could be present in the configuration choices of a node input: AUTOMATIC Source is automatically selected. For example, when a valid signal is present.
ref_type	3.0	String	Reference type, matches type member of a node or subscription. Undefined if reference class is AUTOMATIC.

Member Name	Since	Type	Description
ref_index	3.0	Integer	Reference index, matches index member of a node or subscription. Undefined if reference class is <code>AUTOMATIC</code> .

6.1.6 Source Selection (`SourceSelection`)

The source selection object type is used in both streams and nodes to specify the origin of the data.

```
{
  "value" : Integer,
  "choices" : [SourceSelectionChoice, ...]
}
```

Table 93: Source Selection Object Members

Member Name	Since	Type	Description
value	3.0	Integer	Selected source, must match the <code>value</code> member of one of the enumerated choices.
choices	3.0	Array of <code>SourceSelectionChoice</code>	Enumeration of the allowed source selections. This member is not included in the response of a <code>get settings</code> command.

6.1.7 Source Selection Choice (`SourceSelectionChoice`)

The source selection choice object type is used in streams and nodes to specify the origin of the data.

```
{
  "value" : Integer,
  "description" : String
  "details" : SourceReference
}
```

Table 94: Source Selection Choice Object Members

Member Name	Since	Type	Description
value	3.0	Integer	Value of this choice.
description	3.0	String	Human-readable description of this choice.
details	3.0	SourceReference	Source reference associated with this choice.

6.2 Device Identity (`DeviceIdentity`)

The device identity object type contains information applicable to device identification.

Below is the identity for BlueRiver devices:

```
{
  "chipset_type" : String,
  "engine" : String,
  "vendor_id" : Integer,
  "product_id" : Integer,
  "firmware_comment" : String,
  "firmware_version" : String,
  "firmware_rc" : String,
  "is_receiver" : Boolean,
}
```



```

    "is_transmitter" : Boolean,
    "firmware_details" : [FirmwareDetails],
    "chip_id" : Integer (0)
}

```

Below is the identity for UEX devices, which are reported

- in the json return as a usb_extenders array of one or two items after the BlueRiver identity when the get identity command is sent to target AVP that has one or two attached UEX (UEX1000 or UEX2000)
- in the json return alone when the get identity command is sent to target UEX2000 used standalone:

```

{
    "chipset_type" : String,
    "engine" : String,
    "vendor_id" : Integer,
    "product_id" : Integer,
    "firmware_comment" : String,
    "firmware_version" : String,
    "chip_id" : Integer (0 or 1)
}

```

Table 95: Device Identity Object Members

Member Name	Since	Type	Included In	Description
chipset_type	3.0	String	get identity get settings	Identifies the type of the device chipset. Applicable to BlueRiver and UEX devices.
engine	3.0	String	get identity get settings	Identifies the type of processing engine the device uses: PLETHORA for current BlueRiver devices. BLUERIVER_NT reported for legacy BlueRiver NT chipset. USB2 . 0 reported for UEX chipset.
vendor_id	3.0	Integer	get identity get settings	Device vendor ID. Applicable to BlueRiver and UEX devices. This feature requires firmware support. If such support is not available, the reserved value 0 is reported.
product_id	3.0	Integer	get identity get settings	Device product ID. Applicable to BlueRiver and UEX devices. This feature requires firmware support. If such support is not available, the reserved value 0 is reported.
firmware_comment	3.0	String	get identity get settings	Comment string embedded in firmware file that was programmed in the device. Applicable to BlueRiver and UEX devices. This comment typically contains a version string but the exact format and content is device manufacturer-specific. May contain an empty string if no comment is set.

Member Name	Since	Type	Included In	Description
firmware_version	3.0	String	get hello get identity get settings	Device firmware version. Applicable to BlueRiver and UEX devices.
firmware_rc	3.0	String	get hello get identity get settings	Device firmware release candidate. Applicable to BlueRiver devices. Note: For Semtech AptoVision Product Group internal use only.
is_receiver	3.0	Boolean	get hello get identity get settings	True is the device is a receiver device. Applicable to BlueRiver devices.
is_transmitter	3.0	Boolean	get hello, get identity get settings	True is the device is a transmitter device. Applicable to BlueRiver devices.
firmware_details	3.0	Array of FirmwareDetails	get identity get settings	Provide information regarding the various firmware images of the device. Applicable to BlueRiver devices.
chip_id	3.0	Integer	get identity get settings	Provides the chip ID. Applicable to BlueRiver and UEX devices.

A FirmwareDetails object has the following structure:

```
{
  "type" : String,
  "details_available" : Boolean,
  "firmware_version" : String,
  "firmware_rc" : String
}
```

Table 96: Firmware Details Object Members

Member Name	Since	Type	Description
type	3.0	String	Type of firmware image being described. One of the following: PRIMARY for the primary firmware image. GOLDEN for the golden firmware image.
details_available	3.0	Boolean	Whether information is available for the firmware image.
firmware_version	3.0	String	Version of the firmware image. If the value of the details_available member is false, this value is undefined.
firmware_rc	3.0	String	Release candidate number of the firmware image. If the value of the details_available member is false, this value is undefined. Note: BlueRiver support internal use only.

6.3 Device Capabilities (DeviceCapabilities)

The `device_capabilities` object type contains information applicable to device capabilities. There are two types of capabilities. Boolean capabilities where true means “supported” and false means “not supported”. Integer capabilities indicating the number of physical occurrences of a capability.

Below are the capabilities for BlueRiver devices:

```
{
  "color_generator_force" : Boolean,
  "dhcp_option_43" : Boolean,
  "dot1x" : Boolean,
  "external_video_compression_supported": Boolean,
  "external_video_decompression_supported": Boolean,
  "frame_rate_divider" : Boolean,
  "freeze_video": Boolean, "hdmi_audio_downmix" : Boolean,
  "hdmi_inputs" : Integer,
  "hdmi_outputs" : Integer,
  "hdmi_stream1_auto_stop" : Boolean,
  "infrared_inputs" : Integer,
  "infrared_outputs" : Integer,
  "local_loopback" : Boolean,
  "multich_audio_inputs" : Integer,
  "multich_audio_outputs" : Integer,
  "nbaset" : Boolean,
  "net10gbe_dual_link" : Boolean,
  "overlay" : Boolean,
  "scaler" : Boolean,
  "send_wol" : Boolean,
  "stereo_audio_inputs" : Integer,
  "stereo_audio_input_outputs" : Integer,
  "stereo_audio_outputs" : Integer,
  "thumbnail" : Boolean,
  "uarts" : Integer,
  "usb_hid" : Boolean,
  "usb_extender" : Boolean,
  "video_processing" : Boolean
},
```

Below are the capabilities for UEX devices, which are reported

- in the json return as a `usb_extenders` array of one or two items after the BlueRiver capabilities when the `get capabilities` command is sent to target AVP that has one or two attached UEX (UEX1000 or UEX2000)
- in the json return alone when the `get capabilities` command is sent to target UEX2000 used standalone:

```
{
  "local_role_supported" : Boolean,
  "remote_role_supported" : Boolean,
  "legacy_mode" : Boolean
},
```

Table 97: Device Capabilities Object Members

Member Name	Since	Type	Description
color_generator_force	3.0	Boolean	Whether receiver without video processing capability has the ability to blank its video output. Affects the following property: <code>nodes[COLOR_GENERATOR:index].configuration.force_output</code>
dhcp_option_43	3.7	Boolean	Whether the device supports sending "HELLO" packets to IP of DHCP option 43.
dot1x	3.7	Boolean	Whether device supports 802.1X authentication.
external_video_compression_supported	3.9	Boolean	Whether the external HDMI transceiver chip supports video compression for FlexLinQ (e.g. if Lontium LT86121GXE is used).
external_video_decompression_supported	3.9	Boolean	Whether the external HDMI transceiver chip supports video decompression for FlexLinQ (e.g. if Lontium LT86121GXE is used).
frame_rate_divider	3.0	Boolean	Whether device supports dividing frame rate by a factor of 2. Applicable to the properties of stream 0 or stream 1, as follows: Native stream (stream 0) <code>streams[HDMI:index].configuration.source.value</code> Scaled stream (stream 0) <code>nodes[SCALER:0].inputs[main:0].configuration.source.value</code>
freeze_video	3.9	Boolean	Whether receiver device supports freeze video on command (e.g. to manually force a freeze of the receiver video output during a change of the video source at the HDMI input of the currently subscribed transmitter device).
hdmi_audio_downmix	3.0	Boolean	Whether device produces a downmix of its HDMI audio. Affects: - Whether device can loop out the HDMI audio downmix to one of its local audio outputs. - Whether a receiver subscribed to this devices HDMI_AUDIO stream can output the HDMI audio downmix to one of its audio outputs. Affects the following properties: <code>nodes[HDMI_DECODER:0].inputs[main:0].configuration.source.value</code> <code>nodes[MULTICH_AUDIO_OUTPUT:0].inputs[main:0].configuration.source.value</code> <code>nodes[STEREO_AUDIO_INPUT_OUTPUT:0].inputs[main:0].configuration.source.value</code> <code>nodes[STEREO_AUDIO_OUTPUT:0].inputs[main:0].configuration.source.value</code>
hdmi_inputs	3.0	Integer	Number of HDMI video inputs.
hdmi_outputs	3.0	Integer	Number of HDMI video outputs.
hdmi_stream1_auto_stop	3.0	Boolean	Whether device supports the HDMI stream 1 auto stop feature. Affects the following property: <code>streams[HDMI:1].configuration.auto_stop</code>
infrared_inputs	3.0	Integer	Number of infrared receiver connectors.
infrared_outputs	3.0	Integer	Number of infrared emitter connectors.
local_loopback	3.0	Boolean	Whether device supports subscribing to its own video and audio streams.

Member Name	Since	Type	Description
			Affects: • <code>join</code> command when the source and destination devices are the same.
<code>multich_audio_inputs</code>	3.0	Integer	Number of multichannel audio inputs.
<code>multich_audio_outputs</code>	3.0	Integer	Number of multichannel audio outputs.
<code>nbaset</code>	3.0	Boolean	Whether NBase-T is supported by the device.
<code>net10gbe_dual_link</code>	3.0	Boolean	Whether this is a dual link 10GbE copper+fiber device.
<code>overlay</code>	3.0	Boolean	Whether the monochrome bitmap overlay is supported. Affects: • The <code>set overlay</code> command.
<code>scaler</code>	3.0	Boolean	Supports scaling the video from the HDMI input and sending the scaled video on a separate stream (HDMI stream 1). Affects: • The <code>set scaler</code> command. • The <code>start</code> command when stream type is HDMI and stream index is 1.
<code>send_wol</code>	3.7	Boolean	Whether device supports sending Wake-on-Lan packet
<code>stereo_audio_inputs</code>	3.0	Integer	Number of stereo audio inputs.
<code>stereo_audio_input_outputs</code>	3.0	Integer	Number of stereo audio input/outputs.
<code>stereo_audio_outputs</code>	3.0	Integer	Number of stereo audio outputs.
<code>thumbnail</code>	3.0	Boolean	Whether the device supports sending a thumbnail stream. Affects: • The <code>set thumbnail</code> command. • The <code>start</code> command when the stream type is THUMBNAIL.
<code>uarts</code>	3.0	Integer	Number of enabled UARTs.
<code>usb_hid</code>	3.0	Boolean	Whether USB HID is supported. Affects: • The <code>usb</code> command. • The <code>switch</code> and <code>stop</code> command when stream type is USB_HID.
<code>usb_extender</code>	3.8	Boolean	Whether device has a Semtech UEX or Icron USB extender. Affects: • The <code>usb_extender</code> command. • The <code>switch</code>, <code>stop</code> and <code>pair/unpair</code> commands when stream type is USB_EXTENDER.
<code>video_processing</code>	3.0	Boolean	Whether the receiver supports video processing. This includes support for video processing display modes, such as fast switch, multiview and wall. Affects: • The <code>set video</code> command, specifically the ability to specify a display mode other than genlock. • The <code>set multiview</code> command.
<code>local_role_supported</code>	3.8	Boolean	Whether the UEX device supports the LOCAL role.
<code>remote_role_supported</code>	3.8	Boolean	Whether the UEX device supports the REMOTE role.
<code>legacy_mode</code>	3.8	Boolean	Whether the UEX device is in legacy mode (legacy_mode must be the same between devices for compatibility, if set to true then no advanced features are available).

6.4 Device configuration (DeviceConfig)

The device configuration object holds configurable device, such as the device name.

```
{
    "device_name" : String,
    "locate_mode" : Boolean,
    "secure_av" : String,
    "resume_streaming" : Boolean,
    "claimed" : Boolean,
    "authenticated" : Boolean,
    "device_mode" : String,
    "dot1x_certificate" : Boolean,
    "unidirectional_p2p_only" : Boolean
}
```

Table 98: Device Configuration Object Members

Member Name	Since	Type	Included in	Description
device_name	3.0	String	get identity get settings	Name of the device. Applicable to BlueRiver and UEX devices.
locate_mode	3.0	Boolean	get settings	Whether the device is in locate mode. Applicable to BlueRiver and UEX devices. TIP: When in locate mode, all device LEDs flash so it can be located easily.
secure_av	3.0	String	get settings	secure av “default” or “user_defined”. Applicable to BlueRiver devices.
resume_streaming	3.0	Boolean	get settings	Indicates whether a device is to automatically restart streams and subscriptions after a reboot. Applicable to BlueRiver devices.
claimed	3.0	Boolean	get settings	Indicates if device has been exclusively claimed by a API server. Applicable to BlueRiver and UEX devices.
authenticated	3.2	Boolean	get settings	TRUE indicates device is authenticated bound to an API server. FALSE indicates the device is not. Refer to the SDVoE Control Server Administrators Guide for more information on this option. Applicable to BlueRiver AVP devices.
device_mode	3.1	String	get settings	Indicates whether the configured device mode is TRANSMITTER, RECEIVER or TRANSCEIVER. Applicable to BlueRiver devices.
dot1x_certificate	3.7	Boolean	get settings	Indicates whether the device is configured with a 802.1X certificate. Applicable to BlueRiver AVP devices.
volatile_settings	3.6	Boolean	get settings	Indicates whether the device settings management is configured in volatile mode. Applicable to BlueRiver and UEX devices (when target is AVP it will be applied to the AVP and its attached UEX, for UEX2000 standalone the target shall be the UEX).

Member Name	Since	Type	Included in	Description
unidirectional_p2p_only	3.9	Boolean	get settings	Indicates whether the device only supports unidirectional point to point fiber transmission, allowing transport of video and audio from transmitter device to receiver device and blocking data to return from receiver device to transmitter device.

6.5 Device Status (DeviceStatus)

The `device_status` object contains information about device states, such as if device is active on the network and connection state.

```
{
    "active" : Boolean,
    "point_to_point" : Boolean,
    "point_to_point_peer" : String,
    "boot_status" : String,
    "update_in_progress" : Boolean,
    "igmp_reporting_mode" : String,
    "error_status" : DeviceErrorStatus,
    "temperature" : Integer
}
```

Table 99: Device Status Object Members

Member Name	Since	Type	Description
active	3.0	Boolean	Indicates if device is connected to the network. Applicable to BlueRiver and UEX devices.
point_to_point	3.0	Boolean	Indicates if device is in point-to-point mode. Applicable to BlueRiver and UEX devices.
point_to_point_peer	3.0	String	Device ID of the peer device. Applicable to BlueRiver and UEX devices. Is set to NULL when <code>point_to_point</code> is false or the peer device does not support this feature.
boot_status	3.0	String	Firmware image device booted with. Applicable to BlueRiver devices. One of the following: PRIMARY for the primary image GOLDEN for the golden (fallback) image UNKNOWN if active firmware cannot be determined because device firmware does not support this feature. If the value reported is GOLDEN, this indicates that the primary firmware image is corrupted and device may need to be re-programmed.
update_in_progress	3.0	Boolean	Indicates if a firmware update (<code>update</code> command) is running on the device. Applicable to BlueRiver and UEX devices. Once the firmware update is complete, the device must be rebooted, at which point the value of this

Member Name	Since	Type	Description
igmp_reporting_mode	3.1	String	member is reset to <code>false</code> . Indicates whether the IGMP reporting mode is <code>INTERVAL</code> (sent unsolicited every minute) or <code>QUERY</code> (sent as a response to the IGMP snooping queries from the switch). Applicable to BlueRiver devices.
error_status	3.0	DeviceErrorStatus	Indicates if the device is reporting error code. Applicable to BlueRiver and UEX devices.
temperature	3.0	Integer	Current junction temperature of BlueRiver device, reported in degrees Celsius. Applicable to BlueRiver devices. Note: Only included in response to either a <code>get temperature</code> or <code>get device</code> command.

A `DeviceErrorStatus` object has the following structure:

```
{
  "has_error_code" : Boolean,
  "error_code " : Integer
}
```

Table 100: Device Error Status Object Members

Member Name	Since	Type	Description
has_error_code	3.0	Boolean	Indicates if the device is reporting an error code.
error_code	3.0	Integer	If <code>has_error_code</code> member is <code>true</code> , this indicates that the member contains an error code reported by the device. Otherwise, the value of this member is undefined. Reminder: Meaning of error code device-dependent.

6.6 Streams (StreamObject)

Each individual stream has a `stream` object. This object specifies the stream type, its configuration as well as the streams origin.

```
{
  "type" : String,
  "index" : Integer,
  "configuration" : StreamConfig,
  "status" : StreamStatus,
  "source" : SourceReference
}
```

Table 101: Stream Object Members

Member Name	Since	Type	Description
type	3.0	String	Type of data contained in the stream. STEREO_AUDIO Stereo audio CEC CEC data HDMI HDMI video HDMI_AUDIO HDMI audio

Member Name	Since	Type	Description
			MULTICH_AUDIO Multichannel audio INFRARED Infrared remote-control data RS232 RS-232 data THUMBNAIL Video Thumbnail
index	3.0	Integer	Index of the stream of the given type (streams are indexed individually by type, not necessarily contiguous)
configuration	3.0	StreamConfig	Stream configuration
source	3.0	SourceReference	Identifies the origin of the streamed data

6.6.1 Stream Configuration (StreamConfig)

The `stream configuration` object is a sub-object of the `stream` object and is located within the `stream` object itself.

```
{
  "address" : String,
  "enable" : Boolean,
  "auto_stop" : Boolean,
  "remove_audio" : Boolean,
  "source" : SourceSelection
}
```

Table 102: Stream Configuration Object Members

Member Name	Since	Type	Description
address	3.0	String	IP address that the data is to be sent to.
enable	3.0	Boolean	Whether the stream is enabled or not.
auto_stop	3.0	Boolean	When <code>true</code> , the stream stops automatically whenever the video source is lost or a resolution change occurs. Stream must then be started with <code>start</code> command. Option useful in managing network bandwidth. A video resolution change leads to change in bandwidth usage by the video. It could be appropriate to take an action in response before restarting the stream. For example, enable or disable compression. Always <code>false</code> for streams other than scaled stream (HDMI : 1).
remove_audio	3.0	Boolean	When <code>true</code> , HDMI audio is not sent with the stream. Reminder: audio only present on native stream, HDMI 0.
source	3.0	SourceSelection	Selection of the input source (optional). This member is only present if the input source is configurable on the specified stream.

6.6.2 Stream Status (StreamStatus)

The `stream status` is a sub-object within the `stream` object containing details of a stream's status.

```
{
  "state" : String,
}
```

Table 103: Stream Status Object Member

Member Name	Since	Type	Description
state	3.0	String	State of the subscription: STREAMING Subscription is running STOPPED Subscription is stopped LINK_DOWN The 10-Gigabit network interface is down

6.7 Subscriptions (SubscriptionObject)

The `subscription` object contains information about stream subscriptions, that is which devices have subscribed to receive which streams.

```
{
  "type" : String,
  "index" : Integer,
  "configuration" : SubscriptionConfig
  "status" : SubscriptionStatus
}
```

Table 104: Subscriptions Object Members

Member Name	Since	Type	Description
type	3.0	String	Type of data, same values as for stream type. STEREO_AUDIO Stereo audio CEC CEC data HDMI HDMI video HDMI_AUDIO HDMI audio MULTICH_AUDIO Multichannel audio INFRARED Infrared remote-control data RS232 RS-232 data
index	3.0	Integer	Index of subscription of given type. Note: Subscriptions indexed individually by type, therefore not necessarily contiguous.
configuration	3.0	SubscriptionConfig	Subscription configuration.
source	3.0	SubscriptionStatus	Subscription status.

6.7.1 Subscription Configuration (SubscriptionConfig)

The `subscription configuration` object is a sub-object inside of the subscription object.

```
{
  "address" : String,
  "enable" : Boolean,
  "mute_msec" : Integer,
}
```

Table 105: Subscription Configuration Object Members

Member Name	Since	Type	Description
address	3.0	String	IP address from which to receive the data.
enable	3.0	Boolean	Whether the subscription is enabled or not

Member Name	Since	Type	Description
mute_msec	3.0	Integer	Amount of milliseconds firmware mutes on a HDMI_AUDIO subscription change. Value must be between 0 and 5000 inclusively (present for HDMI_AUDIO subscription only).

6.7.2 Subscription Status (SubscriptionStatus)

The `subscription status` object is also a sub-object inside of the subscription object.

```
{
  "state" : String,
  "source" : String
}
```

Table 106: Subscription Status Object Members

Member Name	Since	Type	Description
state	3.0	String	State of the subscription: STREAMING Subscription is running. STOPPED Subscription is stopped. LINK_DOWN 10-Gigabit network interface is down. Other values could occur, these are transient states that occur immediately after a configuration change and should be ignored.
source	3.6	String	Source data, specifically the <code>device_id</code> , <code>device_name</code> and <code>stream_index</code> , provided in the subscription status node.

6.8 Nodes (Node)

The `node` object describes hardware modules present on the device along with their status and configuration.

```
{
  "type" : String,
  "index" : Integer,
  "configuration" : Object,
  "status" : Object,
  "inputs" : [NodeInput, ...]
}
```

Table 107: Node Object Members

Member Name	Since	Type	Description
type	3.0	String	Type of node. STEREO_AUDIO_INPUT STEREO_AUDIO_INPUT_OUTPUT STEREO_AUDIO_OUTPUT CEC COLOR_GENERATOR FRAME_BUFFER FRAME_RATE_CONVERTER HDMI_DECODER HDMI_ENCODER HDMI_MONITOR MULTICH_AUDIO_INPUT MULTICH_AUDIO_OUTPUT INFRARED_DECODER INFRARED_ENCODER NETWORK_INTERFACE NETWORK_PORT NETWORK_SWITCH SCALER UART USB_HID USB_EXTENDER VIDEO_COMPRESSOR VIDEO_DECOMPRESSOR VIDEO_INPUT
index	3.0	Integer	Index of the node of the given type. Nodes are indexed independently for each type, not necessarily contiguous.
configuration	3.0	Object	Node configuration. The structure of this object varies according to the node type. Refer to the node type-specific sections below.
status	3.0	Object	Node status. The structure of this object varies according to the node type. Refer to the node type-specific sections below.
inputs	3.0	Array of NodeInput	Array describing how the node is connected to upstream nodes and/or subscriptions.

6.8.1 Node Inputs (NodeInput)

The node `input` is a sub-object inside a node object describing how it is connected to other nodes.

```
{
  "name" : String
  "index" : Integer,
  "configuration" : NodeInputConfig,
  "status" : NodeInputStatus,
}
```

Table 108: Node Input Object Members

Member Name	Since	Type	Description
name	3.0	String	Input name, allowable values depend on node type
index	3.0	Integer	Index of the input with the given name. Inputs are indexed independently for each name.
configuration	3.0	NodeInputConfig	Input configuration
source	3.0	NodeInputStatus	Input status

6.8.1.1 Node Input Configuration (NodeInputConfig)

The `node input configuration` member describes the node inputs.

```
{
  "source" : SourceSelection
}
```

Table 109: Node Input Configuration Object Member

Member Name	Since	Type	Description
source	3.0	SourceSelection	(Optional) Selection of the input source. Only present if input source is configurable on specified input.

6.8.1.2 Node Input Status (NodeInputStatus)

The `node input status` member describes the node input status.

```
{
  "source" : SourceReference
}
```

Table 110: Node Input Status Object Member

Member Name	Since	Type	Description
source	3.0	SourceReference	Current input source.

6.8.2 Stereo Audio Input Node (STEREO_AUDIO_INPUT)

This section describes a Node object with `type` member set to `STEREO_AUDIO_INPUT`.

6.8.2.1 Configuration

None

6.8.2.2 Status

None

6.8.2.3 Inputs

None

6.8.2.4 Example

Example shows a stereo audio input node JSON object returned as part of the output of `get device`.

```
{
  "type" : "STEREO_AUDIO_INPUT",
  "index" : 0,
  "configuration" : {},
  "status" : {},
  "inputs" : []
}
```

For more information on the `get device` command, refer to Section 4.7 Command `get` for details.

6.8.3 Stereo Audio Input/Output Node (STEREO_AUDIO_INPUT_OUTPUT)

This section describes a Node object with `type` member set to `STEREO_AUDIO_INPUT_OUTPUT`. This node represents the stereo audio port, which can serve as either an input and an output port.

6.8.3.1 Configuration (StereoAudioInputOutputNodeConfig)

```
{
  "direction" : String,
}
```

Table 111: Stereo Audio Input/Output Object Member

Member Name	Since	Type	Description
direction	3.0	String	Port direction, which is one of the following: INPUT The port is set to be an audio input. OUTPUT The port is set to be an audio output.

6.8.3.2 Status

None

6.8.3.3 Inputs

Table 112: Inputs Supported for Audio Input/Output Node

Input Name	Since	Configurable	Description
main	3.0	Yes	Audio input source. A <code>transmitter</code> may support none, some or all of the following: 2 -- Local HDMI audio (stereo downmix) 8 -- Multichannel audio subscription (audio return channel) 9 -- Local Multichannel audio 13 -- Stereo audio subscription 15 -- HDMI audio subscription (stereo downmix) A <code>receiver</code> may support none, some or all of the following: 2 -- HDMI audio subscription (stereo downmix) 3 -- Stereo audio subscription 8 -- Multichannel audio subscription 9 -- Local Multichannel audio Note: Not all audio types are available to all endpoint devices, supported

Input Name	Since	Configurable	Description
			audio is based on the hardware design and device capabilities. Refer to the SDVoE Developers API Application Note Library for more information. A <code>transceiver</code> device may support none, some or all of the following: 2 -- HDMI audio subscription (stereo downmix) 3 -- Stereo audio subscription 8 -- Multichannel audio subscription 9 -- Local Multichannel audio 12 -- Local HDMI audio (stereo downmix) If device does not support any of above, then following is listed: 0 -- No audio source available Attempting to set this last value as selected input using a <code>set</code> property command results in <code>ILLEGAL_ARGUMENT</code> error.

6.8.3.4 Example

This example shows an stereo audio input/output node JSON object as returned as part of the output of the `get device` command.

```
{
  "type" : "STEREO_AUDIO_INPUT_OUTPUT",
  "index" : 0,
  "configuration" : {
    "direction" : "INPUT"
  },
  "status" : {},
  "inputs" : [
    {
      "name" : "main",
      "index" : 0,
      "configuration" : {},
      "status" : {
        "source" : {
          "ref_class" : "NODE",
          "ref_type" : "HDMI_DECODER",
          "ref_index" : 0
        }
      }
    }
  ]
}
```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.4 Stereo Audio Output Node (STEREO_AUDIO_OUTPUT)

This section describes a Node object with the `type` member set to `STEREO_AUDIO_OUTPUT`.

6.8.4.1 Configuration

None

6.8.4.2 Status

None

6.8.4.3 Inputs

Table 113: Outputs Supported for Audio Output Node

Input Name	Since	Configurable	Description
main	3.0	Yes	Audio source. A transmitter may support none, some or all of the following: 2 -- Local HDMI audio (stereo downmix) 8 -- Multichannel audio subscription (audio return channel) 9 -- Local Multichannel audio 13 -- Stereo audio subscription 15 -- HDMI audio subscription (stereo downmix) A receiver may support none, some or all of the following: 2 -- HDMI audio subscription (stereo downmix) 3 -- Stereo audio subscription 8 -- Multichannel audio subscription 9 -- Local Multichannel audio Note: Not all audio types are available to all endpoint devices, supported audio is based on the hardware design and device capabilities. Refer to the SDVoE Developers API Application Note Library for more information. A transceiver device may support none, some or all of the following: 2 -- HDMI audio subscription (stereo downmix) 3 -- Stereo audio subscription 8 -- Multichannel audio subscription 9 -- Local Multichannel audio 12 -- Local HDMI audio (stereo downmix)

6.8.4.4 Example

Example shows a stereo audio output node JSON object returned as part of the output of `get device`.

```
{
  "type" : "STEREO_AUDIO_OUTPUT",
  "index" : 0,
  "configuration" : {},
  "status" : {},
  "inputs" : [
    {
```



```

"name" : "main",
"index" : 0,
"configuration" : {
  "source" : {
    "value" : 3,
    "choices" : [
      {
        "value" : 2,
        "description" : "HDMI Audio (Downmix)",
        "details" : {
          "ref_class" : "SUBSCRIPTION",
          "ref_type" : "HDMI_AUDIO",
          "ref_index" : 0
        }
      },
      {
        "value" : 3,
        "description" : "Stereo Audio",
        "details" : {
          "ref_class" : "SUBSCRIPTION",
          "ref_type" : "STEREO_AUDIO",
          "ref_index" : 0
        }
      },
      {
        "value" : 8,
        "description" : "Multi Channel Audio",
        "details" : {
          "ref_class" : "SUBSCRIPTION",
          "ref_type" : "MULTICH_AUDIO",
          "ref_index" : 0
        }
      },
      {
        "value" : 9,
        "description" : "Multi Channel Audio Local Loop
Out",
        "details" : {
          "ref_class" : "NODE",
          "ref_type" : "MULTICH_AUDIO_INPUT",
          "ref_index" : 0
        }
      }
    ]
  }
},
"status" : {
  "source" : {
    "ref_class" : "SUBSCRIPTION",
    "ref_type" : "STEREO_AUDIO",
    "ref_index" : 0
  }
}

```

```

    }
  }
]
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.5 Bitmap Overlay Node (**BITMAP_OVERLAY**)

This section describes a Node object with the `type` member set to `BitmapOverlayNodeConfig`. This node is only applicable to NT2000 endpoint devices.

6.8.5.1 Configuration (**BitmapOverlayNodeConfig**)

```

{
  "enable" : Boolean,
  "width" : Integer,
  "height" : Integer,
  "horiz_position" : Integer,
  "vert_position" : Integer,
  "scaling_factor" : Integer,
  "foreground_color" : String,
  "background_color" : String,
  "foreground_transparency" : Integer,
  "background_transparency" : Integer
}

```

Table 114: Bitmap Overlay Node Object Members

Member Name	Since	Type	Description
<code>enable</code>	2.18	Boolean	Whether bitmap overlay is enabled or disabled.
<code>width</code>	2.18	Integer	Width of the overlay bitmap.
<code>height</code>	2.18	Integer	Height of the overlay bitmap.
<code>horiz_position</code>	2.18	Integer	Horizontal position, in pixels, of the overlay
<code>vert_position</code>	2.18	Integer	Vertical position, in pixels, of the overlay
<code>scaling_factor</code>	2.18	Integer	Scaling factor (1, 2 or 4)
<code>foreground_color</code>	2.18	String	Foreground colour in format RRGGBB.
<code>background_color</code>	2.18	String	Background colour in format RRGGBB.
<code>foreground_transparency</code>	2.18	Integer	Foreground transparency level in percent.
<code>background_transparency</code>	2.18	Integer	Background transparency level in percent.

6.8.5.2 Status

None

6.8.5.3 Inputs

None

6.8.5.4 Example

The example below shows a bitmap overlay node JSON object as returned as part of the output of a `get device` request for an NT2000 endpoint device.

```
{
  "type" : "BITMAP_OVERLAY",
  "index" : 0,
  "configuration" : {
    "enable" : true,
    "width" : 100,
    "height" : 100,
    "horiz_position" : 1720,
    "vert_position" : 880,
    "scaling_factor" : 1,
    "foreground_color" : "0000aa",
    "background_color" : "ffffff",
    "foreground_transparency" : 0,
    "background_transparency" : 70
  },
  "status" : {},
  "inputs" : []
}
```

6.8.6 CEC Node (CEC)

Below describes a `Node` object with `type` member set to `CEC`, represents an input source to CEC streams.

6.8.6.1 Configuration

```
{
  "p2p_mode" : String
}
```

Table 115: CEC Node P2P Configuration Object Member

Member	Since	Type	Description
p2p_mode	3.2	String	If BlueRiver device is operating in Point-to-Point (<code>p2p_mode</code>) mode, CEC Node can be configured to operate in API or Passthrough (default) mode. When operating in PASSTHROUGH, CEC data passed between paired devices. When in API mode CEC data can be injected from API to supported logical addresses. The default is "14" (0x0E – Free use). One of the following logical addresses will be reported: 0_tv 1_recording_device_1 2_recording_device_2 3_tuner_1 4_playback_device_1 5_audio_system 6_tuner_2 7_tuner_3 8_playback_device_2 9_recording_device_3

Member	Since	Type	Description
			10_tuner_4
			11_playback_device_3
			12_reserved
			13_reserved
			14_free_use

6.8.6.2 Status

None

6.8.6.3 Inputs

None

6.8.6.4 Example

Example shows a null source node JSON object returned as part of the output of a “get device” command.

```
{
  "type": "CEC",
  "index": 0,
  "configuration": {
    "p2p_mode": "PASSTHROUGH"
    "logical_addresses": {
      "0_tv": true,
      "1_recording_device_1": false,
      "2_recording_device_2": false,
      "3_tuner_1": false,
      "4_playback_device_1": false,
      "5_audio_system": false,
      "6_tuner_2": false,
      "7_tuner_3": false,
      "8_playback_device_2": false,
      "9_recording_device_3": false,
      "10_tuner_4": false,
      "11_playback_device_3": false,
      "12_reserved": false,
      "13_reserved": false,
      "14_free_use": false
    }
  },
  "status": {},
  "inputs": []
},
```

Reminder: For BlueRiver transceiver devices, there is two CEC nodes shown 0 and 1. The p2p_mode value of the second CEC node, index:1.

For more information on the get device command refer to Section 4.7 Command get for details.

6.8.7 Color Generator Node (COLOR_GENERATOR)

This section describes a Node object with `type` member set to `COLOR_GENERATOR`.

6.8.7.1 Configuration

```
{  
  "enable" : Boolean,  
  "color" : String,  
  "force_output" : Boolean  
}
```

Table 116: Color Generator Node Object Members

Member Name	Since	Type	Description
enable	3.0	Boolean	When <code>true</code> , blank video output with a uniform color, used to to implement video mute functionality. Note: Only supported for receivers (RX) operating in a display mode other than genlock, therefore supported by AVP2000(T) devices but not AVP1000.
color	3.0	String	String representation of the color to use for blanking. String format is RRGGBB, where RR, GG and BB are each a pair of hexadecimal digits respectively representing the 8-bit value of the red, green and blue components. These 8-bit values must be interpreted in the context of the default quantization range (full or limited) for the current output video format as specified in standard CEA 861-F. Note: Only supported when receiver (RX) operating in a display mode other than genlock, therefore supported by AVP2000(T) devices but not AVP1000.
force_output	3.0	Boolean	When <code>true</code> , device outputs a blank video output with a uniform color. Output also fixed format of 720p60. Note: Only supported for receiver (RX) devices capable of operating in genlock mode only (AVP1000) not available to devices that support video display modes (AVP2000(T)).

6.8.7.2 Status

None

6.8.7.3 Inputs

None

6.8.7.4 Example

Example shows a color generator node JSON object returned as part of the output of “`get device`”.

```
{  
  "type" : "COLOR_GENERATOR",  
  "index" : 0,  
  "configuration" : {  
    "enable" : false,  
    "color" : "000000"  
  }  
}
```

```

    },
    "status" : {},
    "inputs" : []
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.8 Frame Buffer Node (**FRAME_BUFFER**)

This section describes a Node object with `type` member set to `FRAME_BUFFER`.

6.8.8.1 Configuration (FrameBufferNodeConfig)

```

{
    "display_mode" : String,
    "width" : Integer,
    "height" : Integer,
    "horiz_offset" : Integer,
    "vert_offset" : Integer,
    "frames_per_second" : Integer,
    "keep_width" : Integer,
    "keep_height" : Integer,
    "viewport_horiz" : Integer,
    "viewport_vert" : Integer,
    "viewport_width" : Integer,
    "viewport_height" : Integer,
    "aspect_ratio_mode" : String,
    "has_auto_wall_details" : Boolean,
    "auto_wall_details" : {
        "grid_width" : Integer,
        "grid_height" : Integer,
        "grid_index_horiz" : Integer,
        "grid_index_vert" : Integer,
        "bezel_horiz" : Integer,
        "bezel_vert" : Integer,
    },
    "wall_name" : String,
    "freeze" : Boolean
    "multiview_layout" : String
}

```

Table 117: Frame Buffer Node Object Members

Member Name	Since	Type	Description
display_mode	3.0	String	Processing display mode. Will be one of the following: FAST_SWITCHED Full screen fast switch mode GENLOCK_SCALING Full screen zero-frame latency scaling mode WALL_FAST_SWITCHED Fast switch display wall mode WALL_GENLOCKED Zero-frame latency

Member Name	Since	Type	Description
			display wall mode Multiview mode
width	3.0	Integer	MULTIVIEW Video width
height	3.0	Integer	Video height
horiz_offset	3.0	Integer	Video horizontal offset
vert_offset	3.0	Integer	Video vertical offset
frames_per_second	3.0	Integer	Frame rate
keep_width	3.0	Integer	Width of wall region of interest, in source pixels.
keep_height	3.0	Integer	Height of wall region of interest, in source pixels.
viewport_horiz	3.0	Integer	Viewport horizontal position. Only relevant in wall display mode (fast switched or genlocked wall).
viewport_vert	3.0	Integer	Viewport vertical position. Only relevant in wall display mode (fast switched or genlocked wall).
viewport_width	3.0	Integer	Viewport width. Only relevant in wall display mode (fast switched or genlocked wall).
viewport_height	3.0	Integer	Viewport width. Only relevant in wall display mode (fast switched or genlocked wall).
aspect_ratio_mode	3.2	String	The aspect ratio applied to the output. Options are KEEP, CROP and STRETCH.
has_auto_wall_details	3.2	Boolean	Indicates if the arguments for the Source Independent (auto_wall) are currently applied. TRUE indicates this feature enabled and FALSE that it is not.
grid_width	3.2	Integer	Indicates total number of columns in the wall grid.
grid_height	3.2	Integer	Indicates total number of rows in the wall grid.
grid_index_horiz	3.2	Integer	The horizontal index of the display in the wall grid. Example, if the wall grid is 3x2 and this is the second display mounted horizontally the index would be 2.
grid_index_vert	3.2	Integer	The vertical index of the display in the wall grid. Example, if the wall grid is 3x2 and this is the first display mounted vertically, the index would be 1.
bezel_horiz	3.2	Integer	The horizontal bezel compensation applied, in pixels.
bezel_vert	3.2	Integer	The vertical bezel compensation applied, in pixels.
wall_name	3.5	String	The video wall name.
freeze	3.9	Boolean	Whether manual force freeze is applied to the video output (freeze video on command). Note: This configuration is volatile and only supported for receivers (RX) currently operating in one of the fast switched display modes.

Member Name	Since	Type	Description
multiview_layout	3.4	String or Integer	When a device is operating in multiview mode and a layout has been loaded, this node provides details of the layout. This includes name, size, surfaces, windows (position, size, index and surface assigned).

6.8.8.2 Status (FramebufferNodeStatus)

```
{
  "source_video" : VideoFormat
}
```

Table 118: Frame Buffer Node Status Object Member

Member Name	Since	Type	Description
source_video	3.0	VideoFormat	Format of video source (full screen and wall modes only)

6.8.8.3 Inputs

Table 119: Frame Buffer Node Input Object Members

Input Name	Since	Configurable	Description
main	3.0	No	Video input for full screen and wall modes. Source is a subscription.
multiview	3.0	No	Video inputs for multiview. There are multiple inputs with this name, however each has a unique assigned index. Each input is also connected to a subscription.

6.8.8.4 Example

Below shows a frame buffer node JSON object returned as part of the output of a “get device” command.

```
{
  "type": "FRAME_BUFFER",
  "index": 0,
  "configuration": {
    "display_mode": "FAST_SWITCHED",
    "width": 0,
    "height": 0,
    "horiz_offset": 0,
    "vert_offset": 0,
    "frames_per_second": 0,
    "keep_width": 0,
    "keep_height": 0,
    "viewport_horiz": 0,
    "viewport_vert": 0,
    "viewport_width": 0,
    "viewport_height": 0,
    "aspect_ratio_mode": "NONE",
    "has_auto_wall_details": false,
    "wall_name": "",
    "freeze": false
  }
}
```



```

},
"status": {
  "source_video": {
    "width": 0,
    "height": 0,
    "frames_per_second": 0,
    "color_space": "RGB",
    "bits_per_pixel": 8,
    "scan_mode": "PROGRESSIVE"
  }
},
inputs: [
  {
    name: main,
    index: 0,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 0
      }
    }
  },
  {
    name: multiview,
    index: 0,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 0
      }
    }
  },
  {
    name: multiview,
    index: 1,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 1
      }
    }
  },
  {
    name: multiview,

```

```

        index: 2,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 2
            }
        }
    },
    ...
    {
        name: multiview,
        index: 30,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 30
            }
        }
    },
    {
        name: multiview,
        index: 31,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 31
            }
        }
    },
]
},

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.9 Frame Rate Converter Node (FRAME_RATE_CONVERTER)

This section describes a Node object with `type` member set to `FRAME_RATE_CONVERTER`.

6.8.9.1 Configuration Status (FrameRateConverterNodeConfig)

None

6.8.9.2 Status (FrameRateConverterNodeStatus)

```

{
    "divide_factor" : Integer
}

```

Table 120: Frame Rate Converter Node Object Member

Member Name	Since	Type	Description
divide_factor	3.0	Integer	Factor by which the frame rate is divided

6.8.9.3 Inputs

Table 121: Frame Rate Converter Node Input Object Member

Input Name	Since	Configurable	Description
main	3.0	No	Main video input.

6.8.9.4 Example

Example shows a frame rate converter node JSON object returned as part of the output of “get device”.

```
{
  "type" : "FRAME_RATE_CONVERTER",
  "index" : 0,
  "configuration" : {},
  "status" : {
    "divide_factor" : 2
  },
  "inputs" : [
    {
      "name" : "main",
      "index" : 0,
      "configuration" : {},
      "status" : {
        "source" : {
          "ref_class" : "NODE",
          "ref_type" : "HDMI_DECODER",
          "ref_index" : 0
        }
      }
    }
  ]
}
```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.10 HDMI Decoder Node (HDMI_DECODER)

This section describes a Node object with `type` member set to `HDMI_DECODER`.

Note: This node type is included in the response of a `get edid` command, as well as a `get settings` response. However, in the response of a `get edid` command, the returned node only contains the `configuration` member, which itself only contains the `edid` member.

6.8.10.1 Configuration (HdmiDecoderNodeConfig)

```
{
    "edid" : String,
    "hdcv_support_enable" : Boolean,
    "hdcv_22_support_disable" : Boolean,
    "default_quantization_level_override" :String,
    "max_video_bandwidth" : String
}
```

Table 122: HDMI Decoder Node Object Members

Member Name	Since	Type	Description
edid	3.0	String	Configured EDID seen by connected HDMI sources. Format of member is hexadecimal string with 512-character length representing 256 bytes of data. Note: Not included in response of a <code>get settings</code> but is in response of <code>get edid</code> command.
hdcv_support_enable	3.0	Boolean	Enable HDCP support. When disabled, device reports to the HDMI source that it does not support HDCP. TIPS: * Some laptops and other HDMI sources always encrypt video with HDCP if the sink connected to it supports it, even when not playing back HDCP-protected content. * Can cause problems when trying to display video from HDMI sources on monitors without HDCP support. * Disabling HDCP support in the transmitter device solves this issue.
hdcv_22_support_disable	3.0	Boolean	Disable HDCP 2.2 support. When set <code>true</code> , device reports to HDMI source it only supports HDCP 1.4 (if <code>hdcv_support_enable</code> is also <code>true</code>).
default_quantization_level_override	3.6	String	Override infoframe quantization level default value (unspecified) to <code>disabled</code> (default kept), <code>full</code> or <code>limited</code> .
max_video_bandwidth	3.9	String	Configured maximum allowable video bandwidth. Possible values:

Member Name	Since	Type	Description
			NETWORK_DEFAULT Set for current PHY link operating speed. This is the default value. 1G Force for as if 1G PHY link operating speed. 2_5G Force for as if 2.5G PHY link operating speed. 5G Force for as if 5G PHY link operating speed. 10G Force for as if 10G PHY link operating speed. Note: Only applicable when the HDMI transceiver chip supports compression for FlexLinQ (e.g. Lontium LT86121GXE). The default NETWORK_DEFAULT should only be changed if the user specifically wants to set a limit for video bandwidth lower than the Ethernet link capability (for example if he knows the switch trunk port adds some bandwidth limitation for the overall system or other reason to limit the bandwidth taken by the video data).

6.8.10.2 Status (HdmiDecoderNodeStatus)

```

{
    "hdcp_protected" : Boolean,
    "hdcp_version" : String,
    "hdmi_2_0_support" : Boolean,
    "source_stable" : Boolean,
    "video" : VideoFormat,
    "has_video_details" : Boolean,
    "video_details" : VideoFormatDetails,
    "has_audio_details" : Boolean,
    "audio_details" : AudioFormatDetails,
    "has_audio_details_extended" : Boolean,
    "audio_details_extended" : AudioFormatExtended,
    "audio_downmix_support" : Boolean
}

```

Table 123: HDMI Decoder Node Status Object Members

Member Name	Since	Type	Description
hdcp_protected	3.0	Boolean	Indicates if HDMI source is HDCP protected.
hdcp_version	3.0	String	Version of HDCP content protection: NONE Content is not HDCP-protected. HDCP_1_4 Supports HDCP 2.1 and below (including 1.4). HDCP_2_2 HDCP 2.2.
hdmi_2_0_support	3.0	Boolean	Indicates if HDMI decoder supports HDMI 2.0 resolutions, such as 4K60.
source_stable	3.0	Boolean	Indicates if device is receiving video from a connected source.
video	3.0	VideoFormat	Indicates format of video received from the source, only valid if source_stable is true. Undefined if source_stable is false.
video.external_video_compression_type	3.9	VideoCompression	Indicates if video is compressed by the external HDMI transceiver chip and if so by what ratio. Uncompressed. Compressed by 2 Compressed by 4 Compressed by 8 Note: Only applicable when the HDMI transceiver chip supports compression for FlexLinQ (e.g. Lontium LT86121GXE).
has_video_details	3.0	Boolean	Indicates if detailed video format and timing information available from device.
video_details	3.0	VideoFormatDetails	Detailed video format and timing information. Undefined when source_stable or has_video_details is false.
has_audio_details	3.0	Boolean	Indicates whether audio format information available from device.
audio_details	3.0	AudioFormatDetails	Information regarding audio format. Undefined if source_stable or has_audio_details is false.
has_audio_details_extended	3.0	Boolean	Indicates if extended audio format information available from the device.
audio_details_extended	3.0	AudioFormatExtended	Extended data regarding audio format. Undefined if either source_stable or has_audio_details_extended false.
audio_downmix_support	3.0	Boolean	Indicates if HDMI decoder generates a downmix version of the extracted HDMI audio.

6.8.10.3 Inputs

Table 124: HDMI Decoder Node Input Object Member

Input Name	Since	Configurable	Description
main	3.0	Yes*	Main video input. Video input connector can be selected on devices that have multiple inputs available. Modes can also be selected where device automatically selects an input with a video source present.

* The HDMI decoder node input is only configurable on devices with multiple video inputs.

6.8.10.4 Example

This example shows an HDMI decoder node JSON object returned as part of the output of “get_device”.

```
{
    "type": "HDMI_DECODER",
    "index": 0,
    "configuration": {
        "edid": :
"00ffffffffffffff000614000156524c420816010380341d780a01c1a05747982712
4c4c21080081400101010101010101010101010101010101023a801871382d40582c46004
0846300001e08e80030f2705a80b0588a00ba892100001e000000fc004170746f56
6973696f6e20200a000000fd0018550e853c000a2020202020012002034073580
102030405111213141f2021220607905d5e5f6263646061230d0707830f00006d03
0c001000383c20406801020367d85dc401788003e40f0000c0011d00bc52d01e20b
828554040846300001e023a80d072382d40102c458040846300001e023a80187138
2d40582c450040846300001e00000000000000000000f2",
        "hdcp_support_enable": true,
        "hdcp_22_support_disable": false
        "default_quantization_level_override": disabled,
        "max_video_bandwidth": "NETWORK_DEFAULT
    },
    "status": {
        "video": {
            "width": 0,
            "height": 0,
            "frames_per_second": 0,
            "color_space": "RGB",
            "bits_per_pixel": 10,
            "scan_mode": "PROGRESSIVE",
            "external_video_compression_type": "UNCOMPRESSED",
        },
        "has_video_details": true,
        "video_details": {
            "pixel_clock": 0,
            "total_width": 0,
            "total_height": 0,
            "hsync_width": 0,
            "hsync_front_porch": 0,
            "hsync_negative": false,
```

```

    "vsync_width" : 0,
    "vsync_front_porch" : 0,
    "vsync_negative" : false,
    "vic" : 0,
    "has_hdmi_vic" : false,
    "hdmi_vic" : 0,
    "picture_aspect" : "ASPECT_NO_DATA",
    "has_active_format" : false,
    "active_format" : 0,
    "it_content_type" : "NO_DATA",
    "scan_information" : "NO_DATA",
    "colorimetry" : "RGB",
    "rgb_range" : "DEFAULT",
    "ycc_range" : "LIMITED"
  },
  "has_audio_details" : true,
  "audio_details" : {
    "sampling_frequency" : 48000,
    "number_of_channels" : 8
  },
  "has_audio_details_extended" : true,
  "audio_details_extended" : {
    "channel_allocation" : 0,
    "sample_size" : 24,
    "level_shift_value" : 0,
    "lfe_playback_level" : 0,
    "audio_encoding_type" : "LPCM",
    "is_downmix_inhibited" : false
  },
  "hdcp_protected" : false,
  "hdcp_version" : "NONE",
  "hdmi_2_0_support" : true,
  "source_stable" : false,
  "audio_downmix_support" : true
},
"inputs" : [
  {
    "name" : "main",
    "index" : 0,
    "configuration" : {
      "source" : {
        "value" : 0,
        "choices" : [
          {
            "value" : 0,
            "description" : "HDMI Input 1",
            "details" : {
              "ref_class" : "NODE",
              "ref_type" : "VIDEO_INPUT",
              "ref_index" : 0
            }
          }
        ]
      }
    }
  }
]

```



```

    },
    {
      "value" : 1,
      "description" : "Display Port Input 1",
      "details" : {
        "ref_class" : "NODE",
        "ref_type" : "VIDEO_INPUT",
        "ref_index" : 1
      }
    },
    {
      "value" : 32,
      "description" : "Automatic Input Selection
(HDMI Input 1)",
      "details" : {
        "ref_class" : "AUTOMATIC",
        "ref_type" : null,
        "ref_index" : 0
      }
    }
  ]
},
"status" : {
  "source" : {
    "ref_class" : "NODE",
    "ref_type" : "VIDEO_INPUT",
    "ref_index" : 0
  }
}
}
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.11 HDMI Encoder Node (HDMI_ENCODER)

This section describes a Node object with `type` member set to `HDMI_ENCODER`.

6.8.11.1 Configuration (HdmiEncoderNodeConfig)

```

{
  "hdcp_output_mode" : String
  "hdmi_tx_5v" : String
}

```

Table 125: HDMI Encoder Node Status Object Members

Member Name	Since	Type	Description
hdcp_output_mode	3.0	String	The following nodes supported for the HDCP Output mode:
			UNSUPPORTED Feature is unsupported on this device
			FOLLOW_SINK_1 Sets HDCP output level to the highest HDCP capability of sink.
			FOLLOW_SINK_2 Sets HDCP output level to HDCP 1.4 when

Member Name	Since	Type	Description
hdmi_tx_5v	3.5	String	possible. Sets HDCP output level to same as the source. FOLLOW_SOURCE API control value of AUTO or LOW. AUTO indicates device is currently set to operate in "normal" mode: - In switch mode, the HDMI TX 5V output is always HIGH - In loopback mode, it follows 5V loopback mode rules. - In P2P mode, it follows 5V mirroring rules. LOW indicates set to remain LOW, overriding any other mode. Therefore, regardless of the operating mode (switch, P2P or loopback), when the configuration is set to LOW the HDMI TX 5V is always LOW. Refer to the SDVoE Developers API Application Note Library for further information.

6.8.11.2 Status (HdmiEncoderNodeStatus)

```

{
    "hdcp_protected" : Boolean,
    "hdmi_2_0_support" : Boolean,
    "source_stable" : Boolean,
    "video" : VideoFormat,
    "has_video_details" : Boolean,
    "video_details" : VideoFormatDetails,
}

```

Table 126: HDMI Encoder Node Status Object Members

Member Name	Since	Type	Description
hdcp_protected	3.0	Boolean	Indicates if HDCP protection is enabled. Automatically enabled if source is HDCP protected.
hdmi_2_0_support	3.0	Boolean	Indicates if the HDMI encoder supports HDMI 2.0 resolutions (e.g. 4K60).
source_stable	3.0	Boolean	Indicates device is receiving video from the source.
video	3.0	VideoFormat	Format of video being output by the HDMI encoder. Undefined if source_stable is false.
video.external_video_compression_type	3.9	VideoCompression	Indicates if video is compressed by the external HDMI transceiver chip and if so by what ratio. Uncompressed. Compressed by 2 Compressed by 4 Compressed by 8 Note: Only applicable when the HDMI transceiver chip supports compression for FlexLinQ (e.g. Lontium LT86121GXE).
has_video_details	3.0	Boolean	Indicates whether detailed video format and timing information is available from this device.
video_details	3.0	VideoFormatDetails	Detailed video format and timing information.

Member Name	Since	Type	Description
			Undefined if <code>source_stable</code> or <code>has_video_details</code> is false.

6.8.11.3 Inputs

Table 127: HDMI Encoder Node Input Object Members

Input Name	Since	Configurable	Description
main	3.0	No	Main video input. A device may support one or both of the following values: 0 -- Video from a video subscription (genlock mode) 1 -- Video from a frame buffer (frame buffer mode)
audio	3.0	Yes	HDMI audio source. A receiver and transceiver may support none, some or all of the following values: 2 -- HDMI audio (original from video subscription in genlock mode) 3 -- Stereo audio subscription 6 -- HDMI audio subscription (stereo downmix) 7 -- HDMI audio subscription (all available channels) 8 -- Multichannel audio subscription 9 -- Local Multichannel audio Note: Not all audio types are available to all endpoint devices, supported audio is based on the hardware design and capabilities. Refer to the SDVoE Developers API Application Note Library for more information.

6.8.11.4 Example

Example shows an HDMI encoder node JSON object returned as part of the output of “get device”.

```
{
  "type" : "HDMI_ENCODER",
  "index" : 0,
  "configuration" : {},
  "status" : {
    "video" : {
      "width" : 1920,
      "height" : 1080,
      "frames_per_second" : 60,
      "color_space" : "RGB",
      "bits_per_pixel" : 8,
      "scan_mode" : "PROGRESSIVE",
      "external_video_compression_type" : "UNCOMPRESSED",
    },
    "has_video_details" : true,
    "video_details" : {
      "pixel_clock" : 148500000,
      "total_width" : 2200,
      "total_height" : 1125,
      "hsync_width" : 44,
    }
  }
}
```

```

        "hsync_front_porch" : 88,
        "hsync_negative" : false,
        "vsync_width" : 5,
        "vsync_front_porch" : 4,
        "vsync_negative" : false,
        "vic" : 16,
        "has_hdmi_vic" : false,
        "hdmi_vic" : 0,
        "picture_aspect" : "ASPECT_16_9",
        "has_active_format" : false,
        "active_format" : 0,
        "it_content_type" : "GRAPHICS",
        "scan_information" : "NO_DATA",
        "colorimetry" : "BT709",
        "rgb_range" : "DEFAULT",
        "ycc_range" : "LIMITED"
    },
    "hdcp_protected" : false,
    "hdcp_version" : "NONE",
    "hdmi_2_0_support" : true,
    "source_stable" : false
},
"inputs" : [
    {
        "name" : "main",
        "index" : 0,
        "configuration" : {
            "source" : {
                "value" : 0,
                "choices" : [
                    {
                        "value" : 0,
                        "description" : "Genlock Mode",
                        "details" : {
                            "ref_class" : "SUBSCRIPTION",
                            "ref_type" : "HDMI",
                            "ref_index" : 0
                        }
                    },
                    {
                        "value" : 1,
                        "description" : "Frame Buffer Mode",
                        "details" : {
                            "ref_class" : "NODE",
                            "ref_type" : "FRAME_BUFFER",
                            "ref_index" : 0
                        }
                    }
                ]
            }
        }
    }
]
},

```

```

    "status" : {
        "source" : {
            "ref_class" : "SUBSCRIPTION",
            "ref_type" : "HDMI",
            "ref_index" : 0
        }
    }
},
{
    "name" : "audio",
    "index" : 0,
    "configuration" : {
        "source" : {
            "value" : 2,
            "choices" : [
                {
                    "value" : 2,
                    "description" : "HDMI Audio (from HDMI Video in
Genlock Mode)",
                    "details" : {
                        "ref_class" : "SUBSCRIPTION",
                        "ref_type" : "HDMI_AUDIO",
                        "ref_index" : 0
                    }
                },
                {
                    "value" : 3,
                    "description" : "Stereo Audio",
                    "details" : {
                        "ref_class" : "SUBSCRIPTION",
                        "ref_type" : "STEREO_AUDIO",
                        "ref_index" : 0
                    }
                }
            ],
            {
                "value" : 6,
                "description" : "HDMI Audio (Stereo Downmix)",
                "details" : {
                    "ref_class" : "SUBSCRIPTION",
                    "ref_type" : "HDMI_AUDIO",
                    "ref_index" : 0
                }
            },
            {
                "value" : 7,
                "description" : "HDMI Audio (All Available
Channels)",
                "details" : {
                    "ref_class" : "SUBSCRIPTION",
                    "ref_type" : "HDMI_AUDIO",
                    "ref_index" : 0
                }
            }
        }
    }
}

```

```

    },
    {
      "value" : 8,
      "description" : "Multi channel Audio",
      "details" : {
        "ref_class" : "SUBSCRIPTION",
        "ref_type" : "MULTICH_AUDIO",
        "ref_index" : 0
      }
    },
    {
      "value" : 9,
      "description" : "Multi Channel Audio Local Loop
Out",
      "details" : {
        "ref_class" : "NODE",
        "ref_type" : "MULTICH_AUDIO_INPUT",
        "ref_index" : 0
      }
    }
  ]
},
"status" : {
  "source" : {
    "ref_class" : "SUBSCRIPTION",
    "ref_type" : "HDMI_AUDIO",
    "ref_index" : 0
  }
}
}
]
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.12 HDMI Monitor Node (HDMI_MONITOR)

This section describes a Node object with `type` member set to `HDMI_MONITOR`.

Note: This node type is included in the response to a “`get edid`” command (in addition to `get settings`). However, in the response to a `get edid` command, the returned node only contains the status member.

6.8.12.1 Configuration (HdmiMonitorNodeConfig)

None

6.8.12.2 Status (HdmiMonitorNodeStatus)

```

{
  "connected" : Boolean,
  "hdcp_support" : String,

```

```

    "edid" : String
}

```

Table 128: HDMI Monitor Node Status Object Members

Member Name	Since	Type	Description
connected	3.0	Boolean	Indicates if there is a connected monitor (HPD – Hot-Plug Detect). Included in response of both <code>get edid</code> and <code>get settings</code> commands.
hdcp_support	3.9	String	Indicates the HDCP support level of the currently connected monitor (HDMI sink device), as reported by the external MCU to the AVP I2C registers: HDCP 2.2 Monitor supports up to HDCP 2.2 HDCP 1.4 Monitor supports up to HDCP 1.4 No HDCP Monitor does not support HDCP UNKNOWN Monitor HDCP support level is unknown (feature not supported by the BlueRiver FW version or the EMCU)
edid	3.0	String	EDID of the connected monitor. Format of this member is a hexadecimal string with a length of 512 characters (represents 256 bytes of data). If no monitor is connected (see <code>connected</code> member), member contains an empty string. Note: Not included in response of a <code>get settings</code> command but is included in response of a <code>get edid</code> command.

6.8.12.3 Inputs

Table 129: HDMI Monitor Node Input Object Member

Input Name	Since	Configurable	Description
main	3.0	No	HDMI encoder which sends video to this monitor

6.8.12.4 Example

Example shows an HDMI monitor node JSON object returned as part of the output of “`get device`”.

```

{
  "type" : "HDMI_MONITOR",
  "index" : 0,
  "configuration" : {},
  "status" : {
    "connected" : true,
    "edid" :
    "00ffffffffffff004c2d5706333244590a140103801009782aee91a3544c99260f
    5054230800a9408180814081009500b30001010101023a801871382d40582c4500a
    05a0000001e011d007251d01e206e285500a05a0000001e000000fd00323c1b5111
    000a2020202020200000000fc0053796e634d61737465720a2020013b02031cf3489
    0041f0514130312230907078301000066030c00100080011d80d0721c1620102c25
    80a05a0000009e011d8018711c1620582c2500a05a0000009e011d00bc52d01e20b
    8285540a05a0000001e011d007251d01e206e285500a05a0000001e8c0ad0902040
    31200c405500a05a00000018000000000000000000000075"
  },
  "inputs" : [
    {
      "name" : "main",
      "index" : 0,

```

```

        "configuration" : {},
        "status" : {
            "source" : {
                "ref_class" : "NODE",
                "ref_type" : "HDMI_ENCODER",
                "ref_index" : 0
            }
        }
    }
}
]
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.13 Multichannel Audio Input Node (MULTICH_AUDIO_INPUT)

This section describes a Node object with `type` member set to `MULTICH_AUDIO_INPUT`.

6.8.13.1 Configuration

```

{
    "sampling_frequency" : SamplingFrequencyConfig,
    "number_of_channels" : ChannelsNumberConfig,
    "channel_allocation" : ChannelAllocationConfig,
    "audio_encoding_type" : AudioEncodingTypeConfig
}

```

Table 130: Multichannel Audio Input Node Object Members

Input Name	Since	Type	Description
sampling_frequency	3.0	SamplingFrequencyConfig	Audio sampling rate configuration
number_of_channels	3.0	ChannelsNumberConfig	Number of audio channels configuration
channel_allocation	3.0	ChannelsAllocationConfig	HDMI audio channel allocation as reported in CEA-861-F audio infoframe, from range of 0 to 0x31.
audio_encoding_type	3.0	AudioEncodingTypeConfig	Audio encoding type, can be set to <code>LPCM</code> , <code>ENCODED_HBR</code> or <code>ENCODED_OTHER</code>

Sampling Frequency Configuration (SamplingFrequencyConfig)

```

{
    "value" : Integer,
    "choices" : [SamplingFrequencyChoice, ...]
}

```

Table 131: Multichannel Audio Sampling Frequency Configuration Object Members

Member Name	Since	Type	Description
value	3.0	Integer	Sampling frequency selection. This value is sampling frequency in Hertz and must match value member of an option listed in the choices member. Note: Not all sampling rates are supported by all devices. The valid choices are listed in the choices member.

Member Name	Since	Type	Description
choices	3.0	Array of SamplingFrequencyChoice	Sampling rate choices. Note: Member not included in a <code>get settings</code> response, <code>get device</code> command is required.

Sampling Frequency Choice (SampleFrequencyChoice)

```
{
  "value" : Integer,
  "description" : String,
  "details" : Undefined
}
```

Table 132: Multichannel Audio Sampling Frequency Choice Node Status Object Members

Member Name	Since	Type	Description
value	3.0	Integer	Sampling frequency in Hertz, following values are supported 22050, 24000, 32000, 44100, 48000, 88200, 96000, 176400, 192000 and 768000.
description	3.0	String	Human-readable description of this choice.
details	3.0	Undefined	Undefined. Reserved for future use.

Number of Audio Channels Configuration (ChannelsNumberConfig)

```
{
  "value" : Integer,
  "choices" : [ChannelsNumberChoice, ...]
}
```

Table 133: Multichannel Audio Number of Audio Channels Node Status Object Members

Member Name	Since	Type	Description
value	3.0	Integer	Number of channels selection. Values from 2 to 8 represent number of audio channels. Other values reserved for future use. This value must match the value member listed in the choices member. Note: Not all number of channels supported by all devices. Valid choices are listed in the choices member.
choices	3.0	Array of ChannelsNumberChoice	Number of audio channels choices. Note: Member not included in <code>get settings</code> response, a <code>get device</code> command required.

Channels Number Choice (ChannelNumberChoice)

```
{
  "value" : Integer,
  "description" : String,
  "details" : Undefined
}
```

Table 134: Channels Number Choice Node Status Object Members

Member Name	Since	Type	Description
value	3.0	Integer	Value of this choice. Values from 2 to 8 represent number of audio channels. Other values reserved for future use.
description	3.0	String	Human-readable description of this choice.
details	3.0	Undefined	Undefined. Reserved for future use.

Channel Allocation Configuration (ChannelAllocationConfig)

```

{
    "value" : Integer,
    "choices" : [ChannelAllocationChoice, ...]
}

```

Table 135: Channel Allocation Node Object Members

Member Name	Since	Type	Description
value	3.0	Integer	Channel allocation selection. Values from 0 to 0x31 represent the audio channel allocation of the incoming multichannel audio. The channel allocation value is as reported in the CEA-861-F documentation. Other values are reserved for future use. This value must match the <code>value</code> member of one of the choices listed in the <code>choices</code> member. Note: Not all channel allocations are supported by all devices. The valid choices are listed in the <code>choices</code> member.
choices	3.0	Array of ChannelsAllocationChoice	Channel Allocation choices. Note: This member is not included in the response of a <code>get settings</code> command. A <code>get device</code> command is required.

Channel Allocation Choice (ChannelAllocationChoice)

```

{
    "value" : Integer,
    "description" : String,
    "details" : Undefined
}

```

Table 136: Channel Allocation Choice Object Members

Member Name	Since	Type	Description
value	3.0	Integer	Value of this choice. Values from 0 to 0x31 represent the audio channel allocation configuration. Other values are reserved for future use.
description	3.0	String	Human-readable description of this choice.
details	3.0	Undefined	Undefined. Reserved for future use.

Audio Encoding Configuration (AudioEncodingConfig)

Table 137: Audio Encoding Object Member

Member Name	Since	Type	Description
audio_encoding_type	3.0	String	Audio encoding type. Is one of the following: LPCM Linear PCM audio format. ENCODED_HBR Compressed high-bit-rate audio format. ENCODED_OTHER Compressed non -high-bit-rate audio format.

6.8.13.2 Status

None

6.8.13.3 Inputs

None

6.8.13.4 Example

Example shows a multichannel audio input node JSON object returned as part of the output of a “get device” command.

Reminder: “choice members” are not included in the response of a `get settings` command. A `get device` command is required.

```
{
  "type" : "MULTICH_AUDIO_INPUT",
  "index" : 0,
  "configuration" : {
    "sampling_frequency" : {
      "value" : 48000,
      "choices" : [
        {
          "value" : 22050,
          "description" : "22.05kHz",
          "details" : null
        },
        {
          "value" : 24000,
          "description" : "24kHz",
          "details" : null
        },
        {
          "value" : 32000,
          "description" : "32kHz",
          "details" : null
        },
        {
          "value" : 44100,
          "description" : "44.1kHz",
          "details" : null
        },
        {
          "value" : 48000,
          "description" : "48kHz",
          "details" : null
        }
      ]
    }
  }
}
```

```

    {
      "value" : 88200,
      "description" : "88.2kHz",
      "details" : null
    },
    {
      "value" : 96000,
      "description" : "96kHz",
      "details" : null
    },
    {
      "value" : 176400,
      "description" : "176.4kHz",
      "details" : null
    },
    {
      "value" : 192000,
      "description" : "192kHz",
      "details" : null
    },
    {
      "value" : 768000,
      "description" : "768kHz",
      "details" : null
    }
  ]
},
"number_of_channels" : {
  "value" : 8,
  "choices" : [
    {
      "value" : 2,
      "description" : "2 Channels",
      "details" : null
    },
    {
      "value" : 3,
      "description" : "3 Channels",
      "details" : null
    },
    {
      "value" : 4,
      "description" : "4 Channels",
      "details" : null
    },
    {
      "value" : 5,
      "description" : "5 Channels",
      "details" : null
    },
    {
      "value" : 6,
      "description" : "6 Channels",
      "details" : null
    }
  ]
}

```

```

        "value" : 7,
        "description" : "7 Channels",
        "details" : null
    },
    {
        "value" : 8,
        "description" : "8 Channels",
        "details" : null
    }
]
}
"channel_allocation" : {
    "value" : 0,
    "choices" : [
        {
            "value" : 0,
            "description" : "2 Channels (FL-FR)",
            "details" : null
        },
        {
            "value" : 1,
            "description" : "2.1 Channels (FL-FR-LFE)",
            "details" : null
        },
        {
            "value" : 2,
            "description" : "3 Channels (FL-FR-X-FC)",
            "details" : null
        },
        {
            "value" : 3,
            "description" : "3.1 Channels (FL-FR-LFE-FC)",
            "details" : null
        },
        {
            "value" : 4,
            "description" : "3 Channels (FL-FR-X-X-RC)",
            "details" : null
        },
        {
            "value" : 5,
            "description" : "3.1 Channels (FL-FR-LFE-X-RC)",
            "details" : null
        },
        {
            "value" : 6,
            "description" : "4 Channels (FL-FR-X-FC-RC)",
            "details" : null
        },
        {
            "value" : 7,
            "description" : "4.1 Channels (FL-FR-LFE-FC-RC)",
            "details" : null
        },
        {
            "value" : 8,

```

```

        "description" : "4 Channels (FL-FR-X-X-RL-RR)",
        "details" : null
    },
    {
        "value" : 9,
        "description" : "4.1 Channels (FL-FR-LFE-X-RL-RR)",
        "details" : null
    },
    {
        "value" : 10,
        "description" : "5 Channels (FL-FR-X-FC-RL-RR)",
        "details" : null
    },
    {
        "value" : 11,
        "description" : "5.1 Channels (FL-FR-LFE-FC-RL-RR)",
        "details" : null
    },
    {
        "value" : 12,
        "description" : "5 Channels (FL-FR-X-X-RL-RR-RC)",
        "details" : null
    },
    {
        "value" : 13,
        "description" : "5.1 Channels (FL-FR-LFE-X-RL-RR-RC)",
        "details" : null
    },
    {
        "value" : 14,
        "description" : "6 Channels (FL-FR-X-FC-RL-RR-RC)",
        "details" : null
    },
    {
        "value" : 15,
        "description" : "6.1 Channels (FL-FR-LFE-FC-RL-RR-RC)",
        "details" : null
    },
    {
        "value" : 16,
        "description" : "6 Channels (FL-FR-X-X-RL-RR-RLC-RR)",
        "details" : null
    },
    {
        "value" : 17,
        "description" : "6.1 Channels (FL-FR-LFE-X-RL-RR-RLC-RR)",
        "details" : null
    },
    {
        "value" : 18,
        "description" : "7 Channels (FL-FR-X-FC-RL-RR-RLC-RR)",
        "details" : null
    },
    {
        "value" : 19,
        "description" : "7.1 Channels (FL-FR-LFE-FC-RL-RR-RLC-RR)",

```

```

        "details" : null
    },
    {
        "value" : 20,
        "description" : "4 Channels (FL-FR-X-X-X-X-FLC-FRC)",
        "details" : null
    },
    {
        "value" : 21,
        "description" : "4.1 Channels (FL-FR-LFE-X-X-X-FLC-FRC)",
        "details" : null
    },
    {
        "value" : 22,
        "description" : "5 Channels (FL-FR-X-FC-X-X-FLC-FRC)",
        "details" : null
    },
    {
        "value" : 23,
        "description" : "5.1 Channels (FL-FR-LFE-FC-X-X-FLC-FRC)",
        "details" : null
    },
    {
        "value" : 24,
        "description" : "5 Channels (FL-FR-X-X-RC-X-FLC-FRC)",
        "details" : null
    },
    {
        "value" : 25,
        "description" : "5.1 Channels (FL-FR-LFE-X-RC-X-FLC-FRC)",
        "details" : null
    },
    {
        "value" : 26,
        "description" : "6 Channels (FL-FR-X-FC-RC-X-FLC-FRC)",
        "details" : null
    },
    {
        "value" : 27,
        "description" : "6.1 Channels (FL-FR-LFE-FC-RC-X-FLC-FRC)",
        "details" : null
    },
    {
        "value" : 28,
        "description" : "6 Channels (FL-FR-X-X-RL-RR-FLC-FRC)",
        "details" : null
    },
    {
        "value" : 29,
        "description" : "6.1 Channels (FL-FR-LFE-X-RL-RR-FLC-FRC)",
        "details" : null
    },
    {
        "value" : 30,
        "description" : "7 Channels (FL-FR-X-FC-RL-RR-FLC-FRC)",
        "details" : null
    }

```

```

    },
    {
      "value" : 31,
      "description" : "7.1 Channels (FL-FR-LFE-FC-RL-RR-FLC-FRC)",
      "details" : null
    },
    {
      "value" : 32,
      "description" : "6 Channels (X-FCH-RR-RL-FC-X-FR-FL)",
      "details" : null
    },
    {
      "value" : 33,
      "description" : "6.1 Channels (X-FCH-RR-RL-FC-LFE-FR-FL)",
      "details" : null
    },
    {
      "value" : 34,
      "description" : "6 Channels (TC-X-RR-RL-FC-X-FR-FL)",
      "details" : null
    },
    {
      "value" : 35,
      "description" : "6.1 Channels (TC-X-RR-RL-FC-LFE-FR-FL)",
      "details" : null
    },
    {
      "value" : 36,
      "description" : "6 Channels (FRH-FLH-RR-RL-X-X-FR-FL)",
      "details" : null
    },
    {
      "value" : 37,
      "description" : "6.1 Channels (FRH-FLH-RR-RL-X-LFE-FR-FL)",
      "details" : null
    },
    {
      "value" : 38,
      "description" : "6 Channels (FRW-FLW-RR-RL-X-X-FR-FL)",
      "details" : null
    },
    {
      "value" : 39,
      "description" : "6.1 Channels (FRW-FLW-RR-RL-X-LFE-FR-FL)",
      "details" : null
    },
    {
      "value" : 40,
      "description" : "7 Channels (TC-RC-RR-RL-FC-X-FR-FL)",
      "details" : null
    },
    {
      "value" : 41,
      "description" : "7.1 Channels (TC-RC-RR-RL-FC-LFE-FR-FL)",
      "details" : null
    },
  },

```



```

    {
      "value" : 42,
      "description" : "7 Channels (FCH-RC-RR-RL-FC-X-FR-FL)",
      "details" : null
    },
    {
      "value" : 43,
      "description" : "7.1 Channels (FCH-RC-RR-RL-FC-LFE-FR-FL)",
      "details" : null
    },
    {
      "value" : 44,
      "description" : "7 Channels (TC-FCH-RR-RL-FC-X-FR-FL)",
      "details" : null
    },
    {
      "value" : 45,
      "description" : "7.1 Channels (TC-FCH-RR-RL-FC-LFE-FR-FL)",
      "details" : null
    },
    {
      "value" : 46,
      "description" : "7 Channels (FRH-FLH-RR-RL-FC-X-FR-FL)",
      "details" : null
    },
    {
      "value" : 47,
      "description" : "7.1 Channels (FRH-FLH-RR-RL-FC-LFE-FR-FL)",
      "details" : null
    },
    {
      "value" : 48,
      "description" : "7 Channels (FRW-FLW-RR-RL-FC-X-FR-FL)",
      "details" : null
    },
    {
      "value" : 49,
      "description" : "7.1 Channels (FRW-FLW-RR-RL-FC-LFE-FR-FL)",
      "details" : null
    }
  ]
},
"audio_encoding_type" : "LPCM",
},
"status" : {}, "inputs" : []
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.14 Multichannel Audio Output Node (MULTICH_AUDIO_OUTPUT)

This section describes a Node object with `type` member set to `MULTICH_AUDIO_OUTPUT`.

6.8.14.1 Configuration (MULTICH_AUDIO_OUTPUT)

None

6.8.14.2 Status

None

6.8.14.3 Inputs

Table 138: Multichannel Audio Output Node Object Member

Input Name	Since	Configurable	Description
main	3.0	Yes	Audio source. A <code>transmitter</code> may support none, some or all of the following: 3 -- Local Stereo audio 6 -- Local HDMI audio (stereo downmix) 7 -- Local HDMI audio 8 -- Multichannel audio subscription (audio return channel) 13 -- Stereo audio subscription 14 -- HDMI audio subscription 15 -- HDMI audio subscription (stereo downmix) A <code>receiver</code> may support none, some or all of the following: 3 -- Stereo audio subscription 6 -- HDMI audio subscription (stereo downmix) 7 -- HDMI audio subscription 8 -- Multichannel audio subscription 10 -- Local Stereo audio Note: Not all audio types are available to all endpoint devices, supported audio is based on the hardware design and capabilities. A <code>transceiver</code> may support none, some or all of the following: 3 -- Stereo audio subscription 6 -- HDMI audio subscription (stereo downmix) 7 -- HDMI audio subscription 8 -- Multichannel audio subscription 10 -- Local Stereo audio 11 -- Local HDMI audio 12 -- Local HDMI audio (stereo downmix) If a device has a multichannel audio output but does not support any of the choices above, then the following choice is listed: 0 -- No audio source available Attempting to set this last value as the selected input using a <code>set</code> property request will yield an <code>ILLEGAL_ARGUMENT</code> error.

6.8.14.4 Examples

Example shows a multichannel audio output node JSON object returned as part of the output of the “get device” command called on a transmitter target device.

```
{
  "type" : "MULTICH_AUDIO_OUTPUT",
  "index" : 0,
  "configuration" : {},
  "status" : {},
  "inputs" : [
    {
      "name" : "main",
      "index" : 0,
      "configuration" : {
        "source" : {
          "value" : 7,
          "choices" : [
            {
              "value" : 3,
              "description" : "Stereo Audio Local Loop Out",
              "details" : {
                "ref_class" : "NODE",
                "ref_type" : "STEREO_AUDIO_INPUT_OUTPUT",
                "ref_index" : 0
              }
            },
            {
              "value" : 6,
              "description" : "HDMI Audio Local Loop Out
(Downmix)",
              "details" : {
                "ref_class" : "NODE",
                "ref_type" : "HDMI_DECODER",
                "ref_index" : 0
              }
            },
            {
              "value" : 7,
              "description" : "HDMI Audio Local Loop Out (All
Channels)",
              "details" : {
                "ref_class" : "NODE",
                "ref_type" : "HDMI_DECODER",
                "ref_index" : 0
              }
            }
          ]
        }
      }
    },
    {
      "status" : {
        "source" : {
```

```

        "ref_class" : "NODE",
        "ref_type" : "HDMI_DECODER",
        "ref_index" : 0
    }
}
]
}

```

This second example shows an multichannel audio output node JSON object as returned as part of the output of the “get device” command called on a receiver target device.

```

{
  "type" : "MULTICH_AUDIO_OUTPUT",
  "index" : 0,
  "configuration" : {},
  "status" : {},
  "inputs" : [
    {
      "name" : "main",
      "index" : 0,
      "configuration" : {
        "source" : {
          "value" : 8,
          "choices" : [
            {
              "value" : 3,
              "description" : "Stereo Audio",
              "details" : {
                "ref_class" : "SUBSCRIPTION",
                "ref_type" : "STEREO_AUDIO",
                "ref_index" : 0
              }
            },
            {
              "value" : 6,
              "description" : "HDMI Audio (Stereo Downmix)",
              "details" : {
                "ref_class" : "SUBSCRIPTION",
                "ref_type" : "HDMI_AUDIO",
                "ref_index" : 0
              }
            }
          ],
        },
        {
          "value" : 7,
          "description" : "HDMI Audio (All Available
Channels)",
          "details" : {
            "ref_class" : "SUBSCRIPTION",
            "ref_type" : "HDMI_AUDIO",
            "ref_index" : 0
          }
        }
      ]
    }
  ]
}

```

```

    },
    {
      "value" : 8,
      "description" : "Multi Channel Audio",
      "details" : {
        "ref_class" : "SUBSCRIPTION",
        "ref_type" : "MULTICH_AUDIO",
        "ref_index" : 0
      }
    }
  ]
}
},
"status" : {
  "source" : {
    "ref_class" : "SUBSCRIPTION",
    "ref_type" : "MULTICH_AUDIO",
    "ref_index" : 0
  }
}
}
]
}
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.15 Infrared Decoder Node (INFRARED_DECODER)

This section describes a Node object with `type` member set to `INFRARED_DECODER`.

6.8.15.1 Configuration (INFRARED_DECODER)

None

6.8.15.2 Status

None

6.8.15.3 Inputs

None

6.8.15.4 Example

The example below shows an infrared decoder node JSON object as returned as part of the output of the `"get device"` command.

```

{
  "type" : "INFRARED_DECODER",
  "index" : 0,
  "configuration" : {},
  "status" : {},
  "inputs" : []
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.16 Infrared Encoder Node (INFRARED_ENCODER)

This section describes a Node object with `type` member set to `INFRARED_ENCODER`.

6.8.16.1 Configuration (INFRARED_ENCODER)

None

6.8.16.2 Status

```
{
    "data_length_max" : Integer
}
```

Table 139: Infrared Encoder Node Object Member

Member Name	Since	Type	Description
<code>data_length_max</code>	3.0	Integer	Maximum length, in bytes, of the Pronto code that can be sent to the infrared encoder.

6.8.16.3 Inputs

Table 140: Infrared Encoder Input Node Object Member

Input Name	Since	Configurable	Description
<code>network</code>	3.0	No	Associated network subscription.

6.8.16.4 Example

This example shows an infrared encoder node JSON object as returned as part of the output of “`get device`”.

```
{
  "type" : "INFRARED_ENCODER",
  "index" : 0,
  "configuration" : {},
  "status" : {},
  "inputs" : [{
    "name" : "network",
    "index" : 0,
    "configuration" : {},
    "status" : {
      "source" : {
        "ref_class" : "SUBSCRIPTION",
        "ref_type" : "INFRARED",
        "ref_index" : 0
      }
    }
  }]
}
```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.17 Network Interface Node (NETWORK_INTERFACE)

This section describes a Node object with `type` member set to `NETWORK_INTERFACE`.

6.8.17.1 Configuration (NetworkInterfaceNodeConfig)

```
{
  "hostname" : String,
  "ip" : NetInterfaceIPConfig,
  "dhcp_option_43" : DhcpOption43Config,
}
```

Table 141: Network Interface Node Object Members

Member Name	Since	Type	Description
hostname	3.0	String	Host name for the device on this interface
ip	3.0	NetInterfaceIPConfig	Internet Protocol (IP) configuration
dhcp_option_43	3.7	DhcpOption43Config	DHCP Option 43 configuration

Network Interface IP Configuration (NetInterfaceIPConfig)

```
{
  "address" : String,
  "mode" : String,
  "mask" : String,
  "gateway" : String
}
```

Table 142: Network Interface IP Configuration Node Object Members

Member Name	Since	Type	Description
address	3.0	String	IP address.
mode	3.0	String	One of the following: MANUAL Internet Protocol configuration is fixed and configured manually. DHCP Internet Protocol configuration is obtained from a DHCP server, with fallback on auto-IP if no DHCP server is available. AUTO_IP Internet Protocol configuration is obtained using Auto-IP, regardless of whether an DHCP server is present or not.
mask	3.0	String	IP subnet mask
gateway	3.0	String	IP address of default gateway

DHCP Option 43 Configuration (DhcpOption43Config)

```
{
  "control_server_addresses" : String Array
}
```

If the feature is supported by the device and DHCP server option is configured with IP, IP(s) are returned by API.

6.8.17.2 Status (NetworkInterfaceNodeStatus)

```
{
  "mac_address" : String,
  "ip" : NetInterfaceIPStatus
}
```

Table 143: Network Interface Node Status Object Members

Member Name	Since	Type	Description
mac_address	3.0	String	MAC address for the interface.
ip	3.0	NetInterfaceIPStatus	Internet Protocol (IP) status.

Network Interface IP Status (**NetInterfaceIPStatus**)

```
{
  "address" : String
}
```

Table 144: Network Interface IP Status Object Member

Member Name	Since	Type	Description
address	3.0	String	IP address

6.8.17.3 Inputs

None

6.8.17.4 Example

Example shows network interface node JSON object returned as part of the output of “get device”.

```
{
  "type" : "NETWORK_INTERFACE",
  "index" : 0,
  "configuration" : {
    "hostname" : "PHILIPPE_TX68",
    "ip" : {
      "address" : "10.1.1.37",
      "mode" : "DHCP",
      "mask" : "255.255.255.0",
      "gateway" : "10.1.1.1"
    },
    "dhcp_option_43" : {
      "control_server_addresses" : [
        "10.1.1.101"
      ]
    }
  },
  "status" : {
    "mac_address" : "001ec0f03668",
    "ip" : {
      "address" : "10.1.1.37"
    }
  },
  "inputs" : []
}
```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.18 Network Port Node (NETWORK_PORT)

This section describes a Node object with `type` member set to `NETWORK_PORT` which represents a physical 10GbE port (copper or fiber) on the device.

6.8.18.1 Status (NetworkPortNodeStatus)

```
{
  "active" : Boolean,
  "speed" : Real,
  "supports_speed_10g" : Boolean
  "supports_speed_5g" : Boolean
  "supports_speed_2_5g" : Boolean
  "supports_auto_10g" : Boolean
  "supports_auto_5g" : Boolean
}
```

Table 145: Network Port Node Status Object Members

Member Name	Since	Type	Description
active	3.0	Boolean	Indicates if the port currently selected is active. On dual-10GbE port devices with automatic switchover, only one port is active at a time.
speed	3.0	Real	The current speed of the port in Gbps.
supports_speed_10g	3.0	Boolean	Indicates if the copper Ethernet PHY supports 10Gbps speed configuration.
supports_speed_5g	3.0	Boolean	Indicates if the copper Ethernet PHY supports 5Gbps speed configuration.
supports_speed_2_5g	3.0	Boolean	Indicates if the copper Ethernet PHY supports 2.5Gbps speed configuration.
supports_auto_10g	3.0	Boolean	Indicates if the copper Ethernet PHY supports auto-negotiation up to 10Gbps speed configuration.
supports_auto_5g	3.0	Boolean	Indicates if the copper Ethernet PHY supports auto-negotiation up to 5Gbps speed configuration.

6.8.18.2 Inputs

None

6.8.18.3 Example

Example shows network switch node JSON object returned as part of the output of “`get device`” command.

```
{
  "type" : "NETWORK_PORT",
  "index" : 0,
  "configuration" : {
    "speed" : "AUTO_10G"
  },
  "status" : {
    "active" : true,
    "speed" : 5.0,
    "supports_speed_10g" : true,

```

```

        "supports_speed_5g" : true,
        "supports_speed_2_5g" : true,
        "supports_auto_10g" : true,
        "supports_auto_5g" : true
    },
    "inputs" : []
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.19 Network Switch Node (NETWORK_SWITCH)

This section describes a Node object with `type` member set to `NETWORK_SWITCH`.

6.8.19.1 Configuration (NetworkSwitchNodeConfig)

```

{
    "gigabit_port_1_enable" : Boolean
}

```

Table 146: Network Switch Node Object Member

Member Name	Since	Type	Description
<code>gigabit_port_1_enable</code>	3.0	Boolean	Enable the 1GbE port on the device.

6.8.19.2 Status

None

6.8.19.3 Inputs

None

6.8.19.4 Example

Example shows network switch node JSON object returned as part of the output of “`get device`” command.

```

{
    "type" : "NETWORK_SWITCH",
    "index" : 0,
    "configuration" : {
        "gigabit_port_1_enable" : false
    },
    "status" : {},
    "inputs" : []
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.20 Scaler Node (SCALER)

This section describes a Node object with `type` member set to `SCALER`.

6.8.20.1 Configuration (ScalerNodeConfig)

```

{
    "width" : Integer,
    "height" : Integer
}

```

```
}
```

Table 147: Scaler Node Object Members

Member Name	Since	Type	Description
width	3.0	Integer	Scaled video width, in pixels
height	3.0	Integer	Scaled video height, in pixels

6.8.20.2 Status (ScalerNodeStatus)

```
{  
    "video" : VideoFormat  
}
```

Table 148: Scaler Node Status Node Object Member

Member Name	Since	Type	Description
video	3.0	VideoFormat	Output video format of the scaler.

6.8.20.3 Inputs

Table 149: Scaler Input Node Object Member

Input Name	Since	Configurable	Description
main	3.0	Yes	Main video input. An HDMI decoder node or, for devices that support it, a frame rate converter to select a different frame rate.

6.8.20.4 Example

Example shows a scaler node JSON object as returned as part of the “get device” command.

```
{  
    "type" : "SCALER",  
    "index" : 0,  
    "configuration" : {  
        "width" : 0,  
        "height" : 0  
    },  
    "status" : {  
        "video" : {  
            "width" : 0,  
            "height" : 0,  
            "frames_per_second" : 60,  
            "color_space" : "RGB",  
            "bits_per_pixel" : 8,  
            "scan_mode" : "PROGRESSIVE"  
        }  
    },  
    "inputs" : [  
        {  
            "name" : "main",  
            "index" : 0,  
            "configuration" : {  
                "source" : {
```

```

        "value" : 0,
        "choices" : [
            {
                "value" : 0,
                "description" : "HDMI video input",
                "details" : {
                    "ref_class" : "NODE",
                    "ref_type" : "HDMI_DECODER",
                    "ref_index" : 0
                }
            },
            {
                "value" : 1,
                "description" : "Frame rate converter",
                "details" : {
                    "ref_class" : "NODE",
                    "ref_type" : "FRAME_RATE_CONVERTER",
                    "ref_index" : 0
                }
            }
        ]
    },
    "status" : {
        "source" : {
            "ref_class" : "NODE",
            "ref_type" : "HDMI_DECODER",
            "ref_index" : 0
        }
    }
}
]
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.21 Thumbnail Generator Node (THUMBNAIL_GENERATOR)

This section describes a Node object with type member set to THUMBNAIL_GENERATOR, applicable only the NT1000 and NT2000 endpoint devices.

Refer to Section 6.8 Nodes (Node) for details.

6.8.21.1 Configuration (ThumbnailGeneratorNodeConfig)

```

{
    "frame_rate" : Real,
    "hdcp" : Boolean,
    "ssrc" : Integer,
    "udp_port" : Integer
}

```

Table 150: Thumbnail Generator Node Object Members

Member Name	Since	Type	Description
frame_rate	3.0	Real	The requested frame rate for the thumbnail stream.
hdcp	3.0	Boolean	Whether thumbnails are produced when video input is HDCP protected.
ssrc	3.0	Integer	RTP synchronization source (SSRC) identifier. This 32-bit value must be unique for each thumbnail source. See RFC 3550 section 8 for detail.
udp_port	3.0	Integer	Destination UDP port.

6.8.21.2 Status (ThumbnailGeneratorNodeStatus)

```

{
  "width" : Integer,
  "height" : Integer,
  "frame_rate" : Real,
  "bits_per_comp" : Integer,
  "color_space" : String,
  "colorimetry" : String
}

```

Table 151: Thumbnail Generator Node Status Members

Member Name	Since	Type	Description
width	3.0	Integer	The width of each thumbnail
height	3.0	Integer	The height of each thumbnail.
frame_rate	3.0	Real	The actual frame rate for the thumbnail stream.
bits_per_comp	3.0	Integer	The value 8 which indicates that there are 8 bits per colour component.
color_space	3.0	String	The string YCBCR_422 which indicates the thumbnail stream is in the YCbCr colour space with 4:2:2 chroma subsampling.
colorimetry	3.0	String	The colorimetry of the thumbnail stream, is either BT709 for BT.709 or BT601 for BT.601

6.8.21.3 Inputs

None

6.8.21.4 Example

The example shown below shows a thumbnail generator node JSON object as returned as part of the output of a “get device” request for an NT2000 endpoint device.

```

{
  "type" : "THUMBNAIL_GENERATOR",
  "index" : 0,
  "configuration" : {
    "frame_rate" : 10.00,
    "hdcp" : false,
    "ssrc" : 1234,
    "udp_port" : 6972
  },
  "status" : {
    "width" : 320,
    "height" : 180,
    "frame_rate" : 10.00,

```

```

        "bits_per_comp" : 8,
        "color_space" : "YCBCR_422",
        "colorimetry" : "BT601"
    },
    "inputs" : []
}

```

6.8.22 UART Node (UART)

This section describes a Node object with `type` member set to `UART`.

6.8.22.1 Configuration (UartNodeConfig)

```

{
    "baud_rate" : Integer,
    "data_bits" : Integer,
    "stop_bits" : Integer,
    "parity" : String
}

```

Table 152: UART Node Configuration Object Members

Member Name	Since	Type	Description
baud_rate	3.0	Integer	Baud rate.
data_bits	3.0	Integer	Number of data bits: 6, 7 or 8.
stop_bits	3.0	Integer	Number of stop bits: 1 or 2.
parity	3.0	String	Parity: NONE, ODD or EVEN.

6.8.22.2 Status

```

{
    "usage" : String
}

```

Table 153: UART Node Status Object Member

Member Name	Since	Type	Description
usage	3.0	String	Indicates the UART port usage. Can have one of the following values: EXPOSED Port with an external connector RESERVED Port internally connected, for example to an external MCU.

6.8.22.3 Inputs

Table 154: UART Node Inputs Object Member

Input Name	Since	Configurable	Description
network	3.0	No	Associated network subscription

6.8.22.4 Example

Example shows a UART node JSON object as returned as part of the output of a “`get device`” command.

```

{
    "type" : "UART",
    "index" : 0,
    "configuration" : {
        "baud_rate" : 57600,

```

```

        "data_bits" : 8,
        "stop_bits" : 1,
        "parity" : "NONE"
    },
    "status" : {
        "usage" : "RESERVED"
    },
    "inputs" : [{
        "name" : "network",
        "index" : 0,
        "configuration" : {},
        "status" : {
            "source" : {
                "ref_class" : "SUBSCRIPTION",
                "ref_type" : "RS232",
                "ref_index" : 0
            }
        }
    }]
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.23 USB Human Interface Device (HID) Extender Node (USB_HID)

This section describes a Node object with `type` member set to `USB_HID`, represents a USB Human Interface Device (HID) extender.

6.8.23.1 Configuration (UsbNodeConfig)

```

{
    "role" : String
    "peers" : [
        {
            "pairing_type" : "pairing_type_value",
            "address" : "peer_address_value",
            "device_id" : "device_id_value"
        }
    ],
    "security" : {
        "level" : "security_level_value",
    }
}

```

Table 155: USB HID Configuration Node Object Members

Member Name	Since	Type	Description
role	3.0	String	The role configuration of the device as a USB extender: DISABLED USB support is disabled. LOCAL for a USB device that connects to a host (e.g. PC). REMOTE for a USB host to which devices connect (keyboard, mouse, etc.). OTG (On-the-Go) the device automatically selects the role to be

Member Name	Since	Type	Description
			applied, either REMOTE or LOCAL, according to connected device type: USB device (REMOTE) or USB Host (LOCAL).
peers pairing_type	3.5	String	The USB HID peer pairing type, can either be: DEVICE_ID USB pairing type is by device id. IP_ADDRESS USB pairing type is by IP address.
peers address	3.5	String	The USB HID peer address, can either be the IP address or an empty string if not set.
peers device_id	3.5	String	The USB HID peer device ID, can either be the device id or an empty string if not set.
security level	3.2	String	Indicates USB HID device types to which encryption is to be applied. DISABLED Encryption is not enabled. ENABLED Encryption is added to IP communication for all USB HID devices.

6.8.23.2 Status (UsbNodeStatus)

```
{
  "usb_devices" : [USBDeviceStatus, ...],
  "usb_host" : USBHostStatus,
  "role" : String,
  "peers" : [PeerLinked, ...],
  "remote_extender_statistics" : RemoteExtenderStatistics
}
```

Table 156: USB HID Device Status Node Object Members

Member Name	Since	Type	Description
usb_devices	3.0	Array of USBDeviceStatus	The status of USB devices. - If device configured as a USB host (REMOTE role), member reports status of USB devices connected to it. - If device configured as a USB device (LOCAL role), reports status of USB devices on the device to which it is paired. - If USB support disabled, content of this member unspecified.
usb_host	3.0	USBHostStatus	Status of the USB host (e.g. PC) connected to this device. Member to be ignored if device configured as other than a USB device (i.e. USB host or disabled).
role	3.0	String	The role status of the device as a USB extender: DISABLED USB support is disabled. LOCAL for a USB device that connects to a host (e.g. PC).

Member Name	Since	Type	Description
			REMOTE for a USB host to which devices connect (keyboard, mouse, etc.).
peers	3.5	Array of PeerLinked	The linked status of the USB HID peers.
remote_extender_statistics	3.7	RemoteExtenderStatistics	Statistics related to the remote USB HID extender.

The USBDeviceStatus JSON object has the following members:

```
{
  "status" : String
}
```

Table 157: USB HID Device Status Node Object Member

Member Name	Since	Type	Description
status	3.0	String	One of the following: DISCONNECTED No USB device is connected. CONFIGURING USB device being configured following device connection. This is a transient state. CONNECTED USB device connected, configured and ready to use.

The USBHostStatus JSON object has the following members:

```
{
  "status" : String
}
```

Table 158: USB HID Host Status Node Object Member

Member Name	Since	Type	Description
status	3.0	String	One of the following: CONNECTED Device is connected to USB host and fully configured. DISCONNECTED Device is disconnected from the host. CONFIGURING Device being configured by the USB host following connection. This is a transient state.

The PeerLinked JSON object has the following members:

```
{
  "linked" : Boolean
}
```

Table 159: USB HID Peer Linked Node Object Member

Member Name	Since	Type	Description
linked	3.5	Boolean	The linked status of the peer, true for linked, false otherwise.

The RemoteExtenderStatistics JSON object has the following members:

```
{
  "connected_usb_hub" : Integer,
  "connected_supported_usb_devices" : Integer,
  "connected_unsupported_usb_devices" : Integer,
  "interrupt_pipes" : Integer,
  "bulked_pipes" : Integer
}
```

Table 160: USB HID Remote Extender Statistics Node Object Members

Member Name	Since	Type	Description
connected_usb_hub	3.7	Integer	Number of USB hubs connected to the remote connector.
connected_supported_usb_devices	3.7	Integer	Number of supported USB devices connected to the remote connector.
connected_unsupported_usb_devices	3.7	Integer	Number of unsupported USB devices connected to the remote connector.
interrupt_pipes	3.7	Integer	Number of active interrupt endpoints.
bulked_pipes	3.7	Integer	Number of active bulk endpoints.

6.8.23.3 Inputs

None

6.8.23.4 USB Stream Status

If USB HID encryption is enabled, the encryption pairing status included in the stream status of the USB_HID stream type. Below shows an example of the USB_HID stream node:

```
{
  "type" : "USB_HID",
  "index" : 0,
  "configuration" : {
    "address" : "225.225.225.225",
    "enable" : true,
    "auto_stop" : false
  },
  "status" : {
    "state" : "STREAMING",
    "security" : {
      "status" : "ENCRYPTED"
    }
  },
  "source" : {
    "ref_class" : "NODE",
    "ref_type" : "USB_HID",
    "ref_index" : 0
  }
}
```

6.8.23.5 Example

Example below is a USB HID extender node JSON object returned as part of a `get_device` request.

```

{
  "type" : "USB_HID",
  "index" : 0,
  "configuration" : {
    "role" : "HOST"
  },
  "status" : {
    "usb_devices" : [
      {
        "status" : "CONNECTED"
      },
      {
        "status" : "DISCONNECTED",
      },
      {
        "status" : "DISCONNECTED",
      }
    ],
    "usb_host" : {
      "status" : "CONNECTED"
    },
    "role" : "REMOTE"
  },
  "inputs" : {}
}

```

This example is of the USB HID extender node with IP communication security enabled:

```

{
  "type" : "USB_HID",
  "index" : 0,
  "configuration" : {
    "role" : "REMOTE",
    "security" : {
      "level" : "ENABLED",
      "encryption_key_type" : "DEFAULT"
    }
  },
  "status" : {
    "usb_devices" : [
      {
        "status" : "CONNECTED"
      },
      {
        "status" : "DISCONNECTED",
      },
      {
        "status" : "DISCONNECTED",
      }
    ],
    "usb_host" : {
      "status" : "CONNECTED"
    }
  }
}

```

```

        "role" : "REMOTE",
    }
},
"inputs" : {}
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.24 USB Extender Node (USB_EXTENDER)

This section describes a Node object with `type` member set to `USB_EXTENDER`. It is used for Semtech UEX or Icron USB 2.0 Extender. If a second UEX is attached to the AVP, there will be two `USB_EXTENDER` nodes (one for each UEX).

6.8.24.1 Configuration (UsbExtenderChipNodeConfig)

```

{
    "role" : String,
    "program_mode" : String,
    "rs232_port" : Integer,
    "peers" : [
        {
            "avp_attached" {
                "device_name" : "avp_device_name_value",
                "device_id" : "avp_device_id_value",
            }
            {
                "device_id" : "uex_device_id_value",
                "device_name" : "uex_device_name_value",
                "mac_address" : "mac_address_value"
            }
        }
    ]
    "device_filtering" :
    {
        "block_iso_transfers" : Boolean,
        "class_filerling" : String,
        "allow_api_commands" : Boolean,
        "allow_i2c_control" : Boolean,
    },
    "remote_vbus_ctrl": "AUTO",
    "msa_enable": {
        "bot_protocol" : Boolean,
        "uas_protocol" : Boolean,
    }
}

```

Table 161: USB Extender Configuration Node Object Members

Member Name	Since	Type	Description
role	3.8	String	The role configuration of the USB extender: LOCAL for a local extender (LEX). REMOTE for a remote extender (REX). PIN for role from the UEX pin (only for UEX2000 standalone) DISABLED for USB disable (only for UEX).
program_mode	3.8	String	The enabled programming mode for the USB extender: FIRMWARE for the extender's firmware. EEPROM for the extender's non-volatile configuration memory. Applies to Icron only. NONE if programming is disabled Note: Programming of the Semtech UEX is done with Semtech ProAV programming tool (mainly for first time programming or if network is not available). Icron module is managed by the hardware manufacturer.
rs232_port	3.8	Integer	The (zero-based) number of the RS-232 port through which programming of the USB extender is enabled. If value of the program_mode member is NONE, value is undefined.
peer_avp_attached_device_id	3.8	String	If currently paired to a BlueRiver device, the device ID of the paired BlueRiver device is shown associated with the peer USB Extender device. If the USB Extender is not paired or Semtech UEX or Icron module is not detected or if the BlueRiver device is paired with a standalone UEX device, then the field is not shown.
peer_avp_attached_device_name	3.8	String	If currently paired, the device name of the paired BlueRiver device is shown associated with the peer USB Extender device. If the USB Extender is not paired or Semtech UEX or Icron module is not detected or if the BlueRiver device is paired with a standalone UEX device, then the field is not shown.
peer_device_id	3.8	String	If currently paired, the device ID of the paired UEX device is shown associated with the peer USB Extender device. If the BlueRiver device is paired with Icron or is not paired or Semtech UEX is not detected the field is not shown.
peer_device_name	3.8	String	If currently paired, the device name of the paired UEX device is shown associated with the peer USB Extender device. If the BlueRiver device is paired with Icron or is not paired or Semtech UEX is not detected the field is not shown.
peer_mac_address	3.8	String	If paired, provides MAC address of the peer USB Extender module. If the USB extender is not paired or a Semtech or Icron module has not been detected then this field is blank ("").
device_filtering_block_iso_transfers	3.8	Boolean	Whether USB ISO transfers are blocked or not.

Member Name	Since	Type	Description
device_filtering class_filtering	3.8	String	Whether USB device classes are blocked, options are: ALLOW_ALL to allow all USB device classes BLOCK_MASS_STORAGE to block USB mass storage devices BLOCK_ALL_EXCEPT_HID to block all USB device classes except HID BLOCK_ALL_EXCEPT_HID_SMART_CARD to block all USB device classes except HID and smart card readers BLOCK_ALL_EXCEPT_AUDIO_VENDOR_SPECIFIC to block all USB device classes except audio and vendor specific
device_filtering allow_api_commands	3.8	Boolean	Whether the USB Extender device filtering options are allowed to be configured via API (currently not supported).
device_filtering allow_i2c_control	3.8	Boolean	Whether the USB Extender device filtering options are allowed to be configured via I2C registers (currently not supported).
remote_vbus_ctrl	3.8	String	API control value of AUTO, LOW or HIGH. AUTO indicates VBUS on the remote extender automatically follows the VBUS value seen by the Local extender when the remote extender is linked to a paired local extender. LOW indicates VBUS stays deactivated (0V) regardless of the VBUS state on the local extender or the link connection state. HIGH indicates VBUS stays activated (5V) when linked to a local extender regardless of the VBUS state on the local extender.
msa_enable	3.9	Boolean	Whether Mass Storage Acceleration (MSA) is enabled. When set to this msa_enable field directly, applies to both bot_protocol and uas_protocol subfields for mass storage USB devices using BOT (Bulk-Only Transport) protocol and devices using UAS (USB Attached SCSI) protocol.
msa_enable. bot_protocol	3.9	Boolean	Whether Mass Storage Acceleration (MSA) is enabled for mass storage USB devices using BOT (Bulk-Only Transport) protocol.
msa_enable. uas_protocol	3.9	Boolean	Whether Mass Storage Acceleration (MSA) is enabled for mass storage USB devices using UAS (USB Attached SCSI) protocol.

6.8.24.2 Status (UsbExtenderChipNodeStatus)

```
{
  "chip_present" : Boolean,
  "device_id" : String,
  "device_name" : String,
  "mac_address" : String,
  "chipset_type" : String,
  "reachable" : Boolean,
  "revision" : String,
  "ip_mode" : String,
  "ip_address" : String,
  "link_status_support" : Boolean,
  "sui_enabled" : Boolean,
  "peers" : [
    {
      "linked" : Boolean
    }
  ]
  "role" : String,
}
```

Table 162: USB Extender Status Node Object Members

Member Name	Since	Type	Description
chip_present	3.8	Boolean	TRUE indicates that UEX or an Icron module is detected. Property is FALSE if UEX or Icron USB enabled in the hardware configuration but has not been populated.
device_id	3.8	String	The device ID of UEX USB Extender device. If the USB Extender is an Icron module, this field is not shown.
device_name	3.8	String	The device ID of UEX USB Extender device. If the USB Extender is an Icron module, this field is not shown.
mac_address	3.8	String	MAC address of the Semtech UEX or Icron USB Extender device. If it has not been detected, this field is blank ("").
chipset_type	3.8	String	The USB Extender device model: UEX2000 for Semtech UEX2000 USB Extender UEX1000 for Semtech UEX1000 USB Extender ICRON for Icron USB Extender
reachable	3.8	Boolean	Indicates that the USB extender network interface is reachable (TRUE) or not (FALSE) with an IP address.
revision	3.8	String	The revision level of the Semtech UEX or Icron firmware.
ip_mode	3.8	String	The active IP mode, either DHCP or manual (static).
ip_address	3.8	String	The active IP address.
link_status_support	3.8	Boolean	Indicates whether the USB Extender supports the linked attribute.
sui_enabled	3.8	Boolean	Indicates whether the USB Extender SUI support is enabled.
peers linked	3.8	Boolean	Indicates if bidirectional communication is currently established. Devices need to negotiate the link before they can successfully link. TRUE indicates this negotiation successful. If devices currently are not paired or this link is not successfully

Member Name	Since	Type	Description
			established, the status is FALSE. The field is also FALSE if the link_status_support field reports FALSE.
role	3.8	String	The role status of the USB extender: LOCAL for a local extender (LEX). REMOTE for a remote extender (REX). DISABLED for USB disable (only for UEX).

6.8.24.3 Inputs

None

6.8.24.4 Example

The example provided shows the USB Extender node JSON object returned as part of a get settings request.

```
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "device_node" : [
      {
        "device_id" : "0016c035566e",
        "device_name" : "AVP_0016c035566e",
        ...
      }
    ],
    "type" : "USB_EXTENDER",
    "index" : 0,
    "configuration" : {
      "role" : "REMOTE",
      "program_mode" : "NONE",
      "rs232_port" : 0,
      "peers" : [ {
        "avp_attached" {
          "device_id" : "d8803962e132",
          "device_name" : "AVP_d8803962e132",
        }
      },
      {
        "device_id" : "0016c035566f ",
        "device_name" : "USB_0016c035566f",
        "mac_address" : "0016c035566f"
      }
    ],
    "device_filtering" : {
      "block_iso_transfers" : false,
      "class_filtering" : "ALLOW_ALL",
      "allow_api_commands" : false,
      "allow_i2c_control" : false
    }
  },
  "status" : {
    "chip_present" : true,
  }
}
```



```

        "device_id" : "0016c35571d",
        "device_name" : "USB_0016c35571d",
        "mac_address" : "0016c35571d",
        "chipset_type" : "UEX1000",
        "reachable" : true,
        "revision" : "1.0.0.0",
        "ip_mode" : "DHCP",
        "ip_address" : "192.168.100.22",
        "link_status_support" : true,
        "sui_enabled" : false,
        "peers" : [ {
            "linked" : true
        }
    ],
    "role" : "REMOTE"
},
"inputs" : []
}
},
{
    "device_id" : "d8803962e132",
    "device_name" : "AVP_d8803962e132",
    {
        "type" : "USB_EXTENDER",
        "index" : 0,
        "configuration" : {
            "role" : "LOCAL",
            "program_mode" : "NONE",
            "rs232_port" : 0,
            "peers" : [ {
                "avp_attached" {
                    "device_id" : "0016c035566e ",
                    "device_name" : "AVP_0016c035566e",
                }
                "device_id" : "0016c35571d",
                "device_name" : "USB_0016c35571d",
                "mac_address" : "0016c35571d"
            }
        ],
        "device_filtering" : {
            "block_iso_transfers" : false,
            "class_filtering" : "ALLOW_ALL",
            "allow_api_commands" : false,
            "allow_i2c_control" : false
        }
    },
    "status" : {
        "chip_present" : true,
        "device_id" : "0016c035566f",
        "device_name" : "USB_0016c035566f",
        "mac_address" : "0016c035566f",
    }
}

```

```

        "chipset_type" : "UEX1000",
        "reachable" : true,
        "revision" : "1.0.0.0",
        "ip_mode" : "DHCP",
        "ip_address" : "192.168.100.20",
        "link_status_support" : true,
        "sui_enabled" : false,
        "peers" : [ {
            "linked" : true
        } ],
        "role" : "LOCAL"
    },
    "inputs" : []
}
}
],
"error" : []
},
"error" : null
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.25 Video Input Node (VIDEO_INPUT)

This section describes a Node object with `type` member set to `VIDEO_INPUT`. This node represents a physical video input connector.

6.8.25.1 Configuration (VIDEO_INPUT)

None

6.8.25.2 Status

```

{
    "connector_type" : String
}

```

Table 163: Video Input Node Object Member

Member Name	Since	Type	Description
<code>connector_type</code>	3.0	String	Indicates type of video input connector, is one of the following: HDMI for an HDMI connector. DISPLAY_PORT for a DisplayPort connector.

6.8.25.3 Inputs

None

6.8.25.4 Example

The example below shows a video input node JSON object as returned as part of the output of the “`get device`” command.

```

{

```

```

    "type" : "VIDEO_INPUT",
    "index" : 0,
    "configuration" : {},
    "status" : {
        "connector_type" : "HDMI"
    },
    "inputs" : []
}

```

For more information on the `get` command, refer to Section 4.7 Command `get` for details.

6.8.26 Video Compressor Node (VIDEO_COMPRESSOR)

This section describes a Node object with `type` member set to `VIDEO_COMPRESSOR`.

6.8.26.1 Configuration (VIDEO_COMPRESSOR)

```

{
    "always_compress" : Boolean,
    "version" : Integer
}

```

Table 164: Video Compressor Node Object Members

Member Name	Since	Type	Description
<code>always_compress</code>	3.0	Boolean	When true, HDMI stream 0 is always compressed without regards to the video format. Otherwise, compressed only when it is necessary to keep the video bandwidth under 9Gbit/s. Note: HDMI stream 1 is never compressed.
<code>version</code>	3.0	Integer	Video compression version to apply. Two options available: 2 For improved video compression (recommended). 1 For backward compatibility with devices that only support this version. Refer to the <code>version_support</code> member in Section 6.8.26.2 for the maximum version supported by a device. Note: All devices receiving compressed video, HDMI stream 0 with compression enabled, must be configured to decompress with same compression version. With ASIC firmware version 1.5 or higher, the decoder (RX) can automatically detect and decode both versions of compression (encoder (TX) must also be using firmware version 1.5 or higher). Refer to Section 6.8.27.1 for configuration of the decompressor.

6.8.26.2 Status

```

{
    "version_support" : Integer
}

```

Table 165: Video Compressor Status Node Object Member

Member Name	Since	Type	Description
version_support	3.0	Integer	Maximum compression version supported by this device. Can be either 1 or 2.

6.8.26.3 Inputs

None

6.8.26.4 Example

The example below shows a video compressor node JSON object as returned as part of the output of the “get device” command.

```
{
  "type" : "VIDEO_COMPRESSION",
  "index" : 0,
  "configuration" : {
    "always_compress" : false,
    "version" : 2},
  "status" : {
    "supported_version" : 2},
  "inputs" : []
}
```

For more information on the get command, refer to Section 4.7 Command get for details.

6.8.27 Video Decompressor Node (VIDEO_DECOMPRESSOR)

This section describes a Node object with type member set to VIDEO_DECOMPRESSOR.

6.8.27.1 Configuration (VIDEO_DECOMPRESSOR)

```
{
  "version" : Integer
}
```

Table 166: Video Decompressor Node Object Member

Member	Since	Type	Description
version	3.0	Integer	Video compression version to use for decompression. Can be either: 2 Improved video compression (recommended). 1 Backwards compatibility with devices that only support this version. Refer to the version_support member in Section 6.8.27.2 for the maximum version supported by a device. Must be set to same version as device sending the compressed video, such as HDMI stream 0 if compression enabled. With ASIC firmware version 1.5 or higher, the decoder (RX) can automatically detect and decode both versions of compression (encoder (TX) must also be using firmware version 1.5 or higher). Refer to Section 6.8.26.1 for configuration of compression version for sending device.

6.8.27.2 Status

```
{
    "version_support" : Integer
}
```

Table 167: Video Decompressor Status Node Object Member

Member Name	Since	Type	Description
version_support	3.0	Integer	Maximum compression version supported by device. Can be either 1 or 2. Refer to Table 166: Video Decompressor Node Object Member for definitions.

6.8.27.3 Inputs

None

6.8.27.4 Example

Example shows a video decompressor node JSON object returned as part of the “get device”.

```
{
    "type" : "VIDEO_DECOMPRESSION",
    "index" : 0,
    "configuration" : {
        "version" : 2},
    "status" : {
        "supported_version" : 2},
    "inputs" : []
}
```

For more information on the get command, refer to Section 4.7 Command get for details.

7 Configurable Device Properties

Some device properties are configurable, this section reviews the device properties that can be managed using the `set property` command. Important to note that not all properties are supported by all devices.

As specified in the argument descriptions of the `set property` command, the key is a path that refers to an object member inside the device object, starting at the root of the device object. Refer to Section 4.19 Command `set` and Section 7 Configurable Device Properties for more information.

7.1 Device Configuration

The following table lists the configurable properties in the device configuration section of the device object. Refer to Section 6.3 Device Capabilities (`DeviceCapabilities`) for more information.

Table 168: Device Configuration Properties

Property Key		
configuration.device_name		
Since	3.0	
Type	String	
Description	Device name, maximum 19 characters. All displayable characters in the ASCII character set are allowed, meaning a to z, A to Z, 0 to 9, space, as well as symbols. It is possible to use the device name as an API request instead of the device ID (MAC address).Applicable to BlueRiver and UEX devices. Note: As of the BlueRiver firmware 1.5 the device name is persistent following a factory reset, therefore is not reverted back to the device MAC address anymore. Refer to the SDVoE Developers API Application Note Library for example and details of using the device name. Note: As of the API 3.6 the device name can be reverted back to the device MAC address for AVP, and as of API 3.8 device MAC address with prefix USB_ for UEX, by using the special keyword value <code>mac_address</code> when setting this <code>device_name</code> property. Refer to the SDVoE Developers API Application Note Library, section Resetting Device Name to MAC Address for details and example. Note: This property can only be set for a single device at a time, except if the special keyword value <code>mac_address</code> is used (in this specific case, targets ALL/ALL_RX/ALL_TX are also accepted).	
configuration.resume_streaming		
Since	3.0	
Type	Boolean	
Description	Specifies whether device streams are restarted automatically after a reboot. Applicable to BlueRiver devices. To enable this option specify <code>TRUE</code> (default) and to disable it specify <code>FALSE</code>. Notes: i. Current subscriptions of the device being rebooted remain subscribed. ii. If <code>auto_stop</code> is enabled for the scaled stream (stream 1), then stream 1 is not restarted, refer to Section 7.2 Stream Configuration for details on this option.	
configuration.device_mode		

Property Key		
Since	3.1	
Type	String	
Description	This property used to modify the device mode of an BlueRiver device. Availale options are: TRANSMITTER RECEIVER TRANSCEIVER Refer to Table 169: Valid Device Mode Changes for details and applying this setting to devices.	
configuration.locate_mode		
Since	3.0	
Type	Boolean	
Description	When enabled LEDS flash for identification purposes. Allows the physical device to be located when in a group, such as an equipment rack. Applicable to BlueRiver and UEX devices. To enable this option specify TRUE and to disable it specify FALSE (default).	

Table 169: Valid Device Mode Changes

ASIC	Hardware**	TRANSMITTER	RECEIVER	TRANSCEIVER
AVP2000T	Video in/out	Transmitter	Receiver	Transceiver
	Video in	Transmitter	Invalid, no change	Invalid, no change
	Video out	Invalid, no change	Receiver	Invalid, no change
AVP2000/AVP1000	Video in/out	Transmitter	Receiver	Invalid, no change
	Video in	Transmitter	Invalid, no change	Invalid, no change
	Video out	Invalid, no change	Receiver	Invalid, no change

** Important to note that **hardware** refers to the HDMI connectors present on the BlueRiver ASIC devices, possible that device has only a Video In or Video Out for transmitter and receiver devices, respectively.

7.2 Stream Configuration

The following table lists the configurable properties in the stream configuration section of the device object. Refer to Section 6.6.1 Stream Configuration (StreamConfig) for details.

Table 170: Stream Configuration Properties

Property Key		
streams[stream_type:index].configuration.source.value		
Since	3.0	
Type	Integer	
Description	Stream source. Select between original and framerate converted version of HDMI stream 0. Since applicable to stream 0, the stream_type index is always 0. TIP: Frame rate divider can also be applied to the scaled stream (stream 1), refer to Section 7.15 Scaler Node Configuration. Supported by AVP2000 devices.	
streams[stream_type:index].configuration.remove_audio		
Since	3.0	
Type	Integer	
Description	AUX audio filtering, remove the aux audio from HDMI stream 0. TRUE = enable FALSE = disable Where TRUE enables the feature removing the audio. The index for audio is always 0.	

Property Key		
streams[HDMI:index].configuration.auto_stop		
Since		3.0
Type		Integer
Description		When auto_stop is enabled, if video input becomes unstable scaled stream (stream 1) is stopped. When condition is resolved, scaled stream needs to be started (command start). TRUE = enable FALSE = disable Since applicable to scaled stream only, therefore the HDMI : index is always 1.

7.3 Stereo Audio Input/Output Node Configuration

The following table lists the configurable properties of a stereo audio input/output node of the device object. Refer to Section 6.8.3 Stereo Audio Input/Output Node (STEREO_AUDIO_INPUT_OUTPUT) for details.

Note: The STEREO_AUDIO_INPUT_OUTPUT:index is 0 and applicable only to encoder (TX) or transceiver (TR) devices. Confirm a devices capabilities using the get capabilities command.

Table 171: Stereo Audio Input/Output Node Configuration Properties

Property Key		
nodes[STEREO_AUDIO_INPUT_OUTPUT:index].configuration.direction		
Since		3.0
Type		String
Description		Set the direction of the stereo audio port for a transmitter: INPUT to set the port as an input OUTPUT to set the port as an output
nodes[STEREO_AUDIO_INPUT_OUTPUT:index].inputs[main:0].configuration.source.value		
Since		3.0
Type		Integer
Description		Audio input source. Transmitter (TX) may support none, some or all of following: 2 -- Local HDMI audio (stereo downmix) 8 -- Multichannel audio subscription (audio return channel)

7.4 Stereo Audio Output Node Configuration

The following table lists the configurable properties of a stereo audio output node of the device object. Refer to Section 6.8.4 Stereo Audio Output Node (STEREO_AUDIO_OUTPUT) for details.

Table 172: Stereo Output Node Configuration Properties

Property Key		
nodes[STEREO_AUDIO_OUTPUT:index].inputs[main:0].configuration.source.value		
Since		3.0
Type		Integer
Description		Set stereo audio output source. Device may support none, some or all of following: 2 -- HDMI audio (stereo downmix) 3 -- Stereo audio 8 -- Multichannel audio subscription Note: Index is 0; applicable to all device types, receivers (RX), transmitters (TX) and transceivers (TR). Confirm device capabilities using get capabilities.

7.5 CEC Node Configuration

The following table lists the configurable properties of the CEC node of the device object. Refer to Section 6.8.6 CEC Node (CEC) for details.

Table 173: CEC P2P Node Configuration Property

Property Key	
nodes[CEC:index].configuration.p2p_mode.value	
Since	3.2
Type	String
Description	Two options are available, API and PASSTHROUGH, where latter is the default. When set to API, CEC injection is done via a set property request for ASIC devices operating in Point-to-Point mode. When API is enabled, Passthrough of CEC data is disabled. Similarly when the mode is set to PASSTHROUGH the API mode is disabled. Note: The CEC index is 0 and applicable to all ASIC devices, regardless of if it is a receiver (RX), transmitter (TX) or transceiver (TR). Confirm a devices capabilities using the get capabilities command.
nodes[CEC:index].configuration.logical_addresses	
Since	3.6
Type	Obect
Description	Configure the CEC addresses from the API server. TRUE enables the address and FALSE disables it. Default 14_free_use:true, all other logical addresses are set to false. Supported logical addresses are: 0_tv 1_recording_device_1 2_recording_device_2 3_tuner_1 4_playback_device_1 5_audio_system 6_tuner_2 7_tuner_3 8_playback_device_2 9_recording_device_3 10_tuner_4 11_playback_device_3 12_reserved 13_reserved 14_free_use Note: When issuing API set property request only the addresses to be modified need to specified. For example to set 0_tv to true and 14_free_use to false: <pre>{ "op": "set:property", "key": "nodes[CEC:0].configuration.logical_addresses", "value": { "0_tv": true, "14_free_use": false } }</pre>

Property Key		
		<pre>}</pre> ADDITIONAL INFORMATION: - The index for receivers (RX) and transmitters (TX) is 0. However in the case of transceivers (TR), CEC IN (TX side) is 0 and CEC OUT is 1 (RX side). Single logical address mode: - Logical address used by the CEC module can be changed from default 14_free_use. - On CEC message reception with the new address, the CEC command is acknowledged on the CEC bus and the message is sent to API Server. Multiple logical addresses mode - When more than one logical address is set to true in the CEC configuration, the API indicates this in its return by displaying multiple logical_addresses values as enabled. - In this case, the json return value indicates multiple items are set to true. - In multi logical addresses mode, CEC requests are managed the same as for single logical address mode but the endpoint device acknowledges and forwards to the API Server any logical addresses set to true. - For example, if 0_tv and 14_free_use are set to true, received messages including either of these two logical addresses as the destination address are acknowledged and transferred to the API Server. Reception of CEC messages with any other logical addresses are discarded. - In multi logical addresses mode, unlike single logical address mode, the source address in the message sent by the user from API Server is not overwritten, the message is sent as-is. In this case, the user has the responsibility to control the message source address. TIP: Control Server returns the following error if <u>all</u> values are set to false: INVALID_OPERATION: "Operation would disable all logical addresses, which is not allowed"

7.6 Color Generator Node Configuration

This section describes a Node Object with type member set to COLOR_GENERATOR. Refer to Section 6.8.7 Color Generator Node (COLOR_GENERATOR) for details.

Note: The COLOR_GENERATOR:index is always 0. Only supported for receivers (RX) operating in a display mode other than GENLOCKED.

Table 174: Color Generator Node Configuration Properties

Property Key		
nodes[COLOR_GENERATOR:index].configuration.enable		
	Since	3.0
	Type	Boolean
	Description	When doing a transition it is possible to enable the color generator to blank video output with a uniform color. To enable specify TRUE, to disable specify FALSE.
nodes[COLOR_GENERATOR:index].configuration.color		
	Since	3.0
	Type	String

Property Key		
	Description	When enabling the video mute also necessary to specify the blanking color using the RGB hexadecimal code, [RRGGBB].
nodes[COLOR_GENERATOR:index].configuration.force_output		
	Since	3.0
	Type	Boolean
	Description	When value set as TRUE, a black stream is output with a fixed resolution video signal of 720P60. When set to FALSE device returns to default video mode, Genlock.

7.7 Frame Buffer Node Configuration

The following table lists the configurable properties of a frame buffer node of the device object. Refer to Section 6.8.8 Frame Buffer Node (FRAME_BUFFER) for details.

Note: The index of each of the following nodes, is either 0 for stream 0 (native) or 1 for stream 1 (scaled), applicable to receivers (RX) and transceiver (TR) devices supporting the AV processing engine. You can confirm a devices capabilities using the get capabilities command.

Table 175: Frame Buffer Node Configuration Properties

Property Key		
nodes[FRAME_BUFFER:index].configuration.width		
	Since	3.0
	Type	Integer
	Description	Set the video width, in pixels.
nodes[FRAME_BUFFER:index].configuration.height		
	Since	3.0
	Type	Integer
	Description	Set the video height, in pixels.
nodes[FRAME_BUFFER:index].configuration.horiz_offset		
	Since	3.0
	Type	Integer
	Description	Set the video horizontal offset, in pixels.
nodes[FRAME_BUFFER:index].configuration.vert_offset		
	Since	3.0
	Type	Integer
	Description	Set the Video vertical offset, in pixels.
nodes[FRAME_BUFFER:index].configuration.frames_per_second		
	Since	3.0
	Type	Integer
	Description	Set the frame rate (fps).
nodes[FRAME_BUFFER:index].configuration.freeze		
	Since	3.9
	Type	Boolean
	Description	Set freeze/unfreeze of the video output (freeze video on command).

7.8 HDMI Audio Node Configuration

The following table lists the configurable properties of an HDMI audio of the device object. Refer to Section 6.7.1 Subscription Configuration (`SubscriptionConfig`) for details.

Note: The `HDMI_DECODER:index` is always 0.

Table 176: HDMI Decoder Node Configuration Properties

Property Key		
subscriptions[HDMI_AUDIO:0].configuration.mute_msec		
Since	3.0	
Type	Integer	
Description	This property allows the duration to mute the HDMI audio during a source transition to be modified. The acceptable range is between 0 and 5000 (0ms to 5 seconds).	

7.9 HDMI Decoder Node Configuration

The following table lists the configurable properties of an HDMI decoder node of the device object. Refer to Section 6.8.10 HDMI Decoder Node (`HDMI_DECODER`) for details.

Note: The `HDMI_DECODER:index` is always 0.

Table 177: HDMI Decoder Node Configuration Properties

Property Key	
nodes[HDMI_DECODER:index].inputs[main:0].configuration.source.value	
Since	3.0
Type	Integer
Description	Video source among available video connectors. For the HDMI port this option is 0. Applicable to the HDMI decoder node imbedded in a transmitter device.
nodes[HDMI_DECODER:index].configuration.hdcp_support_enable	
Since	3.0
Type	Boolean
Description	Specifies whether to enable HDCP support, specify <code>TRUE</code> to enable (default) and <code>FALSE</code> to disable. When disabled HDCP support is not supported.
nodes[HDMI_DECODER:index].configuration.hdcp_22_support_disable	
Since	3.0
Type	Boolean
Description	Specifies whether to disable HDCP 2.2 and only support HDCP 1.4. Specify <code>TRUE</code> to disable HDCP 2.2 support and <code>FALSE</code> to enable it.
nodes[HDMI_DECODER:0].configuration.default_quantization_level_override	
Since	3.6
Type	String
Description	In some circumstances, the Quantization level for HDMI sources is incorrectly set in the AVI InfoFrame. Issue related to DP to HDMI conversion chips commonly used in popular cables and adapters. An override is supported for these cases, which can apply an InfoFrame correction on HDMI outputs (RX devices) to reflect the real quantization level of the source.

Property Key		
		Override applied only when quantization is not specified in InfoFrame (default value). Supported values are: disabled No default quantization level override applied, the default (0) is kept. full Default quantization level override set to FULL range. limited Default quantization level override set to LIMITED range.
nodes[HDMI_DECODER:0].configuration.max_video_bandwidth		
	Since	3.9
	Type	String
	Description	Set maximum allowable video bandwidth. NETWORK_DEFAULT Set for current PHY link operating speed. This is the default value. 1G Force for as if 1G PHY link operating speed. 2_5G Force for as if 2.5G PHY link operating speed. 5G Force for as if 5G PHY link operating speed. 10G Force for as if 10G PHY link operating speed. Note: Only applicable when the HDMI transceiver chip supports compression for FlexLinQ (e.g. Lontium LT86121GXE). The default NETWORK_DEFAULT should only be changed if the user specifically wants to set a limit for video bandwidth lower than the Ethernet link capability (for example if he knows the switch trunk port adds some bandwidth limitation for the overall system or other reason to limit the bandwidth taken by the video data).

7.10 HDMI Encoder Node Configuration

The following table lists the configurable properties of an HDMI encoder node of the device object. Refer to Section 6.8.11 HDMI Encoder Node (HDMI_ENCODER) for details.

Note: The HDMI_ENCODER:index is 0 and applicable only to encoder (TX) or transceiver (TR) devices. Confirm a devices capabilities using the get capabilities command.

Table 178: HDMI Encoder Node Configuration Properties

Property Key		
nodes[HDMI_ENCODER:index].configuration.hdcp_output_mode		
	Since	3.0
	Type	String
	Description	Specify HDCP output mode for receivers running in Fast Switch, Multiview or Fast Switch Wall. mode. The supported options are: UNSUPPORTED FOLLOW_SINK_1 FOLLOW_SINK_2 FOLLOW_SOURCE Refer to Table 125: HDMI Encoder Node Status Object Members for details. TIP: More information on managing the HDCP Ouput Mode is included in the HDCP Encyption section of the SDVoE Developers API Application Note Library.
nodes[HDMI_ENCODER:index].inputs[audio:0].configuration.source.value		
	Since	3.0

Property Key	
Type	Integer
Description	HDMI audio source. An encoder node may support none, some or all of the following values: 2 -- HDMI audio (original audio from video subscription in genlock mode) 3 -- Stereo audio 6 -- HDMI audio (stereo downmix) 7 -- HDMI audio (all available channels) 8 -- Multichannel audio subscription 9 -- Multichannel audio local loop out
nodes[HDMI_ENCODER:0].configuration.hdmi_tx_5v	
Since	3.5
Type	String
Description	API Control Value is either <code>AUTO</code> or <code>LOW</code>. <code>AUTO</code> sets the HDMI encoder to operate in “normal” mode: • In <code>switch</code> mode, the HDMI TX 5V output is always <code>HIGH</code> • In <code>loopback</code> mode, it follows 5V loopback mode rules. • In <code>P2P</code> mode, it follows 5V mirroring rules. <code>LOW</code> sets HDMI encoder to remain <code>LOW</code>, overriding any other mode, regardless of the operating mode (<code>switch</code>, <code>P2P</code> or <code>loopback</code>). Refer to the SDVoE Developers API Application Note Library for details.

7.11 Multichannel Audio Input Node Configuration

The following table lists the configurable properties of a multichannel audio input node of the device object. Refer to Section 6.8.13 Multichannel Audio Input Node (MULTICH_AUDIO_INPUT) for details.

Note: The MULTICH_AUDIO_INPUT:index is 0. Confirm a devices capabilities using the get capabilities command.

Table 179: Multichannel Audio Input Node Configuration Properties

Property Key	
nodes[MULTICH_AUDIO_INPUT:index].configuration.sampling_frequency.value	
Since	3.0
Type	Integer
Description	Set the audio sampling frequency. Supported values are following values are supported 22050, 24000, 32000, 44100, 48000, 88200, 96000, 176400, 192000 and 768000.
nodes[MULTICH_AUDIO_INPUT:index].configuration.number_of_channels.value	
Since	3.0
Type	Integer
Description	Specify the number of audio channels. Enter the appropriate value from 2 to 8, represents the number of audio channels.
nodes[MULTICH_AUDIO_INPUT:index].configuration.channel_allocation.value	
Since	3.0

Property Key	
Type	Integer
Description	Specify the audio channel allocation. Enter appropriate value, between 0 and 0x31 to represent the audio channel allocation of the incoming multichannel audio. Reminder: Not all channel allocations supported by all devices. Valid choices are listed in the <code>choices</code> member. Refer to section Channel Allocation Configuration (ChannelAllocationConfig) for details.
nodes[MULTICH_AUDIO_INPUT:index].configuration.audio_encoding_type	
Since	3.0
Type	String
Description	Specify audio encoding type. Options supported are <code>LPCM</code>, <code>ENCODED_HBR</code> and <code>ENCODED_OTHER</code>. Refer to Audio Encoding Configuration (AudioEncodingConfig) for more information.

7.12 Multichannel Audio Output Node Configuration

The following table lists the configurable properties of a multichannel audio output node of the device object. Refer to Section 6.8.14 Multichannel Audio Output Node (MULTICH_AUDIO_OUTPUT) for details.

Table 180: Multichannel Audio Output Node Configuration Properties

Property Key		
nodes[MULTICH_AUDIO_OUTPUT:index].inputs[main:0].configuration.source.value		
Since	3.0	
Type	Integer	
Description	Audio source. A transmitter device may support none, some or all of following values: 3 -- Stereo audio local loop out 6 -- HDMI audio (stereo downmix) local loop out 7 -- HDMI audio (all channels) local loop out 8 -- Multichannel audio subscription (audio return channel) A receiver device may support none, some or all of the following values: 3 -- Stereo audio 6 -- HDMI audio (stereo downmix) 7 -- HDMI audio (all channels) 8 -- Multichannel audio subscription The <code>MULTICH_AUDIO_OUTPUT:index</code> is 0. Confirm a devices capabilities using the <code>get capabilities</code> command.	

7.13 Network Port Node Configuration

The following table lists the configurable properties of a network port node of the device object. Refer to Section 6.8.18 Network Port Node (`NETWORK_PORT`) for details.

Table 181: Network Port Node Configuration Properties

Property Key		
nodes[<code>NETWORK_PORT:index</code>].configuration.speed		
Since	3.0	
Type	String	
Description	Specify the speed configuration for the network port. For list of available options, refer to Error! Reference source not found.. <code>NETWORK_PORT:index</code> is normally 0, unless an NT1000/NT2000 device using an Aquantia 10G Dual PHY.	

7.14 Network Switch Node Configuration

The following table lists the configurable properties of a network switch node of the device object. Refer to Section 7.13 Network Port Node Configuration for details.

Table 182: Network Switch Node Configuration Properties

Property Key		
nodes[<code>NETWORK_SWITCH:index</code>].configuration.gigabit_port_1_enable		
Since	3.0	
Type	Boolean	
Description	Enable/disable the 1GbE port on the device. Only one 1GbE PHY supported, therefore the <code>NETWORK_PORT:index</code> is 0. Reminder: If the device includes a 1GbE interface, the port can be disabled through the API. To enable 1GbE port enter <code>TRUE</code> and to disable it enter <code>FALSE</code> . Confirm a devices capabilities using the get capabilities command.	

7.15 Scaler Node Configuration

The following table lists the configurable properties of a scaler node of the device object. Refer to Section 6.8.20 Scaler Node (`SCALER`) for details.

Table 183: Scaler Node Configuration Properties

Property Key		
nodes[<code>SCALER:index</code>].inputs[main:0].configuration.source.value		
Since	3.0	
Type	Integer	
Description	Specify if the frame rate divider to be active: 0 to enable 1 to disable There is only one scaler node supported, therefore the <code>SCALER:index</code> is 0. Note: Frame rate divider can be enabled on the native stream (stream 0) as well, refer to Section 7.2 Stream Configuration.	

7.16 UART Node Configuration

The following table lists the configurable properties of a UART node of the device object.

Note: The `UART:index` option will be between 0–3. Hardware can support up to four UART ports, each is managed independently. Confirm a devices capabilities using the get capabilities command.

Table 184: UART Node Configuration Properties

Property Key		
nodes[UART:index].configuration.baud_rate		
Since	3.0	
Type	Integer	
Description	Baud rate	
nodes[UART:index].configuration.data_bits		
Since	3.0	
Type	Integer	
Description	Number of data bits: 6, 7 or 8	
nodes[UART:index].configuration.stop_bits		
Since	3.0	
Type	Integer	
Description	Number of stop bits: 1 or 2	
nodes[UART:index].configuration.parity		
Since	3.0	
Type	String	
Description	Parity: NONE, ODD or EVEN	

7.17 USB Extender Node Configuration

The following table lists the configurable properties of a USB Extender node of the device object. Refer to Section 6.8.24 USB Extender Node (`USB_EXTENDER`) for details. For UEX attached to AVP, the `USB_EXTENDER` node with index 0 refers to the first UEX device attached and the `USB_EXTENDER` node with index 1 refers to the first UEX device attached.

Table 185: USB Extender Node Configuration Properties

Property Key		
nodes[USB_EXTENDER:index].configuration.role		
Since	3.8	
Type	String	
Description	role: LOCAL for a local extender (LEX) REMOTE for a remote extender (REX) PIN for the role to be set from the UEX pin DISABLED to disable the USB extender functionality Note: The <code>USB_EXTENDER:index</code> option will be 0. Confirm a devices capabilities using the get capabilities command.	

7.18 Video Compressor Node Configuration

The following table lists the configurable properties of video compressor node of the transmitter device object. Refer to Section 6.8.26 Video Compressor Node (VIDEO_COMPRESSOR) for details.

Note: The VIDEO_COMPRESSOR:index is always 0.

Table 186: Video Compressor Node Configuration Properties

Property Key	
nodes[VIDEO_COMPRESSOR:index].configuration.always_compress	
Since	3.0
Type	Boolean
Description	When <code>true</code> , video on the HDMI stream 0 is always compressed without regard to the video format. Otherwise, it is compressed only when it is necessary to keep the video bandwidth under 9Gbps.
nodes[VIDEO_COMPRESSOR:index].configuration.version	
Since	3.0
Type	Integer
Description	Video compression version used when compressing. Can be either 1 or 2. Refer to Table 164: Video Compressor Node Object Members for more information on the supported compression versions.

7.19 Video Decompressor Node Configuration

The following table lists the configurable properties of video decompressor node of the receiver device object. Refer to Section 6.8.27 Video Decompressor Node (VIDEO_DECOMPRESSOR) for details.

Note: The VIDEO_DECOMPRESSOR:index is always 0.

Table 187: Video Decompressor Node Configuration Properties

Property Key		
nodes[VIDEO_DECOMPRESSOR:index].configuration.version		
Since	3.0	
Type	Integer	
Description	Video compression version to use when decompressing, version either 1 or 2.	

Appendix: Get Command Output Examples

The following sections provide example outputs for the `get` command.

For details on the `get` command, refer to Section 4.7 Command `get`.

A.1 Command “`get device`” for BlueRiver Transceiver Device

The example provided below is the output for the command `get device` called for an transceiver device with the ID of 0016c04c3d9a.

```
{
  status: SUCCESS,
  request_id: 285951,
  result: {
    devices: [
      {
        device_id: 0016c04c40d9,
        device_name: TR_W1BL,
        identity: {
          chipset_type: AVP2000T,
          engine: PLETHORA,
          vendor_id: 1,
          product_id: 88,
          firmware_comment: BlueRiver_RDK_2-1-0-0_evaluation,
          firmware_version: 2.1.0.0,
          firmware_rc: 0,
          is_receiver: true,
          is_transmitter: true,
          firmware_details: [
            {
              type: PRIMARY,
              details_available: true,
              firmware_version: 2.1.0.0,
              firmware_rc: 0
            },
            {
              type: GOLDEN,
              details_available: true,
              firmware_version: 2.0.0.0,
              firmware_rc: 0
            }
          ]
        },
        chip_id: null
      },
      {
        capabilities: {
          color_generator_force: false,
          dhcp_option_43: true,
          dot1x: true,
          external_video_compression_supported: false,
          frame_rate_divider: false,
          freeze_video: true,
          hdmi_audio_downmix: true,
          hdmi_inputs: 1,
          hdmi_outputs: 1,

```

```

    hdmi_stream1_auto_stop: true,
    infrared_inputs: 1,
    infrared_outputs: 1,
    local_loopback: true,
    multich_audio_inputs: 1,
    multich_audio_outputs: 1,
    nbaset: false,
    net10gbe_dual_link: false,
    overlay: false,
    scaler: true,
    send_wol: true,
    stereo_audio_inputs: 0,
    stereo_audio_input_outputs: 1,
    stereo_audio_outputs: 0,
    thumbnail: false,
    uarts: 4,
    usb_hid: true,
    usb_extender: true,
    video_processing: true
},
configuration: {
    device_name: TR_W1BL,
    locate_mode: false,
    secure_av: default,
    resume_streaming: true,
    claimed: false,
    authenticated: false,
    device_mode: TRANSCEIVER
},
status: {
    active: true,
    point_to_point: false,
    point_to_point_peer: ,
    boot_status: PRIMARY,
    update_in_progress: false,
    igmp_reporting_mode: QUERY,
    error_status: {
        has_error_code: false,
        error_code: 0
    },
    temperature: 66
},
streams: [
    {
        type: HDMI,
        index: 0,
        configuration: {
            address: 224.1.1.3,
            enable: true,
            auto_stop: false,
            remove_audio: false
        },
        status: {
            state: STREAMING
        },
        source: {

```

```

        ref_class: NODE,
        ref_type: HDMI_DECODER,
        ref_index: 0
    },
    {
        type: HDMI,
        index: 1,
        configuration: {
            address: 0.0.0.0,
            enable: false,
            auto_stop: false,
            remove_audio: true
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: SCALER,
            ref_index: 0
        }
    },
    {
        type: HDMI_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: HDMI_DECODER,
            ref_index: 0
        }
    },
    {
        type: MULTICH_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: MULTICH_AUDIO_INPUT,
            ref_index: 0
        }
    }
}

```

```

    },
    {
        type: STEREO_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: STEREO_AUDIO_INPUT,
            ref_index: 0
        }
    },
    {
        type: RS232,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            dest_index: 0,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: UART,
            ref_index: 0
        }
    },
    {
        type: RS232,
        index: 1,
        configuration: {
            address: 0.0.0.0,
            dest_index: 1,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: UART,
            ref_index: 1
        }
    },
    {
        type: RS232,

```

```

        index: 2,
        configuration: {
            address: 0.0.0.0,
            dest_index: 2,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: UART,
            ref_index: 2
        }
    },
    {
        type: RS232,
        index: 3,
        configuration: {
            address: 0.0.0.0,
            dest_index: 3,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: UART,
            ref_index: 3
        }
    },
    {
        type: INFRARED,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: INFRARED_DECODER,
            ref_index: 0
        }
    },
    {
        type: CEC,
        index: 0,
        configuration: {
            address: 0.0.0.0,

```

```

        enable: false,
        auto_stop: false
    },
    status: {
        state: STOPPED
    },
    source: {
        ref_class: NODE,
        ref_type: CEC,
        ref_index: 0
    }
},
{
    type: CEC,
    index: 1,
    configuration: {
        address: 0.0.0.0,
        enable: false,
        auto_stop: false
    },
    status: {
        state: STOPPED
    },
    source: {
        ref_class: NODE,
        ref_type: CEC,
        ref_index: 1
    }
},
{
    type: USB_HID,
    index: 0,
    configuration: {
        address: 10.17.252.130,
        enable: true,
        auto_stop: false
    },
    status: {
        state: STREAMING
    },
    source: {
        ref_class: NODE,
        ref_type: USB_HID,
        ref_index: 0
    }
}
],
subscriptions: [
    {
        type: HDMI,
        index: 0,
        configuration: {
            address: 224.1.1.1,
            enable: true
        },
        status: {

```



```

        state: STREAMING
    },
    {
        type: HDMI,
        index: 1,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 2,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 3,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 4,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 5,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    }
}

```

```

    },
    {
      type: HDMI,
      index: 6,
      configuration: {
        address: 0.0.0.0,
        enable: false
      },
      status: {
        state: STOPPED
      }
    },
    {
      type: HDMI,
      index: 7,
      configuration: {
        address: 0.0.0.0,
        enable: false
      },
      status: {
        state: STOPPED
      }
    },
    {
      type: HDMI,
      index: 8,
      configuration: {
        address: 0.0.0.0,
        enable: false
      },
      status: {
        state: STOPPED
      }
    },
    {
      type: HDMI,
      index: 9,
      configuration: {
        address: 0.0.0.0,
        enable: false
      },
      status: {
        state: STOPPED
      }
    },
    {
      type: HDMI,
      index: 10,
      configuration: {
        address: 0.0.0.0,
        enable: false
      },
      status: {
        state: STOPPED
      }
    }
  ],
  {
    type: HDMI,
    index: 10,
    configuration: {
      address: 0.0.0.0,
      enable: false
    },
    status: {
      state: STOPPED
    }
  }
}

```

```

    },
    {
        type: HDMI,
        index: 11,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 12,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 13,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 14,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 15,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    }
},

```

```

{
    type: HDMI,
    index: 16,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 17,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 18,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 19,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 20,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
}
},
{

```

```

        type: HDMI,
        index: 21,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 22,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 23,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 24,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 25,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,

```

```

        index: 26,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 27,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 28,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 29,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 30,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 31,

```

```

        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false,
            mute_msec: 1500
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: STEREO_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: MULTICH_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: RS232,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: RS232,
        index: 1,

```

```

        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: RS232,
        index: 2,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: RS232,
        index: 3,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: INFRARED,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: USB_HID,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    }
],
nodes: [
    {
        type: STEREO_AUDIO_INPUT_OUTPUT,

```


Audio,

Audio,

Audio,

Audio,

```
index: 0,
configuration: {
  direction: INPUT
},
status: {},
inputs: [
  {
    name: main,
    index: 0,
    configuration: {
      source: {
        value: 2,
        choices: [
          {
            value: 2,
            description: Network HDMI Downmix
          },
          {
            value: 3,
            description: Network Stereo Audio,
            details: {
              ref_class: SUBSCRIPTION,
              ref_type: HDMI_AUDIO,
              ref_index: 0
            }
          },
          {
            value: 8,
            description: Network Multi Channel
          },
          {
            value: 9,
            description: Local Multi Channel
          },
          {
            value: 12,
            description: Local HDMI Downmix
          }
        ]
      }
    }
  }
]
```

```

        details: {
            ref_class: NODE,
            ref_type: HDMI_DECODER,
            ref_index: 0
        }
    }
]
},
status: {
    source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI_AUDIO,
        ref_index: 0
    }
}
}
],
{
    type: CEC,
    index: 0,
    configuration: {
        p2p_mode: PASSTHROUGH
    },
    status: {},
    inputs: []
},
{
    type: CEC,
    index: 1,
    configuration: {
        p2p_mode: PASSTHROUGH
    },
    status: {},
    inputs: []
},
{
    type: COLOR_GENERATOR,
    index: 0,
    configuration: {
        enable: false,
        color: 000000,
        force_output: false
    },
    status: {},
    inputs: []
},
{
    type: FRAME_BUFFER,
    index: 0,
    configuration: {
        display_mode: FAST_SWITCHED,
        width: 1280,
        height: 720,
        horiz_offset: 0,

```

```

        vert_offset: 0,
        frames_per_second: 60,
        keep_width: 3840,
        keep_height: 2160,
        viewport_horiz: 0,
        viewport_vert: 0,
        viewport_width: 1280,
        viewport_height: 720,
        aspect_ratio_mode: KEEP,
        has_auto_wall_details: false,
        wall_name: ,
        freeze: false
    },
    status: {
        source_video: {
            width: 3840,
            height: 2160,
            frames_per_second: 60,
            color_space: RGB,
            bits_per_pixel: 8,
            scan_mode: PROGRESSIVE
        }
    },
    inputs: [
        {
            name: main,
            index: 0,
            configuration: {},
            status: {
                source: {
                    ref_class: SUBSCRIPTION,
                    ref_type: HDMI,
                    ref_index: 0
                }
            }
        },
        {
            name: multiview,
            index: 0,
            configuration: {},
            status: {
                source: {
                    ref_class: SUBSCRIPTION,
                    ref_type: HDMI,
                    ref_index: 0
                }
            }
        },
        {
            name: multiview,
            index: 1,
            configuration: {},
            status: {
                source: {
                    ref_class: SUBSCRIPTION,
                    ref_type: HDMI,

```

```

        ref_index: 1
    }
},
{
    name: multiview,
    index: 2,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 2
        }
    }
},
{
    name: multiview,
    index: 3,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 3
        }
    }
},
{
    name: multiview,
    index: 4,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 4
        }
    }
},
{
    name: multiview,
    index: 5,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 5
        }
    }
},
{
    name: multiview,
    index: 6,
    configuration: {},

```

```

        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 6
            }
        }
    },
    {
        name: multiview,
        index: 7,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 7
            }
        }
    },
    {
        name: multiview,
        index: 8,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 8
            }
        }
    },
    {
        name: multiview,
        index: 9,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 9
            }
        }
    },
    {
        name: multiview,
        index: 10,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 10
            }
        }
    }
},

```

```

{
  name: multiview,
  index: 11,
  configuration: {},
  status: {
    source: {
      ref_class: SUBSCRIPTION,
      ref_type: HDMI,
      ref_index: 11
    }
  }
},
{
  name: multiview,
  index: 12,
  configuration: {},
  status: {
    source: {
      ref_class: SUBSCRIPTION,
      ref_type: HDMI,
      ref_index: 12
    }
  }
},
{
  name: multiview,
  index: 13,
  configuration: {},
  status: {
    source: {
      ref_class: SUBSCRIPTION,
      ref_type: HDMI,
      ref_index: 13
    }
  }
},
{
  name: multiview,
  index: 14,
  configuration: {},
  status: {
    source: {
      ref_class: SUBSCRIPTION,
      ref_type: HDMI,
      ref_index: 14
    }
  }
},
{
  name: multiview,
  index: 15,
  configuration: {},
  status: {
    source: {
      ref_class: SUBSCRIPTION,
      ref_type: HDMI,

```

```

        ref_index: 15
    }
},
{
    name: multiview,
    index: 16,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 16
        }
    }
},
{
    name: multiview,
    index: 17,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 17
        }
    }
},
{
    name: multiview,
    index: 18,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 18
        }
    }
},
{
    name: multiview,
    index: 19,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 19
        }
    }
},
{
    name: multiview,
    index: 20,
    configuration: {},

```

```

        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 20
            }
        }
    },
    {
        name: multiview,
        index: 21,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 21
            }
        }
    },
    {
        name: multiview,
        index: 22,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 22
            }
        }
    },
    {
        name: multiview,
        index: 23,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 23
            }
        }
    },
    {
        name: multiview,
        index: 24,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 24
            }
        }
    }
},

```



```

{
  name: multiview,
  index: 25,
  configuration: {},
  status: {
    source: {
      ref_class: SUBSCRIPTION,
      ref_type: HDMI,
      ref_index: 25
    }
  }
},
{
  name: multiview,
  index: 26,
  configuration: {},
  status: {
    source: {
      ref_class: SUBSCRIPTION,
      ref_type: HDMI,
      ref_index: 26
    }
  }
},
{
  name: multiview,
  index: 27,
  configuration: {},
  status: {
    source: {
      ref_class: SUBSCRIPTION,
      ref_type: HDMI,
      ref_index: 27
    }
  }
},
{
  name: multiview,
  index: 28,
  configuration: {},
  status: {
    source: {
      ref_class: SUBSCRIPTION,
      ref_type: HDMI,
      ref_index: 28
    }
  }
},
{
  name: multiview,
  index: 29,
  configuration: {},
  status: {
    source: {
      ref_class: SUBSCRIPTION,
      ref_type: HDMI,

```

```

        ref_index: 29
    }
}
},
{
    name: multiview,
    index: 30,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 30
        }
    }
},
{
    name: multiview,
    index: 31,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 31
        }
    }
}
]
},
{
    type: GPIO,
    index: 0,
    configuration: {
        tristate: 1,
        cfg_val: 0
    },
    status: {
        stat_val: 0
    },
    inputs: []
},
{
    type: GPIO,
    index: 1,
    configuration: {
        tristate: 1,
        cfg_val: 0
    },
    status: {
        stat_val: 0
    },
    inputs: []
},
{
    type: GPIO,

```

```

        index: 2,
        configuration: {
            tristate: 1,
            cfg_val: 0
        },
        status: {
            stat_val: 0
        },
        inputs: []
    },
    {
        type: GPIO,
        index: 3,
        configuration: {
            tristate: 1,
            cfg_val: 0
        },
        status: {
            stat_val: 0
        },
        inputs: []
    },
    {
        type: GPIO,
        index: 4,
        configuration: {
            tristate: 1,
            cfg_val: 0
        },
        status: {
            stat_val: 0
        },
        inputs: []
    },
    {
        type: GPIO,
        index: 5,
        configuration: {
            tristate: 1,
            cfg_val: 0
        },
        status: {
            stat_val: 0
        },
        inputs: []
    },
    {
        type: GPIO,
        index: 6,
        configuration: {
            tristate: 1,
            cfg_val: 0
        },
        status: {
            stat_val: 0
        },
        inputs: []
    }

```

```

        inputs: []
    },
    {
        type: GPIO,
        index: 7,
        configuration: {
            tristate: 1,
            cfg_val: 0
        },
        status: {
            stat_val: 0
        },
        inputs: []
    },
    {
        type: GPIO,
        index: 8,
        configuration: {
            tristate: 1,
            cfg_val: 0
        },
        status: {
            stat_val: 0
        },
        inputs: []
    },
    {
        type: GPIO,
        index: 9,
        configuration: {
            tristate: 1,
            cfg_val: 0
        },
        status: {
            stat_val: 0
        },
        inputs: []
    },
    {
        type: GPIO,
        index: 10,
        configuration: {
            tristate: 1,
            cfg_val: 0
        },
        status: {
            stat_val: 0
        },
        inputs: []
    },
    {
        type: GPIO,
        index: 11,
        configuration: {
            tristate: 1,
            cfg_val: 0
        },

```

```

    },
    status: {
        stat_val: 0
    },
    inputs: []
},
{
    type: HDMI_DECODER,
    index: 0,
    configuration: {
        edid:
00ffffffffffff000614000156524c420816010380341d780a01c1a057479827124c4c2108008140010
101010101010101010101010101010101023a801871382d40582c460040846300001e08e80030f2705a80b0588a
00ba892100001e000000fc00426c756552697665720a20202000000fd0018550e853c000a202020202
020017c02034373580102030405111213141f2021220607905d5e5f6263646061230d0707830f00006d
030c001000383c20406801020367d85dc401788003e40f0000c0e200cf011d00bc52d01e20b82855404
0846300001e023a80d072382d40102c458040846300001e023a801871382d40582c450040846300001e
000000000000003e,

        hdcp_support_enable: true,
        hdcp_22_support_disable: false,
        default_quantization_level_override: DISABLED
    },
    status: {
        video: {
            width: 1920,
            height: 1080,
            frames_per_second: 60,
            color_space: RGB,
            bits_per_pixel: 8,
            scan_mode: PROGRESSIVE
        },
        has_video_details: true,
        video_details: {
            frame_rate: 60,
            pixel_clock: 148495683,
            total_width: 2200,
            total_height: 1125,
            hsync_width: 44,
            hsync_front_porch: 88,
            hsync_negative: false,
            vsync_width: 6,
            vsync_front_porch: 4,
            vsync_negative: false,
            vic: 16,
            has_hdmi_vic: false,
            hdmi_vic: 0,
            picture_aspect: ASPECT_16_9,
            has_active_format: false,
            active_format: 0,
            it_content_type: GRAPHICS,
            scan_information: NO_DATA,
            colorimetry: BT709,
            rgb_range: DEFAULT,
            ycc_range: LIMITED,
            has_hdr: false,
            hdr: SDR
        }
    }
}

```

```

    },
    has_audio_details: true,
    audio_details: {
        sampling_frequency: 48000,
        number_of_channels: 2
    },
    has_audio_details_extended: true,
    audio_details_extended: {
        channel_allocation: 0,
        sample_size: 16,
        level_shift_value: 0,
        lfe_playback_level: 0,
        audio_encoding_type: LPCM,
        is_downmix_inhibited: false
    },
    hdcp_protected: false,
    hdcp_version: NONE,
    hdmi_2_0_support: true,
    source_stable: true,
    audio_downmix_support: true
},
inputs: [
    {
        name: main,
        index: 0,
        configuration: {},
        status: {
            source: {
                ref_class: NODE,
                ref_type: VIDEO_INPUT,
                ref_index: 0
            }
        }
    }
]
},
{
    type: HDMI_ENCODER,
    index: 0,
    configuration: {
        hdcp_output_mode: FOLLOW_SINK_1,
        hdmi_tx_5v: AUTO
    },
    status: {
        video: {
            width: 1280,
            height: 720,
            frames_per_second: 60,
            color_space: RGB,
            bits_per_pixel: 8,
            scan_mode: PROGRESSIVE
        },
    },
    has_video_details: true,
    video_details: {
        frame_rate: 60,
        pixel_clock: 74250000,
    }
}

```

```

        total_width: 1650,
        total_height: 750,
        hsync_width: 40,
        hsync_front_porch: 110,
        hsync_negative: false,
        vsync_width: 5,
        vsync_front_porch: 5,
        vsync_negative: false,
        vic: 4,
        has_hdmi_vic: false,
        hdmi_vic: 0,
        picture_aspect: ASPECT_16_9,
        has_active_format: false,
        active_format: 0,
        it_content_type: NO_DATA,
        scan_information: NO_DATA,
        colorimetry: RGB,
        rgb_range: LIMITED,
        ycc_range: LIMITED,
        has_hdr: false,
        hdr: SDR
    },
    has_audio_details: true,
    audio_details: {
        sampling_frequency: 44100,
        number_of_channels: 2
    },
    has_audio_details_extended: true,
    audio_details_extended: {
        channel_allocation: 0,
        sample_size: 0,
        level_shift_value: 0,
        lfe_playback_level: 0,
        audio_encoding_type: LPCM,
        is_downmix_inhibited: false
    },
    hdcp_protected: false,
    hdcp_version: NONE,
    hdmi_2_0_support: true,
    source_stable: true
},
inputs: [
    {
        name: main,
        index: 0,
        configuration: {
            source: {
                value: 1,
                choices: [
                    {
                        value: 0,
                        description: Genlock Mode,
                        details: {
                            ref_class: SUBSCRIPTION,
                            ref_type: HDMI,
                            ref_index: 0
                        }
                    }
                ]
            }
        }
    }
]

```

(Video Genlock),

Audio,

```
    },
    {
      value: 1,
      description: Frame Buffer Mode,
      details: {
        ref_class: NODE,
        ref_type: FRAME_BUFFER,
        ref_index: 0
      }
    }
  ]
},
status: {
  source: {
    ref_class: NODE,
    ref_type: FRAME_BUFFER,
    ref_index: 0
  }
},
{
  name: audio,
  index: 0,
  configuration: {
    source: {
      value: 2,
      choices: [
        {
          value: 2,
          description: Network HDMI Audio Aux
        }
      ],
      details: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI_AUDIO,
        ref_index: 0
      }
    },
    {
      value: 3,
      description: Network Stereo Audio,
      details: {
        ref_class: SUBSCRIPTION,
        ref_type: STEREO_AUDIO,
        ref_index: 0
      }
    }
  ],
  {
    value: 6,
    description: Network HDMI Downmix
  },
  details: {
    ref_class: SUBSCRIPTION,
    ref_type: HDMI_AUDIO,
    ref_index: 0
  }
}
```


Audio,

Audio,

```
    },
    {
      value: 7,
      description: Network HDMI Audio,
      details: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI_AUDIO,
        ref_index: 0
      }
    },
    {
      value: 8,
      description: Network Multi Channel
      details: {
        ref_class: SUBSCRIPTION,
        ref_type: MULTICH_AUDIO,
        ref_index: 0
      }
    },
    {
      value: 9,
      description: Local Multi Channel
      details: {
        ref_class: NODE,
        ref_type: MULTICH_AUDIO_INPUT,
        ref_index: 0
      }
    }
  ]
},
status: {
  source: {
    ref_class: SUBSCRIPTION,
    ref_type: HDMI_AUDIO,
    ref_index: 0
  }
}
}
],
{
  type: HDMI_MONITOR,
  index: 0,
  configuration: {},
  status: {
    connected: true,
    edid:
```

```
00ffffffffffff00220e173401010101061b0103803c22782a2895a7554ea3260f5054a10800d1c081c
081809500a9c0a940b300d1004dd000a0f0703e803040350055502100001a000000fd00283c87873c00
0a202020202020000000fc00485020454e5659203237730a20000000ff0036434d373036304e52530a2
02001b7020335b14b6160101f04130312021101e2002b6c030c001000383c000000000067d85dc40178
8000681a00000101283cede30f0000565e00a0a0a029503020350055502100001ae26800a0a0402e602
```



```

        value: 96000,
        description: 96kHz,
        details: null
    },
    {
        value: 176400,
        description: 176.4kHz,
        details: null
    },
    {
        value: 192000,
        description: 192kHz,
        details: null
    },
    {
        value: 768000,
        description: 768kHz,
        details: null
    }
]
},
number_of_channels: {
    value: 8,
    choices: [
        {
            value: 2,
            description: 2 Channels,
            details: null
        },
        {
            value: 3,
            description: 3 Channels,
            details: null
        },
        {
            value: 4,
            description: 4 Channels,
            details: null
        },
        {
            value: 5,
            description: 5 Channels,
            details: null
        },
        {
            value: 6,
            description: 6 Channels,
            details: null
        },
        {
            value: 7,
            description: 7 Channels,
            details: null
        },
        {
            value: 8,

```

RC),

```
        description: 8 Channels,
        details: null
    }
]
},
channel_allocation: {
    value: 0,
    choices: [
        {
            value: 0,
            description: 2 Channels (FL-FR),
            details: null
        },
        {
            value: 1,
            description: 2.1 Channels (FL-FR-LFE),
            details: null
        },
        {
            value: 2,
            description: 3 Channels (FL-FR-X-FC),
            details: null
        },
        {
            value: 3,
            description: 3.1 Channels (FL-FR-LFE-FC),
            details: null
        },
        {
            value: 4,
            description: 3 Channels (FL-FR-X-X-RC),
            details: null
        },
        {
            value: 5,
            description: 3.1 Channels (FL-FR-LFE-X-RC),
            details: null
        },
        {
            value: 6,
            description: 4 Channels (FL-FR-X-FC-RC),
            details: null
        },
        {
            value: 7,
            description: 4.1 Channels (FL-FR-LFE-FC-
            details: null
        },
        {
            value: 8,
            description: 4 Channels (FL-FR-X-X-RL-RR),
            details: null
        },
        {
            value: 9,
```

```

RR) ,
    description: 4.1 Channels (FL-FR-LFE-X-RL-
    details: null
  },
  {
    value: 10,
    description: 5 Channels (FL-FR-X-FC-RL-RR),
    details: null
  },
  {
    value: 11,
    description: 5.1 Channels (FL-FR-LFE-FC-RL-
RR) ,
    details: null
  },
  {
    value: 12,
    description: 5 Channels (FL-FR-X-X-RL-RR-
RC) ,
    details: null
  },
  {
    value: 13,
    description: 5.1 Channels (FL-FR-LFE-X-RL-
RR-RC) ,
    details: null
  },
  {
    value: 14,
    description: 6 Channels (FL-FR-X-FC-RL-RR-
RC) ,
    details: null
  },
  {
    value: 15,
    description: 6.1 Channels (FL-FR-LFE-FC-RL-
RR-RC) ,
    details: null
  },
  {
    value: 16,
    description: 6 Channels (FL-FR-X-X-RL-RR-
RLC-RR) ,
    details: null
  },
  {
    value: 17,
    description: 6.1 Channels (FL-FR-LFE-X-RL-
RR-RLC-RR) ,
    details: null
  },
  {
    value: 18,
    description: 7 Channels (FL-FR-X-FC-RL-RR-
RLC-RR) ,
    details: null

```

```

},
{
  value: 19,
  description: 7.1 Channels (FL-FR-LFE-FC-RL-
RR-RLC-RRC) ,
  details: null
},
{
  value: 20,
  description: 4 Channels (FL-FR-X-X-X-X-FLC-
FRC) ,
  details: null
},
{
  value: 21,
  description: 4.1 Channels (FL-FR-LFE-X-X-X-
FLC-FRC) ,
  details: null
},
{
  value: 22,
  description: 5 Channels (FL-FR-X-FC-X-X-
FLC-FRC) ,
  details: null
},
{
  value: 23,
  description: 5.1 Channels (FL-FR-LFE-FC-X-
X-FLC-FRC) ,
  details: null
},
{
  value: 24,
  description: 5 Channels (FL-FR-X-X-RC-X-
FLC-FRC) ,
  details: null
},
{
  value: 25,
  description: 5.1 Channels (FL-FR-LFE-X-RC-
X-FLC-FRC) ,
  details: null
},
{
  value: 26,
  description: 6 Channels (FL-FR-X-FC-RC-X-
FLC-FRC) ,
  details: null
},
{
  value: 27,
  description: 6.1 Channels (FL-FR-LFE-FC-RC-
X-FLC-FRC) ,
  details: null
},
{

```

```

        value: 28,
        description: 6 Channels (FL-FR-X-X-RL-RR-
FLC-FRC) ,
        details: null
    },
    {
        value: 29,
        description: 6.1 Channels (FL-FR-LFE-X-RL-
RR-FLC-FRC) ,
        details: null
    },
    {
        value: 30,
        description: 7 Channels (FL-FR-X-FC-RL-RR-
FLC-FRC) ,
        details: null
    },
    {
        value: 31,
        description: 7.1 Channels (FL-FR-LFE-FC-RL-
RR-FLC-FRC) ,
        details: null
    },
    {
        value: 32,
        description: 6 Channels (X-FCH-RR-RL-FC-X-
FR-FL) ,
        details: null
    },
    {
        value: 33,
        description: 6.1 Channels (X-FCH-RR-RL-FC-
LFE-FR-FL) ,
        details: null
    },
    {
        value: 34,
        description: 6 Channels (TC-X-RR-RL-FC-X-
FR-FL) ,
        details: null
    },
    {
        value: 35,
        description: 6.1 Channels (TC-X-RR-RL-FC-
LFE-FR-FL) ,
        details: null
    },
    {
        value: 36,
        description: 6 Channels (FRH-FLH-RR-RL-X-X-
FR-FL) ,
        details: null
    },
    {
        value: 37,
        description: 6.1 Channels (FRH-FLH-RR-RL-X-

```

```

LFE-FR-FL) ,
                                details: null
                                },
                                {
FR-FL) ,
                                value: 38,
                                description: 6 Channels (FRW-FLW-RR-RL-X-X-
                                details: null
                                },
                                {
LFE-FR-FL) ,
                                value: 39,
                                description: 6.1 Channels (FRW-FLW-RR-RL-X-
                                details: null
                                },
                                {
FR-FL) ,
                                value: 40,
                                description: 7 Channels (TC-RC-RR-RL-FC-X-
                                details: null
                                },
                                {
LFE-FR-FL) ,
                                value: 41,
                                description: 7.1 Channels (TC-RC-RR-RL-FC-
                                details: null
                                },
                                {
FR-FL) ,
                                value: 42,
                                description: 7 Channels (FCH-RC-RR-RL-FC-X-
                                details: null
                                },
                                {
LFE-FR-FL) ,
                                value: 43,
                                description: 7.1 Channels (FCH-RC-RR-RL-FC-
                                details: null
                                },
                                {
FR-FL) ,
                                value: 44,
                                description: 7 Channels (TC-FCH-RR-RL-FC-X-
                                details: null
                                },
                                {
LFE-FR-FL) ,
                                value: 45,
                                description: 7.1 Channels (TC-FCH-RR-RL-FC-
                                details: null
                                },
                                {
X-FR-FL) ,
                                value: 46,
                                description: 7 Channels (FRH-FLH-RR-RL-FC-
                                details: null

```



```

    },
    {
        value: 47,
        description: 7.1 Channels (FRH-FLH-RR-RL-
FC-LFE-FR-FL),
        details: null
    },
    {
        value: 48,
        description: 7 Channels (FRW-FLW-RR-RL-FC-
X-FR-FL),
        details: null
    },
    {
        value: 49,
        description: 7.1 Channels (FRW-FLW-RR-RL-
FC-LFE-FR-FL),
        details: null
    }
]
},
audio_encoding_type: LPCM,
mute_audio: false
},
status: {
    data_lanes: 4,
    sampling_frequency: 0
},
inputs: []
},
{
    type: MULTICH_AUDIO_OUTPUT,
    index: 0,
    configuration: {},
    status: {
        data_lanes: 4
    },
    inputs: [
        {
            name: main,
            index: 0,
            configuration: {
                source: {
                    value: 8,
                    choices: [
                        {
                            value: 3,
                            description: Network Stereo Audio,
                            details: {
                                ref_class: SUBSCRIPTION,
                                ref_type: STEREO_AUDIO,
                                ref_index: 0
                            }
                        }
                    ]
                }
            },
            {
                value: 6,

```

```

Audio,
description: Network HDMI Downmix

details: {
  ref_class: SUBSCRIPTION,
  ref_type: HDMI_AUDIO,
  ref_index: 0
}
},
{
  value: 7,
  description: Network HDMI Audio,
  details: {
    ref_class: SUBSCRIPTION,
    ref_type: HDMI_AUDIO,
    ref_index: 0
  }
},
{
  value: 8,
  description: Network Multi Channel

Audio,
details: {
  ref_class: SUBSCRIPTION,
  ref_type: MULTICH_AUDIO,
  ref_index: 0
}
},
{
  value: 10,
  description: Local Stereo Audio,
  details: {
    ref_class: NODE,
    ref_type:
STEREO_AUDIO_INPUT_OUTPUT,
    ref_index: 0
  }
},
{
  value: 12,
  description: Local HDMI Downmix

Audio,
details: {
  ref_class: NODE,
  ref_type: HDMI_DECODER,
  ref_index: 0
}
},
{
  value: 11,
  description: Local HDMI Audio,
  details: {
    ref_class: NODE,
    ref_type: HDMI_DECODER,
    ref_index: 0
  }
}
}

```

```

        ]
    },
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: MULTICH_AUDIO,
            ref_index: 0
        }
    }
}
],
},
{
    type: INFRARED_ENCODER,
    index: 0,
    configuration: {},
    status: {
        data_length_max: 1032
    },
    inputs: [
        {
            name: network,
            index: 0,
            configuration: {},
            status: {
                source: {
                    ref_class: SUBSCRIPTION,
                    ref_type: INFRARED,
                    ref_index: 0
                }
            }
        }
    ]
},
{
    type: INFRARED_DECODER,
    index: 0,
    configuration: {},
    status: {},
    inputs: []
},
{
    type: LED,
    index: 0,
    configuration: {
        function: {
            value: 0,
            choices: [
                {
                    value: 0,
                    description: (Unknown),
                    details: UNKNOWN
                },
                {
                    value: 1,

```

```

        description: Blink LED,
        details: BLINK
    },
    {
        value: 8,
        description: Default functionality,
        details: DEFAULT
    }
]
},
status: {},
inputs: []
},
{
    type: NETWORK_INTERFACE,
    index: 0,
    configuration: {
        hostname: TR_W1BL,
        ip: {
            address: 10.17.252.109,
            mode: DHCP,
            mask: 255.255.255.0,
            gateway: 10.17.252.254

            dhcp_option_43: {
                control_server_addresses: []
            }
        },
        status: {
            mac_address: 0016c04c40d9,
            ip: {
                address: 10.17.252.109
            }
        },
        inputs: []
    },
    {
        type: NETWORK_PORT,
        index: 0,
        configuration: {
            speed: SPEED_10G
        },
        status: {
            active: true,
            speed: 10,
            supports_speed_10g: true,
            supports_speed_5g: false,
            supports_speed_2_5g: false,
            supports_auto_10g: false,
            supports_auto_5g: false
        },
        inputs: []
    },
    {
        type: NETWORK_SWITCH,

```

```

        index: 0,
        configuration: {
            gigabit_port_1_enable: true
        },
        status: {},
        inputs: []
    },
    {
        type: SCALER,
        index: 0,
        configuration: {
            width: 0,
            height: 0
        },
        status: {
            video: {
                width: 0,
                height: 0,
                frames_per_second: 60,
                color_space: RGB,
                bits_per_pixel: 8,
                scan_mode: PROGRESSIVE
            }
        },
        inputs: [
            {
                name: main,
                index: 0,
                configuration: {},
                status: {
                    source: {
                        ref_class: NODE,
                        ref_type: HDMI_DECODER,
                        ref_index: 0
                    }
                }
            }
        ]
    },
    {
        type: UART,
        index: 0,
        configuration: {
            baud_rate: 57600,
            data_bits: 8,
            stop_bits: 1,
            parity: NONE
        },
        status: {
            usage: EXPOSED
        },
        inputs: [
            {
                name: network,
                index: 0,
                configuration: {},

```

```

        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: RS232,
                ref_index: 0
            }
        }
    }
}
],
},
{
    type: UART,
    index: 1,
    configuration: {
        baud_rate: 57600,
        data_bits: 8,
        stop_bits: 1,
        parity: NONE
    },
    status: {
        usage: EXPOSED
    },
    inputs: [
        {
            name: network,
            index: 1,
            configuration: {},
            status: {
                source: {
                    ref_class: SUBSCRIPTION,
                    ref_type: RS232,
                    ref_index: 1
                }
            }
        }
    ]
},
{
    type: UART,
    index: 2,
    configuration: {
        baud_rate: 57600,
        data_bits: 8,
        stop_bits: 1,
        parity: NONE
    },
    status: {
        usage: EXPOSED
    },
    inputs: [
        {
            name: network,
            index: 2,
            configuration: {},
            status: {
                source: {

```



```

        {
            status: CONNECTED
        }
    ],
    usb_host: {
        status: CONNECTED
    },
    role: LOCAL,
    peers: [
        {
            linked: true
        }
    ]
},
inputs: []
},
{
    type: USB_EXTENDER,
    index: 0,
    configuration: {
        role: LOCAL,
        program_mode: NONE,
        rs232_port: 0,
        peers: [ {
            avp_attached: {
                device_id: 001122334455,
                device_name: AVP_001122334455,
            },
            device_id: 66778899AABB,
            device_name: USB_66778899AABB,
            mac_address: 66778899AABB,
        } ],
        device_filtering: {
            block_iso_transfers: false,
            class_filtering: ALLOW_ALL,
            allow_api_commands: false,
            allow_i2c_control: false,
        }
    },
    remote_vbus_ctrl: AUTO,
    msa_enable: {
        bot_protocol: true,
        uas_protocol: true,
    }
},
},

status: {
    chip_present: true,
    device_id: 0016c0355739,
    device_name: USB_0016c0355739,
    mac_address: 0016c0355739,
    chipset_type: UEX1000,
    reachable: true,
    revision: 1.0.0.0,
    ip_mode: DHCP,
    ip_address: 10.17.252.101,
    link_status_support: true,
    sui_enabled: false,

```



```

        peers: [ {
            linked: true
        } ],
        role: LOCAL
    },
    inputs: []
},
{
    type: VIDEO_COMPRESSOR,
    index: 0,
    configuration: {
        always_compress: false,
        version: 1
    },
    status: {
        version_support: 2
    },
    inputs: []
},
{
    type: VIDEO_DECOMPRESSOR,
    index: 0,
    configuration: {
        version: 1
    },
    status: {
        version_support: 2
    },
    inputs: []
},
{
    type: VIDEO_INPUT,
    index: 0,
    configuration: {},
    status: {},
    inputs: []
}
],
usb_extenders: [
    {
        device_id: 0016c0355739,
        device_name: USB_0016c0355739,
        identity: {
            chipset_type: UEX1000,
            engine: USB2.0,
            vendor_id: 1,
            product_id: 212,
            firmware_comment: UEX 1.0.0.0_build986 uex_fpga,
            firmware_version: 1.0.0.0,
            chip_id: 0
        },
        capabilities: {
            local_role_supported: true,
            remote_role_supported: false,
            legacy_mode: false
        }
    },

```

```

configuration: {
  device_name: USB_0016c0355739,
  locate_mode: false,
  claimed: false,
  volatile_settings: false
},
status: {
  active: true,
  point_to_point: false,
  point_to_point_peer: ,
  update_in_progress: false,
  error_status: {
    has_error_code: false,
    error_code: 0
  }
},
streams: [],
subscriptions: [],
nodes: [
  {
    type: NETWORK_INTERFACE,
    index: 0,
    configuration: {
      hostname: USB_0016c0355739,
      ip: {
        address: 10.17.252.115,
        mode: DHCP,
        mask: 255.255.255.0,
        gateway: 10.17.252.254
      }
    },
    status: {
      mac_address: 0016c0355739,
      ip: {
        address: 10.17.252.115
      }
    },
    inputs: []
  },
  {
    type: USB_EXTENDER,
    index: 0,
    configuration: {
      role: LOCAL,
      peers: [ {
        avp_attached: {
          device_id: 001122334455,
          device_name: AVP_001122334455,
        }
        device_id: 66778899AABB,
        device_name: USB_66778899AABB,
        mac_address: 66778899AABB,
      } ],
      device_filtering: {
        block_iso_transfers: false,
        class_filtering: ALLOW_ALL,
      }
    }
  }
]

```

```

        allow_api_commands: false,
        allow_i2c_control: false,
    },
    remote_vbus_ctrl: AUTO,
    msa_enable: {
        bot_protocol: true,
        uas_protocol: true,
    }
},
status: {
    chip_present: true,
    device_id: 44b7d0cb86a3,
    device_name: USB_44b7d0cb86a3,
    mac_address: 44b7d0cb86a3,
    chipset_type: UEX1000,
    reachable: true,
    revision: 1.0.0.0,
    ip_mode: DHCP,
    ip_address: 10.17.252.115,
    link_status_support: true,
    sui_enabled: false,
    peers: [{
        linked: true
    } ],
    role: LOCAL
},
inputs: []
}
}
]
},
error: []
},
error: null
}

```

A.2 Command “get device” for UEX2000 LOCAL Device

The example provided below is the output for the command `get device` called for a UEX2000 standalone USB 2.0 Extender device the ID of 44b7d0cb86a3.

```
{
  status: SUCCESS,
  request_id: 285951,
  result: {
    devices: [
      {
        device_id: 44b7d0cb86a3,
        device_name: USB_44b7d0cb86a3,
        identity: {
          chipset_type: UEX2000,
          engine: USB2.0,
          vendor_id: 1,
          product_id: 212,
          firmware_comment: UEX 1.0.0.0_build986 uex_fpga,
          firmware_version: 1.0.0.0,
          chip_id: 0
        },
        capabilities: {
          local_role_supported: true,
          remote_role_supported: false,
          legacy_mode: false
        },
        configuration: {
          device_name: USB_44b7d0cb86a3,
          locate_mode: false,
          claimed: false,
          volatile_settings: false
        },
        status: {
          active: true,
          point_to_point: false,
          point_to_point_peer: ,
          update_in_progress: false,
          error_status: {
            has_error_code: false,
            error_code: 0
          }
        },
        streams: [],
        subscriptions: [],
        nodes: [
          {
            type: NETWORK_INTERFACE,
            index: 0,
            configuration: {
              hostname: USB_44b7d0cb86a3,
              ip: {
                address: 10.17.252.109,
                mode: DHCP,
                mask: 255.255.255.0,
                gateway: 10.17.252.254
              }
            }
          }
        ]
      }
    ]
  }
}
```

```

    },
    status: {
        mac_address: 44b7d0cb86a3,
        ip: {
            address: 10.17.252.109
        }
    },
    inputs: []
},
{
    type: USB_EXTENDER,
    index: 0,
    configuration: {
        role: LOCAL,
        peers: [ {
            device_id: 44b7d0cb86a9,
            device_name: USB_44b7d0cb86a9,
            mac_address: 44b7d0cb86a9,
        } ],
        device_filtering: {
            block_iso_transfers: false,
            class_filtering: ALLOW_ALL,
            allow_api_commands: false,
            allow_i2c_control: false,
        },
        remote_vbus_ctrl: AUTO,
        msa_enable: {
            bot_protocol: true,
            uas_protocol: true,
        }
    },
    status: {
        chip_present: true,
        device_id: 44b7d0cb86a3,
        device_name: USB_44b7d0cb86a3,
        mac_address: 44b7d0cb86a3,
        chipset_type: UEX2000,
        reachable: true,
        revision: 1.0.0.0,
        ip_mode: DHCP,
        ip_address: 10.17.252.109,
        link_status_support: true,
        sui_enabled: false,
        peers: [ {
            linked: true
        } ],
        role: LOCAL
    },
    inputs: []
}
]
},
error: []
},

```

[illegible]

```
"result": {  
    "devices": [  
        {  
            "device_id": "d8803962e132",  
            "status": {  
                "active": true  
            },  
            "nodes": [  
                {  
                    "type": "HDMI_DECODER",  
                    "index": 0,  
                    "configuration": {  
                        "edid": "  
00ffffffffffffff0010ac82d0425152302219010380351e78ea2f8180a9c0d1c0010101010101010101023a801871382d40582c450d543935384a305251420a000000fc0044454c4c2053453234313a2020202020200140020317b14c9005040302071601141f1213682c45000f282100001e011d8018711c1620582c25000f2821000f282100001e8c0ad08a20e02d10103e96000f2821000018000000000000000000000000000000003d"  
                                }  
                            },  
                            "status": {}  
                        ]  
                    }  
                ],  
                "error": []  
            },
```

```

"result" : {
  "devices" : [
    {
      "device_id" : "d8803962e132",
      "status" : {
        "active" : true
      },
      "nodes" : [
        {
          "type" : "HDMI_DECODER",
          "index" : 0,
          "configuration" : {
            "edid" :
"00ffffffffffffffff0010ac82d0425152302219010380351e78ea2
f8180a9c0d1c001010101010101010101023a801871382d40582c450
d543935384a305251420a000000fc0044454c4c2053453234313
a20202020200140020317b14c9005040302071601141f12136
82c45000f282100001e011d8018711c1620582c25000f2821000
f282100001e8c0ad08a20e02d10103e96000f282100001800000
00000000000000000000000000000003d"
          },
          "status" : {}
        }
      ]
    }
  ],
  "error" : []
},

```

```
"result": {  
    "devices": [  
        {  
            "device_id": "d8803962e132",  
            "status": {  
                "active": true  
            },  
            "nodes": [  
                {  
                    "type": "HDMI_DECODER",  
                    "index": 0,  
                    "configuration": {  
                        "edid": "  
00ffffffffffffff0010ac82d0425152302219010380351e78ea2f8180a9c0d1c001010101010101010101023a801871382d40582c450d543935384a305251420a000000fc0044454c4c2053453234313a2020202020200140020317b14c9005040302071601141f1213682c45000f282100001e011d8018711c1620582c25000f2821000f282100001e8c0ad08a20e02d10103e96000f28210000180000000000000000000000000000000000000003d"  
                                }  
                            },  
                            "status": {}  
                        }  
                    ]  
                },  
                ],  
            "error": []  
        },
```

```

"result" : {
  "devices" : [
    {
      "device_id" : "d8803962e132",
      "status" : {
        "active" : true
      },
      "nodes" : [
        {
          "type" : "HDMI_DECODER",
          "index" : 0,
          "configuration" : {
            "edid" :
"00ffffffffffffffff0010ac82d0425152302219010380351e78ea2
f8180a9c0d1c001010101010101010101023a801871382d40582c450
d543935384a305251420a000000fc0044454c4c2053453234313
a20202020200140020317b14c9005040302071601141f12136
82c45000f282100001e011d8018711c1620582c25000f2821000
f282100001e8c0ad08a20e02d10103e96000f282100001800000
00000000000000000000000000000003d"
          },
          "status" : {}
        }
      ]
    }
  ],
  "error" : []
},

```

A.4 Command “get edid” for BlueRiver Receiver Device

The following is an example output for the command `get edid` called with a receiver device target argument.

Refer to Section 4.7 Command `get` for details.

```
{
  "status" : "SUCCESS",
  "request_id" : 284,
  "result" : {
    "devices" : [
      {
        "device_id" : "d88039631d77",
        "status" : {
          "active" : true
        },
        "nodes" : [
          {
            "type" : "HDMI_MONITOR",
            "index" : 0,
            "configuration" : {},
            "status" : {
              "connected" : true,
              "edid" :
"00ffffffffffff0009d11e8045540000031a0103803c22782efcd0a6544a9d240e5054a56b80d1c
081c081008180a9c0b300010101014dd000a0f0703e8030203500544f2100001a000000ff004b314
73030343934534c300a20000000fd001e4c1ea03c000a20202020202000000fc0042656e51204c4
3440a2020202001c402033af253101f0102030405060711121314151620615e5f230907078301000
06c030c0020003878200040010267d85dc40178c000e40f0000014dd000a0f0703e8030203500544
f2100001e565e00a0a0a0295030203500544f2100001e00000000000000000000000000000000
00000000000000000000000000000000cf"
            }
          ]
        ]
      }
    ],
    "error" : []
  },
  "error" : null
}
```

A.5 Command “get hello” for BlueRiver and UEX Devices

The following is an example output for the command `get hello` for the device ID `0016c04c3d9a`.

Refer to Section 4.7 Command `get` for details.

```
{
  status: SUCCESS,
  request_id: null,
  result: {
    devices: [
      {
        device_id: 0016c04c3d9a,
        device_name: name_0016c04c3d9a,
        identity: {
          firmware_version: 2.3.0.0,
          firmware_rc: 0,
          is_receiver: true,
          is_transmitter: true,
          chip_id: null
        },
        status: {
          active: true
        },
        nodes: [
          {
            type: NETWORK_INTERFACE,
            index: 0,
            configuration: {},
            status: {
              mac_address: 0016c04c3d9a,
              ip: {
                address: 10.17.252.104
              }
            },
            dhcp_option_43: {
              control_server_addresses: []
            }
          }
        ]
      }
    ],
    error: []
  },
  error: null
}
```

Semtech Confidential Information
Mark Li 02/03/2026

A.6 Command “get identity” for BlueRiver Devices

The following is an example output for the command `get identity` for BlueRiver devices.

Refer to Section 4.7 Command `get` for details.

```
{
  status: SUCCESS,
  request_id: null,
  result: {
    devices: [
      {
        device_id: 0016c04c3d9a,
        device_name: name_0016c04c3d9a,
        identity: {
          chipset_type: AVP2000T,
          engine: PLETHORA,
          vendor_id: 1,
          product_id: 77,
          firmware_comment: BlueRiver_RDK_2-0-0-0_evaluation,
          firmware_version: 2.3.0.0,
          firmware_rc: 0,
          is_receiver: true,
          is_transmitter: true,
          firmware_details: [
            {
              type: PRIMARY,
              details_available: true,
              firmware_version: 2.3.0.0,
              firmware_rc: 0
            },
            {
              type: GOLDEN,
              details_available: true,
              firmware_version: 2.3.0.0,
              firmware_rc: 0
            }
          ],
          chip_id: null
        },
        configuration: {
          device_name: name_0016c04c3d9a
        },
        status: {
          active: true
        },
        nodes: [
          {
            type: NETWORK_INTERFACE,
            index: 0,
            configuration: {},
            status: {
              mac_address: 0016c04c3d9a,
              ip: {
                address: 10.17.252.104
              }
            }
          }
        ]
      }
    ]
  }
}
```

```

        dhcp_option_43: {
            control_server_addresses: []
        }
    },
    usb_extenders: [
        {
            device_id: 0016c0355739,
            device_name: USB_0016c0355739,
            identity: {
                chipset_type: UEX1000,
                engine: USB2.0,
                vendor_id: 1,
                product_id: 212,
                firmware_comment: UEX 1.0.0.0_build986 uex_fpga,
                firmware_version: 1.0.0.0,
                chip_id: 0
            },
            configuration: {
                device_name: USB_0016c0355739,
            },
            status: {
                active: true,
            },
            nodes: [
                {
                    type: NETWORK_INTERFACE,
                    index: 0,
                    configuration: {},
                    status: {
                        mac_address: 0016c0355739,
                        ip: {
                            address: 10.17.252.115
                        }
                    }
                }
            ]
        }
    ],
    error: []
},
error: null
}

```

A.7 Command “get identity” for UEX Devices

The following is an example output for the command `get identity` for UEX devices.

Refer to Section 4.7 Command `get` for details.

```

{
    status: SUCCESS,
    request_id: null,
    result: {

```

```

    devices: [
      {
        device_id: 0016c04c3d9a,
        device_name: USB_0016c04c3d9a,
        identity: {
          chipset_type: UEX2000,
          engine: USB2.0,
          vendor_id: 1,
          product_id: 208,
          firmware_comment: UEX 1.0.0.0_build986 uex20_fpga,
          firmware_version: 1.0.0.0,
          chip_id: 0
        },
        configuration: {
          device_name: 0016C04C3D9A
        },
        status: {
          active: true
        },
        nodes: [
          {
            type: NETWORK_INTERFACE,
            index: 0,
            configuration: {},
            status: {
              mac_address: 0016c04c3d9a,
              ip: {
                address: 10.17.252.104
              }
            }
          }
        ]
      }
    ],
    error: []
  },
  error: null
}

```

A.8 Command “get list” for BlueRiver and UEX Devices

The following is an example output for the command `get list`, with the target of `all`. This command is typically called with `all` as a target argument as below in order to list all BlueRiver and UEX devices.

Refer to Section 4.7 Command `get` for details.

```

{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "devices" : [
      {
        "device_id" : "001ec0f03668",
        "status" : {
          "active" : true
        }
      }
    ]
  },

```

```
{
  {
    "device_id" : "d88039631d77",
    "status" : {
      "active" : true
    }
  },
],
"error" : [
  {
    "device_id" : "d88039634e85",
    "reason" : "DEVICE_DISCONNECTED",
    "message" : "Device is disconnected from the network"
  },
  {
    "device_id" : "d8803962ffa4",
    "reason" : "DEVICE_DISCONNECTED",
    "message" : "Device is disconnected from the network"
  }
]
},
"error" : null
}
```

Semtech Confidential Information
Mark Li
ADV
02/03/2026

A.9 Command “get settings” for BlueRiver Transceiver Device

The following is an example output for the command `get settings` called with a transceiver device.

Refer to Section 4.7 Command `get` for details.

```
{
  status: SUCCESS,
  request_id: null,
  result: {
    devices: [
      {
        device_id: 0016c04c3d9a,
        identity: {
          chipset_type: AVP2000T,
          engine: PLETHORA,
          vendor_id: 1,
          product_id: 77,
          firmware_comment: BlueRiver_RDK_2-0-0-0_evaluation,
          firmware_version: 2.3.0.0,
          firmware_rc: 0,
          is_receiver: true,
          is_transmitter: true,
          firmware_details: [
            {
              type: PRIMARY,
              details_available: true,
              firmware_version: 2.3.0.0,
              firmware_rc: 0
            },
            {
              type: GOLDEN,
              details_available: true,
              firmware_version: 2.2.0.0,
              firmware_rc: 0
            }
          ],
          chip_id: null
        },
        capabilities: {},
        configuration: {
          device_name: 0016C04C3D9A,
          locate_mode: false,
          secure_av: default,
          resume_streaming: true,
          claimed: false,
          authenticated: false,
          device_mode: TRANSCEIVER
        },
        status: {
          active: true,
          point_to_point: false,
          point_to_point_peer: ,
          boot_status: PRIMARY,
          update_in_progress: false,
          igmp_reporting_mode: QUERY,
          error_status: {

```

```

        has_error_code: false,
        error_code: 0
    },
    streams: [
        {
            type: HDMI,
            index: 0,
            configuration: {
                address: 0.0.0.0,
                enable: false,
                auto_stop: false,
                remove_audio: false
            },
            status: {
                state: STOPPED
            },
            source: {
                ref_class: NODE,
                ref_type: HDMI_DECODER,
                ref_index: 0
            }
        },
        {
            type: HDMI,
            index: 1,
            configuration: {
                address: 0.0.0.0,
                enable: false,
                auto_stop: false,
                remove_audio: true
            },
            status: {
                state: STOPPED
            },
            source: {
                ref_class: NODE,
                ref_type: SCALER,
                ref_index: 0
            }
        },
        {
            type: HDMI_AUDIO,
            index: 0,
            configuration: {
                address: 0.0.0.0,
                enable: false,
                auto_stop: false
            },
            status: {
                state: STOPPED
            },
            source: {
                ref_class: NODE,
                ref_type: HDMI_DECODER,
                ref_index: 0
            }
        }
    ]
}

```

```

    },
    {
        type: MULTICH_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: MULTICH_AUDIO_INPUT,
            ref_index: 0
        }
    },
    {
        type: STEREO_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: STEREO_AUDIO_INPUT,
            ref_index: 0
        }
    },
    {
        type: RS232,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            dest_index: 0,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: UART,
            ref_index: 0
        }
    },
    {
        type: RS232,

```

```

        index: 1,
        configuration: {
            address: 0.0.0.0,
            dest_index: 1,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: UART,
            ref_index: 1
        }
    },
    {
        type: RS232,
        index: 2,
        configuration: {
            address: 0.0.0.0,
            dest_index: 2,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: UART,
            ref_index: 2
        }
    },
    {
        type: RS232,
        index: 3,
        configuration: {
            address: 0.0.0.0,
            dest_index: 3,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: UART,
            ref_index: 3
        }
    },
    {
        type: INFRARED,
        index: 0,
        configuration: {

```



```

        address: 0.0.0.0,
        enable: false,
        auto_stop: false
    },
    status: {
        state: STOPPED
    },
    source: {
        ref_class: NODE,
        ref_type: INFRARED_DECODER,
        ref_index: 0
    }
},
{
    type: CEC,
    index: 0,
    configuration: {
        address: 0.0.0.0,
        enable: false,
        auto_stop: false
    },
    status: {
        state: STOPPED
    },
    source: {
        ref_class: NODE,
        ref_type: CEC,
        ref_index: 0
    }
},
{
    type: CEC,
    index: 1,
    configuration: {
        address: 0.0.0.0,
        enable: false,
        auto_stop: false
    },
    status: {
        state: STOPPED
    },
    source: {
        ref_class: NODE,
        ref_type: CEC,
        ref_index: 1
    }
},
{
    type: USB_HID,
    index: 0,
    configuration: {
        address: 0.0.0.0,
        enable: false,
        auto_stop: false
    },
    status: {

```

```

        state: STOPPED
    },
    source: {
        ref_class: NODE,
        ref_type: USB_HID,
        ref_index: 0
    }
},
subscriptions: [
    {
        type: HDMI,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 1,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 2,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 3,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    }
]

```

```

        index: 4,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 5,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 6,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 7,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 8,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 9,

```

```

        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 10,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 11,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 12,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 13,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 14,
        configuration: {

```

```

        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 15,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 16,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 17,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 18,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 19,
    configuration: {
        address: 0.0.0.0,

```

```

        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 20,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 21,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 22,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 23,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 24,
    configuration: {
        address: 0.0.0.0,
        enable: false
    }
}

```

```

    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 25,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 26,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 27,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 28,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },
    status: {
        state: STOPPED
    }
},
{
    type: HDMI,
    index: 29,
    configuration: {
        address: 0.0.0.0,
        enable: false
    },

```

```

        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 30,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 31,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false,
            mute_msec: 1500
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: STEREO_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: MULTICH_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
    },

```



```

        status: {
            state: STOPPED
        }
    },
    {
        type: RS232,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: RS232,
        index: 1,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: RS232,
        index: 2,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: RS232,
        index: 3,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: INFRARED,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {

```

```

        state: STREAMING
    },
    {
        type: USB_HID,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    }
],
nodes: [
    {
        type: STEREO_AUDIO_INPUT_OUTPUT,
        index: 0,
        configuration: {
            direction: INPUT
        },
        status: {},
        inputs: [
            {
                name: main,
                index: 0,
                configuration: {
                    source: {
                        value: 2
                    }
                },
                status: {
                    source: {
                        ref_class: SUBSCRIPTION,
                        ref_type: HDMI_AUDIO,
                        ref_index: 0
                    }
                }
            }
        ]
    },
    {
        type: CEC,
        index: 0,
        configuration: {
            p2p_mode: PASSTHROUGH
        },
        status: {},
        inputs: []
    },
    {
        type: CEC,
        index: 1,
        configuration: {
            p2p_mode: PASSTHROUGH
        }
    }
]

```

```

    },
    status: {},
    inputs: []
  },
  {
    type: COLOR_GENERATOR,
    index: 0,
    configuration: {
      enable: false,
      color: 000000,
      force_output: false
    },
    status: {},
    inputs: []
  },
  {
    type: FRAME_BUFFER,
    index: 0,
    configuration: {
      display_mode: FAST_SWITCHED,
      width: 0,
      height: 0,
      horiz_offset: 0,
      vert_offset: 0,
      frames_per_second: 0,
      keep_width: 0,
      keep_height: 0,
      viewport_horiz: 0,
      viewport_vert: 0,
      viewport_width: 0,
      viewport_height: 0,
      aspect_ratio_mode: NONE,
      has_auto_wall_details: false,
      wall_name: ,
      freeze: false
    },
    status: {
      source_video: {
        width: 0,
        height: 0,
        frames_per_second: 0,
        color_space: RGB,
        bits_per_pixel: 8,
        scan_mode: PROGRESSIVE
      }
    },
    inputs: [
      {
        name: main,
        index: 0,
        configuration: {},
        status: {
          source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 0
          }
        }
      }
    ]
  }
]

```

```

    }
  },
  {
    name: multiview,
    index: 0,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 0
      }
    }
  },
  {
    name: multiview,
    index: 1,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 1
      }
    }
  },
  {
    name: multiview,
    index: 2,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 2
      }
    }
  },
  {
    name: multiview,
    index: 3,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 3
      }
    }
  },
  {
    name: multiview,
    index: 4,
    configuration: {},
    status: {

```

```

        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 4
        }
    },
    {
        name: multiview,
        index: 5,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 5
            }
        }
    },
    {
        name: multiview,
        index: 6,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 6
            }
        }
    },
    {
        name: multiview,
        index: 7,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 7
            }
        }
    },
    {
        name: multiview,
        index: 8,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 8
            }
        }
    }
}

```

```

        name: multiview,
        index: 9,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 9
            }
        }
    },
    {
        name: multiview,
        index: 10,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 10
            }
        }
    },
    {
        name: multiview,
        index: 11,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 11
            }
        }
    },
    {
        name: multiview,
        index: 12,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 12
            }
        }
    },
    {
        name: multiview,
        index: 13,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 13
            }
        }
    }
}

```

```

    }
  },
  {
    name: multiview,
    index: 14,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 14
      }
    }
  },
  {
    name: multiview,
    index: 15,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 15
      }
    }
  },
  {
    name: multiview,
    index: 16,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 16
      }
    }
  },
  {
    name: multiview,
    index: 17,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 17
      }
    }
  },
  {
    name: multiview,
    index: 18,
    configuration: {},
    status: {

```

```

        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 18
        }
    },
    {
        name: multiview,
        index: 19,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 19
            }
        }
    },
    {
        name: multiview,
        index: 20,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 20
            }
        }
    },
    {
        name: multiview,
        index: 21,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 21
            }
        }
    },
    {
        name: multiview,
        index: 22,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 22
            }
        }
    }
}

```



```

        name: multiview,
        index: 23,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 23
            }
        }
    },
    {
        name: multiview,
        index: 24,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 24
            }
        }
    },
    {
        name: multiview,
        index: 25,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 25
            }
        }
    },
    {
        name: multiview,
        index: 26,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 26
            }
        }
    },
    {
        name: multiview,
        index: 27,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 27
            }
        }
    }
}

```

```

        }
    },
    {
        name: multiview,
        index: 28,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 28
            }
        }
    },
    {
        name: multiview,
        index: 29,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 29
            }
        }
    },
    {
        name: multiview,
        index: 30,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 30
            }
        }
    },
    {
        name: multiview,
        index: 31,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 31
            }
        }
    }
]
},
{
    type: HDMI_DECODER,
    index: 0,

```

```

configuration: {
    hdcp_support_enable: true,
    hdcp_22_support_disable: false
    default_quantization_level_override: disabled
},
status: {
    video: {
        width: 0,
        height: 0,
        frames_per_second: 0,
        color_space: RGB,
        bits_per_pixel: 8,
        scan_mode: PROGRESSIVE
    },
    has_video_details: true,
    video_details: {
        frame_rate: 0,
        pixel_clock: 0,
        total_width: 0,
        total_height: 0,
        hsync_width: 0,
        hsync_front_porch: 0,
        hsync_negative: false,
        vsync_width: 0,
        vsync_front_porch: 0,
        vsync_negative: false,
        vic: 0,
        has_hdmi_vic: false,
        hdmi_vic: 0,
        picture_aspect: ASPECT_NO_DATA,
        has_active_format: false,
        active_format: 0,
        it_content_type: NO_DATA,
        scan_information: NO_DATA,
        colorimetry: RGB,
        rgb_range: DEFAULT,
        ycc_range: LIMITED,
        has_hdr: false,
        hdr: SDR
    },
    has_audio_details: true,
    audio_details: {
        sampling_frequency: 44100,
        number_of_channels: 2
    },
    has_audio_details_extended: true,
    audio_details_extended: {
        channel_allocation: 0,
        sample_size: 0,
        level_shift_value: 0,
        lfe_playback_level: 0,
        audio_encoding_type: LPCM,
        is_downmix_inhibited: false
    },
    hdcp_protected: false,
    hdcp_version: NONE,

```

```

        hdmi_2_0_support: true,
        source_stable: false,
        audio_downmix_support: true
    },
    inputs: [
        {
            name: main,
            index: 0,
            configuration: {},
            status: {
                source: {
                    ref_class: NODE,
                    ref_type: VIDEO_INPUT,
                    ref_index: 0
                }
            }
        }
    ]
},
{
    type: HDMI_ENCODER,
    index: 0,
    configuration: {
        hdcp_output_mode: FOLLOW_SINK_1
        hdmi_tx_5v: AUTO
    },
    status: {
        video: {
            width: 0,
            height: 0,
            frames_per_second: 0,
            color_space: RGB,
            bits_per_pixel: 8,
            scan_mode: PROGRESSIVE
        },
        has_video_details: true,
        video_details: {
            frame_rate: 0,
            pixel_clock: 0,
            total_width: 0,
            total_height: 0,
            hsync_width: 0,
            hsync_front_porch: 0,
            hsync_negative: false,
            vsync_width: 0,
            vsync_front_porch: 0,
            vsync_negative: false,
            vic: 0,
            has_hdmi_vic: false,
            hdmi_vic: 0,
            picture_aspect: ASPECT_NO_DATA,
            has_active_format: false,
            active_format: 0,
            it_content_type: NO_DATA,
            scan_information: NO_DATA,
            colorimetry: RGB,

```

```

        rgb_range: DEFAULT,
        ycc_range: LIMITED,
        has_hdr: false,
        hdr: SDR
    },
    has_audio_details: false,
    audio_details: {},
    has_audio_details_extended: false,
    audio_details_extended: {},
    hdcp_protected: false,
    hdcp_version: NONE,
    hdmi_2_0_support: true,
    source_stable: false
},
inputs: [
    {
        name: main,
        index: 0,
        configuration: {
            source: {
                value: 0
            }
        },
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 0
            }
        }
    },
    {
        name: audio,
        index: 0,
        configuration: {
            source: {
                value: 2
            }
        },
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI_AUDIO,
                ref_index: 0
            }
        }
    }
]
},
{
    type: HDMI_MONITOR,
    index: 0,
    configuration: {},
    status: {
        connected: true
    },

```

```

        inputs: [
            {
                name: main,
                index: 0,
                configuration: {},
                status: {
                    source: {
                        ref_class: NODE,
                        ref_type: HDMI_ENCODER,
                        ref_index: 0
                    }
                }
            }
        ]
    },
    {
        type: MULTICH_AUDIO_INPUT,
        index: 0,
        configuration: {
            sampling_frequency: {
                value: 48000
            },
            number_of_channels: {
                value: 8
            },
            channel_allocation: {
                value: 0
            },
            audio_encoding_type: LPCM,
            mute_audio: false
        },
        status: {
            data_lanes: 4,
            sampling_frequency: 0
        },
        inputs: []
    },
    {
        type: MULTICH_AUDIO_OUTPUT,
        index: 0,
        configuration: {},
        status: {
            data_lanes: 4
        },
        inputs: [
            {
                name: main,
                index: 0,
                configuration: {
                    source: {
                        value: 8
                    }
                },
                status: {
                    source: {
                        ref_class: SUBSCRIPTION,

```

```

        ref_type: MULTICH_AUDIO,
        ref_index: 0
    }
}
}
]
},
{
    type: INFRARED_ENCODER,
    index: 0,
    configuration: {},
    status: {
        data_length_max: 1032
    },
    inputs: [
        {
            name: network,
            index: 0,
            configuration: {},
            status: {
                source: {
                    ref_class: SUBSCRIPTION,
                    ref_type: INFRARED,
                    ref_index: 0
                }
            }
        }
    ]
},
{
    type: INFRARED_DECODER,
    index: 0,
    configuration: {},
    status: {},
    inputs: []
},
{
    type: LED,
    index: 0,
    configuration: {
        function: {
            value: 0
        }
    },
    status: {},
    inputs: []
},
{
    type: NETWORK_INTERFACE,
    index: 0,
    configuration: {
        hostname: 0016C04C3D9A,
        ip: {
            address: 10.17.252.104,
            mode: DHCP,
            mask: 255.255.255.0,

```

```

        gateway: 10.17.252.254
    },
    dhcp_option_43: {
        control_server_addresses: []
    },
    status: {
        mac_address: 0016c04c3d9a,
        ip: {
            address: 10.17.252.104
        }
    },
    inputs: []
},
{
    type: NETWORK_PORT,
    index: 0,
    configuration: {
        speed: SPEED_10G
    },
    status: {
        active: true,
        speed: 10,
        supports_speed_10g: true,
        supports_speed_5g: false,
        supports_speed_2_5g: false,
        supports_auto_10g: false,
        supports_auto_5g: false
    },
    inputs: []
},
{
    type: NETWORK_SWITCH,
    index: 0,
    configuration: {
        gigabit_port_1_enable: true
    },
    status: {},
    inputs: []
},
{
    type: SCALER,
    index: 0,
    configuration: {
        width: 0,
        height: 0
    },
    status: {
        video: {
            width: 0,
            height: 0,
            frames_per_second: 0,
            color_space: RGB,
            bits_per_pixel: 8,
            scan_mode: PROGRESSIVE
        }
    }
}

```



```

    },
    inputs: [
        {
            name: main,
            index: 0,
            configuration: {},
            status: {
                source: {
                    ref_class: NODE,
                    ref_type: HDMI_DECODER,
                    ref_index: 0
                }
            }
        }
    ]
},
{
    type: UART,
    index: 0,
    configuration: {
        baud_rate: 57600,
        data_bits: 8,
        stop_bits: 1,
        parity: NONE
    },
    status: {
        usage: EXPOSED
    },
    inputs: [
        {
            name: network,
            index: 0,
            configuration: {},
            status: {
                source: {
                    ref_class: SUBSCRIPTION,
                    ref_type: RS232,
                    ref_index: 0
                }
            }
        }
    ]
},
{
    type: UART,
    index: 1,
    configuration: {
        baud_rate: 57600,
        data_bits: 8,
        stop_bits: 1,
        parity: NONE
    },
    status: {
        usage: EXPOSED
    },
    inputs: [

```

```

        {
            name: network,
            index: 1,
            configuration: {},
            status: {
                source: {
                    ref_class: SUBSCRIPTION,
                    ref_type: RS232,
                    ref_index: 1
                }
            }
        }
    ],
},
{
    type: UART,
    index: 2,
    configuration: {
        baud_rate: 57600,
        data_bits: 8,
        stop_bits: 1,
        parity: NONE
    },
    status: {
        usage: EXPOSED
    },
    inputs: [
        {
            name: network,
            index: 2,
            configuration: {},
            status: {
                source: {
                    ref_class: SUBSCRIPTION,
                    ref_type: RS232,
                    ref_index: 2
                }
            }
        }
    ]
},
{
    type: UART,
    index: 3,
    configuration: {
        baud_rate: 57600,
        data_bits: 8,
        stop_bits: 1,
        parity: NONE
    },
    status: {
        usage: EXPOSED
    },
    inputs: [
        {
            name: network,

```

```

        index: 3,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: RS232,
                ref_index: 3
            }
        }
    }
]
},
{
    type: USB_HID,
    index: 0,
    configuration: {
        role: OTG
    },
    status: {
        usb_devices: [
            {
                status: DISCONNECTED
            },
            {
                status: DISCONNECTED
            },
            {
                status: DISCONNECTED
            }
        ],
        usb_host: {
            status: DISCONNECTED
        },
        role: DISCONNECTED
    },
    inputs: []
},
{
    type: USB_EXTENDER,
    index: 0,
    configuration: {
        role: LOCAL,
        program_mode: NONE,
        rs232_port: 0,
        peers: [ {
            avp_attached: {
                device_id: 001122334455,
                device_name: AVP_001122334455,
            },
            device_id: 66778899AABB,
            device_name: USB_66778899AABB,
            mac_address: 66778899AABB,
        } ],
        device_filtering: {
            block_iso_transfers: false,
            class_filtering: ALLOW_ALL,
        }
    }
}

```

```

        allow_api_commands: false,
        allow_i2c_control: false,
    },
    remote_vbus_ctrl: AUTO,
    msa_enable: {
        bot_protocol: true,
        uas_protocol: true,
    }
},
status: {
    chip_present: true,
    device_id: 0016c03556e8,
    device_name: USB_0016c03556e8,
    mac_address: 0016c03556e8,
    chipset_type: UEX1000,
    reachable: true,
    revision: 1.0.0.0,
    ip_mode: DHCP,
    ip_address: 10.17.252.109,
    link_status_support: true,
    sui_enabled: false,
    peers: [ {
        linked: true
    } ],
    role: LOCAL,
},
inputs: []
},
{
    type: VIDEO_COMPRESSOR,
    index: 0,
    configuration: {
        always_compress: false,
        version: 1
    },
    status: {
        version_support: 2
    },
    inputs: []
},
{
    type: VIDEO_DECOMPRESSOR,
    index: 0,
    configuration: {
        version: 1
    },
    status: {
        version_support: 2
    },
    inputs: []
},
{
    type: VIDEO_INPUT,
    index: 0,
    configuration: {},
    status: {},
}

```

```

        inputs: []
    }
    ]
    },
    ],
    error: []
},
error: null
}

```

A.10 Command “get settings” for UEX Device

The following is an example output for the command `get settings` called with a transceiver device.

Refer to Section 4.7 Command `get` for details.

```

{
  status: SUCCESS,
  request_id: null,
  result: {
    devices: [
      {
        device_id: 44b7d0cb86a3,
        identity: {
          chipset_type: UEX2000,
          engine: USB2.0,
          vendor_id: 1,
          product_id: 212,
          firmware_comment: UEX 1.0.0.0 build986 uex20_fpga,
          firmware_version: 1.0.0.0,
          chip_id: 0
        },
        capabilities: {},
        configuration: {
          device_name: USB_44b7d0cb86a3,
          locate_mode: false,
          claimed: false
        },
        status: {
          active: true,
          point_to_point: false,
          point_to_point_peer: ,
          update_in_progress: false,
          error_status: {
            has_error_code: false,
            error_code: 0
          }
        },
        streams: [],
        subscriptions: [],
        nodes: [
          {
            type: NETWORK_INTERFACE,
            index: 0,
            configuration: {
              hostname: 44b7d0cb86a3,
              ip: {

```

```

        address: 10.17.252.104,
        mode: DHCP,
        mask: 255.255.255.0,
        gateway: 10.17.252.254
    },
    status: {
        mac_address: 44b7d0cb86a3,
        ip: {
            address: 10.17.252.104
        }
    },
    inputs: []
},
{
    type: USB_EXTENDER,
    index: 0,
    configuration: {
        role: LOCAL,
        program_mode: NONE,
        rs232_port: 0,
        peers: [ {
            device_id: 44b7d0cb86a9,
            device_name: USB_44b7d0cb86a9,
            mac_address: 44b7d0cb86a9,
        } ],
        device_filtering: {
            block_iso_transfers: false,
            class_filtering: ALLOW_ALL,
            allow_api_commands: false,
            allow_i2c_control: false,
        },
        remote_vbus_ctrl: AUTO,
        msa_enable: {
            bot_protocol: true,
            uas_protocol: true,
        }
    },
    status: {
        chip_present: true,
        device_id: 44b7d0cb86a3,
        device_name: USB_44b7d0cb86a3,
        mac_address: 44b7d0cb86a3,
        chipset_type: UEX2000,
        reachable: true,
        revision: 1.0.0.0,
        ip_mode: DHCP,
        ip_address: 10.17.252.109,
        link_status_support: true,
        sui_enabled: false,
        peers: [ {
            linked: true
        } ],
        role: LOCAL,
    },
    inputs: []
}

```

```

        }
    ]
    },
    ],
    error: []
},
error: null
}

```

A.11 Command “get path” for BlueRiver Receiver Device

The get command has a “paths” subset that returns all the paths applicable to a specific target (a path being a single source:subscription pair).

```

{
  "status": "SUCCESS",
  "request_id": null,
  "result": {
    "path_info" : [
      {
        "source": {
          "device_id" : "d8803962e132",
          "device_name" : "ROOM_1_TX",
          "stream": {
            "type": "HDMI",
            "index" : 0,
            "ip" : "224.1.1.1",
            "state" : STREAMING
          }
        },
        "subscribers" : [
          {
            "device_id" : "0016c0355869",
            "device_name" : "ROOM_1_RX_1",
            "subscription_index": 0
          },
          {
            "device_id" : "0016c035abcd",
            "device_name" : "ROOM_1_RX_2",
            "subscription_index": 0
          },
          ...
        ]
      },
      {
        "source": {
          ...
        },
        "subscribers"[
          {
            ...
          }
        ]
      }
    ]
  }
}

```

```

    }
    ...
  ],
  "error": []
},
"error": null
}

```

A.12 Command “get usb_tree” for BlueRiver or UEX Device

The get command has a "usb_tree" subset that returns the tree of the USB devices connected downstream of the UEX. The target shall be the AVP if the UEX is attached to an AVP and the UEX if UEX2000 is used standalone. If a second UEX is attached to the AVP, the index shall be 1 and the chip_id field of the return will be 1 (otherwise 0).

```

POST /api/device/{target}

{
  "op": "get",
  "value": "usb_tree"
  "type": "usb_extender", (optional)
  "index": 0 or 1 (optional)
}

```

Below is an example of a get USB tree API json answer:

```

{
  "status": "SUCCESS",
  "request_id": 64,
  "result": {
    "usb_tree" : [
      {
        "device_id" : "001122334455",
        "device_name" : "USB_001122334455"
        "downstream_tier_depth": 3,
        "chip_id": 0,
        "downstream_devices" : [
          {
            "element_type" : "hub",
            "hub_port" : 0,
            "speed": "high",
            "port_count" : 4,
            "hub_port" : 0,
            "downstream_devices" : [
              {
                "element_type" : "device",      <-- e.g. Gamer mouse
                "hub_port" : 1,
                "speed": "full"
              },
              {
                "element_type" : "device",      <-- e.g. Keyboard
                "hub_port" : 3,
                "speed": "low"
              }
            ]
          }
        ]
      }
    ]
  }
}

```

```
    }  
  ]  
},  
]  
},  
],  
"error": []  
},  
"error": null  
}
```

Semtech Confidential Information
Mark Li
ADV
02/03/2026

Appendix: Non Get Command Output Examples

A.1 Command “layout describe”

The following is an example output for the command `layout describe`.

Refer to Section 5.2.5 Command `layout describe` for details.

`layout compatibility_4k_2x2 describe`

```
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "layout_description" : {
      "name" : "compatibility_4k_2x2",
      "width" : 3840,
      "height" : 2160,
      "read_only" : true,
      "surfaces" : [
        {
          "index" : 0,
          "horizontal_position" : 0,
          "vertical_position" : 0,
          "width" : 4096,
          "height" : 2160
        },
        {
          "index" : 1,
          "horizontal_position" : 4160,
          "vertical_position" : 0,
          "width" : 2048,
          "height" : 1080
        },
        {
          "index" : 2,
          "horizontal_position" : 6240,
          "vertical_position" : 0,
          "width" : 2048,
          "height" : 1080
        },
        {
          "index" : 3,
          "horizontal_position" : 8320,
          "vertical_position" : 0,
          "width" : 2048,
          "height" : 1080
        },
        {
          "index" : 4,
          "horizontal_position" : 4160,
          "vertical_position" : 1080,
          "width" : 2048,
          "height" : 1080
        }
      ]
    }
  }
}
```

```

{
  "index" : 5,
  "horizontal_position" : 6240,
  "vertical_position" : 1080,
  "width" : 2048,
  "height" : 1080
},
{
  "index" : 6,
  "horizontal_position" : 8320,
  "vertical_position" : 1080,
  "width" : 2048,
  "height" : 1080
},
{
  "index" : 7,
  "horizontal_position" : 0,
  "vertical_position" : 2160,
  "width" : 2048,
  "height" : 1080
},
{
  "index" : 8,
  "horizontal_position" : 2080,
  "vertical_position" : 2160,
  "width" : 2048,
  "height" : 1080
},
{
  "index" : 9,
  "horizontal_position" : 4160,
  "vertical_position" : 2160,
  "width" : 2048,
  "height" : 1080
},
{
  "index" : 10,
  "horizontal_position" : 6240,
  "vertical_position" : 2160,
  "width" : 2048,
  "height" : 1080
},
{
  "index" : 11,
  "horizontal_position" : 8320,
  "vertical_position" : 2160,
  "width" : 2048,
  "height" : 1080
},
{
  "index" : 12,
  "horizontal_position" : 0,
  "vertical_position" : 3240,
  "width" : 2048,
  "height" : 856
},

```

```

{
  "index" : 13,
  "horizontal_position" : 2080,
  "vertical_position" : 3240,
  "width" : 2048,
  "height" : 856
},
{
  "index" : 14,
  "horizontal_position" : 4160,
  "vertical_position" : 3240,
  "width" : 2048,
  "height" : 856
},
{
  "index" : 15,
  "horizontal_position" : 6240,
  "vertical_position" : 3240,
  "width" : 2048,
  "height" : 856
},
{
  "index" : 16,
  "horizontal_position" : 8320,
  "vertical_position" : 3240,
  "width" : 2048,
  "height" : 856
},
{
  "index" : 17,
  "horizontal_position" : 0,
  "vertical_position" : 0,
  "width" : 2048,
  "height" : 1080
},
{
  "index" : 18,
  "horizontal_position" : 2080,
  "vertical_position" : 0,
  "width" : 2048,
  "height" : 1080
},
{
  "index" : 19,
  "horizontal_position" : 0,
  "vertical_position" : 1080,
  "width" : 2048,
  "height" : 1080
},
{
  "index" : 20,
  "horizontal_position" : 2080,
  "vertical_position" : 1080,
  "width" : 2048,
  "height" : 1080
}

```

```

],
"windows" : [
{
  "index" : 0,
  "horizontal_position" : 0,
  "vertical_position" : 0,
  "width" : 1920,
  "height" : 1080,
  "horizontal_offset" : 0,
  "vertical_offset" : 0,
  "content" : "VIDEO",
  "target_surface" : 0
},
{
  "index" : 1,
  "horizontal_position" : 1920,
  "vertical_position" : 0,
  "width" : 1920,
  "height" : 1080,
  "horizontal_offset" : 0,
  "vertical_offset" : 0,
  "content" : "VIDEO",
  "target_surface" : 1
},
{
  "index" : 2,
  "horizontal_position" : 0,
  "vertical_position" : 1080,
  "width" : 1920,
  "height" : 1080,
  "horizontal_offset" : 0,
  "vertical_offset" : 0,
  "content" : "VIDEO",
  "target_surface" : 2
},
{
  "index" : 3,
  "horizontal_position" : 1920,
  "vertical_position" : 1080,
  "width" : 1920,
  "height" : 1080,
  "horizontal_offset" : 0,
  "vertical_offset" : 0,
  "content" : "VIDEO",
  "target_surface" : 3
}
]
},
"error" : null
}

```

A.2 Command “MULTIVIEW_CHANGED” Event

The example below is the return of a request command sent to retrieve information for a MULTIVIEW_CHANGED event, note that the MULTIVIEW object is part of the FRAME_BUFFER node.

request 28

```
{
  "status" : "SUCCESS",
  "request_id" : 28,
  "result" : {
    "devices" : [
      {
        "device_id" : "0016c035566e",
        "status" : {
          "active" : true
        },
        "nodes" : [
          {
            "type" : "FRAME_BUFFER",
            "index" : 0,
            "configuration" : {
              "display_mode" : "MULTIVIEW",
              "multiview_layout" : {
                "name" : "picture_in_picture",
                "width" : 1920,
                "height" : 1080,
                "read_only" : false,
                "surfaces" : [
                  {
                    "index" : 0,
                    "horizontal_position" : 0,
                    "vertical_position" : 0,
                    "width" : 4096,
                    "height" : 2160
                  },
                  {
                    "index" : 1,
                    "horizontal_position" : 4160,
                    "vertical_position" : 0,
                    "width" : 2048,
                    "height" : 1080
                  },
                  {
                    "index" : 2,
                    "horizontal_position" : 6240,
                    "vertical_position" : 0,
                    "width" : 2048,
                    "height" : 1080
                  },
                  {
                    "index" : 3,
                    "horizontal_position" : 8320,
                    "vertical_position" : 0,
                    "width" : 2048,
                    "height" : 1080
                  }
                ]
              }
            }
          }
        ]
      }
    ]
  }
}
```

```

    },
    {
      "index" : 4,
      "horizontal_position" : 4160,
      "vertical_position" : 1080,
      "width" : 2048,
      "height" : 1080
    },
    {
      "index" : 5,
      "horizontal_position" : 6240,
      "vertical_position" : 1080,
      "width" : 2048,
      "height" : 1080
    },
    {
      "index" : 6,
      "horizontal_position" : 8320,
      "vertical_position" : 1080,
      "width" : 2048,
      "height" : 1080
    },
    {
      "index" : 7,
      "horizontal_position" : 0,
      "vertical_position" : 2160,
      "width" : 2048,
      "height" : 1080
    },
    {
      "index" : 8,
      "horizontal_position" : 2080,
      "vertical_position" : 2160,
      "width" : 2048,
      "height" : 1080
    },
    {
      "index" : 9,
      "horizontal_position" : 4160,
      "vertical_position" : 2160,
      "width" : 2048,
      "height" : 1080
    },
    {
      "index" : 10,
      "horizontal_position" : 6240,
      "vertical_position" : 2160,
      "width" : 2048,
      "height" : 1080
    },
    {
      "index" : 11,
      "horizontal_position" : 8320,
      "vertical_position" : 2160,
      "width" : 2048,
      "height" : 1080
    }
  ]
}

```

```

    },
    {
      "index" : 12,
      "horizontal_position" : 0,
      "vertical_position" : 3240,
      "width" : 2048,
      "height" : 856
    },
    {
      "index" : 13,
      "horizontal_position" : 2080,
      "vertical_position" : 3240,
      "width" : 2048,
      "height" : 856
    },
    {
      "index" : 14,
      "horizontal_position" : 4160,
      "vertical_position" : 3240,
      "width" : 2048,
      "height" : 856
    },
    {
      "index" : 15,
      "horizontal_position" : 6240,
      "vertical_position" : 3240,
      "width" : 2048,
      "height" : 856
    },
    {
      "index" : 16,
      "horizontal_position" : 8320,
      "vertical_position" : 3240,
      "width" : 2048,
      "height" : 856
    },
    {
      "index" : 17,
      "horizontal_position" : 0,
      "vertical_position" : 0,
      "width" : 2048,
      "height" : 1080
    },
    {
      "index" : 18,
      "horizontal_position" : 2080,
      "vertical_position" : 0,
      "width" : 2048,
      "height" : 1080
    },
    {
      "index" : 19,
      "horizontal_position" : 0,
      "vertical_position" : 1080,
      "width" : 2048,
      "height" : 1080
    }
  ]
}

```



```

    },
    {
        "index" : 20,
        "horizontal_position" : 2080,
        "vertical_position" : 1080,
        "width" : 2048,
        "height" : 1080
    }
],
"windows" : [
    {
        "index" : 0,
        "horizontal_position" : 960,
        "vertical_position" : 270,
        "width" : 960,
        "height" : 540,
        "horizontal_offset" : 0,
        "vertical_offset" : 0,
        "content" : "VIDEO",
        "target_surface" : 1
    },
    {
        "index" : 1,
        "horizontal_position" : 0,
        "vertical_position" : 0,
        "width" : 1920,
        "height" : 1080,
        "horizontal_offset" : 0,
        "vertical_offset" : 0,
        "content" : "VIDEO",
        "target_surface" : 0
    }
]
},
"status": {}
}
]
}
],
},
"error" : null
}

```

8 Revision History

Version	Date	Revision
3.9.0.0	December 2025	Removed USB Icron syntax support - Removed usb_icron node, icron command, usb_icron stream_type, usb_icron subscription_type - Affects capabilities, commands pair, unpair, switch, stop, capabilities, netstat, set property, etc. - See PDS-63049 SDVoE_API_Icron_to_USB_Extender_Migration_Guide. Added support of a second UEX USB 2.0 extender attached to an AVP - Affects get capabilities / identity / usb_tree, USB_EXTENDER node, pair / unpair, set ip mode / property, claim / release, factory, reboot. Added freeze on command support (e.g. to manually force a freeze of the receiver video output during a change of the video source at the HDMI input of the currently subscribed transmitter device) - Added freeze_video in capabilities and freeze configuration in FRAME_BUFFER node (volatile configuration only applicable if the receiver device is currently in one of the fast switched display modes) Added HDMI output monitor (sink device) HDCP level support info - Added hdcp_support status in HDMI_MONITOR node Added external HDMI transceiver chip (e.g. Lontium LT86121GXE) video compression support for FlexLinQ - Added external_video_compression/decompression_supported in capabilities, max_video_bandwidth configuration in HDMI_DECODER node and external_video_compression_type status in HDMI_DECODER and HDMI_ENCODER nodes Added unidirectional_point_to_point_only (data diode) in settings, reporting if the device only supports unidirectional point to point fiber transmission (data diode), allowing transport of video and audio from transmitter device to receiver device and blocking data to return from receiver device to transmitter device
3.8.0.1	June 2025	Fixed typo in version number of this document
1.9	February 2025	Added Semtech UEX USB 2.0 extender support (UEX1000 or UEX2000 attached to BlueRiver AVP or UEX2000 used standalone) - new usb_extender node and stream_type replacing usb_icron that will be deprecated in next release, - affects commands pair/unpair/switch/stop, claim/release, event (DEVICE_CONNECTED, DEVICE_DISCONNECTED, DEVICE_INFO_AVAILABLE, FIRMWARE_UPDATE_PROGRESS, SETTINGS_CHANGED and new USB_TREE_CHANGED), factory, get (capabilities, device, hello, identity, list, settings and new usb_tree), list, locate, netstat, reboot, request, set (ip, property), usb_extender, volatile_settings), - affects device identity (chipset_type, engine, vendor_id, product_id, firmware_comment, firmware_vesrion, chip_ip now supported for

Version	Date	Revision
		UEX) - affects device capabilities (usb_extender, local_role_supported, remote_role_supported, legacy_mode now supported for UEX), - affects device_status (active, point_to_point, point_to_point_peer, update_in_progress, error_status now supported for UEX), - added/modify Appendix: Get Command Output Examples for UEX get device, get hello, get identity, get list, get settings and get usb_tree. Added new update_stream command to reload, modify or add a new bitmap image stream supported in BlueRiver Bitmap Streamer version 1.1.0.0.
1.8	April 2024	Added support of the BlueRiver Bitmap Streamer software device 1.0.0.0. Added support of new event_id argument -1 value for API command event in 4.5 Command event 4.5.1 Arguments, allowing to retrieve only the last event. Added new support of new timestamp argument for API command get in 4.7 Command get 4.7.1 Arguments, allowing to retrieve the API time stamp. Support for 802.1X added to the SDVoE solution. For API three options added 4.19.1 Command set dot1x_certificates, Table 10: Subset Arguments for "get" Command updated adding new argument to retrieve list of current certificates and 6.3 Device Capabilities (DeviceCapabilities) added dot1x capability. For details on this feature refer to the SDVoE Developers API Application Note Library. Support for sending WoL added to the SDVoE solution. For API two options added 4.18 Command send_wol and 6.3 Device Capabilities (DeviceCapabilities) added dot1x capability. For details on this feature refer to the SDVoE Developers API Application Note Library. Support for sending "Hello" packets to IP of DHCP option 43 added to the SDVoE solution. For API two options added 6.8.17 Network Interface Node (NETWORK_INTERFACE) updated adding new field to retrieve DHCP option 43 IP and 6.3 Device Capabilities (DeviceCapabilities) added dhcp_option_43 capability. For details on this feature refer to the SDVoE Developers API Application Note Library. Added USB-HID remote extender statistics in 6.8.23 USB Human Interface Device (HID) Extender Node (USB_HID) 6.8.23.2 Status (UsbNodeStatus). Added documentation of already existing peers (pairing_type, address and device_id) in 6.8.23 USB Human Interface Device (HID) Extender Node (USB_HID) 6.8.23.1 Configuration (UsbNodeConfig). Added documentation of already existing role in 6.8.23 USB Human Interface Device (HID) Extender Node (USB_HID) 6.8.23.2 Status (UsbNodeStatus). Added documentation of already existing GPIO functionality of SDVoE solution. For API two options added 4.19.3 Command set gpio and Table 10: Subset Arguments for "get" Command updated adding new argument to retrieve status of GPIO. For details on this feature refer to the Semtech Blueriver HDK Application Note Library. Added documentation of already existing device configuration volatile_settings object member in 6.4 Device configuration (DeviceConfig), allowing user to prevent settings to be saved in flash.

Version	Date	Revision
		Removed set property option streams[HDMI:0].configuration.device_name value mac_address of to the 7.2 Stream Configuration table, corrected to add it instead in the set property option configuration.device_name (new note about special keyword value mac_address to reset the device name to its MAC address) in the 7.1 Device Configuration table.
1.7	September 2023	Add set property key to add ability configure the CEC addresses from the API server, refer to Section 7.5 CEC Node Configuration for details. To the Appendix: Get Command Output Examples added A7 the new get paths. The SFP read command added, 4.20 Command sfp read. New set property option streams[HDMI:0].configuration.device_name value mac_address added to the 7.2 Stream Configuration table.
1.6	March 2022	Added the configuration.locate_mode to section 7.1 Device Configuration table. Details in Table 116: Color Generator Node Object Members clarified regarding device chipset type required. The hdmi_tx_5v node added to section 6.8.11 HDMI Encoder Node (HDMI_ENCODER) and section 7.10 HDMI Encoder Node Configuration.
1.5	February 2023	Updated to add note that the device name is persistent following a factory reset as of the API 3.5 release. Also added HDMI Audio Node Configuration to section the Configurable Device Properties for the mute_msec option. New wall_name added to list of set video arguments.
1.4	June 2022	Updated section “Requests Associated with Event MULTIVIEW_CHANGED” and added Command “MULTIVIEW_CHANGED” event to Appendix: Non Get Command Output Examples.
1.3	May 2022	New API version 3.4 released, modified the command set ip to include the new AUTO_IP option, added new get multiview command and new pair command for Icron stream type. Also, updated the Frame Buffer node and Icron node, as they were impacted by these new commands.
1.2	January 2022	Remove the NULL_SOURCE node from section Nodes (Node) and update table Node Object members to reflect this change. Also, added to section 7.5 CEC Node Configuration that when in API mode, a device allows CEC data to be injected from the API to logical address “14” (0x0E – Free use).
1.1	October 2021	Modified command switch to include Icron pairing, as well modified the Icron node to reflect the new pairing feature.
1.0	July 2021	Initial revision created for the SDVoE API release.

IMPORTANT NOTICE

Information relating to this product and the application or design described herein is believed to be reliable, however such information is provided as a guide only. Semtech and SDVoE Alliance (collectively, "Parties") assume no liability for any errors in this document, or for the application or design described herein. The Parties reserve the right to make changes to their respective products or this document at any time without notice. Buyers should obtain the latest relevant information before placing orders and should verify that such information is current and complete. The Parties warrant performance of their respective products to the specifications applicable at the time of sale, and all sales are made in accordance with their respective standard terms and conditions of sale.

NEITHER PARTY'S PRODUCTS ARE DESIGNED, INTENDED, AUTHORIZED OR WARRANTED TO BE SUITABLE FOR USE IN LIFE-SUPPORT APPLICATIONS, DEVICES OR SYSTEMS, OR IN NUCLEAR APPLICATIONS IN WHICH THE FAILURE COULD BE REASONABLY EXPECTED TO RESULT IN PERSONAL INJURY, LOSS OF LIFE OR SEVERE PROPERTY OR ENVIRONMENTAL DAMAGE. INCLUSION OF EITHER PARTY'S PRODUCTS IN SUCH APPLICATIONS IS UNDERSTOOD TO BE UNDERTAKEN SOLELY AT THE CUSTOMER'S OWN RISK. Should a customer purchase or use either Party's products for any such unauthorized application, the customer shall indemnify and hold such Party and their officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs damages and attorney fees which could arise.

Semtech, the Semtech logo and BlueRiver are registered trademarks of Semtech Corporation or its subsidiaries. The SDVoE Alliance name is a registered trademark of SDVoE Alliance. All other trademarks and trade names mentioned may be marks and names of Semtech, SDVoE Alliance or their respective licensors. The Parties reserve the right to make changes to, or discontinue any products described in this document without further notice. The Parties make no warranty, representation or guarantee, express or implied, regarding the suitability of their respective products for any particular purpose.

© Semtech 2025. All rights reserved.

For more information contact: api@sdvov.org